

GIÁO TRÌNH BẢO TRÌ DỰ ĐOÁN & SINH DỮ LIỆU LỖI

Mục lục

Cho bài toán A1 — DENSO Factory Hacks 2026

Predictive-Maintenance AI Despite Scarce Fault Data

Mở đầu: đọc nhanh trong 5 phút

BẮN ĐỒ TƯ DUY A1 — mục tiêu lớn → bài toán con (đọc trước tiên)

Cách dùng giáo trình này

PHẦN 1 · NỀN TẢNG TOÁN HỌC CHO ML/DL — NỀN TẢNG TOÁN HỌC CHO MACHINE LEARNING / DEEP LEARNING

Sơ đồ: vì sao học sinh toán này lại cần cho A1? (Cây bài toán con)

1.1 Vector và Ma trận

1.1.1 Vector là gì?

1.1.2 Ma trận là gì?

1.1.3 Vì sao ma trận quan trọng với ML?

1.2 Các phép toán cơ bản trên ma trận

1.2.1 Cộng, trừ ma trận

1.2.2 Nhân ma trận với số vô hướng (scalar)

1.2.3 Nhân hai ma trận (matrix multiplication)

1.2.4 Dot product (tích vô hướng) của hai vector

1.2.5 Phép nhân ma trận = tập hợp nhiều dot product

1.2.6 Transpose (chuyển vị)

1.2.7 Ma trận đơn vị và ma trận nghịch đảo

1.3 Norm và chuẩn hóa

1.3.1 Norm L2 (chuẩn Euclid) — hay dùng nhất

1.3.2 Norm L1 (chuẩn Manhattan)

1.3.3 Norm Frobenius — dành cho ma trận

1.3.4 Chuẩn hóa vector (normalization)

1.4 Eigenvalue, Eigenvector và SVD

1.4.1 Eigenvector và eigenvalue — ý tưởng

1.4.2 Ý nghĩa trực quan

1.4.3 SVD — Singular Value Decomposition

1.4.4 SVD → PCA (Principal Component Analysis)

1.5 Vì sao Deep Learning cần matrix math?

1.5.1 Batch — xử lý hàng loạt

1.5.2 Layer tuyến tính = nhân ma trận

1.5.3 GPU và sự song song

1.5.4 Tóm tắt Đại số tuyến tính

2.1 Đạo hàm

2.1.1 Định nghĩa

2.1.2 Bảng đạo hàm hay dùng

2.1.3 Vì sao DL cần đạo hàm?

2.2 Đạo hàm riêng và Gradient

2.2.1 Đạo hàm riêng

2.2.2 Gradient — vector của mọi đạo hàm riêng

2.2.3 Gradient trong ML

2.3 Chain rule — nền tảng của Backprop (TRỌNG TÂM)

2.3.1 Chain rule cho hàm một biến

2.3.2 Chain rule nhiều biến — trọng tâm of backprop

2.3.3 Vì sao backprop = "gọn" hơn tính trực tiếp?

2.3.4 Ví dụ số hoàn chỉnh (1 nơ-ron)

2.3.5 Tổng kết trực quan

2.4 Gradient Descent — trực giác

2.4.1 Thuật toán

2.4.2 Loss landscape — "địa hình" của hàm mất mát

2.4.3 Ảnh hưởng của Learning Rate

2.4.4 Variants — SGD, Mini-batch, Adam

- 2.4.5 Loss landscape trong Predictive Maintenance
- 2.5 Taylor và tính lồi (convexity) — khái niệm ngắn
 - 2.5.1 Khai triển Taylor
 - 2.5.2 Hàm lồi (convex)
- 3.1 Khái niệm cơ bản
- 3.2 Các phân phối phổ biến
 - 3.2.1 Phân phối Uniform (đều)
 - 3.2.2 Phân phối Bernoulli
 - 3.2.3 Phân phối Gaussian (chuẩn / normal)
 - 3.2.4 Quy tắc 68-95-99.7 (chuẩn)
- 3.3 Kỳ vọng, Phương sai
 - 3.3.1 Kỳ vọng (Expected value / Mean)
 - 3.3.2 Phương sai (Variance)
 - 3.3.3 Đối với chuỗi thời gian
- 3.4 Covariance và Correlation
 - 3.4.1 Covariance (hiệp phương sai)
 - 3.4.2 Correlation (tương quan Pearson)
 - 3.4.3 Vì sao quan trọng với chuỗi rung?
- 3.5 MLE — Likelihood và vì sao dùng MSE, Cross-Entropy
 - 3.5.1 Likelihood là gì?
 - 3.5.2 MLE — Maximum Likelihood Estimation
 - 3.5.3 MLE dẫn đến MSE — khi dữ liệu nhiễu Gaussian
 - 3.5.4 MLE dẫn đến Cross-Entropy — khi dữ liệu Bernoulli (phân loại nhị phân)
 - 3.5.5 Mối quan hệ với log-likelihood — tóm gọn
- 3.6 Định lý Bayes
 - 3.6.1 Công thức
 - 3.6.2 Ví dụ số — Predictive Maintenance
 - 3.6.3 Vì sao dùng trong ML?
- 3.7 Bias / Variance và Overfitting / Underfitting
 - 3.7.1 Phân rã bias-variance của sai số
 - 3.7.2 Bốn trạng thái điển hình
 - 3.7.3 Định nghĩa chuẩn
 - 3.7.4 Tradeoff — đánh đổi
 - 3.7.5 K-fold Cross-validation
- 3.8 Bất cân bằng dữ liệu và tổng kết thống kê
 - 3.8.1 Vấn đề lớp hiếm trong Predictive Maintenance
 - 3.8.2 Bảng tóm tắt các công thức thống kê chính
- 4.1 Định nghĩa chuỗi thời gian và tính dừng (Stationarity)
 - 4.1.1 Chuỗi thời gian
 - 4.1.2 Tính dừng (Stationarity) — khái niệm cốt lõi
 - 4.1.3 Kiểm tra tính dừng
 - 4.1.4 Xử lý khi không dừng
- 4.2 Autocovariance và Autocorrelation
 - 4.2.1 Autocovariance
 - 4.2.2 Autocorrelation / ACF
 - 4.2.3 Vì sao ACF hữu ích cho rung?
 - 4.2.4 Partial autocorrelation (PACF) — khái niệm
- 4.3 Rolling Statistics (thống kê cửa sổ trượt)
 - 4.3.1 Định nghĩa
 - 4.3.2 Vì sao dùng?
 - 4.3.3 Chọn độ dài cửa sổ W
- 4.4 Thang đo cửa sổ, lookback: từ quá khứ → tương lai
 - 4.4.1 Vấn đề: phải dùng quá khứ để dự đoán tương lai
 - 4.4.2 Ví dụ số minh họa lookback
 - 4.4.3 Cấu trúc dữ liệu huấn luyện (sliding window)
 - 4.4.4 Bẫy rò rỉ (data leakage) — cực kỳ quan trọng
 - 4.4.5 Baseline timeline cần chú ý

PHẦN 2 · MACHINE LEARNING CỐT LỖI & ANOMALY DETECTION — MACHINE LEARNING CỐT LỖI CHO BÀI TOÁN PREDICTIVE MAINTENANCE & ANOMALY DETECTION

- 1.1. Mô tả bài toán
- 1.2. Thách thức cốt lõi
- 1.3. Vì sao dữ liệu rung lại quan trọng nhất?
- 1.4. Luồng tư duy chung để giải A1
- 2.1. Window và overlap cơ bản (nền tảng trước tiên)
- 2.2. Feature miền thời gian (time-domain features)
 - 2.2.1. Mean (trung bình)
 - 2.2.2. Standard deviation (độ lệch chuẩn) — std
 - 2.2.3. RMS (Root Mean Square) — căn bậc hai trung bình bình phương
 - 2.2.4. Peak (đỉnh) & Peak-to-peak
 - 2.2.5. Skewness (độ lệch bất đối xứng)
 - 2.2.6. Kurtosis (độ nhọn)
 - 2.2.7. Crest factor (hệ số đỉnh)
 - 2.2.8. Shape factor (hệ số hình dạng)
 - 2.2.9. Impulse factor (hệ số xung)
 - 2.2.10. Bảng tổng kết feature thời gian + khoảng giá trị tham khảo
- 2.3. Giới thiệu biến đổi Fourier và feature miền tần số
 - 2.3.1. Vì sao miền tần số?
 - 2.3.2. FFT trong một đoạn ngắn
 - 2.3.3. Spectral centroid (tâm phổ)
 - 2.3.4. Spectral spread (độ trải phổ)
 - 2.3.5. Spectral flatness (độ phẳng phổ)
 - 2.3.6. Band energy (năng lượng theo dải tần)
 - 2.3.7. Window function (hàm cửa sổ) — khái niệm bắt buộc
- 2.4. Tần số lấy mẫu, Nyquist và aliasing (nền tảng — đọc kỹ)
 - 2.4.1. Tần số Nyquist
 - 2.4.2. Aliasing (biệt danh tần số)
 - 2.4.3. Độ phân giải tần số
- 2.5. Chuẩn hóa (normalization / scaling)
 - 2.5.1. Z-score (standardization)
 - 2.5.2. Min-max scaling
 - 2.5.3. RobustScaler
 - 2.5.4. Ví dụ số minh họa
- 2.6. Gợi ý danh sách feature tổng hợp cho A1 (mức mở rộng)
- 3.1. Logistic Regression (hồi quy logistic)
 - 3.1.1. Ý tưởng
 - 3.1.2. Công thức
 - 3.1.3. Loss function (cross-entropy)
 - 3.1.4. Ví dụ số cực ngắn
- 3.2. Decision Tree → Random Forest
 - 3.2.1. Decision Tree: ý tưởng
 - 3.2.2. Độ đo chia (split criterion)
 - 3.2.3. Rủi ro của một cây đơn: overfitting
 - 3.2.4. Random Forest: Bagging + random feature
 - 3.2.5. Out-of-bag (OOB) score
- 3.3. Gradient Boosting / XGBoost
 - 3.3.1. Ý tưởng cốt lõi: "sửa sai từng bước"
 - 3.3.2. Loss function đối với phân loại
 - 3.3.3. Regularization L1 / L2
 - 3.3.4. Vì sao XGBoost mạnh với dữ liệu tabular / feature-engineered?
- 3.4. So sánh Random Forest vs XGBoost — khi dùng cái nào?
 - 3.4.1. Xử lý imbalance trong XGBoost
- 4.1. Accuracy — vì sao gây hiểu lầm khi class lệch?
- 4.2. Confusion matrix (ma trận nhầm lẫn)
- 4.3. Precision, Recall, F1
 - 4.3.1. Ví dụ số quyết định
- 4.4. PR curve vs ROC/AUC — vì sao PR quan trọng cho lớp hiếm
 - 4.4.1. Tư tưởng: model cho xác suất → di chuyển threshold

4.4.2. PR curve
4.4.3. ROC/AUC — cảnh báo
4.5. Threshold tuning & precision–recall tradeoff (chọn theo business)
4.5.1. Precision–recall tradeoff
4.5.2. Chi phí bỏ sót lỗi vs báo động giả — phân tích business
4.5.3. Mẹo thực hành threshold
4.6. MAE / RMSE cho RUL (nếu làm dự báo tuổi thọ)
5.1. One-class SVM (OC-SVM)
5.1.1. Ý tưởng
5.1.2. Kernel trick
5.1.3. Tham số nu
5.2. Isolation Forest
5.2.1. Ý tưởng nghịch đảo
5.2.2. Ưu điểm
5.2.3. Hạn chế
5.3. Autoencoder + reconstruction error (giới thiệu — chi tiết phần Deep Learning)
5.4. Statistical / heuristic: z-score, rolling threshold, EWMA, CUSUM
5.4.1. Z-score (chung cho 1 biến đo)
5.4.2. Rolling threshold (ngưỡng cuộn theo thời gian)
5.4.3. EWMA (Exponential Weighted Moving Average)
5.4.4. CUSUM (Cumulative Sum) — kiểm tra thay đổi phát hiện
5.5. Cài ngưỡng khi KHÔNG có nhãn
5.5.1. Percentile (phân vị)
5.5.2. Heuristic theo vật lý
5.5.3. Kết hợp "score + thời gian": ngưỡng mềm
5.5.4. Self-validation khi dữ liệu normal sạch
5.6. Cross-validation cho anomaly: hạn chế & data leakage
5.6.1. Vấn đề cơ bản với dữ liệu chuỗi thời gian
5.6.2. Quy tắc vàng: KHÔNG shuffle giữa past/future
5.6.3. Hạn chế của CV khi thiếu nhãn/thiếu dữ liệu lỗi
6.1. Oversampling / undersampling
6.1.1. Undersampling (giảm mẫu lớp đông)
6.1.2. Oversampling (nhân mẫu lớp thiếu số)
6.2. SMOTE (Synthetic Minority Over-sampling Technique)
6.2.1. Ý tưởng
6.2.2. Ví dụ số
6.2.3. Ưu / nhược
6.3. Data augmentation bằng dữ liệu tổng hợp (GAN / diffusion) — giới thiệu
6.3.1. GAN (Generative Adversarial Network)
6.3.2. Diffusion model
6.4. Protocol so sánh trước/sau augmentation — vì sao tránh data leakage
6.4.1. Protocol chuẩn mực (khi có MỘT VÀI lỗi thật)
Hiện trạng split của team (cập nhật mới nhất — phải đọc kỹ)
6.4.2. Con số baseline THẬT của team (chạy trên split mới này)
6.4.3. Vì sao tránh data leakage (điểm mẫu chốt)
6.4.4. Kỳ vọng thực tế của augmentation
7.1. Thiết lập bài toán
7.2. Loss functions cho hồi quy
7.3. Đánh giá
7.4. Lưu ý chuỗi thời gian trong RUL
8.1. Sơ đồ tổng thể
8.2. Quyết định quan trọng rút gọn (decision guidance)
8.3. Checklist trước nộp bài (demo day)
PHẦN 3 · DEEP LEARNING & GENERATIVE MODELS — DEEP LEARNING & GENERATIVE MODELS CHO CHUỖI THỜI GIAN CÔNG NGHIỆP + SINH DỮ LIỆU LỖI GIẢ
CÂY BÀI TOÁN CON — định vị từng mô hình sẽ học trong Phần 3
1.1. Từ Perceptron đến MLP (Multi-Layer Perceptron)
1.1.1. Perceptron: nụ cười đầu tiên
1.1.2. MLP: không chỉ một neuron

- 1.2. Các hàm kích hoạt (activation functions)
- 1.3. Loss functions (hàm mất mát)
 - 1.3.1. MSE (Mean Squared Error) — cho hồi quy và tái tạo tín hiệu
 - 1.3.2. Cross-entropy — cho phân loại
 - 1.3.3. Tóm tắt chọn loss cho A1
- 1.4. Backpropagation: quy tắc chuỗi lan ngược
 - 1.4.1. Chain rule
 - 1.4.2. Backward pass từng bước (pseudocode)
- 1.5. Optimizers: cách đi xuống dốc
 - 1.5.1. SGD (Stochastic Gradient Descent)
 - 1.5.2. Momentum
 - 1.5.3. Adam (Adaptive Moment Estimation) — lựa chọn mặc định
- 1.6. Overfitting và các biện pháp chống
- 1.7. PyTorch căn bản: tensor, autograd, vòng lặp huấn luyện
 - 1.7.1. Tensor: dữ liệu đa chiều
 - 1.7.2. Autograd: tự động tính gradient
 - 1.7.3. Vòng lặp huấn luyện chuẩn
- 2.1. Kiến trúc: encoder – bottleneck – decoder
- 2.2. Huấn luyện: tái tạo + reconstruction loss
- 2.3. Anomaly detection với AE
 - 2.3.1. Logic chính
 - 2.3.2. Ví dụ trực giác
 - 2.3.3. Biến thể nhanh
- 2.4. Variational Autoencoder (VAE) — giới thiệu ý tưởng
- 2.5. Code PyTorch — AE trên window feature (khớp src/ae.py)
- 3.1. Mô hình tuần tự và nỗi đau vanishing gradient
 - 3.1.1. Tại sao cần mô hình tuần tự?
 - 3.1.2. Vì sao RNN thuần khó train? — Vanishing / Exploding gradient
- 3.2. LSTM: Long Short-Term Memory
 - 3.2.1. Ý tưởng
 - 3.2.2. Công thức bốn cổng
- 3.3. GRU: Gated Recurrent Unit (ngắn gọn)
- 3.4. Ứng dụng cho A1: hai hướng chính
 - (a) Phân loại window normal vs abnormal (nhị phân hoặc đa-lớp-lỗi)
 - (b) Dự báo RUL (Remaining Useful Life) hồi quy
 - (c) Sequence-to-sequence (nâng cao, cùng ý tưởng)
- 3.5. Code ngắn — LSTM classifier PyTorch
- 4.1. Self-attention: Q, K, V
- 4.2. Multi-head attention & positional encoding
- 4.3. Liên hệ với chuỗi thời gian: attention thay thế temporal dependency
- 4.4. Trong thực tế bài A1 (lời khuyên quan trọng)
- 5.1. Ý tưởng minimax game
- 5.2. Kiến trúc train — hai model đấu nhau
- 5.3. Mode collapse và training instability
- 5.4. GAN cho chuỗi thời gian: TimeGAN (NeurIPS 2019)
 - 5.4.1. Vì sao GAN thuần sinh chuỗi khó?
 - 5.4.2. Ý tưởng cốt lõi TimeGAN
- 5.5. Lưu ý thực hành với chuỗi rung
- 6.1. DDPM (Denoising Diffusion Probabilistic Models) — ý tưởng
 - Forward noising — công thức
 - Reverse denoising — công thức
- 6.2. Mất mát — epsilon-prediction
 - Ví dụ số (bước cô đọng)
- 6.3. Sampling — lặp khử nhiễu
- 6.4. Diffusion-TS (ICLR 2024) — diffusion cho chuỗi thời gian
 - Các điểm chính
 - Công thức gộp loss (ý niệm)
- 6.5. So sánh GAN vs Diffusion cho sinh dữ liệu

6.6. Hướng thứ nhất — Fault Injection / Mô phỏng vật lý: tạo dữ liệu lỗi bằng cách "bơm lỗi"

- 6.6.1. Bản chất là gì (từ gốc): tín hiệu = healthy + fault signature + noise
- 6.6.2. Cơ sở vật lý: chữ ký tần số của từng loại lỗi (có công thức)
- 6.6.3. Toán học của việc "bơm": mô hình tín hiệu + ví dụ số
- 6.6.4. Áp dụng trong code/project của team
- 6.6.5. Ví dụ mã Python (giải thuật bơm lỗi)
- 6.6.6. Ưu / nhược + so sánh nội bộ
- 6.6.7. Bẫy / lưu ý thực hành (rất dễ mất điểm)
- 6.6.8. Kết nối với phần còn lại của giáo trình

6.7. Đối chiếu hai hướng sinh dữ liệu lỗi (vật lý vs generative) & cột "công nghệ gợi ý"

- 6.7.1. Mục tiêu
- 6.7.2. Bối cảnh / bài toán con
- 6.7.3. Bảng so sánh tổng hợp hai hướng
- 6.7.4. Vì sao hai hướng bổ sung nhau, không loại trừ
- 6.7.5. Đối chiếu cột "công nghệ gợi ý" của BTC (quan trọng)
- 6.7.6. Bản đồ / cây quyết định chọn hướng
- 6.7.7. Khuyến nghị chiến lược cho team (khớp SPEC.md)
- 6.7.8. Kết luận / tóm tắt để nhớ

7.1. Protocol đầy đủ (khung làm việc)

7.2. Vì sao sinh trên class hiếm khó? Few-shot generation

7.3. FaultDiffusion (arXiv 2511.15174, 11/2025) — hướng few-shot fault TS generation

7.4. Đánh giá dữ liệu sinh

- (1) Discriminative score (đo "độ giống")
- (2) Predictive score (đo "giữ được động học")
- (3) Visualization — PCA / t-SNE / UMAP

8.1. Tổng quan pipeline

8.2. Code giả định từng khối

8.3. Lưu ý deployment tối giản

PHẦN 4 · KHO TÀI NGUYÊN, HƯỚNG ĐI & KẾ HOẠCH — KHO TÀI NGUYÊN (DATASET / PAPER / REPO) + HƯỚNG ĐI & KẾ HOẠCH CHIẾN THẮNG

BÀI TOÁN A1 — DENSO FACTORY HACKS 2026

⚠ HIỆN TRẠNG TEAM + HÀNH ĐỘNG NGAY (cập nhật 30/08/2026)

Mục lục CHƯƠNG 4

1. Tổng quan bài A1 + đọc lại deliverable + diễn giải giám khảo mong gì

- 1.1 Bài toán trong một câu
- 1.2 Deliverable chính xác của đề (sau 3 tháng)
- 1.3 Giám khảo mong gì? (diễn giải)
- 1.4 Context stack code hiện có (đã đọc mã nguồn)
- 1.5 Bối cảnh mùa trước — SPARK / DiffusionAD (dùng cho slide)

2. Catalog DATASET

- 2.1 Bảng tổng quan
- 2.2 CWRU Bearing — chi tiết
- 2.3 NASA IMS Bearings — chi tiết
- 2.4 FEMTO / PRONOSTIA — chi tiết
- 2.5 NASA Turbofan (C-MAPSS) — chi tiết
- 2.6 Bảng quyết định "dùng dataset nào ở đâu"

3. Catalog PAPER

- 3.1 Bảng tổng quan
- 3.2 FaultDiffusion (arXiv 2511.15174) — giải thích chi tiết
- 3.3 TimeGAN (NeurIPS 2019) — giải thích chi tiết
- 3.4 Diffusion-TS (ICLR 2024) — giải thích chi tiết
- 3.5 Bảng "đọc → làm gì" tóm tắt

4. Catalog REPO

- 4.1 Bảng tổng quan
- 4.2 TimeGAN — cách dùng
- 4.3 Diffusion-TS — cách dùng
- 4.4 Xung đột môi trường — mẹo vận hành

5. HƯỚNG ĐI KỸ THUẬT 2 NHÁNH (đánh giá kỹ + đề xuất)

- 5.1 Nhánh 1 — Unsupervised anomaly + threshold (baseline bắt buộc)

5.2 Nhánh 2 — Augment + supervised classifier (KHẢ NĂNG THĂNG CAO)

5.3 Phân tích khả thi theo thời gian / GPU / nhân lực

6. KẾ HOẠCH CHI TIẾT THEO TUẦN (26/08 → 12/10/2026)

6.1 Bảng timeline tổng quan

6.2 Tuần 1 (26/08–31/08) — Khởi động & ĐĂNG KÝ

6.3 Tuần 2–3 (01/09–11/09) — Feature + AE baseline

6.4 Tuần 4 (11/09) — Bootcamp + Factory Tour: Hỏi kỹ sư DENSO

6.5 Tuần 5–9 (14/09–09/10) — Augmentation + So sánh + Dashboard

6.6 Checklist nộp cuối cùng (đối chiếu deliverable BTC)

7. CHIẾN LƯỢC SLIDE VÒNG 1 (hạn 12/10)

7.1 Nguyên tắc vàng

7.2 Cấu trúc slide đề xuất

7.3 Điểm "ăn điểm" quan trọng

7.4 Câu hỏi nền hỏi kỹ sư DENSO tại Factory Tour (11/09)

8. QUYẾT ĐỊNH NHANH (DECISION GUIDANCE)

8.1 Bảng chọn mô hình theo bài toán con

8.2 Decision guidance — luồng quyết định (đọc theo thứ tự)

PHỤ LỤC A — Các lệnh chạy end-to-end (để tái lập số liệu trong slide)

PHỤ LỤC B — Những cạm bẫy dễ khiến đội "ăn điểm xấu" (đọc 1 lần/tuần)

PHỤ LỤC C — Tài liệu nên đọc lại theo vai

Cho bài toán A1 — DENSO Factory Hacks 2026

Predictive-Maintenance AI Despite Scarce Fault Data

AI nên đọc: thành viên đội thi A1 có nền tảng ML/DL cơ bản, cần hệ thống lại từ toán → machine learning → deep learning → generative models → tài nguyên, kế hoạch.

Mục tiêu học xong: hiểu và triển khai được pipeline "học cái bình thường → sinh thêm dữ liệu lỗi giả → so sánh accuracy trước/sau" — đúng 3 deliverable của đề.

Mở đầu: đọc nhanh trong 5 phút

- **Bài toán:** máy móc nhà máy gần cảm biến (rung/nhiệt/dòng); dữ liệu lỗi cực hiếm; cần cảnh báo sớm + ước tính tuổi thọ còn lại (RUL) mà ít báo động giả.
- **Thách thức lỗi:** không đủ mẫu lỗi thật để học có giám sát.
- **Ý tưởng giải:** (1) học "thế nào là bình thường" rồi gán cờ cái khác thường (anomaly detection); và/hoặc (2) dùng **GAN/Diffusion sinh thêm dữ liệu lỗi giả** rồi luyện mô hình phân loại, và **so sánh độ chính xác trước/sau** (deliverable chấm điểm).
- **Timeline:** đăng ký đội ≤**31/08**, Factory Tour **11/09**, nộp Vòng 1 slide ≤**12/10**, chung kết **16/11**.

⚠ **HÔM NAY LÀ 30/08/2026.** Hạn đăng ký đội là **31/08 — mai**. Làm ngay trước khi đọc tiếp: lên **densohackathon.vn** lập tài khoản đội (3–5 người), điền ý tưởng sơ bộ, chốt owner tài khoản. Quên mốc này thì toàn bộ giáo trình này vô nghĩa.

BẢN ĐỒ TƯ DUY A1 — mục tiêu lớn → bài toán con (đọc trước tiên)

Mục tiêu lớn và 6 bài toán con phải giải. Mỗi bài con ghi rõ **Mục tiêu / Chọn / Vì sao** và trở tới phần tương ứng của giáo trình.

MỤC TIÊU LỚN: "AI bảo trì dự đoán phát hiện lỗi sớm trên chuỗi rung, dù dữ liệu lỗi cực hiếm" (đăng ký ≤31/08 → nộp slide ≤12/10 → chung kết 16/11)

- | | |
|--|-------------------------------|
| —[a] NÉN TÍN HIỆU TẦN SỐ CAO → ĐẶC TRƯNG CÓ NGHĨA | → Phần 2, src/features.py |
| —[b] ĐỊNH NGHĨA "THẾ NÀO LÀ TỐT/XẤU" CHO LỚP HIẾM | → Phần 2 mục 4 |
| —[c] PHÁT HIỆN BẤT THƯỜNG KHI KHÔNG CÓ NHÃN | → Phần 2 mục 5, Phần 3 |
| —[d] PHÂN LOẠI CÓ GIÁM SÁT KHI CÓ ÍT NHÃN | → Phần 2 mục 3 |
| —[e] BÙ ĐÁP LỚP HIẾM = SINH DỮ LIỆU LỖI GIẢ | → Phần 3 mục 5–7, Phần 4 §3–4 |
| —[f] CHỨNG MINH AUGMENTATION CÓ ÍCH = PROTOCOL TRƯỚC/SAU | → Phần 2 mục 6.4, scripts/02 |

Bài toán con	Mục tiêu	Chọn	Vì sao	Trở tới
[a] Nén tín hiệu	Biến chuỗi rung 12–25 kHz thành vector nhỏ gọn mà vẫn giữ dấu hiệu hỏng	Windowing <code>win=512, stride=256</code> + 16 features (thời gian: mean/std/peak/kurtosis... + tần số: spectral centroid/flatness/fft) + z-score	Không học trực tiếp tín hiệu kHz bằng torch/sklearn nhỏ; features đủ cho AE + RF, nhanh và giải thích được	Phần 2 · <code>src/features.py</code> , <code>src/pipeline.py</code>
[b] Định nghĩa tốt/xấu cho lớp hiếm	Đo bằng thước đo không bị "im lặng" khi fault hiếm	Precision / Recall / F1 / PR-AUC ; quét ngưỡng P95–P99.5	Mất cân bằng ~1:6000 làm accuracy/AUC "nói dối" (AUC 0.52 mà bỏ sót gần hết lỗi); DENSO chấm bằng chi phí báo giả vs bỏ lọt	Phần 2 mục 4
[c] Phát hiện bất thường khi không có nhãn	Baseline "học cái bình thường → gán cờ khác thường" khi chưa có lỗi nào	Autoencoder + reconstruction error + ngưỡng P99; nâng cấp: IsolationForest / OC-SVM	Đúng kịch bản nhà máy mới lắp cảm biến chưa thu được lỗi; cho "cột mốc trước" để so sánh	Phần 2 mục 5 · Phần 3 · <code>scripts/01</code>
[d] Phân loại có giám sát với ít nhãn	Biến "khác thường" thành nhãn dùng được với vài mẫu fault thật (20%)	RandomForest / XGBoost trên 16 features	Mạnh với tabular nhỏ, có feature importance để kể chuyện, chạy CPU được	Phần 2 mục 3 · <code>scripts/02</code>
[e] Bù đắp lớp hiếm bằng lỗi giả	Cân bằng train từ ~1:6000 về gần 1:1 bằng lỗi tổng hợp giống thật	TimeGAN (CPU) trước; Diffusion-TS / FaultDiffusion (GPU) khi có máy	Generative nhìn trúng bản chất đề "scarce fault data"; sinh đúng class fault theo ý; metric discriminative chứng minh "giống thật"	Phần 3 mục 5–7 · Phần 4 §3–4
[f] Chứng minh augmentation có ích	Có con số "trước < sau" đáng tin, giám khảo không bắt bài	3-zone split theo thời gian (TRAIN/TEST/GREY) + generator chỉ học train + test = lỗi THẬT chưa thấy	Leakage là lý do duy nhất khiến số đẹp bị loại; đây đúng deliverable 3 BTC chấm	Phần 2 mục 6.4 · <code>src/pipeline.py</code> (3-zone), <code>scripts/02</code>

Đọc cây này thế nào: bất kỳ câu hỏi nào của đội cũng rơi vào 1 trong 6 nhánh trên. Bắt đầu từ nhánh bài toán đang vướng → đọc "Vì sao" trước → mở phần giáo trình được trở tới.

Cách dùng giáo trình này

Vai của bạn	Đọc gì để nhanh	Chi tiết tra thêm ở đâu
Sinh viên muốn nắm gốc rễ	Đọc tuần tự từ Phần 1 (toán) → Phần 2 (ML cổ điển, feature, anomaly, classification) → Phần 3 (DL + generative) → Phần 4 (tài nguyên + kế hoạch)	Mỗi phần tự đứng được; nếu vội, đọc phần được Bản đồ tư duy trở tới rồi quay lại đủ khi rảnh
Kỹ sư cần nhanh / đang code	Bản đồ tư duy A1 ở trên (5 phút) + Phần 4 §8 — Quyết định nhanh (decision guidance) để chọn model/dữ liệu theo tình huống	Mở đúng file được trở tới trong bảng: <code>src/features.py</code> , <code>src/pipeline.py</code> , <code>scripts/01</code> , <code>scripts/02</code> , <code>scripts/03</code>
Người làm slide Vòng 1	Phần 4: §1 (tổng quan + deliverable) · §3 (paper cite đúng ID) · §5 (2 nhánh hướng đi) · §7 (cấu trúc slide) · §1.5 (bối cảnh mùa trước) + số liệu có sẵn trong <code>results/</code>	Đọc bài " HIỆN TRẠNG TEAM + HÀNH ĐỘNG NGAY " dưới đây để lấy con số thật (30/08/2026) chứ không đoán

Quy tắc chung: con số nào đưa vào slide **MUST** có source trong `results/` . Không viết số tay vào slide — tái chạy command ghi ở Phụ lục A để sinh lại.

PHẦN 1 · NỀN TẢNG TOÁN HỌC CHO ML/DL — NỀN TẢNG TOÁN HỌC CHO MACHINE LEARNING / DEEP LEARNING

Mục đích tài liệu: Cung cấp nền tảng toán học vững chắc cho ML/DL, định hướng theo bài toán **Predictive Maintenance** (bảo trì dự đoán) dựa trên **chuỗi thời gian rung/chấn động** (vibration time series) từ cảm biến trên máy móc (motor, bearing, máy bơm, tuabin...).

Đối tượng: Sinh viên/kỹ sư đã có kiến thức cơ bản, cần ôn lại căn bản một cách có hệ thống, hiểu "vì sao" và "dùng khi nào" từng công cụ toán học.

Cấu trúc tài liệu:

- 1. Đại số tuyến tính
- 2. Giải tích (vi phân)
- 3. Thống kê & Xác suất
- 4. Xác suất & thống kê cho chuỗi thời gian

Sơ đồ: vì sao học sinh toán này lại cần cho A1? (Cây bài toán con)

Trước khi đi vào chi tiết, hãy nhìn toàn cảnh một lần. Toàn bộ phần toán học dưới đây tồn tại vì **một bài toán lớn duy nhất**:

MỤC TIÊU LỚN: Bảo trì dự đoán trên chuỗi rung từ cảm biến — học "cái bình thường", phát hiện lỗi **hiếm** và ước lượng thời gian sống còn lại (RUL).

Bài toán lớn này tách thành **4 bài toán con**. Mỗi bài con cần đúng một chương toán của giáo trình:

MỤC TIÊU LỚN: PREDICTIVE MAINTENANCE trên chuỗi rung

(1) gắn cờ mẫu bất thường → phân loại nhị phân (lỗi/normal)

(2) ước lượng RUL / mức hư hại → hồi quy

[1] BIỂU DIỄN DỮ LIỆU: biến tín hiệu rung thành con số

- có cấu trúc để máy tính xử lý → VECTOR, MA TRẬN, norm
- 1 cửa sổ rung 64 mẫu = 1 vector; cả bộ dữ liệu = 1 ma trận
- trọng số mạng = ma trận; GPU chạy nhanh nhờ nhân ma trận song song

→ cần Chương 1: ĐẠI SỐ TUYẾN TÍNH

[2] HỌC = TỐI ƯU: điều chỉnh trọng số để giảm sai số dần dần

- ĐẠO HÀM, GRADIENT, CHAIN RULE, GRADIENT DESCENT
- đạo hàm cho biết "đi hướng nào để sai số giảm"
- chain rule lan truyền gradient xuyên qua từng lớp (backprop)

→ cần Chương 2: GIẢI TÍCH (VI PHÂN)

[3] ĐO SAI SỐ & SỰ BẤT THƯỜNG bằng ngôn ngữ xác suất

- PHÂN PHỐI, KỶ VỌNG/PHƯƠNG SAI, COVARIANCE, MLE, BAYES
- nhiễu rung ≈ Gaussian → chọn MSE khi dự đoán RUL
- lỗi/nhị phân ≈ Bernoulli → chọn Cross-Entropy khi gắn cờ hỏng
- Bayes cảnh báo về dương tính giả khi sự kiện hỏng hiếm

→ cần Chương 3: THỐNG KÊ & XÁC SUẤT

[4] DỮ LIỆU CÓ THỨ TỰ THỜI GIAN – không thể xáo trộn thoải mái

- CHUỖI THỜI GIAN, STATIONARITY, ACF, ROLLING, LOOKBACK
- kiểm tra tính dừng; ACF phát hiện chu kỳ quay của máy
- rolling RMS là chỉ báo hư hại sớm (chuẩn ISO 10816)
- lookback/sliding window tạo mẫu huấn luyện; chặn data leakage

→ cần Chương 4: CHUỖI THỜI GIAN

Quy tắc đọc: gặp bất kỳ khái niệm nào ở các chương sau, hãy tự hỏi "*bài toán con nào (1–4) đang cần nó?*" — trả lời được câu đó, bạn đã biết công thức vì sao tồn tại và dùng khi nào.

1. ĐẠI SỐ TUYẾN TÍNH

Mục tiêu: Nắm cách biểu diễn dữ liệu rung thành vector/ma trận, vận hành các phép toán cốt lõi (nhân ma trận, norm, chuẩn hóa), và hiểu eigenvalue/SVD/PCA để nén và giảm chiều tín hiệu — nền tảng cho mọi layer của mạng nơ-ron chạy trên GPU.

1.1 Vector và Ma trận

Mục tiêu: Hiểu vì sao "một mẫu dữ liệu là một vector" và "cả bộ dữ liệu là một ma trận", cùng quy ước kích thước (n_{samples} , n_{features}).

1.1.1 Vector là gì?

Một **vector** là một danh sách (mảng) có thứ tự các số. Trong ML, một điểm dữ liệu thường được biểu diễn là một vector.

Bài toán con đang giải: biến một phép đo rung tại một thời điểm thành con số có cấu trúc để máy tính tính toán được.

Ví dụ: mỗi giá trị đo rung tại thời điểm t có thể có nhiều thành phần:

- $ax = 0.52$ — gia tốc rung theo trục X (đơn vị g)
- $ay = 0.61$ — gia tốc rung theo trục Y
- $az = 0.44$ — gia tốc rung theo trục Z
- $temp = 47.3$ — nhiệt độ (°C)
- $rpm = 1498$ — tốc độ quay (vòng/phút)

Ta gộp thành vector đặc trưng:

```
x = [0.52, 0.61, 0.44, 47.3, 1498]
```

Số thành phần của vector gọi là **số chiều** (dimension). Ở đây vector x có 5 chiều, ký hiệu $x \in \mathbb{R}^5$ (nằm trong không gian thực 5 chiều).

Trong bài toán rung/chấn động, một **cửa sổ** (window) 64 mẫu tín hiệu rung trên 1 trục chính là một vector 64 chiều. Cả triệu cửa sổ như vậy được lưu trong một ma trận lớn.

1.1.2 Ma trận là gì?

Một **ma trận** là một bảng số hình chữ nhật gồm m hàng và n cột. Ký hiệu kích thước là $m \times n$.

Bài toán con đang giải: nhiều mẫu dữ liệu — làm sao gom lại để xử lý một lần?

Ví dụ: 3 cửa sổ, mỗi cửa sổ 4 đặc trưng ($ax, ay, az, temp$):

```
X = [ 0.52  0.61  0.44  47.3 ]
     [ 0.55  0.63  0.45  47.8 ]
     [ 0.60  0.70  0.49  48.5 ]
```

Đây là ma trận 3×4 : 3 hàng = 3 mẫu quan sát, 4 cột = 4 đặc trưng.

Trong ML, ký ước phổ biến: **mỗi hàng là một mẫu dữ liệu, mỗi cột là một đặc trưng**. Ma trận dữ liệu X có kích thước $(n_{\text{samples}}, n_{\text{features}})$.

1.1.3 Vì sao ma trận quan trọng với ML?

- Bộ dữ liệu rung chấn động có thể có hàng chục triệu dòng — ma trận là cách lưu trữ duy nhất khả thi.
- Mọi phép toán trong mạng nơ-ron (linear layer, attention, convolution) đều là phép toán ma trận.
- GPU được thiết kế để làm phép nhân ma trận song song hàng loạt.

1.2 Các phép toán cơ bản trên ma trận

Mục tiêu: Nắm các phép toán nền (cộng/trừ, nhân vô hướng, nhân ma trận = nhiều dot product, transpose, đơn vị/ngược đảo) — chúng là "khối lắp ráp" của mọi layer trong mạng.

1.2.1 Cộng, trừ ma trận

Cộng/trừ hai ma trận **cùng kích thước**, thực hiện theo từng phần tử:

```
[ 1  2 ]   [ 5  6 ]   [ 1+5  2+6 ]   [ 6  8 ]
[ 3  4 ] + [ 7  8 ] = [ 3+7  4+8 ] = [10 12]
```

1.2.2 Nhân ma trận với số vô hướng (scalar)

Nhân mọi phần tử với số đó:

$$3 * \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

1.2.3 Nhân hai ma trận (matrix multiplication)

Cho ma trận A kích thước $(m \times k)$ và B kích thước $(k \times n)$. Tích $C = A \cdot B$ có kích thước $(m \times n)$.

Phần tử hàng i , cột j của C được tính bằng **tổng có trọng số** (dot product) của hàng i của A với cột j của B :

$$C[i][j] = A[i][1]*B[1][j] + A[i][2]*B[2][j] + \dots + A[i][k]*B[k][j]$$

Điều kiện tiên quyết: số cột của A phải bằng số hàng của B (cùng bằng k). Nếu không, phép nhân không tồn tại.

Ví dụ số:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad (2 \times 2)$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \quad (2 \times 2)$$

$$C = A \cdot B = \begin{bmatrix} 1*5+2*7 & 1*6+2*8 \\ 3*5+4*7 & 3*6+4*8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Lưu ý: **phép nhân ma trận không giao hoán**, nghĩa là $A \cdot B \neq B \cdot A$ nói chung. Nhưng nó **kết hợp**: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$.

1.2.4 Dot product (tích vô hướng) của hai vector

Cho hai vector cùng chiều $a = [a_1, a_2, \dots, a_n]$ và $b = [b_1, b_2, \dots, b_n]$:

$$a \cdot b = a_1*b_1 + a_2*b_2 + \dots + a_n*b_n$$

Dot product là một **số vô hướng** (scalar).

Ví dụ số:

$$a = [1, 2, 3]$$

$$b = [4, 5, 6]$$

$$a \cdot b = 1*4 + 2*5 + 3*6 = 4 + 10 + 18 = 32$$

Dot product có ý nghĩa hình học: đo mức **tương đồng về hướng** giữa hai vector.

- Hai vector cùng hướng: dot product lớn (dương).
- Hai vector vuông góc: dot product bằng 0.
- Hai vector ngược hướng: dot product âm.

Công thức liên hệ với góc θ giữa hai vector:

$$a \cdot b = |a| * |b| * \cos(\theta)$$

Trong ML, dot product được dùng ở khắp nơi: **correlation** (tương quan), **attention mechanism** trong transformer, và là khối cơ bản của mọi layer tuyến tính.

Ví dụ ứng dụng: muốn biết cửa sổ rung hiện tại giống với "mẫu lỗi" đã biết như thế nào, ta tính dot product giữa đặc trưng chuẩn hóa của chúng. Giá trị càng cao $\rightarrow$ càng giống.

1.2.5 Phép nhân ma trận = tập hợp nhiều dot product

Nhân ma trận chỉ là cách viết gọn của việc tính nhiều dot product cùng lúc:

- Mỗi phần tử $C[i][j]$ là dot product của hàng i (của A) với cột j (của B).
- Với ma trận dữ liệu X ($n_samples \times n_features$) và ma trận trọng số W ($n_features \times n_outputs$), tích $X \cdot W$ tính **đồng thời** đầu ra cho tất cả mẫu.

Đây chính là lý do GPU "nổ máy" cho DL: hàng triệu dot product độc lập chạy song song.

1.2.6 Transpose (chuyển vị)

Chuyển vị của ma trận A , ký hiệu A^T , là ma trận đổi hàng thành cột:

$$A = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \quad (1 \times 3) \\ A^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad (3 \times 1)$$

Với ma trận 2 chiều:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \quad (3 \times 2) \\ A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} \quad (2 \times 3)$$

Quy tắc: phần tử $A[i][j]$ sau chuyển vị thành $A^T[j][i]$.

Vì sao dùng transpose?

- Đổi cách biểu diễn hàng/cột cho khớp kích thước khi nhân ma trận.
- Tính covariance và trong SVD.
- Ký hiệu gọn cho dot product: $a \cdot b = a^T * b$.

1.2.7 Ma trận đơn vị và ma trận nghịch đảo

Ma trận đơn vị (identity) ký hiệu I : có số 1 trên đường chéo chính, 0 ở chỗ khác. Nhân bất kỳ ma trận nào với I đều ra chính nó: $A \cdot I = A$.

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Ma trận nghịch đảo của A , ký hiệu A^{-1} , thỏa mãn:

$$A \cdot A^{-1} = I$$

Nghịch đảo chỉ tồn tại cho ma trận **vuông** và **không suy biến** (determinant khác 0). Trong ML ta gần như không tính nghịch đảo trực tiếp (tốn kém, bất ổn định số) mà dùng các phương pháp ổn định hơn (chuẩn hóa, pseudo-inverse, gradient descent).

1.3 Norm và chuẩn hóa

Mục tiêu: Hiểu khái niệm "độ dài của vector" (norm), phân biệt L1/L2/Frobenius và vì sao bắt buộc chuẩn hóa khi các đặc trưng rung có thang đo rất khác nhau (0.5 g vs 1500 RPM).

1.3.1 Norm L2 (chuẩn Euclid) — hay dùng nhất

Norm là độ dài của một vector. Norm L2:

$$|x|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

Ví dụ số:

$$x = [3, 4] \\ |x|_2 = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Chính là định lý Pythagoras mở rộng nhiều chiều. Khoảng cách thông thường giữa hai điểm a và b là $|a - b|_2$.

1.3.2 Norm L1 (chuẩn Manhattan)

$$|x|_1 = |x_1| + |x_2| + \dots + |x_n|$$

Ví dụ số:

$$x = [3, -4] \\ |x|_1 = |3| + |-4| = 3 + 4 = 7$$

So với L2 = 5, cùng vector nhưng L1 lớn hơn vì nó cộng giá trị tuyệt đối, không bình phương.

Vì sao phân biệt L1 và L2? L1 và L2 phạt trọng số theo hai triết lý khác nhau, quyết định hành vi của mô hình khác hẳn:

Tiêu chí	Norm L1 (Manhattan)	Norm L2 (Euclid)
Công thức	$\sum x_i $	$\sqrt{\sum x_i^2}$
Cách nhìn thành phần	coi mọi thành phần như nhau (tuyến tính)	bình phương → phạt nặng thành phần lớn
Nhạy với outlier	ít bị một giá trị bất thường chi phối	bị một giá trị lớn kéo lệch mạnh
Dùng trong regularization	$\text{loss} + \lambda \cdot \sum w_i \rightarrow$ kéo trọng số về chính xác 0 (sparse, tự chọn đặc trưng)	$\text{loss} + \lambda \cdot \sum w_i^2$ (weight decay) → kéo về gần 0, không bằng 0
Phù hợp khi	ngghi ngờ nhiều đặc trưng vô dụng, muốn loại bỏ	hầu hết đặc trưng đều có ích, chỉ cần ép nhỏ

Nói gọn: **L1 "tuyển chọn", L2 "cân chỉnh"**. Với feature rung (hàng chục đặc trưng thống kê/tần số), L1 giúp biết đặc trưng nào thật sự dự báo lỗi; L2 giúp mọi đặc trưng mượt hơn khi có nhiều.

1.3.3 Norm Frobenius — dành cho ma trận

Frobenius norm của ma trận A = căn bậc hai của tổng bình phương **tất cả phần tử**:

$$|A|_F = \sqrt{\sum_i \sum_j A[i][j]^2}$$

Ví dụ số:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \\ |A|_F = \sqrt{1 + 4 + 9 + 16} = \sqrt{30} \approx 5.48$$

Frobenius norm giống "độ lớn" của cả ma trận; được dùng để đo độ lớn của ma trận trọng số khi regularization.

1.3.4 Chuẩn hóa vector (normalization)

Chuẩn hóa = chia vector cho norm của nó để có **norm = 1** (vector đơn vị):

$$x_{\text{norm}} = x / |x|_2$$

Ví dụ số:

$$x = [3, 4] \\ x_{\text{norm}} = [3/5, 4/5] = [0.6, 0.8] \\ |x_{\text{norm}}|_2 = \sqrt{0.36 + 0.64} = \sqrt{1} = 1 \checkmark$$

Vì sao dùng?

- Các đặc trưng rung có thang đo khác nhau (gia tốc ~0.5 g, nhiệt độ ~47 °C, RPM ~1500). Nếu không chuẩn hóa, RPM sẽ "áp đảo" gia tốc trong tính toán khoảng cách/dot product.
- Khi chuẩn hóa, dot product giữa hai vector chuẩn hóa **chính là cosin của góc giữa chúng** (cosine similarity):

$$\text{cos_sim}(a, b) = (a \cdot b) / (|a|_2 * |b|_2)$$

Được dùng để so sánh mức na ná nhau của hai cửa sổ tín hiệu / hai vec-tơ nhúng.

Lưu ý phân biệt: Chuẩn hóa vector (chia theo norm) ≠ **chuẩn hóa đặc trưng** (feature scaling) như $z\text{-score} = (x - \text{mean}) / \text{std}$. Cả hai đều quan trọng ở phần 3.3 và 3.4 — và với chuỗi rung thì nơi này còn phát sinh bẫy data leakage (chỉ dùng mean/std của quá khứ, xem mục 4.4).

1.4 Eigenvalue, Eigenvector và SVD

Mục tiêu: Hiểu eigenvalue/eigenvector là "hướng biến thiên chính" của dữ liệu, SVD nén ma trận và PCA giảm chiều — để biến cửa sổ rung 64 chiều thành vài thành phần chính mà vẫn giữ phần lớn thông tin.

1.4.1 Eigenvector và eigenvalue — ý tưởng

Cho ma trận vuông A . Một vector khác 0 v được gọi là **eigenvector** của A nếu nhân A với v chỉ làm **giãn/kéo dài** v (đổi độ dài, đổi dấu) chứ không đổi hướng:

$$A \cdot v = \lambda \cdot v$$

λ (lambda) là **eigenvalue** tương ứng — hệ số giãn.

Ví dụ số:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$A \cdot v = \begin{bmatrix} 2 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 1 + 2 \cdot 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\rightarrow \lambda = 3, v = [1, 1]$$

Có eigenvector khác:

$$v = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$A \cdot v = \begin{bmatrix} 2 \cdot 1 + 1 \cdot (-1) \\ 1 \cdot 1 + 2 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1 \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$\rightarrow \lambda = 1, v = [1, -1]$$

1.4.2 Ý nghĩa trực quan

- Eigenvector = **hướng** mà phép biến đổi A "giữ nguyên" (chỉ co/giãn).
- Eigenvalue lớn = dọc theo hướng đó, A **khuếch đại mạnh**.
- Mỗi ma trận vuông $n \times n$ đối xứng có n eigenvector trực giao (vuông góc nhau).

Trong phân tích rung chấn: eigenvalue đo mức năng lượng dao động theo từng hướng. Hướng có eigenvalue lớn = hướng biến thiên chính của tín hiệu.

1.4.3 SVD — Singular Value Decomposition

Mọi ma trận X ($m \times n$) (không cần vuông) đều phân tích được thành:

$$X = U \cdot \Sigma \cdot V^T$$

Trong đó:

- U ($m \times m$): ma trận các vector cột trực giao — biểu diễn "chiều của các mẫu" (trái singular vector).
- V^T ($n \times n$): ma trận các vector cột trực giao — biểu diễn "chiều của các đặc trưng" (phải singular vector).
- Σ : ma trận chéo chứa **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ — to bằng mức độ quan trọng của mỗi "thành phần".

Vai trò trực quan của SVD — nén (compression):

Các singular value giảm nhanh. Nếu chỉ giữ k giá trị lớn nhất, ta được xấp xỉ:

$$X \approx U_k \cdot \Sigma_k \cdot V_k^T$$

với sai số nhỏ. Đây chính là **nén ma trận**: lưu ít số hơn nhưng giữ hầu hết thông tin.

Ví dụ: ma trận chứa 1000 cửa sổ $\times$ 64 đặc trưng (64.000 số). Sau SVD, giữ $k = 5$ thành phần chính $\rightarrow$ chỉ cần $1000 \cdot 5 + 5 + 5 \cdot 64 \approx 5325$ số (~8% dung lượng) mà vẫn giữ được phần lớn biến thiên của dữ liệu.

1.4.4 SVD $\rightarrow$ PCA (Principal Component Analysis)

PCA dùng SVD để **giảm chiều** dữ liệu, giữ lại phương sai lớn nhất (nơi dữ liệu phân tán nhất):

- Chuẩn hóa dữ liệu (mean = 0).
- Tính SVD (hoặc eigenvalue decomposition của covariance matrix).
- Giữ k singular vector ứng với k singular value lớn nhất.
- Chiếu dữ liệu xuống k chiều mới (principal components).

Vì sao dùng trong Predictive Maintenance?

- Tín hiệu rung 64 chiều có thể nén về 3–10 chiều để: giảm nhiễu, giảm thời gian huấn luyện, trực quan hóa cụm lỗi/normal theo thời gian.
- Các "thành phần chính" thường tương ứng với chế độ dao động chính của máy (tần số quay chính, bội số 2f, harmonic).

So sánh hướng tiếp cận giảm chiều — cái nào cho A1?

Cách giảm chiều	Nguyên lý	Đặc điểm	Khi nào hợp cho bài rung
SVD/PCA (tuyến tính)	giữ hướng phương sai lớn nhất	nhanh, diễn giải được (tần số quay, harmonic), là baseline vững	khử nhiễu, trực quan hóa, nén feature trước khi đưa vào model cổ điển
Autoencoder (phi tuyến)	học biểu diễn nén thông qua mạng nơ-ron	mạnh hơn nhưng khó diễn giải, cần nhiều dữ liệu	tự học "hình ảnh bình thường" khi hết dữ liệu lỗi (anomaly detection)
Không giảm chiều	giữ nguyên 64 chiều cho mạng sâu	mạng nơ-ron tự học biểu diễn phi tuyến	thường được ưu tiên khi dùng CNN/LSTM hiện đại

Nói gọn: dùng PCA khi muốn diễn giải + nhanh; nhường công việc "nén" cho chính mạng nơ-ron khi dự thi kiểu DL. Nhưng SVD vẫn là công cụ nền tảng: nén mô hình, low-rank approximation, và trong attention (biểu diễn low-rank).

1.5 Vì sao Deep Learning cần matrix math?

Mục tiêu: Chỉ ra rằng toàn bộ tính toán của DL quy về nhân ma trận trên batch — nền tảng vì sao GPU nhanh, và cách đọc kích thước của mọi layer.

1.5.1 Batch — xử lý hàng loạt

Huấn luyện không xử lý từng mẫu một mà theo batch (mini-batch): ví dụ 32, 64, 128 mẫu mỗi lần.

Giả sử mỗi cửa sổ rung là vector 64 chiều. Một batch 32 cửa sổ được gộp vào ma trận:

X_batch (32 × 64)

Mọi phép toán (tính đầu ra, tính gradient) được tính cho cả batch cùng lúc bằng một phép nhân ma trận.

1.5.2 Layer tuyến tính = nhân ma trận

Một fully-connected layer với trọng số W và bias b là:

Z = X · W + b

- X : đầu vào (batch × in_features)
- W : trọng số (in_features × out_features)
- b : bias (out_features)
- Z : đầu ra trước activation

Ví dụ: X (32×64) nhân W (64×16) → Z (32×16) : ánh xạ 32 cửa sổ rung thành 16 "đặc trưng trung gian". Quy tắc khớp kích thước: số cột của vế đầu (64) = số hàng của ma trận sau (64).

1.5.3 GPU và sự song song

- GPU có hàng nghìn lõi tính đơn giản (khác CPU vài chục lõi mạnh).
- Phép nhân ma trận phân rã thành vô số phép nhân-cộng độc lập, chạy song song tuyệt vời trên GPU.
- Do đó hiệu năng DL = hiệu năng nhân ma trận. Tăng batch → tận dụng băng thông bộ nhớ GPU tốt hơn → huấn luyện nhanh hơn (đến một giới hạn).

1.5.4 Tóm tắt Đại số tuyến tính

Khái niệm	Công thức	Dùng khi nào
Dot product	$a \cdot b = \sum(a_i \cdot b_i)$	Đo tương đồng, khối cơ bản của layer
Nhân ma trận	$C[i][j] = \sum_k A[i][k] \cdot B[k][j]$	Batch forward/backward trên GPU
Transpose	$A^T[j][i] = A[i][j]$	Khớp kích thước, tính covariance
Norm L1	$\sum(x_i)$	Regularization thưa, ít nhạy outlier
Norm L2	$\sqrt{\sum(x_i^2)}$	Độ dài, khoảng cách, weight decay

Khái niệm	Công thức	Dùng khi nào
Norm Frobenius	$\sqrt{\sum(A[i][j]^2)}$	Độ lớn ma trận trọng số
Chuẩn hóa	$x / x _2$	Cosine similarity
Eigen	$A \cdot v = \lambda \cdot v$	Hiệu hướng biến thiên chính
SVD	$X = U \cdot \Sigma \cdot V^T$	Nén, PCA, giảm chiều

2. GIẢI TÍCH (VI PHÂN)

Mục tiêu: Hiểu đạo hàm/gradient dùng để "đi về hướng sai số giảm", chain rule là lỗi của lan truyền ngược (backprop), gradient descent là vòng lặp học của mọi mô hình, còn Taylor và tính lồi giải thích vì sao bước nhỏ mới hội tụ.

2.1 Đạo hàm

Mục tiêu: Đạo hàm là "tốc độ thay đổi" (độ dốc) của hàm tại một điểm; nắm bảng đạo hàm cơ bản và quy tắc tuyến tính.

2.1.1 Định nghĩa

Đạo hàm của hàm $f(x)$ tại điểm x đo **tốc độ thay đổi** của f khi x thay đổi một chút:

$$f'(x) = \lim_{h \rightarrow 0} (f(x+h) - f(x)) / h$$

Trực giác: đạo hàm là **độ dốc** (slope) của tiếp tuyến tại x .

Ví dụ số cơ bản:

$$f(x) = x^2, \text{ tại } x = 3:$$

$$\begin{aligned} f'(x) &= 2x \\ f'(3) &= 2 \cdot 3 = 6 \end{aligned}$$

Nghĩa là quanh $x = 3$, nếu tăng x thêm 0.01 thì f tăng thêm khoảng $6 \cdot 0.01 = 0.06$.

2.1.2 Bảng đạo hàm hay dùng

$f(x) = c$	$\rightarrow f'(x) = 0$	(hằng số)
$f(x) = x^n$	$\rightarrow f'(x) = n \cdot x^{n-1}$	(lũy thừa)
$f(x) = e^x$	$\rightarrow f'(x) = e^x$	
$f(x) = \ln(x)$	$\rightarrow f'(x) = 1/x$	
$f(x) = \sin(x)$	$\rightarrow f'(x) = \cos(x)$	
$f(x) = ax + b$	$\rightarrow f'(x) = a$	(tuyến tính)

Quy tắc tính nhanh (linearity of derivative):

$$d/dx [c_1 f_1(x) + c_2 f_2(x)] = c_1 f_1'(x) + c_2 f_2'(x)$$

2.1.3 Vì sao DL cần đạo hàm?

- Huấn luyện mạng = **tối ưu hóa**: tìm trọng số w sao cho hàm mất mát $L(w)$ nhỏ nhất.
- Đạo hàm cho biết hướng tăng của L . Muốn giảm, ta đi **ngược dấu đạo hàm**.
- Từ một trọng số duy nhất đến hàng triệu trọng số, ta cần đạo hàm riêng và gradient (mục 2.2).

2.2 Đạo hàm riêng và Gradient

Mục tiêu: Mở rộng đạo hàm sang hàm nhiều biến — đạo hàm riêng và gradient, vector chỉ hướng tăng nhanh nhất. Ngược dấu gradient = hướng giảm loss.

2.2.1 Đạo hàm riêng

Khi hàm có **nhiều biến** $f(x_1, x_2, \dots, x_n)$, **đạo hàm riêng** theo x_i là đạo hàm thông thường khi **cố định tất cả biến khác**:

$$\partial f / \partial x_i = \lim_{h \rightarrow 0} (f(\dots, x_i + h, \dots) - f(\dots, x_i, \dots)) / h$$

Ký hiệu ∂ (đọc là "đô") thay cho d khi có nhiều biến.

Ví dụ số: $f(x, y) = x^2 + x \cdot y + y^3$, tại điểm $(x, y) = (2, 1)$:

$$\begin{aligned} \partial f / \partial x &= 2x + y && \rightarrow \text{tại } (2,1): 2 \cdot 2 + 1 = 5 \\ \partial f / \partial y &= x + 3y^2 && \rightarrow \text{tại } (2,1): 2 + 3 \cdot 1 = 5 \end{aligned}$$

2.2.2 Gradient — vector của mọi đạo hàm riêng

Gradient của hàm nhiều biến là vector chứa toàn bộ đạo hàm riêng:

$$\nabla f = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$$

Gradient **chỉ hướng tăng nhanh nhất** của hàm tại điểm đó. Ngược dấu gradient = hướng giảm nhanh nhất = hướng ta cần đi để giảm loss.

Ví dụ số:

$$\begin{aligned} f(x, y) &= (x - 1)^2 + (y - 2)^2 \\ \nabla f &= [2(x-1), 2(y-2)] \end{aligned}$$

$$\text{Tại } (3, 2): \nabla f = [2 \cdot (2), 2 \cdot 0] = [4, 0]$$

Hàm đạt giá trị nhỏ nhất tại $(1, 2)$ (cả hai thành phần gradient = 0). Điểm đạo hàm = 0 gọi là **điểm tới hạn** (critical point) — đây là ứng viên của cực tiểu/cực đại.

2.2.3 Gradient trong ML

Với mạng nơ-ron, hàm mất mát phụ thuộc vào hàng triệu trọng số:

$$L(w_1, w_2, \dots, w_N)$$

Gradient ∇L là vector N chiều. **Backpropagation** (lan truyền ngược) chính là thuật toán tính gradient này hiệu quả, dựa trên **chain rule** (mục 2.3).

2.3 Chain rule — nền tảng của Backprop (TRỌNG TÂM)

Mục tiêu: Hiểu chain rule ghép các đạo hàm cục bộ qua từng lớp để tạo ra gradient toàn cục — đây chính là thuật toán backprop, thứ biến "cập nhật trọng số" trở nên khả thi với hàng triệu tham số.

2.3.1 Chain rule cho hàm một biến

Nếu $z = f(g(x))$ thì:

$$dz/dx = df/dg \cdot dg/dx$$

Nói dễ hiểu: tốc độ thay đổi của z theo x = tích của (tốc độ thay đổi của z theo g) $\times$ (tốc độ thay đổi của g theo x).

Ví dụ số từng bước:

$$\begin{aligned} f(g) &= g^2, \quad g(x) = 3x + 1, \quad \text{tại } x = 2 \\ g(2) &= 7 \\ df/dg &= 2g = 2 \cdot 7 = 14 \\ dg/dx &= 3 \\ dz/dx &= 14 \cdot 3 = 42 \end{aligned}$$

Kiểm chứng trực tiếp: $z = (3x + 1)^2 = 9x^2 + 6x + 1 \rightarrow dz/dx = 18x + 6$. Tại $x = 2$: $18 \cdot 2 + 6 = 42$. ✓ Hai cách cho cùng kết quả.

2.3.2 Chain rule nhiều biến — trọng tâm of backprop

Mạng nơ-ron là **chuỗi hàm lồng nhau**. Ví dụ mạng 1 lớp ẩn chuẩn:

$$\begin{aligned} x \rightarrow z_1 &= W_1 \cdot x + b_1 && (\text{linear 1}) \\ &\rightarrow a_1 = \phi(z_1) && (\text{activation, ví dụ ReLU/tanh}) \\ &\rightarrow z_2 = W_2 \cdot a_1 + b_2 && (\text{linear 2}) \\ &\rightarrow \hat{y} = z_2 && (\text{kết quả dự đoán}) \\ &\rightarrow L = (\hat{y} - y)^2 && (\text{MSE loss}) \end{aligned}$$

Loss phụ thuộc vào w qua nhiều "tầng trung gian". Muốn $\partial L / \partial w_1$, ta lần ngược:

$$\partial L / \partial w_1 = \partial L / \partial \hat{y} * \partial \hat{y} / \partial a_1 * \partial a_1 / \partial z_1 * \partial z_1 / \partial w_1$$

Đây chính là **tích của các đạo hàm riêng qua từng khâu** — chain rule nhiều biến:

$$\partial z / \partial x = \sum_k (\partial z / \partial y_k * \partial y_k / \partial x)$$

(Trong đó y_k là tất cả đường trung gian nối x đến z .)

2.3.3 Vì sao backprop = "gọn" hơn tính trực tiếp?

- Mạng 10 layer, 1000 nơ-ron mỗi layer → hàng triệu trọng số.
- Nếu tính $\partial L / \partial w$ từng cái riêng lẻ bằng định nghĩa → tốn phí cực lớn (rất nhiều lần chạy forward).
- Backprop: một lần **forward** để tính các giá trị trung gian, một lần **backward** để truyền gradient ngược và tái sử dụng kết quả của lớp sau cho lớp trước.
- Gradient $\partial L / \partial w_i$ bằng nhau ở các lớp cuối được tính một lần, dùng lại cho nhiều lớp — đây là sức mạnh của chain rule.

2.3.4 Ví dụ số hoàn chỉnh (1 nơ-ron)

Giả lập một tế bào nơ-ron: $z = w * x + b$, $a = \tanh(z)$ (đạo hàm $\tanh' = 1 - a^2$), loss $L = (a - y)^2$.

Dữ liệu: $x = 2$, $w = 0.5$, $b = 0.1$, $y = 1$.

Bước forward:

$$\begin{aligned} z &= 0.5 * 2 + 0.1 = 1.1 \\ a &= \tanh(1.1) \approx 0.8005 \\ L &= (0.8005 - 1)^2 = (-0.1995)^2 \approx 0.0398 \end{aligned}$$

Bước backward (chain rule từ cuối về đầu):

$$\begin{aligned} \partial L / \partial a &= 2 * (a - y) = 2 * (-0.1995) \approx -0.3990 \\ \partial a / \partial z &= 1 - a^2 = 1 - 0.6408 \approx 0.3592 \\ \partial z / \partial w &= x = 2 \\ \partial z / \partial b &= 1 \end{aligned}$$

$$\begin{aligned} \partial L / \partial w &= \partial L / \partial a * \partial a / \partial z * \partial z / \partial w = (-0.3990) * 0.3592 * 2 \approx -0.2866 \\ \partial L / \partial b &= \partial L / \partial a * \partial a / \partial z * \partial z / \partial b = (-0.3990) * 0.3592 * 1 \approx -0.1433 \end{aligned}$$

Cập nhật trọng số với learning rate $\eta = 0.1$:

$$\begin{aligned} w_{\text{new}} &= w - \eta * \partial L / \partial w = 0.5 - 0.1 * (-0.2866) \approx 0.5287 \\ b_{\text{new}} &= b - \eta * \partial L / \partial b = 0.1 - 0.1 * (-0.1433) \approx 0.1143 \end{aligned}$$

Gradient âm → loss giảm khi tăng w → ta tăng w . Đúng chiều.

2.3.5 Tổng kết trực quan

Forward: $x \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow a_2 \rightarrow \dots \rightarrow \hat{y} \rightarrow L$ (tính giá trị)
Backward: $\partial L / \partial w_1 \leftarrow \partial L / \partial w_2 \leftarrow \dots \leftarrow \partial L / \partial w_n$ (tính gradient ngược)

Mỗi khâu của mạng chỉ cần biết hai thứ: giá trị đạo hàm cục bộ của nó và gradient truyền về từ phía sau. Chain rule ghép chúng lại = gradient toàn cục.

2.4 Gradient Descent — trực giác

Mục tiêu: Nắm vòng lặp "tính gradient → cập nhật trọng số" và vai trò quyết định của learning rate; phân biệt các biến thể optimizer và trực giác loss landscape.

2.4.1 Thuật toán

Hàm mất mát $L(w)$ — mục tiêu: tìm w làm L nhỏ nhất. Gradient descent dựa trên việc **đi ngược hướng gradient**:

Lặp lại:
1. Tính gradient $g = \nabla L(w)$
2. Cập nhật $w \leftarrow w - \eta * g$

Với η (eta) là **learning rate** — bước đi mỗi lần.

2.4.2 Loss landscape — "địa hình" của hàm mất mát

Hình dung $L(w)$ như một **bề mặt địa hình**: trục ngang là trọng số, trục dọc là giá trị loss.

- Gradient descent giống **người đi bộ mù** trong sương mù: chỉ cảm nhận được độ dốc dưới chân, cứ đi xuống dốc.
- **Global minimum** — đáy trũng thấp nhất toàn vùng: điểm mong muốn.
- **Local minimum** — đáy trũng cục bộ, thấp hơn xung quanh nhưng không phải thấp nhất toàn cầu. Gradient = 0 tại đó → thuật toán "mắc kẹt".
- **Saddle point** — điểm yên ngựa: gradient = 0 nhưng không phải cực trị (một hướng xuống, một hướng lên).

2.4.3 Ảnh hưởng của Learning Rate

Với hàm một biến $L(w) = (w - 3)^2$ (cực tiểu tại $w = 3$), bắt đầu $w = 8$, $\nabla L = 2(w - 3)$:

- $\eta = 0.1$: bước $w \leftarrow 8 - 0.1 \cdot 10 = 7$, rồi 6.2, 5.6, 5.1... hội tụ chậm nhưng chắc chắn.
- $\eta = 0.5$: bước $w \leftarrow 8 - 0.5 \cdot 10 = 3$ — tới đích chỉ trong 1 bước (may mắn).
- $\eta = 1.0$: $w \leftarrow 8 - 10 = -2$, rồi $-2 - 1.0 \cdot 2 \cdot (-5) = 8$ — **dao động qua lại**, không hội tụ.
- $\eta = 1.2$: $w \leftarrow 8 - 12 = -4$, rồi $-4 - 1.2 \cdot 2 \cdot (-7) = 12.8$ — **nổ tung** (diverge), loss mỗi lúc một lớn.

Quy luật:

- η quá nhỏ → hội tụ chậm (tốn thời gian, có thể kẹt local minimum).
- η quá lớn → dao động / bùng nổ, không hội tụ.
- Trong thực tế dùng **learning rate schedule** (giảm dần) hoặc optimizers thích ứng (Adam, RMSProp) tự điều chỉnh bước đi theo từng trọng số.

2.4.4 Variants — SGD, Mini-batch, Adam

Biến thể	Tính gradient trên	Ưu điểm	Hạn chế
Batch GD	toàn bộ dữ liệu mỗi bước	chính xác, ổn định	chậm, tốn bộ nhớ
SGD / Mini-batch GD	batch nhỏ (32/64/128 mẫu)	nhanh, thêm "nhiều" ngẫu nhiên để thoát local minimum hẹp	nhiều hơn, cần tinh chỉnh lr
Adam	batch nhỏ + momentum + adaptive lr	tự điều chỉnh bước theo từng tham số, hội tụ nhanh, ít tinh chỉnh	nhiều siêu tham số hơn, một số báo cáo khái quát kém hơn SGD

Trong thực tế thi A1: khởi điểm thường dùng **Adam** với `learning_rate = 0.001`; nếu muốn kết quả tinh, cuối quá trình chuyển sang SGD momentum để hội tụ sâu hơn.

2.4.5 Loss landscape trong Predictive Maintenance

- Với signal có nhiễu, loss landscape có thể nhiều local minimum nông.
- Khi dùng MSE với dữ liệu noisy, landscape khá trơn. Khi dùng Cross-Entropy + softmax, landscape phức tạp hơn nhưng thực tế huấn luyện dễ hơn (mục 3.10 giải thích lý do).

2.5 Taylor và tính lồi (convexity) — khái niệm ngắn

Mục tiêu: Hiểu khai triển Taylor (xấp xỉ hàm bằng parabol cục bộ) lý giải vì sao gradient descent chỉ an toàn với bước nhỏ, và khái niệm lồi cho biết giới hạn của lý thuyết đối với mạng sâu.

2.5.1 Khai triển Taylor

Taylor cho phép **xấp xỉ hàm phức tạp tại gần điểm x_0** bằng đa thức:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2} f''(x_0)(x - x_0)^2$$

(xấp xỉ bậc 1 — đường thẳng)
(thêm bậc 2 — parabol)

Vì sao hữu ích cho DL?

- Giải thích vì sao gradient descent hội tụ với η nhỏ: quanh một điểm, ta xấp xỉ loss bằng parabol, mỗi bước gradient descent thực chất là "rơi xuống đáy của parabol xấp xỉ".
- Nói rõ vì sao η lớn làm hỏng: xấp xỉ bậc 2 chỉ đúng trong lân cận nhỏ; bước quá xa làm xấp xỉ sai hẳn.
- Trong object detection/pose còn dùng Taylor để refine dự đoán.

2.5.2 Hàm lồi (convex)

Hàm f gọi là **lồi** nếu đoạn thẳng nối hai điểm bất kỳ trên đồ thị luôn **nằm trên đồ thị**:

$$f(t \cdot a + (1-t) \cdot b) \leq t \cdot f(a) + (1-t) \cdot f(b) \quad \text{với mọi } 0 \leq t \leq 1$$

Hoặc (với hàm trơn 1 biến): $f''(x) \geq 0$ với mọi x (độ cong không âm).

- Hàm lồi: **mọi local minimum đều là global minimum**. Gradient descent từ điểm nào cũng về đích.
- Ví dụ: x^2 , $-\ln(x)$, hàm MSE theo trọng số. Logistic loss cũng lồi theo tham số.

Vì sao liên quan?

- Các mô hình tuyến tính + loss lồi (hồi quy ridge/lasso) $\rightarrow$ dù dùng gradient descent hay thuật toán nào cũng tìm được nghiệm tối ưu toàn cục.
- Mạng sâu = hàm **không lồi** $\rightarrow$ không có bảo đảm cực tiểu toàn cục. Trong thực tế gradient descent vẫn hoạt động tốt (local minima "đủ tốt", nhiều tham số giúp né saddle point).
- Convexity giúp hiểu: không nên kỳ vọng lý thuyết chặt chẽ cho DL, mà dùng trực giác + thực nghiệm.

3. THỐNG KÊ & XÁC SUẤT

Mục tiêu: Dùng xác suất để (a) mô hình hóa nhiễu rung và chọn đúng hàm loss (MSE hay Cross-Entropy), (b) đo "bất thường" bằng kỳ vọng/phương sai/tương quan, (c) hiểu bất dương tính giả qua Bayes và đánh giá đúng khi lớp lỗi cực hiếm.

3.1 Khái niệm cơ bản

Mục tiêu: Năm biến ngẫu nhiên, phân phối rời rạc/liên tục và trực giác của hàm mật độ xác suất.

- **Biến ngẫu nhiên (random variable):** đại lượng nhận giá trị phụ thuộc kết quả ngẫu nhiên. X = "giá trị amplitude rung tại thời điểm ngẫu nhiên" là một biến ngẫu nhiên.
- **Phân phối xác suất:** mô tả xác suất mỗi giá trị xuất hiện.
- Biến **rời rạc**: hàm khối xác suất $P(X = x)$, ví dụ "số lần cảnh báo trong 1 giờ".
- Biến **liên tục**: hàm mật độ $p(x)$, ví dụ "giá trị rung (real)".

Với hàm mật độ $p(x)$ liên tục:

$$P(a \leq X \leq b) = \int_a^b p(x) dx \quad (\text{từ } a \text{ đến } b)$$

đồng thời $\int p(x) dx = 1$ trên toàn miền (tổng xác suất = 1).

3.2 Các phân phối phổ biến

Mục tiêu: Nhận diện 3 phân phối nền tảng — Uniform (khởi tạo trọng số), Bernoulli (bài toán lỗi/nhị phân), Gaussian (nhiều và MLE $\rightarrow$ MSE) — và biết dùng cái nào cho vấn đề nào.

3.2.1 Phân phối Uniform (đều)

Mọi giá trị trong khoảng $[a, b]$ có **xác suất bằng nhau**.

$$p(x) = 1/(b - a) \quad \text{với } a \leq x \leq b, \text{ ngược lại } p(x) = 0$$

Ví dụ: rung nền nhiều tầng lý tưởng không phải uniform, nhưng **khởi tạo trọng số** thường dùng uniform trong khoảng nhỏ: $w \sim U(-0.05, 0.05)$. Điều này giúp tránh trọng số ban đầu quá lớn (làm bùng nổ) hoặc mọi nơ-ron giống nhau (đối xứng).

3.2.2 Phân phối Bernoulli

Biến rời rạc **0/1** (thành công/thất bại). Một tham số $p = P(X = 1)$:

$$\begin{aligned} P(X = 1) &= p \\ P(X = 0) &= 1 - p \end{aligned}$$

Ví dụ: "mẫu rung này có chuẩn bị hỏng hóc hay không?" — đây chính là bài toán **bài toán phân loại nhị phân** trong Predictive Maintenance. Sản phẩm ra mô hình phóng ra $\hat{p}$ (xác suất lỗi) rồi so sánh với ngưỡng.

3.2.3 Phân phối Gaussian (chuẩn / normal)

Phân phối liên tục quan trọng nhất. Hai tham số: **mean μ** (tâm) và **phương sai σ^2** (độ rộng):

$$p(x) = (1 / (\sigma * \sqrt{2\pi})) * \exp(-0.5 * ((x - \mu) / \sigma)^2)$$

Hình dạng chuông đối xứng quanh μ ; ký hiệu $X \sim N(\mu, \sigma^2)$.

Vì sao Gaussian phổ biến?

- **Định lý giới hạn trung tâm (CLT):** tổng của nhiều biến ngẫu nhiên độc lập có xu hướng phân phối chuẩn, dù phân phối gốc là gì. Rất nhiều hiện tượng vật lý (nhiều đo lường, dao động ngẫu nhiên) tuân theo điều này.
- Nhiều rung nền (không có lỗi) thường xấp xỉ Gaussian quanh giá trị trung bình nhỏ.
- Là nền tảng của MLE dẫn đến **loss MSE** (mục 3.4–3.5).

Trong Predictive Maintenance với dữ liệu rung, ta thường kiểm tra xem phân phối amplitude ở trạng thái "ổn định" có chuẩn không; nếu không (lệch, nhiều đuôi) → nghi ngờ có hiện tượng bất thường (pitting, spalling, lỏng vít).

So sánh nhanh hai thái cực dùng cho A1:

Đặc điểm	Bernoulli	Gaussian
Kiểu biến	rời rạc 0/1	liên tục (real)
Tham số	$p = P(X=1)$	μ (tâm), σ^2 (độ rộng)
Vai trò trong đầu ra model	gắn cờ lỗi/normal	dự đoán giá trị liên tục: RUL, mức hư hại
Loss tương ứng (qua MLE)	Cross-Entropy	MSE

3.2.4 Quy tắc 68-95-99.7 (chuẩn)

Với Gaussian:

$$\begin{aligned} P(\mu - \sigma \leq X \leq \mu + \sigma) &\approx 0.68 \quad (68\%) \\ P(\mu - 2\sigma \leq X \leq \mu + 2\sigma) &\approx 0.95 \quad (95\%) \\ P(\mu - 3\sigma \leq X \leq \mu + 3\sigma) &\approx 0.997 \quad (99.7\%) \end{aligned}$$

Ứng dụng trực tiếp: cảnh báo bất thường. Nếu giá trị rung vượt $\mu + 3\sigma$, xác suất nó chỉ là nhiễu ngẫu nhiên rất nhỏ (<0.2%) → khả năng đang có bệnh tăng lên. Nhưng lưu ý: nếu phân phối không chuẩn thì quy tắc này không chính xác.

3.3 Kỳ vọng, Phương sai

Mục tiêu: Nắm hai "thước đo" trung tâm: kỳ vọng (trung bình theo xác suất) và phương sai/RMS (mức phân tán) — chỉ báo sớm hư hại bearing vì phương sai tăng trước khi biên độ trung bình đổi.

3.3.1 Kỳ vọng (Expected value / Mean)

Kỳ vọng = **trung bình có trọng số theo xác suất**:

$$\begin{aligned} \text{Rời rạc: } E[X] &= \sum x * P(X = x) \\ \text{Liên tục: } E[X] &= \int x * p(x) dx \\ \text{Mẫu thực tế: } E[X] &\approx (1/N) \sum x_i \quad (\text{trung bình cộng của } N \text{ mẫu}) \end{aligned}$$

Ví dụ số (rời rạc): số lần cảnh báo/giờ:

$$\begin{aligned} P(X=0) &= 0.5, \quad P(X=1) = 0.3, \quad P(X=2) = 0.2 \\ E[X] &= 0 * 0.5 + 1 * 0.3 + 2 * 0.2 = 0.7 \quad (\text{cảnh báo/giờ}) \end{aligned}$$

Tính chất tuyến tính: $E[aX + b] = aE[X] + b$ và $E[X + Y] = E[X] + E[Y]$ (kể cả khi X, Y không độc lập).

3.3.2 Phương sai (Variance)

Phương sai đo **mức phân tán** quanh kỳ vọng:

$$\text{Var}(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2$$

với $\mu = E[X]$. Độ lệch chuẩn (standard deviation):

$$\sigma = \sqrt{\text{Var}(X)}$$

Ví dụ số: dãy 5 mẫu rung: [1.0, 1.1, 0.9, 1.02, 0.98]

```

$$\mu = (1.0 + 1.1 + 0.9 + 1.02 + 0.98) / 5 = 1.0$$

Phương sai mẫu đúng (chia N-1):
Các độ lệch: 0; 0.1; -0.1; 0.02; -0.02
→ bình phương: 0; 0.01; 0.01; 0.0004; 0.0004

$$\text{Var} = (0 + 0.01 + 0.01 + 0.0004 + 0.0004) / 4 = 0.0208 / 4 = 0.0052$$


$$\sigma = \sqrt{0.0052} \approx 0.0721$$
 (rung ổn định, độ lệch nhỏ)
```

So với dãy [1.0, 1.5, 0.5, 1.3, 0.7] (cùng $\mu = 1.0$) nhưng $\text{Var} = 0.17$, $\sigma \approx 0.412$ — **phương sai tăng mạnh khi có chấn động**. Đây là một chỉ báo phát hiện sớm hỏng hóc: bệnh lý bearing thường làm **RMS/phương sai của rung tăng trước khi biên độ trung bình thay đổi**.

3.3.3 Đối với chuỗi thời gian

- Mean chuỗi: trung bình theo thời gian của tín hiệu.
- RMS (Root Mean Square)** — tiêu chuẩn công nghiệp cho rung:

$$\text{RMS} = \sqrt{(1/N) \sum x_i^2}$$

Với tín hiệu có mean ≈ 0 , thì $\text{RMS} \approx \sigma$ (phương sai). RMS là chỉ báo năng lượng dao động; tiêu chuẩn ISO 10816 dùng RMS velocity để phân loại tình trạng máy (tốt/chấp nhận/bất thường/nguy hiểm).

3.4 Covariance và Correlation

Mục tiêu: Đo quan hệ tuyến tính giữa 2 đại lượng bằng covariance và correlation; áp dụng cho feature selection và chẩn đoán lệch trục/bearing.

3.4.1 Covariance (hiệp phương sai)

Đo mức **thay đổi cùng chiều** của hai biến:

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

- Dương: X cao thì Y có xu hướng cao. Âm: ngược lại. Gần 0: ít liên quan (tuyến tính).
- Đơn vị: tích đơn vị của X và Y, khó so sánh.

Ví dụ số: X = nhiệt độ (°C), Y = mức rung:

```
Điểm: X = [40, 45, 50, 55]
      Y = [0.50, 0.55, 0.60, 0.66]

$$\mu_X = 47.5, \mu_Y = 0.5775$$


$$\text{Cov} \approx [(40-47.5)(0.5-0.5775) + (45-47.5)(0.55-0.5775) + (50-47.5)(0.6-0.5775) + (55-47.5)(0.66-0.5775)] / 4$$


$$\approx [0.58125 + 0.06875 + 0.05625 + 0.61875] / 4$$


$$\approx 0.331$$

```

Dương → nhiệt độ tăng, rung tăng theo. (Có thể là cả hai đều phản ánh tải/hư hại.)

3.4.2 Correlation (tương quan Pearson)

Chuẩn hóa covariance theo độ lệch chuẩn → **không thứ nguyên, trong [-1, 1]**:

$$\rho(X, Y) = \text{Cov}(X, Y) / (\sigma_X \cdot \sigma_Y)$$

- $\rho = 1$: tương quan dương hoàn hảo ($Y = a + bX$, $b > 0$).
- $\rho = -1$: tương quan âm hoàn hảo.

- $\rho = 0$: không tương quan tuyến tính (vẫn có thể phụ thuộc phi tuyến!).

Ví dụ số về ρ : X là mức tải tăng dần, Y là một đặc trưng rung đo được:

```
X = [1, 2, 3, 4]
Y = [2, 3, 5, 4]

μX = 2.5 → độ lệch: -1.5, -0.5, 0.5, 1.5 → σX ≈ 1.118
μY = 3.5 → độ lệch: -1.5, -0.5, 1.5, 0.5 → σY ≈ 1.118
Cov = ((-1.5)(-1.5) + (-0.5)(-0.5) + (0.5)(1.5) + (1.5)(0.5))/4 = 4/4 = 1.0
ρ = 1.0 / (1.118 * 1.118) = 0.8
```

$\rho = 0.8$: quan hệ tuyến tính khá mạnh (Y vẫn tăng khi X tăng) nhưng chưa hoàn hảo — đúng loại "gần như cùng hướng" thường thấy giữa tải và mức rung. (Lưu ý: μ và σ đều được tính đồng nhất theo cùng công thức chia N, nên ρ không bị lệch do mẫu số.)

3.4.3 Vì sao quan trọng với chuỗi rung?

- **Tương quan giữa các trục X/Y/Z:** lệch trục, mất cân bằng làm thay đổi mối tương quan giữa các trục so với trạng thái bình thường.
- **Feature selection:** loại bỏ các đặc trưng tương quan quá cao ($|\rho| > 0.95$) để giảm độ dư thừa và overfitting.
- **Autocorrelation** (tương quan theo lag) là khái niệm trọng tâm cho chuỗi thời gian — xem mục 4.2.
- Lưu ý: correlation $\neq$ causation. Tương quan cao chỉ gợi ý, không chứng minh quan hệ nhân quả.

3.5 MLE — Likelihood và vì sao dùng MSE, Cross-Entropy

Mục tiêu: Trả lời câu hỏi "vì sao hồi quy dùng MSE, phân loại dùng Cross-Entropy?" — bằng con đường MLE chọn tham số làm dữ liệu quan sát có khả năng nhất.

3.5.1 Likelihood là gì?

Giả sử dữ liệu quan sát $x_1, x_2, \dots, x_N$ sinh ra từ phân phối có tham số chưa biết θ . **Likelihood** = xác suất (mật độ) của dữ liệu như một hàm của θ :

$$L(\theta) = p(x_1 | \theta) * p(x_2 | \theta) * \dots * p(x_N | \theta)$$

Nhấn mạnh sự khác biệt:

- $p(x | \theta)$: cố định θ (đã biết), hàm theo x — hàm mật độ xác suất.
- $L(\theta) = p(x | \theta)$: dữ liệu x cố định (đã đo), hàm theo θ — **likelihood function**.

3.5.2 MLE — Maximum Likelihood Estimation

MLE: chọn θ làm cho **xác suất quan sát được dữ liệu này là lớn nhất**:

$$\hat{\theta}_{MLE} = \operatorname{argmax}_{\theta} L(\theta)$$

Vì tích của nhiều số nhỏ (<1) tràn số, thực tế tối ưu **log-likelihood** (log giữ thứ tự đơn điệu):

$$\log L(\theta) = \sum \ln p(x_i | \theta)$$

Tối đa hóa $\log L$ = tối thiểu hóa $-\log L$ → **negative log-likelihood (NLL)** — đây chính là cái mọi framework DL tối thiểu hóa!

3.5.3 MLE dẫn đến MSE — khi dữ liệu nhiễu Gaussian

Giả sử dự đoán $\hat{y} = f(x, w)$ đúng mean, quan sát $y = \hat{y} + \epsilon$ với $\epsilon \sim N(0, \sigma^2)$:

$$p(y | x, w) = (1/(\sigma \sqrt{2\pi})) * \exp(-0.5 * ((y - \hat{y})/\sigma)^2)$$

Log-likelihood của N mẫu:

$$\log L = \sum [-\ln(\sigma \sqrt{2\pi}) - 0.5 * ((y_i - \hat{y}_i)/\sigma)^2]$$

Các hạng thức hằng (không phụ thuộc w) bỏ đi, tối đa $\log L$ tương đương tối thiểu:

$$MSE(w) = (1/N) \sum (y_i - \hat{y}_i)^2$$

Kết luận: dùng MSE = chính là MLE dưới giả định nhiễu Gaussian, phương sai cố định. Đây là lý do MSE tự nhiên cho **hồi quy** (dự đoán giá trị liên tục như "thời gian sống còn lại RUL", "mức hư hại").

Ví dụ số: $\hat{y} = [1.0, 2.0, 3.0]$, $y = [1.1, 1.9, 3.2]$

$$MSE = ((0.1)^2 + (-0.1)^2 + (-0.2)^2) / 3 = (0.01+0.01+0.04)/3 = 0.02$$

3.5.4 MLE dẫn đến Cross-Entropy — khi dữ liệu Bernoulli (phân loại nhị phân)

Cho bài toán lỗi/không lỗi: $y \in \{0, 1\}$. Đầu ra mô hình là xác suất $\hat{p} = P(y=1)$. Model dùng Bernoulli:

$$p(y | \hat{p}) = \hat{p}^y * (1 - \hat{p})^{(1 - y)}$$

Kiểm tra: nếu $y=1 \rightarrow p(1) = \hat{p}$; nếu $y=0 \rightarrow p(0) = 1 - \hat{p}$. Đúng như định nghĩa Bernoulli.

NLL (âm log-likelihood):

$$NLL = -\sum [y_i * \ln(\hat{p}_i) + (1 - y_i) * \ln(1 - \hat{p}_i)]$$

Đây chính là **binary cross-entropy (BCE)**.

Ví dụ số: mẫu lỗi $y=1$, mô hình dự đoán $\hat{p} = 0.9$:

$$BCE = -[1 * \ln(0.9) + 0 * \ln(0.1)] = -\ln(0.9) \approx 0.105$$

Nếu mô hình kém, $\hat{p} = 0.2$: $BCE = -\ln(0.2) \approx 1.609$ — lớn hơn nhiều $\rightarrow$ bị phạt nặng, đúng ý muốn.

Tại sao Cross-Entropy (> MSE) cho phân loại?

Tiêu chí	MSE cho phân loại	Cross-Entropy (BCE) cho phân loại
Giả định sinh dữ liệu	nhiều Gaussian (không hợp khi y là 0/1)	Bernoulli (đúng bản chất biến nhị phân)
Khi mô hình "tự tin sai" ($\hat{p}=0$ cho $y=1$)	phạt hữu hạn, có hạn	phạt theo log $\rightarrow$ tiến tới ∞ , ép mô hình không dám vừa sai vừa tự tin
Gradient cuối softmax khi saturate	rất nhỏ (gradient vanishing) — học chậm	ổn định — học nhanh
Kết luận cho A1	ok cho RUL (hồi quy), dở cho gắn cờ lỗi	dùng cho nhãn lỗi/normal

Tóm gọn: biến nhị phân không có giả định Gaussian cho sai số; mô hình Bernoulli phù hợp bản chất, và hàm log phạt nặng lỗi "tự tin sai".

3.5.5 Mối quan hệ với log-likelihood — tóm gọn

Dạng bài	Giả định sinh dữ liệu	Loss tối ưu
Hồi quy	Nhiều Gaussian	$MSE = (1/N) \sum (y - \hat{y})^2$
Phân loại nhị phân	Bernoulli	$BCE = -\sum y \ln \hat{p} + (1-y) \ln(1-\hat{p})$
Phân loại đa lớp	Categorical	$Cross-Entropy = -\sum y_k \ln \hat{p}_k$

Dùng đúng loss dựa trên **bản chất của đầu ra** — đừng dùng MSE cho bài toán phân loại. Trong A1: dự đoán RUL $\rightarrow$ MSE; gắn cờ lỗi sắp xảy ra $\rightarrow$ Cross-Entropy.

3.6 Định lý Bayes

Mục tiêu: Kết hợp niềm tin ban đầu (prior) với dữ liệu (likelihood) thành xác suất có điều kiện (posterior); giải thích vì sao cảnh báo trên sự kiện hiếm thường là dương tính giả — bài học trực tiếp cho lớp lỗi cực hiếm của A1.

3.6.1 Công thức

$$P(A | B) = P(B | A) * P(A) / P(B)$$

Các thành phần:

- $P(A | B)$: **posterior** — xác suất A sau khi biết B (có dữ liệu).
- $P(A)$: **prior** — xác suất A trước khi có dữ liệu (niềm tin ban đầu).
- $P(B | A)$: **likelihood** — khả năng gặp B nếu A đúng.

- $P(B)$: **evidence** — hằng số chuẩn hóa (xác suất toàn thể của B).

3.6.2 Ví dụ số — Predictive Maintenance

Trong nhà máy, 2% máy đang "sắp hỏng" (sự kiện F). Test rung dương tính cho F với xác suất 90%; tỷ lệ dương tính giả (báo lỗi khi tốt) là 5%.

Hỏi: một máy có **kết quả đo XẤU (dương)** thì xác suất thực sự sắp hỏng là bao nhiêu?

Cho: $P(F) = 0.02$
 $P(+ | F) = 0.90$
 $P(+ | OK) = 0.05$
 $P(+) = P(+|F)P(F) + P(+|OK)P(OK) = 0.9*0.02 + 0.05*0.98 = 0.067$

$P(F | +) = 0.90 * 0.02 / 0.067 \approx 0.269$

Chỉ ~27%! Dù test "90% chính xác", đa số cảnh báo vẫn là **dương tính giả** — vì sự kiện hỏng hóc (prior nhỏ). F1-score và precision phải được đánh giá cẩn thận trong bài toán hiếm (class imbalance) như thế này.

3.6.3 Vì sao dùng trong ML?

- **Bayesian machine learning**: thay vì 1 điểm θ , ta giữ phân phối $P(\theta | \text{data})$. Hữu ích khi ít dữ liệu, cần ước lượng **bất định** (với cảnh báo bảo trì, biết độ tin cậy của dự đoán quan trọng: tốn tiền dừng máy).
- Giải thích thiên lệch khi dữ liệu mất cân bằng (mục 3.7-3.8): prior mạnh kéo dự đoán về lớp đa số.
- Naive Bayes — baseline đơn giản hiệu quả cho phân loại đặc trưng rung.

3.7 Bias / Variance và Overfitting / Underfitting

Mục tiêu: Phân rõ sai số thành bias/variance/noise để nhận diện underfitting và overfitting; biết cách dùng K-fold cross-validation — kèm lưu ý riêng cho chuỗi thời gian.

3.7.1 Phân rõ bias-variance của sai số

Sai số kỳ vọng của mô hình trên mẫu mới tách thành 3 phần:

$E[\text{error}] = \text{bias}^2 + \text{variance} + \text{irreducible noise}$

với:

- **Bias** = sai số hệ thống: mô hình quá đơn giản, không bắt được pattern thật.
- **Variance** = mô hình quá nhạy với dữ liệu huấn luyện cụ thể: thay đổi ít dữ liệu → thay đổi lớn mô hình.
- **Irreducible noise**: nhiễu không thể loại (vd nhiễu đo về bản chất).

3.7.2 Bốn trạng thái điển hình

Giả sử bản chất quan hệ giữa tốc độ quay và rung là $y = 0.0004 * \text{RPM}^2 + \text{nhiều}$:

1. **Underfitting (high bias)**: dùng mô hình tuyến tính $\hat{y} = w * \text{RPM}$. Không bắt được độ cong → sai số lớn ở cả train và test.
2. **Khớp tốt**: dùng polynomial bậc 2 → sai số train nhỏ, test nhỏ.
3. **Overfitting (high variance)**: dùng polynomial bậc 10 → sai số train gần bằng 0, nhưng test **rất lớn** — mô hình học cả nhiễu và biến động ngẫu nhiên.
4. **Khớp dữ liệu thực tế**: luôn có irreducible noise → sai số tốt nhất $\approx$ mức nhiễu.

3.7.3 Định nghĩa chuẩn

- **Overfitting**: mô hình nhớ dữ liệu huấn luyện thay vì học quy luật tổng quát. Dấu hiệu: loss train nhỏ, loss validation/test cao.
- **Underfitting**: mô hình quá đơn giản, không đủ sức biểu diễn pattern. Dấu hiệu: loss train và test đều cao.

3.7.4 Tradeoff — đánh đổi

- Tăng độ phức tạp mô hình (thêm layer, thêm feature) → bias giảm, variance tăng.
- Điểm cân bằng tốt nhất nằm ở giữa, nơi **tổng** sai số nhỏ nhất.

- Trong DL: chọn đủ lớn để bias thấp, rồi dùng **regularization** để giảm variance (weight decay, dropout, data augmentation, early stopping, cross-validation mục 3.7.5) thay vì hoàn toàn tránh phức tạp.

Mẹo phát hiện: vẽ đường cong loss theo epochs:

Underfit: train và val đều bắt đầu cao, vẫn giảm dần đều → cần mô hình mạnh hơn
Overfit: train giảm, val tăng lên tiếp → dừng (early stopping) tại point val thấp nhất

3.7.5 K-fold Cross-validation

Chia dữ liệu thành K phần, huấn luyện K lần, mỗi lần dùng 1 phần làm validation và phần còn lại làm train; lấy trung bình kết quả.

```
Fold 1: valid [1]   train [2 3 4 5]
Fold 2: valid [2]   train [1 3 4 5]
...
Kết quả = mean của K lần
```

Lưu ý chuỗi thời gian: KHÔNG shuffle ngẫu nhiên khi cross-validation (rò rỉ quá khứ-tương lai). Phải dùng time-based split: validate trên dữ liệu **sau** thời điểm train (xem mục 4.4).

3.8 Bất cân bằng dữ liệu và tổng kết thống kê

Mục tiêu: Đối mặt với lớp lỗi hiếm — bỏ accuracy thuần, đánh giá bằng precision/recall/F1, và so sánh các chiến lược cân bằng lớp (oversampling, SMOTE, class weights, threshold).

3.8.1 Vấn đề lớp hiếm trong Predictive Maintenance

- Hầu hết thời gian máy chạy **bình thường**; sự kiện hỏng rất hiếm. Tỷ lệ có thể 1 lỗi / 10.000 mẫu.
- Mô hình luôn tiên đoán "bình thường" đạt accuracy 99.99% nhưng **vô dụng**. Đánh giá phải dùng precision/recall/F1, ROC-AUC, không dùng accuracy thuần.
- Prior trong Bayes (mục 3.6) giải thích độ lệch tự nhiên về lớp đa số → cần cân nhắc oversampling, class weights, hoặc threshold tối ưu.

So sánh các chiến lược cân bằng lớp — chọn gì cho A1?

Chiến lược	Cơ chế	Ưu	Hạn chế
Oversampling (copy)	nhân bản thêm mẫu lỗi hiếm	đơn giản, tức thì	không thêm thông tin mới; dễ overfit nếu copy nguyên bản
SMOTE	tạo mẫu tổng hợp: nội suy giữa mẫu lỗi và k-láng giềng gần của nó	đa dạng hóa không gian đặc trưng, giảm overfit hơn copy	sinh mẫu có thể vô nghĩa nếu đặc trưng nhiễu; phải làm trong fold
Class weights	phạt sai trên lớp hiếm nặng hơn trong loss	không đụng dữ liệu, không lo leakage	phải tinh chỉnh tỉ lệ; có thể làm mô hình "khó nhằn"
Threshold tối ưu	hạ/nâng ngưỡng quyết định trên $\hat{p}$	không đụng model, tinh chỉnh ở bước dự đoán	cần metric mục tiêu rõ (F1, tỉ lệ báo động giả)

Điểm chung quan trọng với chuỗi thời gian: **oversampling/SMOTE phải làm trong từng fold**, sau khi chia train/valid, để tránh cùng một mẫu (hoặc mẫu bơm từ nó) lọt cả hai phía (xem mục 4.4.4).

3.8.2 Bảng tóm tắt các công thức thống kê chính

Kỳ vọng (rời rạc)	$E[X] = \sum x \cdot P(X=x)$
Phương sai	$Var(X) = E[(X-\mu)^2]$
Độ lệch chuẩn	$\sigma = \sqrt{Var(X)}$
Covariance	$Cov(X, Y) = E[(X-\mu_X)(Y-\mu_Y)]$
Correlation	$\rho = Cov(X, Y) / (\sigma_X \cdot \sigma_Y)$
MSE	$(1/N) \cdot \sum (y_i - \hat{y}_i)^2$
Binary Cross-Entropy	$-\sum [y_i \ln(\hat{p}_i) + (1-y_i) \ln(1-\hat{p}_i)]$
Bayes	$P(A B) = P(B A) \cdot P(A) / P(B)$

4. XÁC SUẤT & THỐNG KÊ CHO CHUỖI THỜI GIAN

(Phần đặc thù cho bài toán Predictive Maintenance bằng rung/chấn động)

Mục tiêu: Nắm các khái niệm riêng của dữ liệu có thứ tự theo thời gian: tính dừng, ACF/PACF, rolling statistics, lookback/horizon và bẫy data leakage — phần đặc thù và quyết định nhất của bài toán A1.

4.1 Định nghĩa chuỗi thời gian và tính dừng (Stationarity)

Mục tiêu: Biết chuỗi thời gian là gì, "tính dừng" nghĩa là gì, cách kiểm tra và làm dừng một chuỗi rung.

4.1.1 Chuỗi thời gian

Chuỗi thời gian = dãy quan sát theo thời gian đều nhau của một biến:

$x_1, x_2, x_3, \dots, x_T$

Với rung/chấn động:

- Tần số lấy mẫu điển hình: 10–25.6 kHz (đến vài chục kHz) cho gia tốc kế.
- 1 giây dữ liệu 25.6 kHz = 25.600 mẫu. Cần **cửa sổ** (chunk) để phân tích, thường 512–4096 mẫu.

Ký hiệu t : chỉ số thời gian/rời rạc.

4.1.2 Tính dừng (Stationarity) — khái niệm cốt lõi

Chuỗi gọi là **dừng (stationary)** (nghĩa hẹp, weak stationarity) nếu các đặc trưng thống kê **không đổi theo thời gian**:

- Mean không đổi: $E[x_t] = \mu$ (với mọi t)
- Phương sai không đổi: $\text{Var}(x_t) = \sigma^2$ (với mọi t)
- Autocovariance chỉ phụ thuộc lag k , không phụ thuộc thời điểm t :
 $\text{Cov}(x_t, x_{t+k}) = \gamma(k)$

Vì sao stationarity quan trọng?

- Hầu hết lý thuyết chuỗi thời gian (ACF, mô hình AR/ARIMA, chẩn đoán quang phổ) đòi hỏi dừng.
- Hầu hết mô hình ML/DL học quy luật chung; nếu phân phối trôi (drifting), mô hình phải được huấn luyện lại, hoặc phải dùng features không đổi theo thời gian (vd. spectral features của cửa sổ ngắn thường gần dừng hơn thô signal).
- Ví dụ không dừng:** máy khởi động → nhiệt độ tăng dần → rung trung bình tăng dần theo thời gian. Trend không dừng.

4.1.3 Kiểm tra tính dừng

- Trực quan:** vẽ chuỗi — có trend (trôi mean) hoặc "hình phễu" (phương sai thay đổi theo thời gian)?
- Rolling statistics** (mục 4.3): mean/std trượt xấp xỉ hằng số?
- ADF test (Augmented Dickey-Fuller):** kiểm định giả thuyết "chuỗi không dừng". p-value thấp (vd < 0.05) → bác bỏ → chuỗi dừng.
- ACF giảm nhanh về 0** → hầu như dừng.

4.1.4 Xử lý khi không dừng

Take difference: $y_t = x_t - x_{t-1}$ (loại trend tuyến tính)
Log hoặc Box-Cox: $y_t = \ln(x_t)$ (ổn định phương sai)
Detrend bằng hồi quy: $y_t = x_t - (a + b \cdot t)$
Chuẩn hóa theo cửa sổ: $y_t = (x_t - \mu_t) / \sigma_t$ (z-score trượt)

Với chuỗi rung máy: vì có chu kỳ máy (bắt đầu-ổn định-dừng), thường tách các "runs" ổn định, hoặc dùng cửa sổ ngắn (512 mẫu) để mỗi cửa sổ xem như gần dừng.

4.2 Autocovariance và Autocorrelation

Mục tiêu: ACF đo tự tương quan theo lag → phát hiện chu kỳ quay và "độ nhớ" của hệ; PACF giúp chọn số lag dùng làm đặc trưng.

4.2.1 Autocovariance

Autocovariance tại lag k = covariance giữa chuỗi với chính nó, dịch đi k bậc:

$$y(k) = \text{Cov}(x_t, x_{t+k}) = E[(x_t - \mu)(x_{t+k} - \mu)]$$

- $y(0) = \text{Var}(x_t)$.
- $|y(k)| \geq |y(k+1)|$ thường khá đúng với chuỗi "trơn".

4.2.2 Autocorrelation / ACF

Autocorrelation function (ACF) — chuẩn hóa autocovariance:

$$\rho(k) = y(k) / y(0) \in [-1, 1]$$

- $\rho(0) = 1$.
- $\rho(k) > 0$: giá trị tại t và $t+k$ có xu hướng cùng dấu so với mean (trơn).
- $\rho(k) < 0$: đảo dấu qua lại (dao động nhanh).

Ví dụ số: chuỗi `[1, 2, 3, 2, 1, 2, 3, 2]` (dạng sóng):

$$\begin{aligned} \mu &= 2.0 \\ x_t - \mu &: [-1, 0, 1, 0, -1, 0, 1, 0] \\ y(0) &= (1+0+1+0+1+0+1+0)/8 = 0.5 \\ y(1) &= [(-1)(0) + (0)(1) + (1)(0) + (0)(-1) + (-1)(0) + (0)(1) + (1)(0)]/8 = 0 \\ \rho(1) &= 0/0.5 = 0 \end{aligned}$$

Ồ chạy từng ô dương/âm đuổi nhau → lag 1 không tương quan.

4.2.3 Vì sao ACF hữu ích cho rung?

- **Tính chu kỳ:** chuỗi rung có thành phần tuần hoàn ở tần số quay → ACF đạt đỉnh lại đúng các lag bội của chu kỳ. Điểm **đỉnh ACF** → **phát hiện chu kỳ/tần số chia sẻ**, đơn giản mà hiệu quả cho fault detection.
- **Độ dài nhớ của hệ thống:** ACF giảm chậm → hệ "nhớ dài", trạng thái hiện tại có thông tin về tương lai → số bước lookback cần lớn (mục 4.4).
- Độ trơn/bất thường: khi bệnh phát triển, thay đổi cấu trúc xung → ACF thay đổi (vd xuất hiện đỉnh tại tần số $2 \times \text{RPM}$, sidebands).

4.2.4 Partial autocorrelation (PACF) — khái niệm

PACF $\alpha(k)$ = autocorrelation tại lag k **sau khi loại ảnh hưởng của các lag trung gian** (1..k-1).

$$\begin{aligned} \alpha(1) &= \rho(1) \\ \alpha(2) &= (\rho(2) - \rho(1)^2) / (1 - \rho(1)^2) \end{aligned}$$

Dùng để chọn bậc mô hình AR (số lag cần dùng làm đặc trưng), thay vì ACF (dễ nhiễu chéo giữa các lag).

So sánh ACF vs PACF — dùng cái nào?

Tiêu chí	ACF	PACF
Đo	tương quan ở lag k tổng hợp (gồm cả ảnh hưởng gián tiếp qua các lag trung gian)	tương quan ở lag k sau khi đã trừ ảnh hưởng của lag 1..k-1
Hướng dùng	nhận diện chu kỳ, tính mùa, kiểm tra dừng (đỉnh tại lag bội của chu kỳ)	chọn số bậc p cho mô hình AR/ số lag làm đặc trưng
Trong A1	phát hiện tần số quay + harmonic	tránh đưa các lag gần nhau trùng thông tin vào model

4.3 Rolling Statistics (thống kê cửa sổ trượt)

Mục tiêu: Tóm tắt một cửa sổ hàng nghìn mẫu thô thành vài đại lượng (mean/std/RMS) làm feature cấp cửa sổ; biết cách chọn độ dài W và xử lý ngoại lệ.

4.3.1 Định nghĩa

Với cửa sổ độ dài W (vd 256, 512, 1024 mẫu), **rolling statistic** tại thời điểm t tính trên cửa sổ **kết thúc tại t** :

$$\begin{aligned} \text{Mean trượt: } \mu_t &= (1/W) \sum_{i=t-W+1..t} x_i \\ \text{Std trượt: } \sigma_t &= \sqrt{(1/(W-1)) \sum_{i=t-W+1..t} (x_i - \mu_t)^2} \end{aligned}$$

RMS trượt: $\text{rms_t} = \sqrt{(1/W) \sum_{i=t-W+1..t} x_i^2}$

Ví dụ số: chuỗi [1, 2, 1, 2], $W = 2$:

t=2: $\mu = (1+2)/2 = 1.5$
t=3: $\mu = (2+1)/2 = 1.5$
t=4: $\mu = (1+2)/2 = 1.5$

4.3.2 Vì sao dùng?

- Các chỉ số này là **feature cấp cửa sổ** — tóm tắt hàng nghìn mẫu thô thành vài số đầu vào cho model.
- Chuẩn công nghiệp ISO 10816 dựa trên **RMS velocity**. Rolling RMS tăng dần → phát hiện hư hại sớm (năng lượng rung tăng trước khi biên độ đạt ngưỡng cắt).
- Detrend + làm trơn nhiễu ngẫu nhiên; phát hiện đổi trạng thái (drift) một cách trực quan.

4.3.3 Chọn độ dài cửa sổ W

- W nhỏ (64–256)**: phản ứng nhanh, nhiều hơn trong ước lượng; dễ sinh false alarm.
- W lớn (1024–8192)**: ước lượng vững (var giảm theo $1/W$), nhưng chậm phản ứng (lag); có thể bỏ sót lỗi nhanh bất ngờ.
- Cân nhắc**: cho phép đủ số vòng quay trong cửa sổ. Ví dụ máy 1500 RPM = 25 vòng/s. Muốn 10 vòng/cửa sổ → cửa sổ ≥ 0.4 s → ≈ 25.6 kHz $\approx \geq 10240$ mẫu, hoặc dùng feature tần số thay vì thời gian.
- Rolling std quá lớn khi có spike → sử dụng **median + MAD** (Median Absolute Deviation) nếu bị nhiễu xung (impulse noise) làm phân phối fat-tail.

4.4 Thang đo cửa sổ, lookback: từ quá khứ → tương lai

Mục tiêu: Tạo dữ liệu huấn luyện TỪ CHUỖI THEO THỨ TỰ bằng lookback/horizon và sliding window, tuân thủ chiều nhân quả và chặn data leakage.

4.4.1 Vấn đề: phải dùng quá khứ để dự đoán tương lai

Model chuỗi thời gian học ánh xạ:

$[x_{t-\text{window}}, \dots, x_t] \rightarrow$ dự đoán ở $t+1, t+\text{horizon}$

với:

- Lookback** (window size, context length): số mẫu/bước trong quá khứ dùng làm đầu vào.
- Horizon**: khoảng cách tới mốc cần dự đoán (vd dự đoán 1 giờ trước sự cố, hay RUL đến hỏng).
- Strata/time alignment**: khi tạo tập hỗ trợ cho huấn luyện.

Ví dụ cụ thể:

Nhận cửa sổ $W = 1024$ mẫu rung (lookback = 1024), dự đoán xác suất lỗi trong 24 giờ (horizon = 24h). Quyết định bảo trì dựa trên dự đoán này.

4.4.2 Ví dụ số minh họa lookback

Cần ước lượng ban đầu dùng trung bình cửa sổ trị $a=1, b=2$ trượt bậc 2 (đơn giản, minh họa ý tưởng):

$x = [1, 2, 1, 2, \dots]$ (chu kỳ rung giả định)
Lookback $W=2$: dự đoán $x_{t+1} = \text{ave}(x_t, x_{t-1})$
t=3: $\text{ave}(x_2, x_3) = \text{ave}(2, 1) = 1.5 \rightarrow$ dự đoán 1.5 cho x_4 (=2) — sai số 0.5

Vì chu kỳ 2 mẫu, lookback đủ 2 mẫu là tối thiểu; muốn dự đoán chu kỳ chuẩn hơn cần lookback $\geq$ chu kỳ. (Nói nôm na: đối với rung máy, lookback phải phủ **hiều vòng quay**, không chỉ vài mẫu liền kề.)

Quy tắc thực hành: lookback phải **đủ dài**, nhưng dài quá làm tăng số tham số của model (trừ dùng RNN/attention xử lý chuỗi dài); feature tóm tắt cấp cửa sổ (RMS, spectral) giúp giữ thông tin với lookback ngắn.

4.4.3 Cấu trúc dữ liệu huấn luyện (sliding window)

Từ chuỗi dài, tạo ra nhiều mẫu bằng cửa sổ trượt:

X (input, lookback=W)	y (target)
[x1 x2 ... xW]	[x_{W+horizon} hoặc nhãn lỗi tại W+horizon]
[x2 x3 ... x_{W+1}]	[x_{W+1+horizon} hoặc nhãn ...]
[x3 x4 ... x_{W+2}]	[x_{W+2+horizon} ...]

Ví dụ: chuỗi x1..x8, W=3, horizon=1, hồi quy:

[x1 x2 x3]	→ y = x5
[x2 x3 x4]	→ y = x6
[x3 x4 x5]	→ y = x7
[x4 x5 x6]	→ y = x8

4.4.4 Bẫy rò rỉ (data leakage) — cực kỳ quan trọng

Trong huấn luyện chuỗi thời gian, **không được để tương lai lọt vào đầu vào**:

- Chuẩn hóa/scale **chỉ dựa trên cửa sổ quá khứ** (hoặc trên phần train), KHÔNG dùng mean/std của toàn bộ chuỗi (gồm cả tương lai).
- Train/valid/test split theo **thời gian** (valid sau train), không shuffle.
- K-fold: dùng **expanding/rolling origin** (train tích lũy, valid kế tiếp), lặp lại K lần như mục 3.7.5 nhưng giữ trật tự thời gian.
- Oversampling để cân bằng lớp hiếm phải làm **trong fold**, không trước khi chia (tránh cùng mẫu xuất hiện cả train lẫn valid).

Sơ so sánh hai cách chia dữ liệu — vì sao time-based thắng cho A1?

Tiêu chí	Random shuffle K-fold	Time-based split
Nguyên tắc	trộn ngẫu nhiên rồi chia K phần	chia theo mốc thời gian: train ≤ mốc chia < valid ≤ test
Rủi ro	data leakage : mẫu tương lai lọt vào train	thấp: mô hình chỉ thấy quá khứ
Tính tổng quát	nếu chuỗi dừng thì ok	đúng bản chất dự báo, chấp nhận valid bị chệch nếu trend trôi
Kết luận	dùng khi chắc chắn chuỗi đủ dừng	mặc định cho chuỗi rung máy

4.4.5 Baseline timeline cần chú ý

Quá khứ (x1..x_t) —————> Mốc t —> Tương lai (t+1.. t+horizon)

Dự đoán tốt yêu cầu: tín hiệu quá khứ **có chứa thông tin có ích** (tương quan cao với tương lai — ACF giúp kiểm tra điều này), và mô hình học đúng chiều nhân quả (past → future, không reverse).

TỔNG KẾT

Toàn bộ toán nền tảng này kết nối với pipeline Predictive Maintenance như sau:

- Thu thập**: đo gia tốc 3 trục, nhiệt độ, RPM → dữ liệu = ma trận (mục 1.1–1.2).
- Tiền xử lý**: chuẩn hóa đặc trưng (z-score), loại trend, kiểm tra dừng (mục 1.3, 4.1), tính cách rolling/RMS (mục 4.3).
- Trích đặc trưng**: từ miền thời gian (RMS, std, peak) + miền tần số (FFT → SVD/PCA có thể dùng để nén) (mục 1.4).
- Huấn luyện**: forward → loss (MSE cho RUL, Cross-Entropy cho lỗi/không lỗi) → backprop (chain rule) → gradient descent. Dùng đúng loss theo mục 3.5.
- Đánh giá**: bias-variance, cross-validation time-based, precision/recall cho lớp hiếm (mục 3.7, 3.8).
- Đưa ra quyết định**: dự đoán theo lookback/horizon đúng chuỗi nhân quả (mục 4.4).

Cẩm nang kỹ thuật nhanh:

Đồng bộ dữ liệu	→ matrix batch, GPU (mục 1.5)
Điều chỉnh w	→ gradient descent + learning rate (mục 2.4)
Huấn luyện mạng	→ backprop = chain rule (mục 2.3)
Chọn loss	→ phân loại: CrossEntropy; hồi quy: MSE (mục 3.5)
Chống overfit	→ bias/variance, regularization, early stopping (mục 3.7)
Chuỗi thời gian	→ dừng, ACF, rolling, lookback, thời gian-split (mục 4.x)

(Hết tài liệu Phần 1 — Toán nền tảng. Tiếp theo có thể là Phần 2: Xử lý tín hiệu & Trích đặc trưng, và Phần 3: Mô hình ML/DL cho chuỗi thời gian.)

PHẦN 2 · MACHINE LEARNING CỐT LỖI & ANOMALY DETECTION — MACHINE LEARNING CỐT LỖI CHO BÀI TOÁN PREDICTIVE MAINTENANCE & ANOMALY DETECTION

Giáo trình luyện thi — Cuộc thi **DENSO Factory Hacks 2026** Bài toán **A1**: "AI bảo trì dự đoán khi thiếu dữ liệu lỗi" — dữ liệu là chuỗi thời gian rung của máy móc. Đối tượng: sinh viên / kỹ sư AI. Mức độ: từ nền tảng đến ứng dụng trực tiếp. Bản này được viết khớp với hiện trạng code của team (`src/pipeline.py` , `src/features.py` , `scripts/01` , `scripts/02`).

Cây bài toán con — bản đồ toàn Phần 2

Trước khi vào chi tiết, hãy nhìn bức tranh tổng. Bài toán A1 không phải "một việc duy nhất" mà là chuỗi bài toán con nối nhau. Biết mình đang ở nhánh nào sẽ biết mình đang giải gì:

MỤC TIÊU LỚN: phát hiện **LỖI SỚM** trên chuỗi rung, trong khi dữ liệu lỗi **CỰC HIẾM**

- (1) Tín hiệu dài hàng nghìn điểm → vài con số có nghĩa?
→ Mục 2. Feature engineering (window/overlap, FFT, 16 feature, chuẩn hoá)
- (2) Có nhãn lỗi → phân loại "bình thường hay lỗi"?
→ Mục 3. Học có giám sát (LogReg → Decision Tree/RF → XGBoost)
- (3) **KHÔNG** có nhãn lỗi → tách "cái gì khác bình thường"?
→ Mục 5. Anomaly detection (OC-SVM / Isolation Forest / Autoencoder / statistical)
- (4) "Thế nào là tốt" khi lớp hiếm — chọn mô hình nào, ngưỡng nào?
→ Mục 4. Metrics (Confusion / Precision-Recall-F1 / PR vs ROC / threshold theo chi phí)
- (5) Lớp lỗi quá ít → làm sao bù?
→ Mục 6. Imbalance + augmentation (undersampling / SMOTE / GAN / diffusion)
- (6) Làm sao **CHỨNG MINH** model thật sự có giá trị (không "chém gió")?
→ Mục 6.4. Protocol trước/sau augmentation + chống data leakage
- (7) (Mở rộng) còn bao lâu nữa thì hỏng? Không chỉ 0/1?
→ Mục 7. RUL dự báo ngắn
- (8) Gom mọi nhánh vào một luồng chạy được
→ Mục 8. Pipeline thực hành A1 (sơ đồ 8.1, quyết định 8.2, checklist 8.3)

Đọc nhanh theo tư duy đúng: - Kéo mũi tên từ trái sang phải: **tín hiệu** → **feature** → **model** → **metrics** → (**bù lớp**) → **protocol** → (**RUL**) → **pipeline**. - Nhánh (2) và (3) là hai con đường song song phụ thuộc vào một câu hỏi duy nhất: "**ta có bao nhiêu nhãn lỗi?**". Có nhiều → đi nhánh (2); gần như không → đi nhánh (3). - Nhánh (6) là "bộ chia trung thực" — nó quyết định mọi con số bạn báo cáo có đáng tin hay không. Đây là phần giám khảo chấm điểm mạnh nhất ở đề A1.

Mục lục Phần 2:

1. Overview bài toán A1
2. Feature engineering cho tín hiệu rung
3. Học có giám sát cổ điển
4. Metrics (trọng tâm — bài toán mất cân bằng mạnh)
5. Anomaly detection / học không giám sát
6. Xử lý dữ liệu mất cân bằng + thiếu dữ liệu lỗi
7. RUL dự báo ngắn
8. Pipeline thực hành cho A1 - Phụ lục A: Bảng công thức nhanh · Phụ lục B: Từ vựng Anh-Việt

1. Overview bài toán A1

Mục tiêu: nắm chính xác ta đang giải bài toán gì — đầu vào, đầu ra, thứ khó nhất — để không lạc hướng khi chọn feature và model.

1.1. Mô tả bài toán

Bài toán A1 của cuộc thi DENSO Factory Hacks 2026 xoay quanh **bảo trì dự đoán (predictive maintenance)**:

- **Input**: chuỗi thời gian của các cảm biến gắn trên máy móc nhà máy:
- **Rung (vibration)** — cảm biến gia tốc, thường đo ở tần số lấy mẫu cao (kHz trở lên). Đây là tín hiệu CHÍNH.
- **Nhiệt độ (temperature)** — nhiệt độ động cơ, ổ trục, vỏ máy.
- **Dòng điện (current)** — dòng tiêu thụ của motor.
- Có thể thêm: áp suất, tiếng ồn, điện áp...
- **Output**: nhãn trạng thái cho từng cửa sổ thời gian (window):
- **0** = bình thường (normal).
- **1** = bất thường / sắp hỏng (anomaly / fault).

Nói cách khác: **phát hiện bất thường** (fault detection / anomaly detection) trên dữ liệu cảm biến. Mô hình cần trả lời câu hỏi: "Tại thời điểm này, máy có hoạt động đúng không?"

1.2. Thách thức cốt lõi

Vấn đề nan giải nhất của bài toán này — và cũng chính là tên đề bài A1 — là:

Dữ liệu lỗi cực hiếm (fault data is extremely scarce).

Cụ thể hơn:

1. **Mất cân bằng nghiêm trọng (severe class imbalance)**: Trong nhà máy thực tế, máy chạy bình thường 99%+ thời gian. Số mẫu bất thường có thể chỉ chiếm 0.1%–2% dữ liệu.
2. **Khan hiếm nhãn lỗi (label scarcity)**: Máy hiếm khi hỏng, và hỏng thì người ta cũng thường không ghi nhãn đầy đủ. Thu thập dữ liệu lỗi là đắt, nguy hiểm và mất thời gian.
3. **Lỗi đa dạng**: Lỗi có thể là mài mòn ổ trục, lệch trục (misalignment), mất cân bằng rotor, nới lỏng ốc, chạm stato — mỗi loại có "dấu vân tay" tín hiệu khác nhau. Có lỗi phát triển từ từ (mài mòn), có lỗi đột ngột (chập, vỡ).
4. **Chuỗi thời gian liên quan đến thời gian thực**: Không thể shuffle ngẫu nhiên train/test như dữ liệu bảng thông thường vì sẽ gây **data leakage** (rò rỉ dữ liệu tương lai vào quá khứ).

Hiện trạng team đang làm đúng tinh thần này: file `src/pipeline.py` mô phỏng đúng tỉ lệ "normal nhiều, fault hiếm" — với dataset IMS (run-to-failure), mỗi test-run được chia theo thời gian thành: 0 → 70% NORMAL, 70 → 90% bỏ qua (vùng mơ hồ, nhãn không rõ), 90 → 100% FAULT. Vùng FAULT chỉ chiếm một phần nhỏ cuối chuỗi — đó chính là "dữ liệu lỗi hiếm" của đề bài.

1.3. Vì sao dữ liệu rung lại quan trọng nhất?

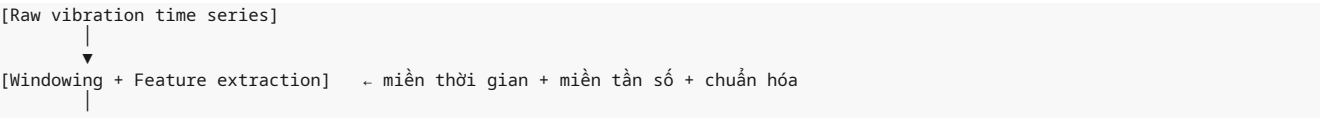
Tham số máy móc phản ứng rất nhanh với trục trặc cơ khí:

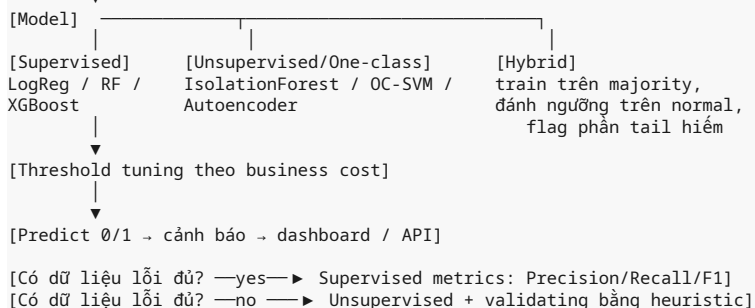
- **Lệch trục / mất cân bằng** → khung 1× (vòng quay), 2×... thành phần tần số thay đổi.
- **Hỏng ổ trục (bearing)** → xung lực lặp lại ở tần số đặc trưng BPFO/BPFI/BSF/FTF.
- **Mài mòn** → năng lượng dao động tăng, dạng sóng từ sin thuần túy thành nhiễu phức tạp.

Tại sao phải lấy feature? Vì dữ liệu rung thô thường có tần số lấy mẫu hàng chục kHz (tức hàng chục nghìn điểm mỗi giây). Đưa trực tiếp raw signal vào mô hình truyền thống là bất khả thi — số chiều quá lớn và nhạy nhiễu. **Feature engineering** nén chuỗi dài hàng nghìn điểm thành vài chục con số có ý nghĩa vật lý, giúp các mô hình cổ điển "ăn" được.

Ví dụ minh họa tổng quan: một chuỗi rung 1 giây ở 20 kHz = 20.000 điểm. Sau khi gom cửa sổ 0,1 s, mỗi cửa sổ (2.000 điểm) được nén thành ~40 feature. Dữ liệu từ "20.000 chiều" thành "40 chiều", rất nhẹ và hiệu quả cho logistic regression / XGBoost. (Trong code hiện tại của team, mỗi window được nén thành đúng **16 feature** — bảng đầy đủ ở Mục 2.6.)

1.4. Luồng tư duy chung để giải A1





👉 Mục 8 sẽ đưa pipeline chi tiết hơn, khớp với luồng chạy thực tế của 3 script trong repo.

2. Feature engineering cho tín hiệu rung

Mục tiêu: nén chuỗi thời gian thô (hàng nghìn điểm) thành các đại lượng vô hướng có ý nghĩa vật lý, tối đa sức phân biệt normal vs anomaly. Đây là nơi **kiếm được nhiều điểm nhất** ở A1 — một feature tốt còn hơn một model xịn trên feature dởm.

Bài toán con mà mục này giải: nhánh (1) của cây — "tín hiệu dài → vài con số có nghĩa". Nếu bỏ qua nhánh này, mọi model phía sau (supervised lẫn unsupervised) đều vận hành trên dữ liệu quá chiều, quá nhiễu và không mang thông tin vật lý.

2.1. Window và overlap cơ bản (nền tảng trước tiên)

Mục tiêu: chia chuỗi dài thành các khối ngắn để mỗi khối cho ra một bộ feature + một nhãn 0/1 riêng — đây là "đơn vị dự đoán" của A1.

Trước khi tính feature, ta phải chia chuỗi thời gian dài thành **cửa sổ (window)** ngắn:

- **Cửa sổ (window):** một khối liên tiếp các mẫu. Ví dụ chuỗi rung 60 s @ 20 kHz = 1.200.000 mẫu. Nếu lấy window = 0,1 s → 2.000 mẫu/window → 600 windows.
- **Overlap (chồng lấp):** các cửa sổ kế nhau được phép đè lên nhau một phần.

Nếu hai cửa sổ kế nhau cách nhau `stride` mẫu: $\text{overlap_ratio} = (\text{window_len} - \text{stride}) / \text{window_len}$

Ví dụ: window = 2.000 mẫu, stride = 200 mẫu → overlap 90%.

Vì sao cần overlap?

- Nhãn của cảm biến thường được gán theo khoảng thời gian (ví dụ: "từ giây 30 đến giây 40 là lỗi"). Nếu cửa sổ nằm lọt vào khoảng bình thường nhưng lại gồm đúng ranh giới lỗi, ta sẽ "lỡ" sự kiện.
- Overlap tăng số lượng mẫu huấn luyện → sinh thêm dữ liệu từ cùng một tín hiệu (rất hữu ích khi dữ liệu ít).
- Giúp dự đoán mượt hơn theo thời gian: thay vì nhảy từng nhịp, ta có thể lấy trung bình/đa số phiếu của các cửa sổ liền kề.
- Hạn chế: overlap làm các cửa sổ **phụ thuộc lẫn nhau** → khi tách train/test phải chia theo thời gian (không shuffle), và tránh để một sự kiện lỗi xuất hiện trong cả train lẫn test.

Cỡ window chọn sao?

- Đủ nhỏ để "bắt kịp" sự thay đổi nhanh của tín hiệu (nếu window quá dài sẽ làm mịn nhòe cú sốc/đợt rung bất thường ngắn).
- Đủ lớn để có đủ mẫu cho ước lượng thống kê ổn định và đủ độ phân giải tần số (xem 2.4.3).
- Quy tắc ngón tay cái: window lớn hơn **một chu kỳ xoay của máy** là hợp lý (nếu 1.800 vòng/phút ≈ 30 vòng/s, một chu kỳ ≈ 0,033 s → window ≥ 0,05–0,1 s).

Ví dụ số: - Tần số lấy mẫu `fs = 20_000` Hz, window 2.000 mẫu = 0,1 s. - Đủ thời gian cho Fourier phân biệt tần số bội 20 kHz/2000 = 10 Hz (khoảng hai đỉnh gần nhau tách được nếu cách ≥ 10 Hz).

Ở hiện trạng team: `src/pipeline.py` dùng `win=512`, `stride=256` (overlap 50%) trên dữ liệu IMS có `fs = 12_000` Hz; nếu dataset quá dài, stride được tự nới để tổng số window không vượt mốc an toàn cho máy tính.

2.2. Feature miền thời gian (time-domain features)

Mục tiêu: từ từng window, trích các đại lượng mô tả **hình dạng và mức độ** dao động trong miền thời gian — đây là lớp feature trực quan nhất, ai cũng tính được và gần như luôn có ích.

Bài toán con: mỗi window là N mẫu; ta cần nén chúng thành vài con số. Bối cảnh: có feature "bắt năng lượng" (std, RMS), có feature "bắt sự kiện tức thời" (peak), có feature "bắt hình dạng phân bố" (skewness, kurtosis), có feature "bắt tỉ lệ" (crest/shape/impulse factor). Dùng đồng bộ cả bốn họ để bao phủ đủ loại lỗi.

So sánh nhanh vai trò hai feature "năng lượng" hay bị nhầm: **std vs RMS** — về mặt toán học, nếu tín hiệu đã trừ mean thì `RMS == std`. Nhưng trong thực tế bộ lọc tiền xử lý trừ mean không hoàn toàn, và nhiều dataset tính RMS trên tín hiệu CÓ thành phần DC → không bằng std. **Kết luận cho A1: tính cả hai, giữ cả hai — không mất gì, thêm độ an toàn.**

Đặt chuỗi trong window có N mẫu: $x = [x_1, x_2, \dots, x_N]$. (Ở đây N là số mẫu trong một cửa sổ.)

2.2.1. Mean (trung bình)

Công thức: `mean(x) = (1/N) * sum(xi)` với $i = 1..N$ hay `mean = (x1 + x2 + ... + xN) / N`

Ý nghĩa vật lý: mức DC của tín hiệu. Với gia tốc kế lý tưởng đặt trên máy rung đối xứng, $\text{mean} \approx 0$. Nếu mean dạt khỏi 0 (bias) có thể là lỗi cảm biến, drift nhiệt độ, hoặc lệch trục tính. Nó gần như không bắt được "mạnh yếu" của rung — nên thường được loại bỏ trước khi tính các feature khác.

2.2.2. Standard deviation (độ lệch chuẩn) — std

Công thức: `std = sqrt( (1/N) * sum((xi - mean)^2) )`

Ý nghĩa vật lý: mức độ phân tán của tín hiệu quanh giá trị trung bình — nói lên "**mạnh hay yếu**" của dao động xoay quanh điểm cân bằng. Khi máy mòn, chi tiết bị lỏng, lệch, phản lực tăng → biên độ dao động lan rộng → std tăng. Đây là feature thô nhất để nhận biết "máy rung hơn bình thường".

2.2.3. RMS (Root Mean Square) — căn bậc hai trung bình bình phương

Công thức: `RMS = sqrt( (1/N) * sum(xi^2) )`

Đây chính là tổng năng lượng trung bình của tín hiệu theo thời gian.

Ý nghĩa vật lý: RMS là thước đo **độ lớn hiệu dụng** của dao động — đại lượng tiêu chuẩn trong kỹ thuật rung (tiêu chuẩn ISO 10816 đánh giá độ rung máy dựa trên RMS tốc độ/vận tốc). RMS tăng đều khi đặc tính mòn/lệch trục dần tiến triển, do đó là feature tốt cho cả **fault detection** lẫn **RUL prediction**.

Lưu ý: nếu tín hiệu đã được trừ mean, `RMS == std` về mặt toán học (vì $\text{sum}(xi^2)/N - \text{mean}^2 = \text{variance}$). Nhưng về mặt kỹ thuật TA NÊN tính cả hai, vì: 1. Việc trừ mean thường xảy ra trong bộ lọc tiền xử lý — có thể không hoàn toàn. 2. Trong một số dataset, RMS được tính trên tín hiệu CÓ mean (bao gồm cả thành phần DC) → không bằng std. Giữ cả hai sẽ không mất gì.

2.2.4. Peak (đỉnh) & Peak-to-peak

Công thức: `peak = max(|xi|)` — biên độ tuyệt đối lớn nhất trong window. `peak_to_peak = max(xi) - min(xi)` — khoảng dao động đỉnh-đỉnh.

Ý nghĩa vật lý: **Bắt sự kiện tức thời** — cú va đập, xung và chạm, khe hở cơ khí. Một ổ trục nứt vỡ sẽ sinh ra xung lực lớn dù RMS chưa tăng nhiều. Peak rất nhạy nhưng cũng dễ bị nhiễu đo (một spike nhiễu làm tăng peak). Vì vậy peak thường kết hợp với chuẩn hóa và ngưỡng hợp lý để tránh báo động giả vì nhiễu.

2.2.5. Skewness (độ lệch bất đối xứng)

Công thức: `skewness = ( (1/N) * sum((xi - mean)^3) ) / std^3`

Ý nghĩa vật lý: độ bất đối xứng của phân bố biên độ. Tín hiệu rung đối xứng (sin + nhiễu đối xứng) → skewness ≈ 0. Khi có hiện tượng **chạm/cọ xát** một chiều (rubbing), hoặc lỗi tạo ra xung lực một phía → phân bố lệch → skewness dương hoặc âm rõ rệt. Skewness giúp phát hiện loại lỗi tạo ra biên độ bất đối xứng.

2.2.6. Kurtosis (độ nhọn)

Công thức (kurtosis dư, excess kurtosis): $kurtosis = \left(\frac{1}{N} \sum (x_i - mean)^4 \right) / std^4 - 3$

(-3 để chuẩn hóa: phân phối chuẩn Gaussian có kurtosis = 0.)

Ý nghĩa vật lý: mức độ "nhọn/thừa đuôi" của phân bố. Tín hiệu rung bình thường gần Gaussian → kurtosis ≈ 0. Khi xuất hiện **xung lực lặp lại (impulses)** — dấu hiệu kinh điển của ổ trục hỏng, vỡ răng bánh răng — phân bố trở nên nhọn, đuôi dày → **kurtosis tăng vọt** (lên 5, 10, 20+). Đây là một trong những feature **nhạy nhất** với hỏng ổ trục ở giai đoạn sớm.

2.2.7. Crest factor (hệ số đỉnh)

Công thức: $crest_factor = peak / RMS$

Ý nghĩa vật lý: tỉ lệ giữa đỉnh và mức năng lượng. Tín hiệu sin thuần túy có crest factor = $\sqrt{2} \approx 1,414$. Tín hiệu có xung lực sắc nhọn (giàu impulse) → crest factor cao (3, 5, 10...). Khi máy mòn dần, đôi khi RMS tăng nhanh hơn peak → crest factor **giảm** — một dạng tương đối hóa giúp phát hiện sự thay đổi bản chất tín hiệu.

2.2.8. Shape factor (hệ số hình dạng)

Công thức: $shape_factor = RMS / mean_abs$ với $mean_abs = \frac{1}{N} \sum (|x_i|)$

Ý nghĩa vật lý: quan hệ giữa mức năng lượng và mức tuyệt đối trung bình — phản ánh "hình dạng" dạng sóng. Sin thuần túy có shape factor ≈ 1,11. Khi tín hiệu chứa nhiều xung (spiky) thì shape factor cao hơn. Dùng kết hợp với crest factor để phân biệt dạng sóng.

2.2.9. Impulse factor (hệ số xung)

Công thức: $impulse_factor = peak / mean_abs$

Ý nghĩa vật lý: tương tự crest factor nhưng so với mức trung bình tuyệt đối thay vì RMS. Nhạy với xung đột ngột (impulsive faults): khi có va đập, peak tăng nhanh hơn mean_abs → impulse factor tăng. Rất hữu ích cho phát hiện lỗi dạng impact (khe hở, nứt vỡ bề mặt).

2.2.10. Bảng tổng kết feature thời gian + khoảng giá trị tham khảo

Feature	Công thức	Ý nghĩa	Bắt loại lỗi gì
mean	$\frac{1}{N} \sum (x_i)$	mức DC, drift	lỗi cảm biến, lệch tĩnh
std	$\sqrt{\frac{1}{N} \sum (x_i - mean)^2}$	mức năng lượng dao động	mòn tăng dần (mọi loại)
RMS	$\sqrt{\frac{1}{N} \sum (x_i^2)}$	năng lượng tích phân	tiêu chuẩn ISO, mòn tiến triển
peak	$\max(x_i)$	biên độ đỉnh	xung, va đập
peak-to-peak	$\max(x_i) - \min(x_i)$	khoảng dao động	xung lớn, clearance
skewness	$\left(\frac{1}{N} \sum (x_i - mean)^3 \right) / std^3$	bất đối xứng	chạm một chiều (rubbing)
kurtosis	$\left(\frac{1}{N} \sum (x_i - mean)^4 \right) / std^4 - 3$	độ nhọn/đuôi dày	ổ trục hỏng, xung lặp lại
crest factor	$peak / RMS$	tỉ đỉnh/năng lượng	lỗi dạng impulse
shape factor	$RMS / mean_abs$	hình dạng dạng sóng	thay đổi dạng sóng
impulse factor	$peak / mean_abs$	tỉ đỉnh/trung bình tuyệt đối	lỗi impact

Ví dụ số minh họa kurtosis:

- Tín hiệu sin thuần: $x = [1, 0, -1, 0, \dots]$ → phân bố cong hình chữ M, kurtosis sau trừ 3 âm nhẹ (≈ -1,5).
- Tín hiệu nhiễu Gaussian lý tưởng → kurtosis = 0.
- Tín hiệu bình thường + 3 xung lực cực lớn → phân bố có đuôi dài → kurtosis 8–20.

Kinh nghiệm: theo dõi đồng thời RMS (năng lượng) + kurtosis (độ nhọn). Nếu cả hai cùng tăng → lỗi cơ khí tiến triển. Nếu RMS ổn định mà kurtosis tăng → xuất hiện xung rời rạc, nghi ngờ lỗi loại impact.

2.3. Giới thiệu biến đổi Fourier và feature miền tần số

Mục tiêu: thêm lớp feature miền tần số để bắt "năng lượng tập trung ở đâu" — thứ mà feature thời gian gần như mù — nhất là với lỗi ổ trục (BPFO/BPFI) và mài mòn lan tỏa.

Bài toán con: nhiều lỗi cơ khí mang "dấu vân tay tần số" rất riêng (đỉnh ở 1x, 2x, BPFO...). Feature thời gian chỉ thấy biên độ tăng; feature tần số thấy NHỮNG TẦN SỐ NÀO tăng → phân biệt được loại lỗi.

2.3.1. Vì sao miền tần số?

Tín hiệu rung thường là "chồng chập" của nhiều thành phần dao động ở các tần số khác nhau: - Tần số quay cơ bản f_1 (1x), hài bậc 2x f_1 , 3x f_1 ... - Tần số đặc trưng của ổ trục (BPFO, BPFI...) — giá trị phụ thuộc số viên bi, đường kính. - Nhiễu nền trắng.

Miền tần số cho ta nhìn thấy "năng lượng tập trung ở đâu" — giúp phân biệt các loại lỗi cực rõ. Ví dụ:

- **Mất cân bằng rotor (unbalance):** đỉnh lớn ở đúng 1x.
- **Lệch trục (misalignment):** đỉnh ở 1x, 2x thậm chí 3x.
- **Ổ trục hỏng:** các đỉnh nhỏ ở BPFO/BPFI cộng với "mặt nằng" nhiều tần số cao.

2.3.2. FFT trong một đoạn ngắn

FFT biến đổi N mẫu miền thời gian thành N (phức) hệ số tần số. Cách dùng gọn:

```
X[k] = FFT(x)[k]          # k = 0..N-1
Amp[k] = |X[k]|           # biên độ (magnitude) — dùng trong code src/features.py
Power[k] = |X[k]|^2        # công suất (power spectrum)
freq[k] = k * fs / N       # tần số tương ứng (Hz)
```

- $k = 0$ là tần số DC.
- $k = N/2$ tương ứng tần số Nyquist (xem 2.4).
- Chỉ nửa phổ đầu (từ $k=0$ đến $k=N/2$) chứa thông tin mới; nửa sau đối xứng với nửa đầu (với tín hiệu thực). Code team dùng `rfft` (chỉ lấy nửa phổ thực) — đúng tinh thần này.

Lưu ý nhanh: không dùng feature "toàn bộ N hệ số FFT" làm input mô hình — quá chiều, quá nhạy nhiễu, và không dịch bất biến theo pha. Thay vào đó ta nén phổ thành các **feature vô hướng** dưới đây.

2.3.3. Spectral centroid (tâm phổ)

Công thức: $\text{centroid} = \text{sum}(\text{freq}[k] * \text{Power}[k]) / \text{sum}(\text{Power}[k])$ với k chạy trên miền tần số quan tâm.

Ý nghĩa vật lý: "trọng tâm" của phổ — tần số trung bình có trọng số theo năng lượng. Centroid nhích lên cao khi tín hiệu chứa nhiều năng lượng ở tần số cao (nghi ngờ mài mòn, hỏng viên bi, phát ra tiếng rít tần số cao). Đây là feature "định hướng" nhanh: bình thường centroid thấp, khi lỗi mài mòn → nhích cao.

2.3.4. Spectral spread (độ trải phổ)

Công thức: $\text{spread} = \sqrt{\text{sum}(\text{Power}[k] * (\text{freq}[k] - \text{centroid})^2) / \text{sum}(\text{Power}[k])}$

Ý nghĩa vật lý: mức độ "loang rộng" của năng lượng quanh centroid — tương tự std nhưng trên miền tần số. Tín hiệu hẹp băng (nhiều hài rời rạc rõ) có spread nhỏ; tín hiệu nhiều broadband (mài mòn lan tỏa, kêu rè) có spread lớn. Kết hợp: centroid nói "ở đâu", spread nói "rộng hay hẹp".

2.3.5. Spectral flatness (độ phẳng phổ)

Công thức: $\text{flatness} = \exp((1/K) * \text{sum}(\ln(\text{Power}[k] + \text{epsilon}))) / ((1/K) * \text{sum}(\text{Power}[k]) + \text{epsilon})$

với `+epsilon` tránh $\log(0)$ khi $\text{Power} = 0$ (epsilon thường $1e-12$ trong code).

Ý nghĩa vật lý: tỉ số trung bình hình học / trung bình số học của phổ, nằm trong (0, 1]:

- flatness → 1: phổ "phẳng" như nhiều trắng (không có đỉnh nổi bật) → âm thanh ồn rè, mang tính ngẫu nhiên.
- flatness → 0: phổ "nhọn" với vài đỉnh sắc (tonal, harmonic) → máy chạy gọn rõ ràng.

Khi lỗi kiểu ồn/mài mòn lan rộng, flatness **tăng**. Khi xuất hiện các hài mới từ lỗi, có thể giảm. Đây là feature hữu ích phân biệt lỗi rời rạc vs nhiễu lan tỏa.

2.3.6. Band energy (năng lượng theo dải tần)

Chia phổ thành các dải tần và tính năng lượng từng dải:

$E[\text{band } b] = \sum(\text{Power}[k])$ với k thuộc dải b

Ví dụ dải (Hz): [0, 50), [50, 100), [100, 200), [200, 500), [500, 1000), [1000, 2000), [2000, 5000), [5000, 10000].

Ý nghĩa vật lý và cách dùng: - Tần số quay $f_1 < 100$ Hz thường; năng lượng khổng lồ ở dải thấp = cân bằng/lệch. - Năng lượng ở vùng BPFO-BPFI (thường hàng trăm Hz - vài kHz) tăng = ổ trục hỏng. - Năng lượng ở cực cao (mid-high kHz) tăng = mài mòn, khô dầu, chậm. Đây có lẽ là nhóm feature dễ "ăn điểm" nhất vì tương đối trực quan với thợ máy — có thể tỉ lệ hóa: $E_{\text{rel}}[b] = E[\text{band } b] / \text{total_energy}$.

So sánh trong code hiện tại: `src/features.py` chọn kiểu "tổng hợp gọn" hơn thay vì 8 dải tần — nó lấy `spec_energy` (tổng năng lượng phổ) + `spec_centroid` + `spec_spread` + `spec_flatness` + `freq_mean` + `freq_std`. Nếu bạn mở rộng, thêm band energy 8 dải (theo bảng trên) là hướng rõ ràng nhất để tăng điểm.

2.3.7. Window function (hàm cửa sổ) — khái niệm bắt buộc

Mục tiêu: trước FFT, nhân tín hiệu với hàm cửa sổ để giảm "rò rỉ phổ" — nếu bỏ qua, đỉnh phổ bị loang sang tần số kế cận và feature miền tần số bị nhiễu.

Trước FFT nên nhân tín hiệu với **window function**:

- `hann[k] = 0.5 * (1 - cos(2*pi*k/(N-1)))`
- `hamming[k] = 0.54 - 0.46*cos(2*pi*k/(N-1))`
- Rectangular (không nhân gì) — đơn giản nhất nhưng gây rò rỉ phổ (spectral leakage): một đỉnh sắc sẽ "loang" sang tần số kế cận.

Vì sao? Vì chuỗi ngắn N mẫu cắt đột ngột tạo ra "sườn" gián đoạn ở hai đầu → xuất hiện các tần số giả. Window làm mềm hai đầu → giảm leakage, đỉnh phổ sắc hơn. **Tổng năng lượng sẽ đổi** do window — nếu dùng window, hãy dùng đồng nhất cho mọi window và mọi file để so sánh công bằng. (Code team dùng `np.hanning(win)` nhất quán cho mọi window — đúng.)

2.4. Tần số lấy mẫu, Nyquist và aliasing (nền tảng — đọc kỹ)

Mục tiêu: biết giới hạn tin cậy của phổ ($0 \rightarrow f_s/2$) và tránh cái bẫy "đỉnh ảo" do aliasing — một sai lầm nhỏ ở đây làm mọi feature tần số vô nghĩa.

Bài toán con: kết quả FFT chỉ đúng trong một dải tần giới hạn. Làm feature miền tần số mà không biết dải này thì có thể "phát hiện lỗi" ở tần số không tồn tại.

2.4.1. Tần số Nyquist

Định lý lấy mẫu (Shannon-Nyquist): để tái dựng chính xác một tín hiệu liên tục, tần số lấy mẫu phải **lớn hơn hai lần** tần số cao nhất chứa trong tín hiệu.

Công thức: $f_{\text{Nyquist}} = f_s / 2$

Ví dụ: $f_s = 20 \text{ kHz} \rightarrow f_{\text{Nyquist}} = 10 \text{ kHz}$. Nghĩa là: ta chỉ tin tưởng FFT ở vùng 0–10 kHz. Mọi thành phần năng lượng được FFT "báo" ở dải trên 10 kHz là ẢO (do aliasing, xem 2.4.2), trừ khi phần cứng đã có anti-aliasing filter.

2.4.2. Aliasing (biệt danh tần số)

Khi tín hiệu thực chứa tần số $> f_{\text{Nyquist}}$ nhưng vẫn bị lấy mẫu với f_s , các tần số cao này sẽ "ngụy trang" thành tần số thấp hơn trong dải hợp lệ:

$f_{\text{gia}} = |f_{\text{thuc}} - n * f_s|$ với n là số nguyên làm kết quả nằm trong $[0, f_s/2]$.

Ví dụ số: $f_s = 1000$ Hz $\rightarrow$ Nyquist 500 Hz. Một thành phần thực ở **700 Hz** sẽ được FFT hiển thị ở: $|700 - 1000| = 300$ Hz. Nhà phân tích tưởng có dao động 300 Hz — sai bản chất, không thể phát hiện lỗi vì đỉnh "bị dời chỗ".

Hệ quả thực hành: - Luôn đọc metadata để biết f_s chính xác của từng file. - Khi tính FFT: `freq_max_hop_tin =  $f_s/2$` . Feature band energy chỉ tính trong $[0, f_s/2]$. - Nếu AI trước xử lý giảm lấy mẫu (downsample) $2\times$, nhớ lọc thông thấp trước khi downsample (vì sau khi giảm f_s , tần số cao cũ sẽ gây aliasing). - Với chuỗi khác f_s giữa các file: đưa feature band energy theo **tỷ lệ f/f_1** (hoặc chuẩn hóa theo harmonic) để so sánh được.

2.4.3. Độ phân giải tần số

Với window N mẫu @ f_s , bước tần số giữa hai bin liên tiếp: `Delta_f =  $f_s / N$`

Ví dụ: `$f_s=20\_000$` , `$N=2\_000$` $\rightarrow$ `Delta_f = 10` Hz. Hai đỉnh cách nhau dưới 10 Hz sẽ bị gộp làm một. Muốn phân giải tốt hơn: tăng N (lấy window dài hơn).

Quan hệ bất định trong thực hành: window dài $\rightarrow$ phân giải tần số tốt, nhưng nhòe theo thời gian; window ngắn $\rightarrow$ bắt nhanh sự kiện nhưng phổ thô. Đây là lý do overlap được dùng: giữ window đủ dài cho FFT mượt mà vẫn có nhiều decision point theo thời gian.

2.5. Chuẩn hóa (normalization / scaling)

Mục tiêu: đưa các feature khác đơn vị về cùng thang đo để mô hình (đặc biệt là mô hình gradient/khoảng cách) không bị chiều có biên độ lớn "áp đảo".

Bài toán con: các feature có "đơn vị" rất khác nhau (RMS $\sim 1e-2$ g, kurtosis ~ 10 , band energy $\sim 1e4$). Mô hình như Logistic Regression (gradient) hoặc khoảng cách (kNN, SVM) **sẽ bị méo** nếu không chuẩn hóa — chiều có biên độ lớn "áp đảo" chiều nhỏ.

So sánh ba cách chuẩn hóa — và vì sao chọn RobustScaler cho rung:

Cách	Phép	Chống outlier	Khuyến nghị A1
Z-score	<code>(x-mu)/sigma</code>	Trung bình	nếu dùng chuẩn hóa trước RF/XGB ok
Min-max	<code>(x-min)/(max-min)</code>	Kém	ít dùng với rung
RobustScaler	<code>(x-median)/IQR</code>	Tốt	★ mạnh nhất cho rung

Kết luận cho A1: tín hiệu rung có đặc điểm về bản chất là chứa **spike/outlier** (xung va đập, cú sốc). Min-max bị một outlier kéo toàn dải $\rightarrow$ mọi giá trị bình thường dồn cục vào một chỗ $\rightarrow$ mất hết sức phân biệt. RobustScaler dùng median + IQR nên "không bị kéo" — giữ được hình dạng cụm lẫn làm lộ outlier.

Lưu ý quan trọng theo hiện trạng team: pipeline dùng **z-score** (`fit_zscore` + `zscore` trong `src/features.py`) thay vì RobustScaler — vì z-score đủ tốt sau khi đã có feature 16 chiều và chuẩn hóa bằng mean/std fit TRÊN TRAIN. Nếu bạn muốn nâng cấp, thay bằng RobustScaler là điều chỉnh nhỏ và hoàn toàn hợp lý cho rung.

2.5.1. Z-score (standardization)

Công thức: `$z = (x - \mu) / \sigma$`

- `mu`, `sigma` khai báo train; áp dụng cho test với CÙNG `mu`, `sigma` đó (không tính lại trên test — nếu không sẽ giới thiệu leakage thông tin test vào quy trình).
- Kết quả: trung bình 0, std 1.
- Khi dùng:** mô hình giả định phân bố Gauss (Logistic, SVM); khi feature có thể có độ lệch ít nghiêm trọng; khi không rõ nên dùng gì $\rightarrow$ chọn mặc định.

2.5.2. Min-max scaling

Công thức: $x_{scaled} = (x - x_{min}) / (x_{max} - x_{min}) \rightarrow$ nằm trong $[0, 1]$.

- Nhạy cảm với outlier: một outlier kéo toàn dải $\rightarrow$ mọi giá trị bình thường dồn cục vào 0,1–0,3. Kém nếu dữ liệu có giá trị cực (thường gặp ở rung — có spike).
- **Khi dùng:** dữ liệu đã biết có biên độ ràng buộc (nhiệt độ 20–80°C); khi bắt buộc feature nằm trong $[0, 1]$ (neural có activation bão hòa như sigmoid).

2.5.3. RobustScaler

Công thức: $x_{scaled} = (x - median) / IQR$ với $IQR = Q3 - Q1$ (tứ phân vị), hoặc $(x - median) / 1.349 * MAD$.

- Trung vị & IQR **chống nhiễu** — không bị kéo bởi vài outlier vài nghìn lần.
- **Khi dùng:** dữ liệu thống kê có nhiều outlier/xung — **điển hình của tín hiệu rung**. Rất khuyên dùng cho A1.

2.5.4. Ví dụ số minh họa

Dãy RMS của 5 window: $[0.5, 0.6, 0.55, 8.0, 0.7]$ (có outlier 8.0 do một cú sốc thật hoặc nhiễu).

- min-max: $min=0.5, max=8.0 \rightarrow 0.55 \rightarrow (0.55-0.5)/7.5 = 0.0067 \rightarrow$ bị "ép" gần 0, các giá trị bình thường không phân biệt được.
- z-score: $\mu \approx 2.07, \sigma \approx 3.24 \rightarrow 0.55 \rightarrow (0.55-2.07)/3.24 \approx -0.47$.
- RobustScaler: $median = 0.6, Q1=0.55, Q3=0.7, IQR=0.15 \rightarrow 0.55 \rightarrow (0.55-0.6)/0.15 \approx -0.33$; chiều 8.0 $\rightarrow (8-0.6)/0.15 \approx 49$ rất cao $\rightarrow$ dễ phát hiện ngay.

RobustScaler giữ được cả "hình dạng cụm" lẫn làm lộ outlier.

2.6. Gợi ý danh sách feature tổng hợp cho A1 (mức mở rộng)

Bảng 16 feature mà `src/features.py` đang tính cho mỗi window (RAW, chưa chuẩn hoá):

```
-- Miền thời gian (10)
mean, std, rms, peak, peak2peak, skewness, kurtosis,
crest_factor, shape_factor, impulse_factor

-- Miền tần số (6) – FFT (rfft) với window function hann
spec_centroid, spec_spread, spec_flatness, spec_energy,
freq_mean, freq_std
```

Nếu mở rộng để ăn thêm điểm, bổ sung nhóm gợi ý sau:

```
-- Miền thời gian mở rộng
zero_crossing_rate (số lần đổi dấu – đo "tần suất" nhanh)

-- Miền tần số mở rộng (trước FFT với window function hann)
spectral_rolloff,
band_energy[8 dải] (chuẩn hóa tổng = 1),
harmonic_ratio = E( tại f1, 2f1, 3f1 ) / total_energy

-- Chỉ số thống kê phụ
diff_features: std của sai phân bậc nhất (đo tốc độ thay đổi);
range_ratio = peak_to_peak / rms
```

Mẹo mạnh: **tính feature theo cả chuỗi raw lẫn chuỗi đã lọc bandpass** (dải quanh tần số quay) gặp được khả năng phân tách lỗi. Chọn feature sẵn tốt hơn là dùng toàn bộ và để model tự gán.

3. Học có giám sát cổ điển

Mục tiêu của cả mục: trả lời nhánh (2) của cây bài toán con — khi ta ĐÃ CÓ nhãn lỗi (một phần), dựng mô hình phân loại 0/1 đạt điểm cao nhất. Giả định: ta đã có **nhãn** (0/1) cho từng window — hoặc từ dữ liệu thi (kèm ít dữ liệu lỗi), hoặc từ dữ liệu bổ sung sau khi khai thác.

Bài toán con: bài toán phân loại nhị phân trên dữ liệu bảng feature đã nén. Bối cảnh đặc thù A1: ít mẫu lỗi, dữ liệu nhiễu, cần diễn giải được. Vì vậy thứ tự đi là: mô hình đơn giản và diễn giải được (LogReg) $\rightarrow$ mô hình mạnh dạng cây (RF) $\rightarrow$ mô hình thường

thắng giải (XGBoost).

3.1. Logistic Regression (hồi quy logistic)

Mục tiêu: có một baseline NHANH, diễn giải được, để đối chiếu mọi model khác — đồng thời "đọc" được feature nào ảnh hưởng lỗi mạnh qua trọng số w .

3.1.1. Ý tưởng

Dù tên là "regression", đây là thuật toán **phân loại nhị phân**. Ý tưởng: dự đoán **xác suất** thuộc lớp 1 từ tổ hợp tuyến tính của các feature, rồi "bóp" vào khoảng $(0,1)$ bằng hàm sigmoid.

3.1.2. Công thức

Linear score: $z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_p \cdot x_p$

Sigmoid: $p = 1 / (1 + \exp(-z))$

Hàm sigmoid có tính chất: - $z = 0 \rightarrow p = 0.5$ - $z \rightarrow +\infty \rightarrow p \rightarrow 1$ - $z \rightarrow -\infty \rightarrow p \rightarrow 0$

Quyết định (decision boundary): $\text{prediction} = 1$ nếu $p \geq \text{threshold}$, ngược lại 0 , threshold mặc định = 0.5.

Decision boundary trong không gian feature: nghiệm của $z = 0$, tức $w_0 + w_1 \cdot x_1 + \dots = 0$ — là một **siêu phẳng** trong không gian feature. Vì là siêu phẳng tuyến tính, Logistic **không bắt được quan hệ phi tuyến** tốt — nhưng với thư viện scikit-learn và data chuẩn hóa tốt, nó vẫn là mô hình "benchmark nhanh + diễn giải được" — bạn có thể đọc trọng số w để biết feature nào ảnh hưởng mạnh (hệ số dương $\rightarrow$ tăng xác suất lỗi).

3.1.3. Loss function (cross-entropy)

Với N mẫu, nhãn $y_i \in \{0,1\}$:

$\text{Loss} = -(1/N) \cdot \sum (y_i \cdot \ln(p_i) + (1 - y_i) \cdot \ln(1 - p_i))$

Giải thích trực quan: - Nếu $y_i=1$ nhưng mô hình đoán p_i nhỏ $\rightarrow \ln(p_i)$ rất âm $\rightarrow$ loss lớn $\rightarrow$ bị phạt nặng. - Nếu $y_i=0$ nhưng mô hình đoán p_i cao $\rightarrow \ln(1-p_i)$ rất âm $\rightarrow$ loss lớn. Nhờ đó mô hình học để đưa p gần nhãn thật.

Logistic tối ưu hóa bằng gradient descent (thường là phiên bản L-BFGS trong library). Nên thêm **regularization** (xem 3.4).

3.1.4. Ví dụ số cực ngắn

Có 1 feature: RMS. Học được $z = -3.5 + 40 \cdot \text{RMS}$ (sau chuẩn hóa). Với window có RMS=0.08 (chuẩn hóa $\rightarrow \sim 0$): $z = -3.5 + 40 \cdot 0 \approx -3.5 \rightarrow p = 1/(1+e^{3.5}) \approx 0.03 \rightarrow$ dự đoán 0 (bình thường). Với window có RMS lớn (chuẩn hóa ≈ 0.15): $z = -3.5 + 40 \cdot 0.15 = 2.5 \rightarrow p = 1/(1+e^{-2.5}) \approx 0.92 \rightarrow$ dự đoán 1 (bất thường).

3.2. Decision Tree $\rightarrow$ Random Forest

Mục tiêu: đi từ một cây đơn (dễ hiểu nhưng dễ overfit) đến Random Forest — model dạng cây đầu tiên thực sự mạnh với dữ liệu tabular ít mẫu, kèm "validation miễn phí" bằng OOB.

3.2.1. Decision Tree: ý tưởng

Cây quyết định chia không gian feature theo các **luật if-else**: "Nếu $\text{kurtosis} > 6$ và $\text{RMS} > 0.05 \rightarrow$ lỗi". Mỗi nút chọn một feature + ngưỡng phân tách tốt nhất sao cho sau khi tách, các nhánh con **thuần nhất hơn** (ít lẫn lớp).

3.2.2. Độ đo chia (split criterion)

Gini impurity của một tập hợp: $\text{Gini} = 1 - \sum_c (p_c)^2$, với p_c là tỉ lệ lớp c . - Tập thuần 1 lớp $\rightarrow$ Gini = 0. - Hai lớp đều 50/50 $\rightarrow$ Gini = 0.5.

Thuật toán thử mọi (feature, ngưỡng), chọn phân tách làm **giảm Gini nhiều nhất**: $gain = Gini(parent) - (n_l/n) * Gini(left) - (n_r/n) * Gini(right)$

Một tiêu chí khác: entropy $H = -\sum_c p_c \ln(p_c)$, gain theo information gain.

3.2.3. Rủi ro của một cây đơn: overfitting

Cây đơn sâu đến mức mỗi lá thuần 100% thường **quá khớp**: nhớ nhiều của train. Kết quả: trên test có thể tệ hơn Logistic. Vì vậy người ta dùng rừng cây — **ensemble**.

3.2.4. Random Forest: Bagging + random feature

- **Bagging** (Bootstrap aggregating): 1. Từ N mẫu train, lấy mẫu **có hoàn lại** (bootstrap) N mẫu → tập con cho mỗi cây (mỗi cây nhìn một phiên bản khác của train, vài cây thiếu mẫu này, vài cây thiếu mẫu kia). 2. Huấn luyện M cây; mỗi cây khi tách chỉ được xét **ngẫu nhiên subset các feature** (thường $\sqrt{p}$). 3. Dự đoán = **đa số phiếu** (bình chọn) giữa các cây.

Vì sao hiệu quả?: - Mỗi cây thiên về (variance cao) nhưng thiên lệch theo các hướng nhiều khác nhau → khi lấy trung bình, **sai số phương sai giảm** mà bias gần như không đổi. - Random feature làm cây giảm tương quan nhau hơn (không cùng nhiều), tăng lợi ích của ensemble.

3.2.5. Out-of-bag (OOB) score

Vì bootstrap lấy mẫu có hoàn lại, với N mẫu, xác suất một mẫu không được chọn trong một cây $\approx (1-1/N)^N \approx e^{-1} \approx 0.368$. Các mẫu "bỏ lỡ" (out-of-bag) của một cây KHÔNG tham gia train cây đó → có thể dùng để đánh giá ngay trong lúc train mà **không cần tách thêm validation set**:

- Với mỗi mẫu, dự đoán bằng chỉ các cây mà mẫu đó thuộc OOB.
- Gộp tất cả → OOB score $\approx$ ước lượng generalization không chệch.

OOB rất tiện trong bài toán thiếu dữ liệu: tận dụng toàn bộ dữ liệu train mà vẫn có hồi kiểm (cross-validation style) miễn phí. Lưu ý: vẫn phải tách **test theo thời gian** riêng để đánh giá cuối cùng (tránh leak về chuỗi thời gian — xem 5.6).

3.3. Gradient Boosting / XGBoost

Mục tiêu: model mạnh nhất họ cây cho dữ liệu tabular đã feature-engineer — có regularization chống overfit, bất biến scale, và là điểm khởi đầu thắng giải cho A1.

3.3.1. Ý tưởng cốt lõi: "sửa sai từng bước"

Khác với Random Forest (xây M cây song song rồi lấy trung bình), **boosting** xây cây **tuần tự**: cây tiếp theo được huấn luyện để **bù đắp sai lệch mà các cây trước để lại**.

Đặc biệt gradient boosting:

Bắt đầu: mô hình khởi tạo $F_0(x) = \text{mean}(y)$ (hồi quy) hoặc $\log(\text{odds})$ (phân loại)
Vòng lặp $m = 1..M$:
1. Tính "phần dư" (residual) $r_i = y_i - F_{\{m-1\}}(x_i)$ (hồi quy)
Hoặc: tính gradient của loss theo dự đoán hiện tại (tổng quát)
2. Fit một cây nhỏ h_m để dự đoán r_i từ x_i
3. Cập nhật: $F_m(x) = F_{\{m-1\}}(x) + \text{lr} * h_m(x)$
Kết quả: $F_M(x) = F_0(x) + \text{lr}*h_1(x) + \text{lr}*h_2(x) + \dots + \text{lr}*h_M(x)$

Trực giác: cây 1 bắt xu hướng lớn; sai chỗ nào, cây 2 nhắm đúng chỗ còn sai; tiếp tục... Mỗi cây "thợ sửa" bé (thường nông, $\text{max_depth} = 3-6$), nhiều cây cộng lại thành mô hình mạnh và đúng hơn. **learning rate (lr)** kiểm soát mức độ mỗi cây được hiệu chỉnh — lr bé + nhiều cây thường hội tụ tốt hơn lr lớn.

3.3.2. Loss function đối với phân loại

Phân loại (mục tiêu xác suất) cho A1 thường dùng **binary logistic loss**:

$Loss = -y*\ln(p) - (1-y)*\ln(1-p)$ (với N mẫu thì trung bình loss trên N).

XGBoost còn tối ưu bằng **second-order** thông tin: dùng gradient $g = dL/dp$ và Hessian $h = d^2L/dp^2$. Ý tưởng: không chỉ biết "lỗi bao nhiêu" (gradient) mà còn biết "lỗi" biến thiên thế nào (Hessian) → xác định lá cây và giá trị mỗi lá chính xác hơn so với naive gradient. Đây là một phần vì sao XGBoost nhanh+chắc chắn so với boosting thể hệ đầu (GBM).

3.3.3. Regularization L1 / L2

Công thức loss XGBoost tổng: $L_{total} = \sum_i L(y_i, p_i) + \lambda \sum_l w_l^2 + \alpha \sum_l |w_l|$

Hai số hạng phạt trên **trọng số lá** w_l : - **L2** ($\lambda \sum w_l^2$): phạt giá trị lá lớn → lá ôn hòa, giảm overfit (giống ridge). - **L1** ($\alpha \sum |w_l|$): phạt phi tuyến tính, có thể **ép số lá về 0** → kiểu sparsity.

Cùng với `max_depth`, `min_child_weight`, `subsample`, `colsample` — những tham số này giúp chống overfit mạnh mẽ, cực kỳ quan trọng vì dữ liệu A1 ít và mất cân bằng.

3.3.4. Vì sao XGBoost mạnh với dữ liệu tabular / feature-engineered?

- Xử lý phi tuyến + interaction**: tự động tìm ra luật phức hợp giữa feature (kurtosis cao AND RMS tăng) mà không cần ta thủ công khai báo.
- Bất biến với scale**: cây chỉ so sánh ngưỡng nên không cần chuẩn hóa feature (có thể dùng trực tiếp) — tiết kiệm bước tiền xử lý.
- Robust với outlier và dữ liệu thiếu**: ngưỡng chia dựa trên thứ hạng giá trị, không bị vài outlier kéo; tự học hướng xử lý missing theo phân phối.
- Nhạy tốt số mẫu**: với đúng số lượng thì, ensemble tree học được cấu trúc nhưng ít chịu overfit nếu regularization tốt.
- Đo importance**: `gain` và `frequency` của từng feature → diễn giải được, giúp rút gọn chọn feature.

Kinh nghiệm: XGBoost/LightGBM thường là **điểm khởi đầu thắng giải** cho bài toán tabular lỗi hiếm ở A1.

3.4. So sánh Random Forest vs XGBoost — khi dùng cái nào?

Mục tiêu: biết khi nào chọn RF, khi nào chọn XGB — cả hai đều là ứng viên cuối cùng của A1, nhưng tính chất khác nhau.

Tiêu chí	Random Forest	XGBoost / Gradient Boosting
Cách xây cây	Song song, lấy trung bình	Tuần tự, sửa lỗi
Bias/variance	Giảm variance	Giảm bias tuần tự
Khả năng fit dữ liệu nhiễu	Tốt hơn, ít overfit tự nhiên	Cần regularization cẩn thận
Tốc độ train	Nhanh (song song được dễ)	Chậm hơn khi lặp tuần tự (LightGBM nhanh hơn)
Feature importance	Có (impurity / permutation)	Có (gain, cover, frequency)
Hiệu chỉnh ngưỡng chính xác trên dữ liệu hiếm	Dễ bí (ít mẫu lỗi)	Hiệu quả hơn với nhiều cây nhỏ
Khi nào dùng	Dữ liệu nhiễu nhiều, cần nhanh, cần OOB validation	Dữ liệu tabular feature-engineered, cần maximize chính xác

Kết luận cho A1 — vì sao chọn cái này hơn cái kia: - Nếu dữ liệu lỗi thực sự chỉ vài chục mẫu: **RF an toàn hơn** — ít tham số, ít overfit mạnh, có OOB sạch. - Nếu có cỡ vài trăm mẫu lỗi trở lên: **XGBoost gần như luôn đạt top** với GridSearch nhẹ. - Quy trình đề xuất (đúng tinh thần "đi từ đơn giản"): 1. Bắt đầu bằng **Logistic** (chạy nhanh, đánh giá feature, baseline metrics). 2. Rồi **Random Forest** với OOB để có ước lượng sạch. 3. Rồi **XGBoost** (với GridSearch nhẹ trên `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `scale_pos_weight`) — gần như luôn đạt top cho A1. - Test cả hai và chọn theo **threshold-independent metric** (PR-AUC) trên test theo thời gian.

Đúng với hiện trạng: `scripts/02_compare_augmentation.py` dùng **RandomForest** làm model chấm trước/sau augmentation — hợp lý vì đây là mốc nhanh, ổn định với ít lỗi thật; nâng lên XGBoost là bước tối ưu tiếp theo.

3.4.1. Xử lý imbalance trong XGBoost

`scale_pos_weight = n_negative / n_positive`. Ví dụ 100.000 normal, 500 lỗi → `scale ≈ 200`. Điều này nhân trọng số loss cho lớp thiểu số — hướng mô hình học kỹ lớp lỗi.

4. Metrics cho bài toán mất cân bằng (trọng tâm)

Mục tiêu của cả mục: trả lời nhánh (4) — "thế nào là tốt" khi lớp hiếm. Định nghĩa sai "tốt" = chọn sai model = thất bại dù model giỏi. Metrics ở đây ảnh hưởng TRỰC TIẾP tới cách bạn báo cáo cho giám khảo.

⚠ Trong A1, chọn sai metric = chọn sai mô hình. Accuracy là "cạm bẫy" lớn nhất. Mục này phải nằm gốc.

4.1. Accuracy — vì sao gây hiểu lầm khi class lệch?

Mục tiêu: biết vì sao accuracy KHÔNG dùng được khi lỗi hiếm.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Với dữ liệu 99% bình thường, một mô hình "ngu" luôn đoán 0 sẽ đạt **99% accuracy**.

Ví dụ số: dữ liệu có 10.000 normal, 100 lỗi. Model luôn dự đoán 0: - TP=0, TN=10.000, FP=0, FN=100. - Accuracy = $10000/10100 = 99,0\%$ — nghe rất hay nhưng chẳng bắt được **một** lỗi nào! Trong bảo trì dự đoán, đó là thảm họa: máy vỡ mà không báo.

Vì vậy phải dùng metrics tập trung vào lớp quan tâm: lỗi (positive).

4.2. Confusion matrix (ma trận nhầm lẫn)

Mục tiêu: có bức tranh bốn ô đầy đủ — nền tảng của mọi metric chống mất cân bằng.

Ta quy ước "positive = lớp bất thường/lỗi".

	Dự đoán 0	Dự đoán 1
Thực tế 0	TN (True Neg)	FP (False Pos - báo động giả)
Thực tế 1	FN (False Neg - bỏ sót lỗi)	TP (True Pos - bắt đúng lỗi)

Bốn ô này nền tảng cho mọi metric: - **TP**: lỗi thật được báo → tốt. - **FP**: báo lỗi khi thật bình thường → **báo động giả** → chi phí dừng máy vô ích. - **FN**: lỗi thật bị bỏ sót → **nguy hiểm nhất** (máy tiếp tục chạy rồi vỡ). - **TN**: bình thường và được xác nhận bình thường.

4.3. Precision, Recall, F1

Mục tiêu: đo đúng hai mặt của chất lượng dự đoán lớp hiếm: "bắt được bao nhiêu lỗi thật" (recall) và "báo bao nhiêu % là đúng" (precision), rồi gộp bằng F1.

Công thức: `Precision = TP / (TP + FP)` `Recall = TP / (TP + FN)` `F1 = 2 * Precision * Recall / (Precision + Recall)` (harmonic mean)

Ý nghĩa thực tế: - **Precision**: "Trong những gì mô hình BÁO là lỗi, bao nhiêu % là thật lỗi?" → đo mức báo động giả. Precision cao = ít làm phiền nhà máy. - **Recall**: "Trong tất cả lỗi THẬT, mô hình bắt được bao nhiêu %?" → đo độ an toàn. Recall thấp = sót lỗi = nguy hiểm. - **F1**: cân bằng hài hòa hai mục tiêu trên khi nhãn mất cân bằng.

4.3.1. Ví dụ số quyết định

Giả thiết: 10.000 normal, 100 lỗi. Mô hình A dự đoán: - TP=80, FN=20 → bỏ sót 20 lỗi. - FP=500, TN=9500 → 500 báo động giả.

Tính: - Precision = $80/(80+500) = 80/580 = 13,8\%$ - Recall = $80/(80+20) = 80/100 = 80\%$ - Accuracy = $(80+9500)/10100 = 9580/10100 = 94,9\%$ - F1 = $2(0.138 \cdot 0.8)/(0.138+0.8) = 0.21044/0.938 = 0.22088/0.938 \approx 23,5\%$

Nhận xét: accuracy cao (94,9%) nhưng precision tệ — mô hình báo ò ạt, gây 500 báo động giả. Trong nhà máy, 500 lần dừng máy vô lý là không thể chấp nhận.

Kiểm tra song song: Model B dự đoán TP=70, FN=30, FP=100, TN=9900: - Precision = $70/170 = 41,2\%$; Recall = $70/100 = 70\%$; $F1 = 2(0.412 \cdot 0.7)/(0.412+0.7) \approx 51,9\%$.

Model B recall thấp hơn chút nhưng precision tốt hơn nhiều → F1 cao hơn gấp đôi. **Bài học: chọn mô hình theo F1/PR, không theo accuracy.**

4.4. PR curve vs ROC/AUC — vì sao PR quan trọng cho lớp hiếm

Mục tiêu: biết PR-AUC/AP là thước đo chính của A1, ROC/AUC chỉ là tham khảo phụ — vì ROC bị "che mắt" bởi số normal khổng lồ.

Bài toán con: một metric duy nhất mà không cần chọn threshold trước (threshold-independent), để so sánh model công bằng.

4.4.1. Tư tưởng: model cho xác suất → di chuyển threshold

Hầu hết model (Logistic, RF, XGB) output ra **xác suất** $p \in [0, 1]$, rồi ta so với threshold t (mặc định 0.5) để quyết định 0/1. **Thay đổi t → thay đổi cặp (Precision, Recall).** Ví dụ:

- t thấp (0,1): gần như mọi mẫu bị báo lỗi → Recall $\approx 100\%$ nhưng Precision rất thấp.
- t cao (0,9): chỉ báo khi thực sự chắc chắn → Precision cao, Recall thấp.

4.4.2. PR curve

Vẽ **Precision (trục đứng) vs Recall (trục ngang)** khi quét t từ 1 → 0:

- Điểm (0, 1) khi t cao → Precision tối đa, Recall 0.
- Điểm (Recall=1, Precision thấp) khi t thấp.

PR-AUC = diện tích dưới đường PR. PR-AUC cao = model giữ Precision cao khi recall tăng — đúng thứ ta cần khi lớp hiếm.

4.4.3. ROC/AUC — cảnh báo

ROC vẽ **TPR (Recall) vs FPR ($FP / (FP+TN)$)** — FPR có mẫu số gồm toàn normal (rất nhiều) nên **dễ lạc quan** với lớp hiếm:

Ví dụ: 10.000 normal, 100 lỗi. Model báo 500 lỗi (FP=500): - FPR = $500/10500 = 4,76\%$ (trông bé nhỏ). - Nhưng 500 báo động giả là 500 lần dừng máy — chi phí thực tế KHỔNG LỒ. FPR nhìn "bé" vì mẫu số gồm toàn normal (nhiều) — hay cheat số đo. **ROC/AUC bị ảnh hưởng bởi TN**, còn PR chỉ quan tâm TP/FP/FN → nhạy hơn với lớp thiểu số.

So sánh nhanh:

Tiêu chí	PR-AUC / AP	ROC-AUC
Nhạy với lớp hiếm	Cao — nhìn thẳng vào TP/FP/FN	Thấp — TN khổng lồ "nuốt" FPR
Diễn giải ổn định khi class lệch	Tốt	Dễ lạc quan giả tạo
Vai trò ở A1	★ metric chính	Tham khảo phụ

Kết luận cho A1 (class hiếm): ưu tiên **PR-AUC, Average Precision (AP)** và báo cáo Precision/Recall tại một vài threshold cụ thể. ROC/AUC chỉ mang tính tham khảo phụ. (Các script team vẫn in AUC như một con số "phân biệt tổng quát" — đọc kèm với recall/precision, đừng xem nó là số quyết định.)

4.5. Threshold tuning & precision-recall tradeoff (chọn theo business)

Mục tiêu: chọn threshold không phải "mặc định 0.5" mà theo chi phí thật của nhà máy — đây là câu chuyện khiến giám khảo tin bạn hiểu vấn đề.

4.5.1. Precision-recall tradeoff

Bạn không thể muốn cả hai cùng cao với một model cố định: giữa chúng có **đánh đổi (tradeoff)**. Khi tăng threshold → precision tăng, recall giảm (và ngược lại). Chọn điểm nào phụ thuộc **chi phí**:

4.5.2. Chi phí bỏ sót lỗi vs báo động giả — phân tích business

Trong nhà máy, hãy ước lượng: - **C_{cost}**: chi phí BỎ SÓT một lỗi thật (máy hỏng, dừng sản xuất, sửa chữa lớn, mất an toàn) — thường rất lớn. - **C_{fp}**: chi phí mỗi lần báo động giả (dừng máy kiểm tra vô ích, nhân công kiểm tra lại, mất nhịp sản xuất) — nhỏ hơn nhưng không bằng 0. - **C_{tp}**: lợi ích bắt được một lỗi thật (sửa chủ động gọn nhẹ, tránh hỏng lớn).

Quy tắc chọn threshold: chọn t làm cho **tổng chi phí kỳ vọng nhỏ nhất**: $Cost = FP * C_{fp} + FN * C_{fault}$

Ví dụ số: mỗi tuần máy có ~2 lỗi thật cần bắt. Nếu $C_{fault} = 50 \text{ triệu VND}$, $C_{fp} = 2 \text{ triệu VND}$: - Model at $t=0.5$: $FN=0$, $FP=8/\text{tuần}$ → $Cost = 0.50tr + 82tr = 16tr$. - Model at $t=0.8$: $FN=0.3$, $FP=1/\text{tuần}$ (mỗi ~3 tuần sót 1 lỗi) → $Cost = 0.350tr + 12tr = 15tr + 2tr = 17tr$. - Model at $t=0.65$: $FN=0.1$, $FP=3$ → $Cost = 0.150tr + 32tr = 5tr + 6tr = 11tr$ ← tối ưu.

Thay vì "cố định threshold mặc định", đọc đường PR, tìm điểm lướt trên-trái (precision giữ cao khi recall vẫn chấp nhận được), rồi chốt vài threshold liên tiếp cho 3 mức cảnh báo (warning / alarm / critical).

4.5.3. Mẹo thực hành threshold

- XGBoost output `predict_proba`; không dùng `predict` mặc định khi class lệch.
- Tìm threshold tối ưu trên **validation theo thời gian** (không tinh trên test — sẽ chủ quan).
- Với A1: chọn theo `max F1`, theo `min cost`, hoặc theo ràng buộc $recall \geq 90\%$ → precision tối đa được trong điều kiện đó.

4.6. MAE / RMSE cho RUL (nếu làm dự báo tuổi thọ)

Mục tiêu: biết bộ metric RIÊNG khi chuyển từ phân loại 0/1 sang hồi quy RUL — Precision/Recall không còn đúng nữa.

Khi chuyển sang bài toán **RUL (Remaining Useful Life)** — dự đoán số giờ (hoặc số chu kỳ) còn lại trước khi hỏng — đây là **hồi quy**, không còn dùng Precision/Recall nữa.

$$MAE = (1/N) * \sum(|y_i - \hat{y}_i|) \quad RMSE = \sqrt{(1/N) * \sum((y_i - \hat{y}_i)^2)}$$

So sánh: - **MAE**: trung bình độ lệch tuyệt đối — trực quan, đơn vị nguyên bản (giờ), không phạt mạnh outlier. - **RMSE**: bình phương rồi căn — **phạt nặng lỗi lớn**. Nếu vài dự đoán lệch cực xa (dự đoán 500 giờ nhưng thật 2 giờ) → RMSE kéo lên rất cao.

Cho RUL, **lỗi âm (dự đoán ít hơn thật) là an toàn hơn lỗi dương** (dự đoán 2 giờ nhưng thật 500 giờ → máy vỡ trong lúc ta đang chờ). Một vài contest dùng **asymmetric score**:

$score = \sum(\exp(-\hat{y}_i/13) - 1)$ nếu dự đoán muộn (over-predict, tức báo lỗi trễ — nguy hiểm), $\sum(\exp(\hat{y}_i/10) - 1)$ nếu dự đoán sớm.

Lưu ý: model A1 thường trả về 0/1 trước, RUL là mở rộng — chỉ khi đề yêu cầu cụ thể về thời gian còn lại mới cần (chi tiết thêm ở Mục 7).

Kết thúc mục 4 — checklist: - [] Báo cáo Confusion matrix + Precision/Recall/F1 tại vài threshold. - [] Báo PR-AUC (hoặc AP) thay cho accuracy. - [] Chọn threshold theo business cost, không mặc định 0.5. - [] Nếu làm RUL: dùng MAE/RMSE + asymmetric score nếu contest quy định.

5. Anomaly detection / học KHÔNG giám sát

Mục tiêu của cả mục: trả lời nhánh (3) — khi KHÔNG có (hoặc gần như không có) nhãn lỗi, chỉ học trên dữ liệu bình thường rồi "soi" cái gì khác bình thường.

Đúng tinh thần A1: **thiếu dữ liệu lỗi**. Khi không có (hoặc gần như không có) nhãn lỗi, ta không thể train supervised. Giải pháp: chỉ học trên dữ liệu bình thường rồi "soi" cái gì khác bình thường.

Làm rõ phân biệt 3 hướng trong mục này: 1. **Unsupervised**: không có nhãn gì → group/cluster, lọc outlier. 2. **One-class / semi-supervised**: train CHỈ trên normal → đánh dấu bất kỳ thứ gì lệch khỏi normal. 3. **Statistical**: phân tích phân bố → ngưỡng động.

5.1. One-class SVM (OC-SVM)

Mục tiêu: dựng một "biên bao" quanh cụm normal trong không gian feature; điểm nằm ngoài biên = bất thường. Chọn khi số mẫu normal vừa phải (hàng nghìn), dimension thấp-trung bình.

5.1.1. Ý tưởng

Thay vì tìm siêu phẳng phân tách 2 lớp, OC-SVM tìm một **vùng chứa hầu hết dữ liệu normal** và đánh dấu những điểm nằm ngoài là bất thường. Nói nôm na: một "quả cầu" mềm bao quanh cụm dữ liệu normal trong không gian feature.

5.1.2. Kernel trick

Không gian feature thật có thể phức hợp — không một siêu phẳng (hay quả cầu) đơn giản nào bao được. **Kernel** ánh xạ dữ liệu lên không gian chiều cao hơn để tìm biên dễ hơn:

- Linear: không gian giữ nguyên.
- RBF: $K(x, z) = \exp(-\gamma \|x - z\|^2)$ — phổ biến nhất; đo "độ tương tự" giảm dần theo khoảng cách. Kiểm soát "độ mềm" biên bằng γ & ν .

5.1.3. Tham số ν

$\nu \in (0, 1]$ là tỉ lệ lỗi tối đa được phép (upper bound) và là tỉ lệ support vectors tối thiểu. Nói nôm na: ν nói với model "khoảng % dữ liệu normal có thể bị coi là outlier nếu cần".

- $\nu=0.05$: cho phép coi ~5% normal là biên lỏng → biên rộng, ít báo động giả.
- $\nu=0.001$: biên ép sát dữ liệu → nhạy bắt lỗi nhưng dễ báo giả. Chọn ν bằng kinh nghiệm với chi phí business (giống threshold).

5.2. Isolation Forest

Mục tiêu: tách outlier cực nhanh bằng cách so "độ sâu cô lập" trong cây ngẫu nhiên — lựa chọn hàng đầu khi dữ liệu chiều cao (nhiều feature) và cần tốc độ.

5.2.1. Ý tưởng nghịch đảo

Isolation Forest đi ngược logic "đo khoảng cách đến tâm": nó cố tìm một **cây quyết định ngẫu nhiên** phân rời dữ liệu càng nhanh càng tốt.

- Mỗi cây: chọn ngẫu nhiên feature + ngưỡng ngẫu nhiên để tách dữ liệu thành 2 nửa, lặp lại đến khi mỗi điểm nằm riêng lá.
- Điểm dị biệt (outlier) bị tách ra CHỈ sau vài bước** (nằm trong vùng thưa xa, dễ chia) → **độ sâu (path length) ngắn**.
- Điểm normal nằm trong cụm đông, cần nhiều lần chia mới cô lập** → path length dài.

Điểm bất thường = điểm có **path length trung bình ngắn** trên nhiều cây.

5.2.2. Ưu điểm

- Rất nhanh và nhẹ (không tính khoảng cách $O(N^2)$).
- Hoạt động tốt trên dữ liệu dimension cao (hàng chục feature).
- Không cần giả định phân bố Gauss.

5.2.3. Hạn chế

- Nhạy với số cây & kích thước; có thể bỏ sót outlier cụm (nếu lỗi tạo cụm riêng đông thì lại không "cô lập nhanh").
- Ngưỡng đánh giá (contamination) vẫn phải đặt bằng tay — ta phải cài ~ tỉ lệ lỗi ước lượng.

So sánh OC-SVM vs Isolation Forest vs Autoencoder — chọn cái nào cho A1?

Tiêu chí	OC-SVM	Isolation Forest	Autoencoder
Ý tưởng cốt lõi	Biên bao quanh normal (kernel)	Độ sâu cô lập trong cây ngẫu nhiên	Sai số tái dựng của mạng nén-giải nén
Giả định phân bố	Không (nhờ kernel)	Không	Không
Dữ liệu chiều cao / nhiều feature	Kém (slow, cần dimension vừa)	Tốt	Tốt (học biểu diễn)
Số mẫu normal cần	Hàng nghìn (không hàng triệu)	Ít hơn	Khá nhiều, dễ overfit nếu normal thiếu đa dạng
Tốc độ	Chậm ở quy mô lớn	Rất nhanh	Phụ thuộc mạng, cần GPU khi lớn
Diễn giải	Tương đối	Tương đối	Khó
Robustness với nhiễu nghiệp vụ	Medium	Cao (thứ hạng ngưỡng)	Cao nếu regularize tốt

Kết luận cho A1: kết hợp IsolationForest như một **tầng 1 nhanh nhạy** (loại bất thường quá rõ), OC-SVM hoặc Autoencoder làm **tầng 2 chi tiết** hơn; nếu đủ dữ liệu normal, Autoencoder cho biểu diễn mạnh nhất (chi tiết ở Phần 3). Ở hiện trạng team, `scripts/01_baseline_anomaly.py` chọn **Autoencoder** làm mốc vô giám sát — chính là "tầng chi tiết" này, và `results/baseline_ae.json` lưu con số baseline (xem Mục 6.4).

5.3. Autoencoder + reconstruction error (giới thiệu — chi tiết phần Deep Learning)

Autoencoder là mạng neural: nén dữ liệu về không gian ẩn chiều thấp (encoder), rồi giải nén về gần giống đầu vào (decoder). Nó **chỉ được huấn luyện trên dữ liệu NORMAL**.

Công thức tổng quát:

```
z = f_enc(x)          # encode x thành vector ẩn
x_hat = g_dec(z)       # tái dựng x
reconstruction_error = || x - x_hat ||^2  (thường là MSE)
```

Khi test: - Mẫu **normal** → tái dựng tốt → reconstruction error thấp. - Mẫu **bất thường** (dạng chưa từng thấy) → không nén được → error cao → nhận diện là lỗi.

Trực giác: autoencoder "nhớ" khuôn dạng số liệu normal; thứ không giống normal sẽ không khớp.

Ứng dụng A1: - Input có thể là **vector feature** (như 16 feature của `src/features.py` — đúng cách `scripts/01` dùng) hoặc **đoạn spectrogram/waveform** của cửa sổ. - Output = `reconstruction_error` → tiếp tục dùng **statistical threshold** (5.4) để quyết định 0/1. - Điểm mạnh: học biểu diễn mạnh mẽ, mở rộng theo dữ liệu; nhược điểm: cần khá nhiều dữ liệu normal, dễ overfit nếu normal thiếu đa dạng, khó giải thích. Đi sâu về architecture, loss, đánh giá sẽ ở phần **Deep Learning (part 3)**.

5.4. Statistical / heuristic: z-score, rolling threshold, EWMA, CUSUM

Mục tiêu: phát hiện bất thường CHỈ bằng ngưỡng trên một-không-ít biến (thường RMS/band energy) — nhanh, cài ngay, nhà máy hiểu được. Là họ "vũ khí" rẻ và minh bạch nhất; không cần train model.

Đây là các kỹ thuật "chỉ dùng 1 biến (thường là RMS/band energy)" — nhanh, cài được ngay, có ý nghĩa với nhà máy.

5.4.1. Z-score (chung cho 1 biến đo)

Mục tiêu: chênh bao nhiêu độ lệch chuẩn so với mức bình thường — ngưỡng theo thang σ , trực giác với kỹ sư.

Cách: tính μ , σ trên **toàn bộ dữ liệu normal** (hoặc cửa sổ đầu record xem là ổn định). Ngưỡng: nếu $|z| = |x - \mu| / \sigma > T$ với T thường 3–4 → báo lỗi. Z-score T=3 nghĩa "chỉ ~0,3% mẫu normal nằm ngoài" (theo xấp xỉ Gauss).

Giới hạn: phải biết dữ liệu gần Gauss (RMS gần vậy); nếu có nhiều cụm chế độ vận hành khác nhau (idle vs full load) thì scale toàn cục kém.

5.4.2. Rolling threshold (ngưỡng cuộn theo thời gian)

Thay vì 1 μ/σ toàn cục, dùng **rolling window**: với điểm tại thời gian t , tính μ , σ từ cửa sổ trước đó (ví dụ 5 phút lui về).

Ngưỡng: `alarm` khi `x[t] > mu_roll + k * sigma_roll`

Ưu: bám theo drift chậm của máy. Nhược: nếu lỗi diễn biến từ từ (càng tăng dần), rolling cũng "ăn theo" lỗi → chậm bắt; cần **baseline cố định** từ dữ liệu sạch khi máy mới.

5.4.3. EWMA (Exponential Weighted Moving Average)

Công thức đệ quy: `EWMA[t] = alpha * x[t] + (1 - alpha) * EWMA[t-1]`

- `alpha` ($0 < \alpha < 1$): trọng số cho mẫu mới; α gần 0 → trơn lâu (ít nhạy), α gần 1 → phản ứng nhanh (dễ nhiễu).
- EWMA làm mịn nhiễu nhanh để bắt **xu hướng** nhẹ. Ngưỡng: báo khi `EWMA[t] > mu + lambda * sigma_ewma` (λ ví dụ 3).

5.4.4. CUSUM (Cumulative Sum) — kiểm tra thay đổi phát hiện

CUSUM tích lũy chênh lệch nhỏ có hướng của biến theo thời gian — cực nhạy với **drift nhỏ nhưng kéo dài** (lỗi từ từ) mà z-score/EWMA bỏ lỡ.

Công thức hai hướng:

```
S_plus[t] = max(0, S_plus[t-1] + (x[t] - mu0) - K)
S_minus[t] = max(0, S_minus[t-1] + (mu0 - x[t]) - K)
Báo lỗi khi S_plus[t] > h hoặc S_minus[t] > h
```

- `mu0`: giá trị trung bình kỳ vọng (bình thường).
- `K`: mức dịch chuyển tối thiểu muốn phát hiện (mức độ nhạy); thường $K = 0.5 * \sigma$.
- `h`: ngưỡng báo (thường vài lần K , ví dụ $h = 4-5 * \sigma_{\text{shift}}$).

So sánh 4 kỹ thuật statistical — chọn cái nào?

Kỹ thuật	Bắt lỗi gì	Nhạy drift nhỏ kéo dài	Phức tạp cài đặt	Ảnh hưởng outlier đơn lẻ
Z-score	Nhảy vọt đột ngột	Kém	Rất thấp	Cao
Rolling	Drift chậm, theo cục bộ	Trung bình (có thể "ăn theo" lỗi)	Thấp	Trung bình
EWMA	Xu hướng nhẹ, làm mịn nhiễu	Tốt	Thấp	Thấp
CUSUM	Drift nhỏ kéo dài	Rất tốt	Trung bình	Thấp

Kết luận cho A1: bắt đầu bằng z-score hoặc EWMA (đơn giản, dễ giải thích với giám khảo); nếu biết lỗi phát triển từ từ (mài mòn) → thêm CUSUM hoặc ngưỡng percentile ổn định trên baseline.

Cảnh báo với mọi phương pháp statistical: phải xác định rõ liệu "normal" có phải là một chế độ ổn định hay có nhiều chế độ (vận hành khác nhau). Nếu có nhiều chế độ, hãy tách theo chế độ (clustering hoặc chia dòng) trước khi tính ngưỡng.

5.5. Cài ngưỡng khi KHÔNG có nhãn

Mục tiêu: khi không có nhãn lỗi thật, vẫn phải chốt được ngưỡng "báo động" — bằng percentile, heuristic vật lý, hoặc quy tắc thời gian.

Vì không có nhãn lỗi thật, ta phải "đoán" ngưỡng. Các cách:

5.5.1. Percentile (phân vị)

Cách: tính mọi anomaly score (reconstruction error, isolation factor, khoảng cách...), rồi chọn ngưỡng ở phân vị cao: `threshold = percentile(scores, 99.0)` (hoặc `99.5`, `99.9`).

Chỉ số: 1% hay 0,5% dữ liệu bị coi là bất thường. Nếu ta biết từ kinh nghiệm nhà máy "1% thời gian là có vấn đề nhỏ", dùng percentile 99. Nếu lỗi hiếm hơn, 99.9. (Đúng cách `scripts/01_baseline_anomaly.py` làm: `--thr 99.0` → ngưỡng = percentile 99 của reconstruction error trên tập normal.)

5.5.2. Heuristic theo vật lý

- Với RMS rung: so với giá trị trong datasheet máy (ví dụ ISO: máy "tốt" < 2,8 mm/s; "cảnh báo" 2,8–4,5; "dừng" > 7,1). → lấy ngưỡng từ tiêu chuẩn.
- Với band energy ở tần số ổ trục: so sánh độ tăng cho phép (ví dụ tăng > 10× so với baseline normal).
- Với multiple sensors: ngưỡng theo mức tăng tương đối so với chính chuỗi từng máy (khác nhau giữa máy mới và máy cũ).

5.5.3. Kết hợp "score + thời gian": ngưỡng mềm

Tránh báo động do spike nhiễu đơn: quyết định "lỗi" chỉ khi score vượt ngưỡng trong **k window liên tiếp** (ví dụ 5/10). Hoặc dùng trung bình 3 điểm gần nhau → giảm báo động giả mà không bỏ lỡ chuỗi lỗi kéo dài.

5.5.4. Self-validation khi dữ liệu normal sạch

Nếu ta biết 2–3 chuỗi dữ liệu đầu là sạch (máy vừa kiểm định): dùng nó làm **reference baseline** để calibrate mu, sigma, ngưỡng percentile. Nhưng KHÔNG dùng nó làm "test tự do" — vẫn phải dành một phần để đánh giá cuối (xem 5.6).

5.6. Cross-validation cho anomaly: hạn chế & data leakage

Mục tiêu: hiểu vì sao CV cổ điển (shuffle) sai với chuỗi thời gian, và cách chia **train = quá khứ**, **test = tương lai** là bất biến cho A1.

5.6.1. Vấn đề cơ bản với dữ liệu chuỗi thời gian

K-fold CV cổ điển **shuffle ngẫu nhiên** các mẫu → trộn mẫu trong quá khứ với mẫu trong tương lai. Điều này là **data leakage**: model "nhìn thấy" tương lai trong quá trình train → điểm trên CV cao giả tạo, nhưng khi deploy (chỉ biết quá khứ), model tệ hơn nhiều — **kết quả đánh giá bị lệch nghiêm trọng**.

Hệ quả với anomaly detection: nếu các mẫu normal giai đoạn đầu giống nhau và mẫu lỗi nằm cuối, shuffle sẽ "lén" cho model biết dạng lỗi ở phase hoặc window kế — **bất công**.

5.6.2. Quy tắc vàng: KHÔNG shuffle giữa past/future

Đúng nhất với deploy: **train = quá khứ**, **test = tương lai**.

```
[data theo thời gian]
|----- train (past) -----||----- test (future) -----|
```

- Với **anomaly score** (không nhãn): dùng **TimeSeriesSplit** — chia theo khối thời gian liên tiếp: lần 1 train tháng 1 → test tháng 3; lần 2 train tháng 1–3 → test tháng 4;...
- Khi làm **sliding block**: vùng train phải **TẤT CẢ** trước vùng test trong cùng fold. Không có bất kỳ điểm nào trong test nằm trước thời điểm train lân cận.
- Đặc biệt với smooth features (rolling mean, EWMA): rolling window **chỉ được tính từ dữ liệu quá khứ**, không tính từ tương lai. Nói cách khác, tính feature bằng cách "chỉ nhìn về trước" (causal). Với offline dataset, dễ mắc lỗi âm thầm dùng rolling giữa tâm → phải kiểm tra code.

5.6.3. Hạn chế của CV khi thiếu nhãn/thiếu dữ liệu lỗi

1. Không có metrics chuẩn vàng: không biết số lỗi thật → mọi ngưỡng percentile chỉ là giả định; kết quả CV "tốt" không đảm bảo bắt đúng lỗi thật.
2. Ít đại diện quần thể: nếu chỉ 1–2 chuỗi lỗi thật, chia fold có thể làm test chỉ chứa 1 chuỗi → độ tin cậy thấp (variance cao).
3. Nếu thống kê học (z-score/Rolling) dùng rolling window để tính ngưỡng trên train thì sẽ "đúng" trên train nhưng chưa chắc generalize.

Cách giảm thiểu: - Báo cáo **phân phối anomaly score** trên train/test riêng (histogram) — giúp nhìn trực giác liệu ngưỡng có hợp lý. - Đánh giá trên **chuỗi thời gian thật** (file thứ n so với file thứ 1) — so sánh khi máy mới (baseline) vs khi máy cũ. - Nếu có nhãn ít ỏi, dùng nó **CHỈ** cho test cuối, tuyệt đối không cho chọn threshold (tránh overfit vào vài chuỗi lỗi hiếm đó).

6. Xử lý dữ liệu mất cân bằng + thiếu dữ liệu lỗi (nhánh chính)

Mục tiêu của cả mục: trả lời nhánh (5) và (6) — làm sao giúp model học lớp lỗi hiếm tốt hơn (augmentation), và làm sao chứng minh điều đó bằng protocol trung thực.

Giả định: có TOÀN BỘ normal + MỘT PHẦN rất nhỏ lỗi thật (a few). Mục này trả lời: làm sao để mô hình học hiệu quả và đánh giá trung thực.

6.1. Oversampling / undersampling

Mục tiêu: cân bằng tỉ lệ lớp (50/50 hoặc gần đó) để model không "lười" chỉ đoán lớp đông — nhưng phải biết cái giá của từng cách.

6.1.1. Undersampling (giảm mẫu lớp đông)

- Loại ngẫu nhiên mẫu normal để cân bằng (50/50 hoặc gần đó).
- Lợi: nhẹ, nhanh, bias về lớp lỗi chứ không bị áp đảo.
- Hại: **vứt bỏ dữ liệu normal quý** → normal ít hơn dẫn đến học phân bố normal kém, mất ổn định. Chỉ nên dùng khi dữ liệu normal cực nhiều và lỗi có nghĩa.

6.1.2. Oversampling (nhân mẫu lớp thiểu số)

- Nhân bản ngẫu nhiên** các mẫu lỗi cho bằng số normal.
- Lợi: giữ mọi dữ liệu.
- Hại: nhân bản tạo **trùng lặp chính xác** → model có thể "thuộc lòng" từng mẫu lỗi → overfit, và CV shuffle sẽ bị "trùng mẫu giữa train/test" (leakage!) nếu nhân bản xong rồi mới chia train/test.

⚠ **Quy tắc quan trọng:** thực hiện resampling **TRONG fold** (chỉ trên train split) — không bao giờ resample toàn bộ rồi mới tách, nếu không test sẽ chứa bản sao của các mẫu train → metric giả tạo. (Đúng cơ chế `scripts/02`: lỗi giả chỉ ghép vào `X_base / y_base` — tức vào TRAIN; `X_test / y_test` không bao giờ chạm.)

6.2. SMOTE (Synthetic Minority Over-sampling Technique)

Mục tiêu: sinh thêm mẫu lỗi TỔNG HỢP (không phải nhân bản) ở khoảng giữa các mẫu lỗi thật — đa dạng hóa lớp hiếm hơn oversampling đơn thuần.

6.2.1. Ý tưởng

Thay vì nhân bản y hệt, SMOTE sinh **mẫu tổng hợp nằm trên đoạn thẳng nối giữa các mẫu lỗi gần nhau**:

Cách: 1. Với mỗi mẫu lỗi a , chọn ngẫu nhiên k láng giềng cùng lớp lỗi (k -NN trong không gian feature, k thường 5). 2. Chọn một láng giềng b , tạo điểm mới: $x_{\text{new}} = a + \lambda * (b - a)$, với $\lambda \sim \text{Uniform}(0,1)$. 3. Lặp cho đến đủ số mẫu mong muốn.

6.2.2. Ví dụ số

Feature (kurtosis=5.0, RMS=0.4) và láng giềng (kurtosis=7.0, RMS=0.6). Với $\lambda=0.5$: $x_{\text{new}} = (5.0 + 0.5 * (7.0 - 5.0), 0.4 + 0.5 * (0.6 - 0.4)) = (6.0, 0.5)$. Mẫu mới nằm "ở giữa" hai mẫu lỗi thật.

6.2.3. Ưu / nhược

- Ưu: đa dạng hóa lớp thiểu số hơn nhân bản; thường cải thiện recall/F1 khi dữ liệu lỗi hiếm (trong trường hợp số mẫu lỗi đủ để ước lượng cụm).
- Nhược / nguy hiểm: 1. **Nhảy nhiều:** nếu một mẫu lỗi là nhiễu đo, SMOTE sinh thêm mẫu nhiễu quanh nó → model học lỗi giả. 2. **Nội suy sai vị trí:** với feature không smooth (kurtosis có thể nhảy), nội suy tuyến tính tạo mẫu "không có thật" nằm

giữa cụm normal → có thể gây lẫn lộn. 3. **Nguồn dữ liệu lỗi quá ít** (như 5–20 mẫu) thì SMOTE kém; ít nhất nên có vài chục mẫu. 4. Vẫn phải resample trong fold.

Biến thể đáng biết: **Borderline-SMOTE** (tập trung sinh quanh biên), **SMOTE-ENN / SMOTE-Tomek** (kết hợp undersampling để dọn nhiễu và làm sạch biên).

6.3. Data augmentation bằng dữ liệu tổng hợp (GAN / diffusion) — giới thiệu

Mục tiêu: khi vài mẫu lỗi thật là quá ít cho mọi heuristic, dùng generative model học phân bố lớp lỗi để "bơm" thêm mẫu — là deliverable nổi bật của đề A1 ("bộ sinh lỗi giả").

Khi đã có vài mẫu lỗi thật nhưng quá ít, ta có thể **sinh thêm mẫu lỗi bằng generative model** (huấn luyện để "mô phỏng" phân bố lớp lỗi). Hai họ phổ biến:

6.3.1. GAN (Generative Adversarial Network)

- Generator G sinh ra giả; Discriminator D đoán giả/thật. Đấu nhau (adversarial) cho đến khi G sinh ra mẫu không phân biệt được với thật.
- Sinh mẫu lỗi giả từ mẫu lỗi ít ỏi: G học phân bố lớp lỗi → sinh thêm.
- Nhược: khó train (mode collapse — G chỉ sinh vài dạng), cần dữ liệu đủ để "huấn luyện ổn định"; với 5 mẫu lỗi thì GAN thường không ăn — chỉ nên dùng khi $\geq$ vài trăm mẫu lỗi.

6.3.2. Diffusion model

- Học cách **khử nhiễu (denoise)**: thêm nhiễu dần vào mẫu thật đến khi gần như trắng, rồi học ngược để tái dựng → sinh mẫu mới.
- Thường ổn định hơn GAN, chất lượng tốt, nhưng nặng tính toán → thực dụng hơn khi data ít? Vẫn cần lượng mẫu đáng kể cho phân bố lỗi.

So sánh SMOTE vs GAN/diffusion — chọn cái nào?

Tiêu chí	SMOTE	GAN	Diffusion
Nguyên lý	Nội suy tuyến tính giữa mẫu thiếu số	Đấu G-D để học phân bố	Học khử nhiễu ngược để sinh
Số mẫu lỗi tối thiểu	Vài chục	Vài trăm	Vài trăm (nặng hơn)
Chất lượng mẫu	Chạy theo cụm, nhảy nhiều	Tốt nhưng dễ mode collapse	Tốt, ổn định hơn
Chi phí	Rẻ, chạy CPU	Trung bình–cao	Cao
Rủi ro	Nội suy vào vùng "không có thật"	Train mất ổn định	Nặng, cần tài nguyên
Vai trò ở A1	★ Khởi điểm nhanh, đủ dùng	Tăng năng cao, cần nhiều lỗi thật	Xu hướng, deliverable "xịn"

Kết luận cho A1: SMOTE là điểm khởi đầu rẻ và đủ dùng; khi muốn điểm cộng "bộ sinh lỗi giả" (đúng yêu cầu đề), nâng lên **GAN/diffusion** — nhưng luôn nhớ: mẫu sinh chỉ phụ trợ, con số quyết định là protocol trung thực (6.4). (Trong repo, `scripts/03` sinh `synthetic_faults.npy` bằng TimeGAN rồi `scripts/02 --gen npy` dùng lại — đúng hướng này.)

Chi tiết triển khai: **phần Deep Learning (part 3)**. Ở đây chỉ ghi nhận: augmentation tổng hợp THƯỜNG là phụ (chỉ giúp), không thay thế được một protocol đánh giá trung thực. Nếu nguyên bản có 50 mẫu lỗi thật, hãy giữ nguyên cho test.

6.4. Protocol so sánh trước/sau augmentation — vì sao tránh data leakage

Mục tiêu của mục này (quan trọng nhất trong Phần 2): định nghĩa một giao thức đánh giá TRUNG THỰC để chứng minh augmentation có giá trị hay không — và cập nhật đúng hiện trạng split của team.

6.4.1. Protocol chuẩn mực (khi có MỘT VÀI lỗi thật)

1. Tách theo thời gian TRƯỚC KHI làm bất cứ điều gì khác:
train_norm_windows = normal windows (thời gian trước, vùng khỏe)
train_fault_windows = vài windows lỗi THẬT (giữ nguyên, không sinh thêm)
test_future_windows = windows (cả normal cả lỗi THẬT) nằm SAU cùng deadline
2. Augmentation (SMOTE / GAN / copy):
CHỈ áp dụng lên train (train_norm + train_fault)
test_future_window KHÔNG BAO GIỜ bị sinh thêm, không bị trộn
3. Fit scaler / feature transform CHỈ trên train → áp lên test.
(scaler trước augmentation cho công bằng; không fit trên toàn bộ)
4. Đánh giá cuối: Precision/Recall/F1 (hoặc PR-AUC) TRÊN test_future chỉ.
5. Báo cáo cả hai:
 - baseline (train KHÔNG augmentation)
 - augmented (train CÓ augmentation)→ soi được việc augmentation có thật sự giúp generalization hay chỉ "khớp" mỗi.

Hiện trạng split của team (cập nhật mới nhất — phải đọc kỹ)

Trước đây một nhánh thí nghiệm cũ lấy **test normal là 20% NORMAL CUỐI** (phần normal sát vùng hỏng). Đó là một **test bất công**: phần normal ấy thực ra đã suy giảm, gần với lỗi thật → model bị đánh giá trên vùng mơ hồ → recall thấp giả tạo (có lần chỉ ~0.013). Không phải model yếu mà là TEST SAI CHỖ.

src/pipeline.py hiện tại đã sửa và định nghĩa split đúng như sau — **bạn báo cáo theo đúng cách này**:

Mỗi test-run theo thời gian:
[0% ---- 70% ---- 90% ---- 100%]
[NORMAL khỏe | vùng mơ hồ | FAULT]
- 0 → 70% : NORMAL (toàn bộ)
- 70 → 90% : BỎ QUA (nhân không rõ, tránh nhân nhiều)
- 90 → 100% : FAULT (dữ liệu lỗi THẬT, chính là thứ hiếm)

Trong riêng vùng NORMAL của MỖI run (tính trên phần 0-70%):
[0% — 70% | 70% — 90% | 90% — 100%]
[TRAIN | TEST | GREY (loại)]
- TRAIN : normal thật khỏe → model học "bình thường là gì"
- TEST : normal chưa thấy NHƯNG VẪN KHỎE → test đánh giá
- GREY : normal đã suy giảm, sát vùng hỏng → LOẠI khỏi cả train lẫn test
(giữ GREY trong test → model khó phân biệt normal-gần-hỏng với lỗi thật → recall/AUC thấp do test "bất công", không phải do model yếu)

FAULT: trộn ngẫu nhiên trong cụm lỗi → 20% TRAIN / 80% TEST.
Test dùng lỗi THẬT chưa từng thấy khi train → chống data leakage.

Chuẩn hoá z-score: fit mean/std TRÊN TRAIN (normal + fault train) rồi áp cho train + test + synthetic. Scaler không bao giờ nhìn test.

6.4.2. Con số baseline THẬT của team (chạy trên split mới này)

Chạy trên dataset IMS (--limit 300, win=512, stride=256), kết quả lưu trong results/ :

Baseline vô giám sát — AutoEncoder (scripts/01, results/baseline_ae.json): - Ngưỡng (percentile 99 của reconstruction error trên normal): **0.0164** - Precision **0.387** · Recall **0.031** · F1 **0.057** · AUC **0.524**

Đọc: AE thuần "học normal rồi đo độ lạ" bắt được rất ít lỗi thật (recall ~3%), AUC gần 0.5 (đoán bừa). Đây chính là lý do đề bài muốn ta **thêm dữ liệu lỗi giả để model giám sát học tốt hơn** — tức bài toán của script 2.

So sánh trước/sau augmentation — RandomForest (scripts/02 --gen heuristic, results/compare_augmentation.json):

metric	trước aug	sau aug (heuristic)	Δ
precision	0.416	0.418	+0.002
recall	0.477	0.482	+0.005
f1	0.444	0.447	+0.003
auc	0.582	0.580	-0.002

Semantics: recall = % lỗi thật bắt được (0.477 nghĩa ~48/100); precision = % cảnh báo đúng; AUC = phân biệt tổng quát (0.5 = đoán bừa).

Hai điều rút ra: 1. **Split MỚI (test normal khỏe) cho recall thực tế cao** — chuyển từ ~0.013 (test bất công, trùng vùng suy giảm) lên **0.477** — vì test giờ hỏi đúng câu: "phân biệt normal khỏe với lỗi thật". 2. So với mốc AE vô giám sát (recall 0.031), RF có nhãn +

chút lỗi thật đã **nhảy vọt lên recall 0.477 → 0.482**. Heuristic augmentation chỉ đẩy thêm nhẹ (+0.005 recall); bản --gen npy (TimeGAN) và tinh chỉnh feature/threshold là hướng để tăng tiếp.

6.4.3. Vì sao tránh data leakage (điểm mẫu chốt)

1. **Nếu shuffle 80/20 ngẫu nhiên**: mẫu lỗi ở file4 có thể rơi vào train, còn bản sao ở phần khác của CHÍNH file đó rơi vào test → mô hình "gặp" mẫu tương tự nhau → recall cao giả tạo. Khi deploy máy mới, không có "bản sao" đó → hỏng thật.
2. **Nếu fit scaler trên toàn bộ**: mu/sigma chứa thông tin test (thống kê tương lai) → feature trong train "bị nhìn trước" → mọi metric tin cậy thấp.
3. **Nếu SMOTE trên toàn bộ rồi tách**: mẫu sinh ra giữa hai mẫu lỗi thật có thể sinh ra một điểm gần với test → leak.

Nguyên tắc tóm gọn: **"test tương lai là vùng đất thiêng — augmentation, scaling, chọn threshold, chọn feature đều chỉ thực hiện trong train."**

6.4.4. Kỳ vọng thực tế của augmentation

- Tốt cho: giúp model học biên lớp lỗi ổn định hơn, giảm variance với mẫu lỗi ít.
- Không thần kỳ: nếu 5 mẫu lỗi, không thể sinh ra "lỗi mới thuộc loại khác"; SMOTE/GAN chỉ nội suy quanh các mẫu đã có.
- Kết luận: augmentation là **phụ trợ**, không phải thuốc chữa bách bệnh. Ưu tiên số 1 vẫn là feature engineering + threshold tối ưu + model tốt trên dữ liệu thật. Báo cáo so sánh trước/sau (như 6.4.2) đúng chuẩn "deliverable A1" mà giám khảo chấm.

7. RUL (Remaining Useful Life) dự báo ngắn

Mục tiêu: khi đề mở rộng từ "0/1" sang "còn bao lâu nữa hỏng", biết cách chuyển bài toán thành hồi quy và chọn loss/phương án đánh giá đúng.

Khi A1 mở rộng: không chỉ trả 0/1 mà muốn biết **còn bao nhiêu giờ trước khi hỏng** → bài toán **hồi quy**.

7.1. Thiết lập bài toán

- Input: window các feature tại thời điểm t.
- Output: RUL = số giờ (hoặc chu kỳ hoặc vòng quay) còn lại. Ví dụ dataset hình thành từ chuỗi hư hỏng: file từ khi máy mới (RUL=365 ngày) giảm dần đến 0 (hỏng).
- Nếu nhấn chỉ "một khoảng thời gian trước khi hỏng" → gán tuyến tính hoặc chập sần (capping) RUL tại giá trị tối đa quan tâm: $RUL_capped = \min(RUL, RUL_max)$ với RUL_max như 30 ngày → model dễ học hơn.

7.2. Loss functions cho hồi quy

- **MSE**: $(1/N) * \sum((y - \hat{y})^2)$ — dùng XGBRegressor / Neural. Phạt lỗi lớn nặng.
- **MAE**: $(1/N) * \sum(|y - \hat{y}|)$ — robust hơn với outlier, trực quan.
- **Huber loss**: MSE khi lỗi nhỏ, MAE khi lỗi lớn — dung hòa 2 loại trên, rất ổn với chuỗi thật (outlier nặng).
- **Asymmetric / quantile loss**: phạt lỗi âm và dương khác nhau — phù hợp business RUL (muốn không trì hoãn báo hỏng).
Quantile loss: $L = \max(\tau * (y - \hat{y}), (\tau - 1) * (y - \hat{y}))$ với τ là quantile muốn ($\tau=0.5$ → giống MAE; $\tau=0.8$ → phạt mạnh khi $\hat{y} < y$, tức dự đoán sớm quá).

So sánh nhanh: MSE phạt lỗi lớn nặng (muốn không có cú lệch cực xa), MAE trực quan nhất (đơn vị giờ), quantile loss cho ta "phương án an toàn" của RUL. **Cho A1 (nếu làm RUL)**: bắt đầu bằng MAE để diễn giải, bổ sung quantile/asymmetric nếu contest quy định score riêng.

7.3. Đánh giá

- **MAE**: dễ hiểu (trung bình lệch ± giờ).
- **RMSE**: phạt chệch xa.
- Nếu contest có **score riêng** (exponential như NASA PHM 2008): dùng hẳn score contest: $Score = \sum(\exp(-(y - \hat{y})/13) - 1)$ nếu $y - \hat{y} < 0$ (dự đoán sớm — ít nguy hiểm), và $\sum(\exp((y - \hat{y})/10) - 1)$ nếu $y - \hat{y} > 0$ (dự đoán muộn —

nguy hiểm hơn).

7.4. Lưu ý chuỗi thời gian trong RUL

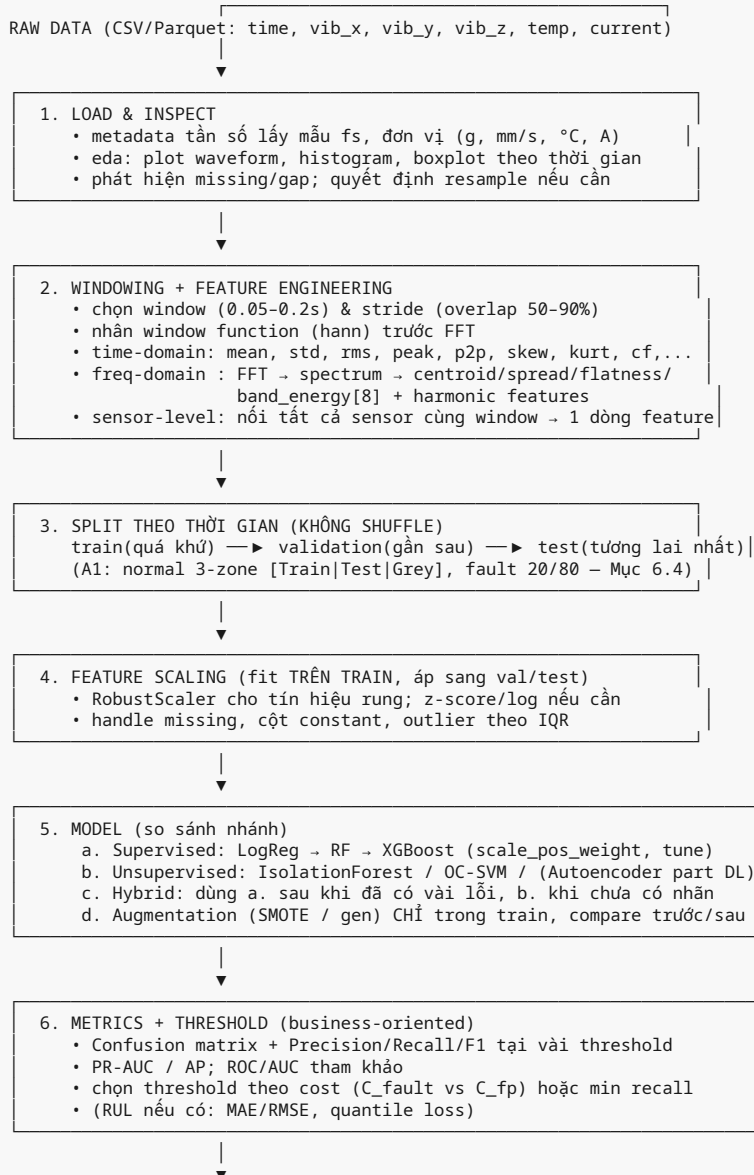
1. RUL các window cùng một máy **không độc lập** — ta không thể shuffle. Phải chia theo máy/chuỗi: train trên máy lẻ, test trên máy chẵn — hoặc theo thời gian.
2. Feature hướng về tương lai: rolling statistics phải causal.
3. RUL đánh giá bằng xu hướng đơn điệu tăng của error khi gần hỏng — vẽ `predicted_vs_true` theo thời gian để kiểm tra drift.

Khuyến nghị nhanh: nếu đề A1 chủ yếu là 0/1, đừng lẫn tẩn RUL — chắc chắn feature + protocol + metrics (mục 4 & 6) trước; RUL chỉ làm khi có thời gian và yêu cầu rõ.

8. Pipeline thực hành cho A1

Mục tiêu của cả mục: gom toàn bộ nhánh trên cây bài toán con thành một luồng chạy được từ raw signal đến báo cáo — kèm quyết định nhanh và checklist trước nộp.

8.1. Sơ đồ tổng thể



7. CHẠY TRÊN TEST CHƯA TỪNG CHẠM, GHI KẾT QUẢ, DIỄN GIẢI
- báo cáo ví dụ: hệ thống "alarm" tại giây X – đúng/giả?
 - feature importance → vẽ top features (kurtosis, E_band_hi,...)



8. TRIỂN KHAI (demo + dashboard)
- batch thao tác trên file mới → ghi kết quả CSV
 - dashboard: theo thời gian (vib + anomaly score + alert)
 - API stream: nhận window → trả (prob, class, rms, kurtosis)

Đối chiếu repo hiện tại: bước 3–5 được gom sẵn trong `src/pipeline.py` (`split_and_scale`, `to_train_test_arrays`) để ba script dùng chung; bước 2 trong `src/features.py`; bước 5b/6 trong `scripts/01`; bước 5d/6 trong `scripts/02`; bước 5a–6 kết hợp trong `scripts/03`. Muốn nộp bài "reproducible một cú", hãy giữ mọi tham số qua CLI (như các script đang làm).

8.2. Quyết định quan trọng rút gọn (decision guidance)

Câu hỏi	Câu trả lời ngắn
Có ≥ vài trăm mẫu lỗi nhãn?	Ưu tiên supervised (XGBoost)
Có 0 – vài chục mẫu lỗi?	Unsupervised/one-class + ngưỡng percentile; hoặc supervised trên normal-only + score
Feature miền nào quan trọng nhất?	band_energy vùng ổ trục tần số cao, kurtosis, crest factor, spectral centroid
Nên chuẩn hóa gì?	RobustScaler (rung có nhiều outlier); hiện tại repo dùng z-score fit trên train — hợp lệ
Threshold chọn sao?	Theo business cost, không mặc định 0.5
Chia dữ liệu sao?	Theo thời gian, KHÔNG shuffle; test normal lấy từ vùng KHỎE (Mục 6.4)
Test normal lấy ở đâu?	Từ vùng khỏe của riêng vùng NORMAL ([Train
Fault chia sao khi rất hiếm?	20% train / 80% test, đúng một cụm lỗi thật, chống leakage
Augmentation đáng tin?	Chỉ khi có bảng trước/sau trên CÙNG test (results/compare_augmentation.json)

8.3. Checklist trước nộp bài (demo day)

1. File EDA + notebook sạch: plot waveform, spectrum normal vs fault.
2. Feature bảng (địa chỉ cột rõ ràng, đơn vị ghi chú).
3. Model cuối + hyperparams + lý do chọn.
4. Bảng metrics trên test: confusion, precision/recall/F1, PR-AUC.
5. Threshold được chọn có rationale business (đừng nói "vi F1 max" — hãy nói chi phí bỏ sót lỗi bao nhiêu).
6. Baseline vs augmentation so sánh được (chứng minh augmentation có hiệu quả hay không) — dùng trước/sau trên CÙNG test khỏe (như bảng 6.4.2).
7. Dashboards: cảnh báo thời gian thực demo bằng file thực (kể cả khi chạy offline).
8. Pipeline reproducible: script build train/val/test theo thời gian, train, eval một cú.
9. Kể rõ câu chuyện split (normal 3-zone, fault 20/80, scaler fit trên train) — giám khảo chấm điểm phần "hiểu leakage".

Phụ lục A: Bảng công thức nhanh dùng mọi lúc

Đại lượng	Công thức
RMS	$\text{RMS} = \sqrt{(1/N) * \sum(x_i^2)}$
std	$\text{std} = \sqrt{(1/N) * \sum((x_i - \text{mean})^2)}$
Skewness	$\text{skew} = ((1/N) * \sum((x_i - \text{mean})^3)) / \text{std}^3$
Kurtosis (dư)	$\text{kurt} = ((1/N) * \sum((x_i - \text{mean})^4)) / \text{std}^4 - 3$
Crest factor	$\text{crest} = \text{peak} / \text{RMS}$
Shape factor	$\text{shape} = \text{RMS} / \text{mean_abs}$

Đại lượng	Công thức
Impulse factor	$\text{impulse} = \text{peak} / \text{mean_abs}$
Spectral centroid	$C = \sum(f_k \cdot P_k) / \sum(P_k)$
Spectral spread	$S = \sqrt{\sum(P_k \cdot (f_k - C)^2) / \sum(P_k)}$
Spectral flatness	$F = (\prod(P_k)^{(1/K)}) / (\text{mean}(P_k))$
Band energy (dải b)	$E_b = \sum(P_k)$ với k thuộc dải b
Z-score	$z = (x - \mu) / \sigma$
Min-max	$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$
RobustScaler	$x' = (x - \text{median}) / \text{IQR}$
Sigmoid	$p = 1 / (1 + \exp(-z))$
Cross-entropy	$-\sum_i (y_i \ln p_i + (1 - y_i) \ln(1 - p_i)) / N$
Gini	$G = 1 - \sum_c p_c^2$
EWMA	$E_t = \alpha \cdot x_t + (1 - \alpha) \cdot E_{t-1}$
CUSUM	$S_t = \max(0, S_{t-1} + (x_t - \mu_0) - K)$
Precision	$\text{Precision} = TP / (TP + FP)$
Recall	$\text{Recall} = TP / (TP + FN)$
F1	$F1 = 2 \cdot P \cdot R / (P + R)$
Accuracy	$\text{Acc} = (TP + TN) / (TP + TN + FP + FN)$
MAE	$(1/N) \cdot \sum(y - \hat{y})$
RMSE	$\sqrt{(1/N) \cdot \sum((y - \hat{y})^2)}$
Huber	MSE nếu

Phụ lục B: Các từ vựng Anh-Việt qua lại hay gặp

- Predictive maintenance — bảo trì dự đoán
- Fault / Anomaly — lỗi / bất thường
- Window / Stride / Overlap — cửa sổ / bước nhảy / chồng lấp
- Sampling frequency — tần số lấy mẫu
- Nyquist frequency — tần số Nyquist
- Aliasing — biệt danh tần số
- Feature engineering — kỹ thuật trích đặc trưng
- Spectral centroid / spread / flatness — tâm phổ / độ trải phổ / độ phẳng phổ
- Band energy — năng lượng dải tần
- Normalization / Standardization — chuẩn hóa / chuẩn hóa z
- Class imbalance — mất cân bằng lớp
- Confusion matrix — ma trận nhầm lẫn
- Precision / Recall / F1 — độ chính xác dương / độ phủ / F1
- Threshold — ngưỡng
- False alarm — báo động giả
- One-class SVM — SVM một lớp
- Isolation Forest — rừng cô lập
- Reconstruction error — sai số tái dựng
- Data leakage — rò rỉ dữ liệu
- SMOTE — kỹ thuật quá mẫu tổng hợp thiếu số
- Remaining Useful Life (RUL) — tuổi thọ còn lại
- Cross-validation — kiểm định chéo

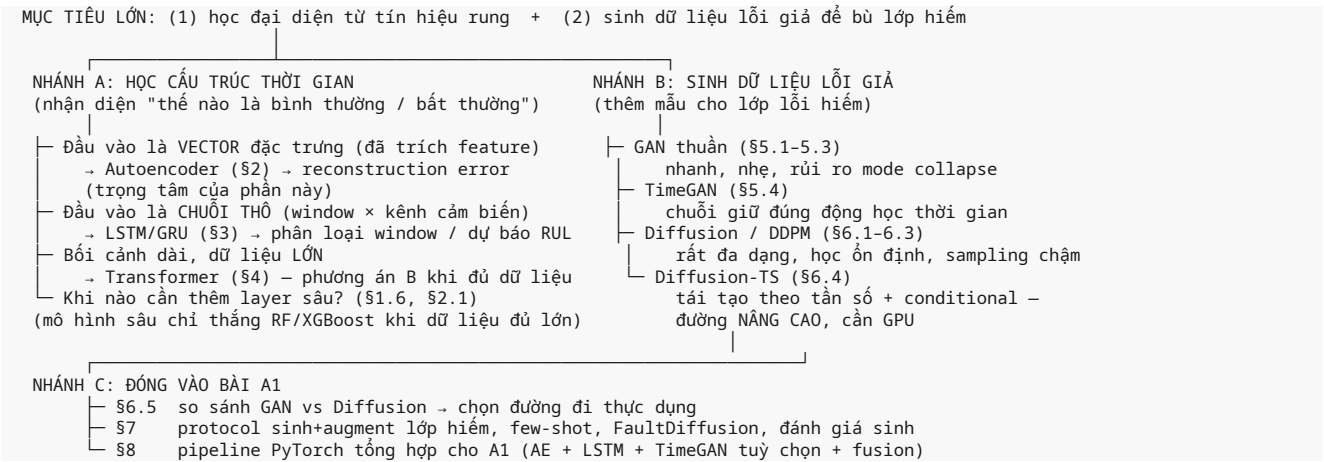
Hết giáo trình phần 2. Bước tiếp theo: phần 3 về Deep Learning (RNN/LSTM, Autoencoder chi tiết, GAN/diffusion cho augmentation) với nền tảng của giáo trình này.

PHẦN 3 · DEEP LEARNING & GENERATIVE MODELS — DEEP LEARNING & GENERATIVE MODELS
CHO CHUỖI THỜI GIAN CÔNG NGHIỆP + SINH DỮ LIỆU LỖI GIẢ

Giáo trình luyện thi — Cuộc thi **DENSO Factory Hacks 2026** Bài toán **A1**: "AI bảo trì dự đoán khi thiếu dữ liệu lỗi" — dữ liệu rung chuỗi thời gian. Đối tượng: sinh viên AI đã biết ML cơ bản. Phần này trọng tâm vào: **neural network nền tảng** → **autoencoder** → **LSTM** → **transformer (giới thiệu)** → **GAN** → **diffusion** → **sinh dữ liệu lỗi giả để augment lớp hiếm**.

CÂY BÀI TOÁN CON — định vị từng mô hình sẽ học trong Phần 3

Một mô hình chỉ đáng dùng khi ta biết nó giải **nhánh nào** của bài toán. Cây dưới đây phân rã MỤC TIÊU LỚN của Phần 3 thành các nhánh con; mỗi nhánh trở thẳng tới mục giải thích.



Nguyên tắc đọc: mọi thứ đều quy về một trong hai động lực trên. AE/LSTM học **hình dạng + thứ tự** của tín hiệu; TimeGAN/Diffusion học **phân phối** của dữ liệu để bơm thêm mẫu lỗi giả. Phần còn lại (activation, loss, backprop, optimizer) chỉ là "thịt" chung cho mọi mô hình sâu.

1. Neural network nền tảng (MLP, activation, loss, backprop, optimizer)
2. Autoencoder (bảo trì bất thường — trọng tâm)
3. RNN / LSTM / GRU cho chuỗi thời gian
4. Transformer (giới thiệu đúng mức cần thiết)
5. Generative Adversarial Network (GAN) + TimeGAN chi tiết
6. Diffusion models (DDPM, Diffusion-TS, so sánh GAN vs Diffusion)
7. Sinh dữ liệu để augment lớp hiếm (gắn bài A1)
8. Thực hành PyTorch cho A1: pipeline tổng hợp

1. Neural network nền tảng

Mục tiêu: xây khối "tế bào thần kinh" đầu tiên biết học từ dữ liệu rung — một MLP có khả năng: phân loại window bình thường/bất thường, dự báo RUL, hoặc làm encoder/decoder cho Autoencoder ở §2. Hiểu xong bạn nắm được ba thứ mọi mô hình sâu đều dùng: **activation** (nguồn phi tuyến), **loss** (thước đo sai), **backprop + optimizer** (cỗ máy học).

Bối cảnh (bài toán con): Phần 2 đã có RF/XGBoost và Isolation Forest trên feature thủ công (RMS, kurtosis, phổ...). Câu hỏi đặt ra: *khi nào cần chuyển sang mô hình sâu?* Trả lời gọn: khi (a) ta muốn mô hình **tự học đặc trưng** thay vì trích tay, và (b) dữ liệu đủ lớn để đặc trưng tự học đó ổn định.

So sánh — MLP/deep so với RF/XGBoost trên feature, vì sao chọn mô hình sâu:

Tiêu chí	RF / XGBoost trên feature tay	MLP / mô hình sâu
Đặc trưng	Người trích trước (RMS, spectral...)	Tự học từ dữ liệu thô
Số lượng dữ liệu cần	Ít (vài trăm mẫu vẫn tạm)	Nhiều (thường > vài nghìn)
Phi tuyến phức tạp	Tốt ở mức "cây" ghép	Rất tốt, đặc biệt tín hiệu tuần hoàn
Chi phí huấn luyện	Rẻ, ngay trên CPU	Đắt, cần GPU cho mô hình lớn
Vai trò trong A1	Baseline nhanh để so sánh	Lỗi của AE/lớp LSTM/GAN — học đại diện

Kết luận cho A1: RF/XGBoost giữ vai trò **baseline** (chạy 1 phút, cho con số để đối chứng). Mô hình sâu chỉ trở thành lựa chọn khi ta cần thứ RF không có: **không gian ẩn** (để AE phát hiện bất thường, để GAN sinh dữ liệu). Toàn bộ Phần 3 xoay quanh "đặc trưng tự học trong không gian ẩn".

1.1. Từ Perceptron đến MLP (Multi-Layer Perceptron)

1.1.1. Perceptron: nụ cười đầu tiên

Perceptron là đơn vị tính toán nhỏ nhất của mạng nơ-ron. Với vector đầu vào $x = (x_1, x_2, \dots, x_n)$, trọng số $w = (w_1, \dots, w_n)$ và bias b , perceptron tính **tổng có trọng số** rồi cho qua một **hàm kích hoạt**:

$$z = w \cdot x + b = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$$
$$a = f(z)$$

- z : giá trị "thuần" trước kích hoạt (tổng có trọng số cộng thêm bias).
- a : đầu ra của neuron, là kết quả đã "kích hoạt".
- Bootstrap: $y = \text{sign}(w \cdot x + b)$.

Trực giác bias: không có b , neuron luôn đi qua gốc tọa độ — không học được ranh giới lệch so với gốc. Bias cho phép dịch chuyển ranh giới quyết định. Ví dụ: muốn phân biệt "mức rung < 3 mm/s là bình thường theo tiêu chuẩn", đường phân cách cần nằm ở $x = 3$, không phải $x = 0$; bias chịu trách nhiệm cho sự dịch chuyển đó ($w \cdot x + b = 0 \rightarrow 3w + b = 0 \rightarrow b = -3w$).

1.1.2. MLP: không chỉ một neuron

Một neuron chỉ phân loại được **dữ liệu tuyến tính phân tách**. Tín hiệu rung khi bị lỗi thường có dạng phi tuyến phức tạp (xung, điều chế tần số), không cắt bằng một đường thẳng. MLP xếp thành **nhiều lớp (layers)**, mỗi lớp có nhiều neuron:

$$\begin{array}{l} \text{Lớp vào (input)} \rightarrow \text{Lớp ẩn 1 (hidden)} \rightarrow \text{Lớp ẩn 2} \rightarrow \text{Lớp ra (output)} \\ x \rightarrow a_1 = f(W_1 \cdot x + b_1) \rightarrow a_2 = f(W_2 \cdot a_1 + b_2) \rightarrow \hat{y} = g(W_3 \cdot a_2 + b_3) \end{array}$$

Trong đó W là **ma trận trọng số**, b là vector bias, f là activation phi tuyến, g là activation lớp ra (sigmoid/softmax cho phân loại, identity cho hồi quy).

Forward pass (lan truyền xuôi) — từ dữ liệu sang dự đoán:

```
# pseudocode forward pass cho mạng 1 lớp ẩn (batch = 1 mẫu)
a1 = relu(W1 @ x + b1) # lớp ẩn, shape (hidden,)
y_hat = sigmoid(W2 @ a1 + b2) # lớp ra, shape (1,)
```

Cơ chế từ gốc — vì sao cần hàm kích hoạt phi tuyến? Nếu chỉ dùng phép tính tuyến tính (không có f), thì dù xếp bao nhiêu lớp, tổng hợp vẫn là một phép biến đổi tuyến tính — mạng "xẹp" thành đúng một neuron. Chính f phi tuyến tạo nên sức mạnh biểu diễn. **Universal Approximation Theorem** nói: với đủ neuron và activation phi tuyến, MLP có thể xấp xỉ bất kỳ hàm liên tục nào. Một MLP đủ rộng cũng có thể làm encoder của Autoencoder (§2) hoặc chồng lên LSTM (§3) — đây là "gạch cơ bản" của mọi kiến trúc sâu.

1.2. Các hàm kích hoạt (activation functions)

Tên	Công thức	Miền	Trực giác / lưu ý
ReLU	$\max(0, z)$	$[0, +\infty)$	Phổ biến nhất cho lớp ẩn. Đạo hàm 1 khi $z > 0$, 0 khi $z \leq 0$. Rẻ, giảm vanishing gradient. Nhược: neuron "chết" nếu luôn nhận đầu vào âm.
Sigmoid	$\sigma(z) = 1 / (1 + e^{-(z)})$	$(0, 1)$	Ép kết quả về xác suất. Dùng cho lớp ra nhị phân. Đạo hàm cực đại 0.25 $\rightarrow$ khi xếp nhiều lớp, nhân liên tiếp đạo hàm nhỏ $\rightarrow$ gradient mờ dần (vanishing).

Tên	Công thức	Miền	Trực giác / lưu ý
tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	$(-1, 1)$	Giống sigmoid nhưng đối xứng quanh 0 → đầu ra trung bình gần 0, giúp gradient ổn định hơn.
Softmax	$p_i = e^{z_i} / \sum_j e^{z_j}$	$(0, 1)$, tổng = 1	Cho phân loại đa lớp. Chuyển logits thành phân phối xác suất.

Trực giác ReLU: neuron "tắt" (0) đối với kích thích âm, "bật" tuyến tính với kích thích dương — giống cách tế bào nơ-ron sinh học chỉ phát xung khi kích thích vượt ngưỡng. ReLU giải quyết vanishing gradient vì đạo hàm là hằng số 1 trên nhánh dương: thông tin gradient không bị "nhân nhỏ dần" qua nhiều lớp.

1.3. Loss functions (hàm mất mát)

Loss đo **mức sai** giữa dự đoán $\hat{y}$ và nhãn thật y . Mô hình tối ưu bằng cách giảm loss.

1.3.1. MSE (Mean Squared Error) — cho hồi quy và tái tạo tín hiệu

$$L = (1/N) \cdot \sum (y_i - \hat{y}_i)^2$$

- Dùng khi dự đoán **giá trị liên tục**: RUL (máy còn sống bao lâu nữa), nhiệt độ dự báo, hoặc **tái tạo lại tín hiệu** (autoencoder).
- MSE phạt sai lệch theo **bình phương** → sai lớn bị phạt rất nặng → gradient lớn ở sai lệch lớn → học nhanh lúc đầu.
- Đạo hàm $dL/dy = -(2/N) \sum (y - \hat{y})$, nhân với chain rule sẽ lan về trọng số.

Ví dụ số: $y = 2.0$, $\hat{y} = 0.4$ → đóng góp vào MSE là $(2.0 - 0.4)^2 = 2.56$; ngược lại nếu $\hat{y} = 1.9$ thì chỉ $(0.1)^2 = 0.01$. Sai lệch gấp 4 lần bị phạt 256 lần — đây là lý do MSE chạy nhanh khi mô hình còn sai to, và nhạy với **outlier** (một điểm đo hỏng có thể kéo cả loss lên).

1.3.2. Cross-entropy — cho phân loại

Binary Cross-Entropy (BCE) — nhãn 0/1:

$$L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

- $p = \sigma(z)$ là xác suất mô hình dự đoán lớp dương (ví dụ: "bất thường").
- Nếu $y=1$: $L = -\log(p)$ → muốn $p \rightarrow 1$. Nếu $y=0$: $L = -\log(1-p)$ → muốn $p \rightarrow 0$.
- Vì sao cross-entropy thay vì MSE cho phân loại?** Với sigmoid đầu ra, MSE có đạo hàm chứa $\sigma'(z)$ → gradient gần 0 khi mô hình sai trầm trọng (học cực chậm). Cross-entropy khử $\sigma'(z)$, gradient tỉ lệ $(p - y)$ — sai càng to, bước cập nhật càng lớn. Hội tụ nhanh hơn hẳn.

Categorical Cross-Entropy (CE) — nhiều lớp. Với mã one-hot $y = (y_1..y_K)$ và logits $z = (z_1..z_K)$:

$$L = -\sum_k y_k \cdot \log(\text{softmax}(z)_k)$$

Một cách viết gọn: $L = -\log(p_c)$ với c là lớp đúng — tức chỉ chịu phạt khi xác suất của lớp đúng thấp.

1.3.3. Tóm tắt chọn loss cho A1

Bài toán	Loss
Phân loại window normal/anomaly (2 lớp)	BCE
Phân loại nhiều loại lỗi	CE
Dự báo RUL (số giờ còn lại)	MSE (hoặc MAE)
Autoencoder tái tạo window rung	MSE trên tín hiệu tái tạo

1.4. Backpropagation: quy tắc chuỗi lan ngược

Mục tiêu con: trả lời câu hỏi "lỗi này do trọng số nào gây ra, phải chỉnh thế nào?". Ý tưởng: dùng **chain rule** (đạo hàm hàm hợp) để tính gradient của loss theo từng trọng số, rồi cập nhật trọng số ngược hướng gradient.

1.4.1. Chain rule

Nếu $L = f(g(h(w)))$ thì:

$$dL/dw = (dL/da) \cdot (da/dz) \cdot (dz/dw)$$

Ví dụ mạng 1 neuron: $z = w \cdot x + b$, $a = \sigma(z)$, $L = \text{BCE}(y, a)$.

$$\begin{aligned} dL/dw &= dL/da \cdot da/dz \cdot dz/dw \\ &= (a - y) \cdot a(1-a) \cdot x \end{aligned}$$

1.4.2. Backward pass từng bước (pseudocode)

Mạng 1 lớp ẩn: $x \rightarrow a1$ (ReLU) $\rightarrow \hat{y}$ (sigmoid), loss BCE.

```
# 1) Forward (tính sai số)
z1 = W1 @ x + b1
a1 = relu(z1)
z2 = W2 @ a1 + b2
y_hat = sigmoid(z2)
L = bce_loss(y, y_hat)

# 2) Backward (đạo hàm lớp ra trước)
dL_dz2 = y_hat - y          # đạo hàm BCE + sigmoid gộp lại
dL_dW2 = dL_dz2 * a1^T      # shape của W2
dL_db2 = dL_dz2
dL_da1 = W2^T @ dL_dz2

# 3) Lan tiếp qua hidden
dL_dz1 = dL_da1 * relu'(z1) # relu'=1 nếu z1>0 ngược lại 0
dL_dW1 = dL_dz1 * x^T
dL_db1 = dL_dz1

# 4) Cập nhật (SGD)
W1 = W1 - lr * dL_dW1
b1 = b1 - lr * dL_db1
W2 = W2 - lr * dL_dW2
b2 = b2 - lr * dL_db2
```

Thực giác từng bước: 1. Luôn tính đạo hàm **từ lớp ra về lớp vào** (ngược chiều forward) vì loss chỉ phụ thuộc vào đầu ra, ta duyệt ngược để biết "lỗi do ai gây ra". 2. Ở mỗi lớp, gradient = gradient từ lớp trên $\times$ đạo hàm cục bộ (activation + linear). 3. "Cập nhật ngược hướng gradient": nếu $dL/dW > 0$, tăng W sẽ tăng loss $\rightarrow$ phải giảm W . Công thức $W = W - lr \cdot dL/dW$.

Trong PyTorch, toàn bộ việc này chỉ là `loss.backward()` — autograd (tự động đạo hàm) làm hết.

1.5. Optimizers: cách đi xuống dốc

Mục tiêu con: sau khi có gradient, Chung hỏi "bước đi bao lớn và theo hướng nào?". Optimizer trả lời.

1.5.1. SGD (Stochastic Gradient Descent)

$$\theta = \theta - \eta \cdot \nabla L(\theta)$$

- θ : toàn bộ tham số mô hình, η : learning rate, ∇ : gradient.
- Thực giác: đi một bước ngược hướng dốc. Mỗi bước trên một **batch nhỏ** ngẫu nhiên (stochastic) $\rightarrow$ nhanh, nhiễu giúp thoát cực tiểu cục bộ nhỏ.
- Nhược: dao động vuông góc khi "thung lũng" lõm mạnh theo một hướng; không biết đổi nhịp khi sắp tới cực tiểu.

Ví dụ lr: `lr = 0.001` cho Adam là khởi điểm an toàn cho mạng huấn luyện trên dữ liệu chuẩn hóa (feature scale ~ 1).

1.5.2. Momentum

$$\begin{aligned} v &= \gamma \cdot v + \eta \cdot \nabla L \\ \theta &= \theta - v \end{aligned}$$

- γ thường `0.9`. Thực giác: quả bóng lăn xuống dốc — tích lũy "đà" qua các bước, hướng chuyển động trung bình hóa các gradient nhiễu.
- Lợi ích: giảm dao động zíc-zắc, tăng tốc xuống đáy thung lũng.

1.5.3. Adam (Adaptive Moment Estimation) — lựa chọn mặc định

Adam giữ trung bình di động của gradient (m , "momentum") và bình phương gradient (v , "tỉ lệ học tự thích ứng");

```

m_t =  $\beta_1 \cdot m_{(t-1)} + (1-\beta_1) \cdot g_t$  # trung bình gradient
v_t =  $\beta_2 \cdot v_{(t-1)} + (1-\beta_2) \cdot g_t^2$  # trung bình gradient bình phương
m_t = m_t / (1 -  $\beta_1^t$ ) # hiệu chỉnh lệch lúc đầu
v_t = v_t / (1 -  $\beta_2^t$ )
 $\theta = \theta - \eta \cdot m_t / (\sqrt{v_t} + \epsilon)$  #  $\epsilon \approx 1e-8$  tránh chia 0

```

Tham số mặc định: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$.

Thực giác: mỗi tham số có **learning rate riêng** — tham số gradient dao động mạnh (v lớn) thì bước nhỏ; gradient ổn định (v nhỏ) thì bước lớn. Adam "tự tay lái" cho từng tham số → ít phải chỉnh lr, hội tụ tốt trong đa số bài toán, gồm cả GAN/diffusion. Nhược: cần nhớ m , v → tốn bộ nhớ (~2× tham số), và có thể overfit noise ở cuối train.

1.6. Overfitting và các biện pháp chống

Overfitting: mô hình nhớ "thuộc lòng" tập train (kể cả nhiễu) nhưng kém trên dữ liệu mới. Nguy cơ rất cao khi **dữ liệu lỗi hiếm** như bài A1 — mô hình chỉ cần vài mẫu lỗi là học vẹt. (Nhánh "khi nào nên dùng mô hình sâu/sâu hơn" trong cây bài toán con: mô hình càng sâu càng nhiều tham số → càng dễ overfit → với dữ liệu ít, dùng mô hình **nhỏ hơn** thay vì sâu hơn.)

Phương pháp	Công thức / cơ chế	Thực giác
Dropout	Ngẫu nhiên gán 0 cho một phần neuron mỗi batch	Mạng "mù" vài đường nối mỗi lần → nhiều tập hợp con mô hình cùng học → trung bình hóa ẩn, giảm phụ thuộc vào đơn neuron
Weight decay (L2)	$L' = L + (\lambda/2) \sum w^2$	Phạt trọng số lớn → ép trọng số nhỏ, mô hình "đơn giản" (ít gồ ghề), tránh vừa khít nhiễu. Gradient thêm $\lambda \cdot w$.
Early stopping	Theo dõi validation loss, dừng khi bắt đầu tăng	Dừng đúng lúc mô hình còn tổng quát — học chậm lại vùng piano nhưng chưa học vẹt nhiễu
Batch normalization	Chuẩn hóa từng batch: $\hat{x} = (x - \mu_b) / \sigma_b$, rồi nhân γ cộng β	Giữ activation cùng phân phối qua các lớp → gradient ổn định, cho phép lr cao hơn và train nhanh hơn

Nguyên tắc quan trọng: **dropout/weight decay hoạt động kém hiệu quả khi dữ liệu lỗi quá ít** — mô hình không có gì để học từ. Khi mọi thứ khác thất bại, chiến lược kiên cường nhất là **(1) xử lý imbalance ví mô** (loss có trọng số, oversampling có kiểm soát) và **(2) generative augmentation** — chủ đề chính của phần 5–7.

1.7. PyTorch căn bản: tensor, autograd, vòng lặp huấn luyện

Mục tiêu con: biến toán học §1.1–1.6 thành code chạy được — nền tảng cho mọi mô hình ở các phần sau.

1.7.1. Tensor: dữ liệu đa chiều

Tensor giống numpy array nhưng có tính năng auto-diff. Data A1: window rung dạng `(batch, time_steps, n_signals)`.

```

import torch
x = torch.randn(8, 128, 3) # 8 windows, 128 time steps, 3 tín hiệu (x,y,z)
print(x.shape) # torch.Size([8, 128, 3])

```

1.7.2. Autograd: tự động tính gradient

```

x = torch.tensor([2.0], requires_grad=True)
w = torch.tensor([3.0], requires_grad=True)
loss = (w * x) ** 2
loss.backward() # lan ngược
print(w.grad) # tensor([24.]) = 2*w*x**2 = 2*3*4

```

Chỉ các tensor có `requires_grad=True` mới nhận `.grad`. Mọi toán tử PyTorch đều đăng ký đạo hàm của mình vào đồ thị tính toán.

1.7.3. Vòng lặp huấn luyện chuẩn

```

import torch, torch.nn as nn

model = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 1))
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.BCEWithLogitsLoss() # BCE + sigmoid gộp, ổn định số

for epoch in range(20):
    for x_batch, y_batch in dataloader: # x:(B,64), y:(B,)
        optimizer.zero_grad() # xóa gradient cũ

```

```
logits = model(x_batch).squeeze()
loss   = loss_fn(logits, y_batch)
loss.backward()           # tính gradient
optimizer.step()          # cập nhật trọng số
print(epoch, loss.item())
```

Lưu ý: đặt model ở chế độ `model.train()` khi train và `model.eval()` khi đánh giá (quan trọng với dropout/batchnorm).

2. Autoencoder — công cụ bảo trì bất thường TRỌNG TÂM

Mục tiêu: HỌC "THẾ NÀO LÀ BÌNH THƯỜNG" — một mô hình **không cần nhãn**, nén dữ liệu rung normal thành một không gian ẩn nhỏ rồi tái tạo lại. Dùng cái "sai số tái tạo" (reconstruction error) làm điểm bất thường để phát hiện lỗi — kể cả lỗi **chưa từng thấy**. Đây là vũ khí số 1 cho A1 vì ta chỉ có dữ liệu normal dồi dào.

Bối cảnh (bài toán con): phân loại window thành normal/bất thường khi **không có nhãn lỗi đủ**. So với MLP có giám sát (§1) cần cả hai lớp, AE chỉ cần một lớp để học "phong cách" normal.

So sánh — Autoencoder vs PCA vs bin-wise feature + IsolationForest, vì sao chọn AE:

Tiêu chí	PCA + threshold	Bin-wise feature + IsolationForest (Phần 2)	Autoencoder
Đặc trưng	Chiều tuyến tính → không bắt phi tuyến	Đường histogram/share feature do người chọn	Học phi tuyến tự động
Không gian ẩn	Có (principal components)	Không	Có (latent) — vừa nén vừa tái tạo được
Bắt cấu trúc phức tạp (xung, điều chế)	Kém	Trung bình	Tốt nhờ DNN encoder
Vai trò trong A1	Baseline tuyến tính	Baseline nhanh	Phương án chính (src/ae.py)

Cả ba đều "học normal"; AE thắng vì mũi tên nén–giải nén phi tuyến bắt được cấu trúc tín hiệu (dáng sóng, đỉnh xung) mà PCA tuyến tính không làm nổi.

2.1. Kiến trúc: encoder – bottleneck – decoder

Autoencoder (AE) học cách **nén rồi khôi phục** dữ liệu mà không cần nhãn:

$$x \rightarrow [\text{Encoder } \phi] \rightarrow z \text{ (bottleneck, chiều nhỏ)} \rightarrow [\text{Decoder } \psi] \rightarrow \hat{x}$$

- **Encoder** $z = \varphi(x)$: nén $x \in \mathbb{R}^d$ thành vector tiềm ẩn $z \in \mathbb{R}^p$ với $p < d$. Các lớp DNN nén dần thông tin.
- **Bottleneck / latent space**: điểm giữa mỏng nhất — buộc mô hình giữ lại **những gì quan trọng nhất** để tái tạo, bỏ đi nhiễu/dư thừa. Đây chính là **bản nén thông tin** chứa "đặc trưng nền tảng" của dữ liệu.
- **Decoder** $\hat{x} = \psi(z)$: căng ngược z thành đầu ra cùng chiều với x .

Thực giác tổng quát: như nén file ảnh — giữ lại "bản chất" bức ảnh, xả ra gần giống bản gốc. Chiều bottleneck càng nhỏ → mô hình càng phải khái quát hóa mạnh (chỉ giữ đặc trưng phổ biến của dữ liệu train). Trong code repo của bạn (`src/ae.py`), encoder là `Linear(-32) → ReLU → Linear(-8)`, bottleneck = 8; decoder là `Linear(-32) → ReLU → Linear(-d_in)`. Với `d_in = 16` feature mỗi window, ta có chuỗi chiều rộng **16 → 32 → 8 → 32 → 16** — tức "phình ra rồi nén về 8 rồi phình lại": mô hình phải gói toàn bộ "bản chất normal" vào 8 con số.

vẽ mạch feature AE $16 \rightarrow 32 \rightarrow 8 \rightarrow 32 \rightarrow 16$:

x:[16] -L16-32- [32] -ReLU- [32] -L32-8- [8] z -L8-32- [32] -ReLU- [32] -L32-16- x̂:[16]

ENCODER bottleneck DECODER

2.2. Huấn luyện: tái tạo + reconstruction loss

Mục tiêu: đầu ra $\hat{x}$ giống đầu vào x nhất. Với tín hiệu rung (số thực liên tục), loss tái tạo thường dùng **MSE**:

$$L_{\text{recon}} = (1/N) \cdot \sum_i ||x_i - \hat{x}_i||^2 = (1/N) \cdot \sum_i \sum_k (x_{i,k} - \hat{x}_{i,k})^2$$

Trong đó $\hat{x} = \psi(\varphi(x))$. Loss nhỏ $\rightarrow$ decoder khôi phục được tín hiệu từ latent $\rightarrow$ encoder đã bắt được "phong cách" dữ liệu học.

Ví dụ số (một mẫu, 2 chiều): $x = [1.0, 2.0]$, $\hat{x} = [0.9, 0.4] \rightarrow L = (0.1)^2 + (1.6)^2 = 0.01 + 2.56 = 2.57$. Gradient sẽ đẩy decoder xuất ra chiều thứ hai gần 2.0 hơn — cụ thể, đạo hàm theo thành phần thứ hai là $2 \cdot (\hat{x}_2 - x_2) = 2 \cdot (0.4 - 2.0) = -3.2$, nên $\hat{x}_2$ sẽ được đẩy lên.

Điểm mấu chốt: **AE train hoàn toàn không dùng nhãn** → dùng được khi chỉ có dữ liệu "bình thường", đúng hoàn cảnh A1.

2.3. Anomaly detection với AE

2.3.1. Logic chính

1. **Train AE chỉ trên dữ liệu NORMAL** (phong phú, dễ thu).
2. Vì chỉ học "về ngoài của normal", AE tái tạo mẫu normal rất tốt, nhưng **tái tạo kém mẫu lạ** — lỗi rung, mòn trục, xung bất thường có cấu trúc khác hẳn.
3. Điểm bất thường = **reconstruction error**:

Anomaly score: $e = ||x - \varphi, \psi(x)||^2$ (MSE giữa chuỗi gốc và tái tạo)

4. Ngưỡng **th**: có thể lấy từ phân vị 99% của **e** trên tập validation normal (thủ công) hoặc dùng Otsu / double-check bằng dữ liệu lỗi thật nếu có. Trong `src/ae.py` có sẵn `threshold_from_err(rec_err, percentile=99.0)` làm đúng việc này.

2.3.2. Ví dụ trực giác

Mẫu	Tái tạo	MSE	Kết luận
Normal: sin ổn định 1× 50Hz	gần như giống	0.01	normal
Bắt đầu mài mòn ổ trục: xung cộng vào	chỉ tái tạo phần "sin mượt", bỏ xung	0.85	bất thường → báo sớm
Va chạm / chập: méo hoàn toàn	tái tạo ra thứ "lạ" không giống	3.2	bất thường mạnh

Trực giác sâu: AE là "chuyên gia về normal". Khi gặp thứ nó chưa từng thấy, nó phải "nói bừa" → sai lệch lớn. Điều này khiến AE **phát hiện được cả lỗi chưa từng gặp** (unseen faults) — vô giá trong nhà máy vì ta không thể dự trù hết mọi kiểu lỗi.

2.3.3. Biến thể nhanh

- **AE trên window:** chia rung thành window (vd 128 điểm × 3 trục), mỗi window là một "ảnh 2D" — AE xử lý như ảnh nhỏ.
- **AE trên feature (khuyến nghị đầu tiên):** trích nhẹ feature (RMS, kurtosis, spectral...) mỗi window rồi cho vào FeatureAE nhỏ — chạy CPU được, ổn định (nền tảng của `src/ae.py`).
- **Conv-AE:** dùng Conv1d cho encoder/decoder → bắt cấu trúc cục bộ (đỉnh xung, hình dạng sóng).

2.4. Variational Autoencoder (VAE) — giới thiệu ý tưởng

Mục tiêu: thêm "bước đệm" từ AE sang generative models: vừa giữ khả năng anomaly detection, vừa có **latent có xác suất mượt** để vẽ sau sinh dữ liệu mới (bàn đạp cho GAN/Diffusion §5–6).

VAE thay vì map $x \rightarrow z$ (một điểm), map $x \rightarrow$ phân bố z (trung bình μ + độ lệch σ):

encoder: $z \sim N(\mu(x), \text{diag}(\sigma^2(x)))$
generator: $\hat{x} \sim p(x | z)$ (thường decoder cho ra giá trị trung bình)

Mẫu z được lấy bằng **reparametrization trick**:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim N(0, I)$$

(trick này làm cho gradient chảy xuyên qua phép lấy mẫu — phép lấy mẫu trở thành phép nhân khả vi).

ELBO (Evidence Lower Bound) — ý tưởng (không cần chứng minh đầy đủ): VAE cực đại hóa cận dưới xác suất log của dữ liệu:

$$L = E_{\{z \sim q\}}[\log p(x|z)] - KL(q(z|x) || p(z))$$

Hai số hạng đóng vai trò: 1. **Số hạng tái tạo** $E[\log p(x|z)]$: gần $-MSE(x, \hat{x})$, yêu cầu $\hat{x}$ giống x . 2. **Số hạng KL**: ép $q(z|x)$ gần phân phối chuẩn tiên nghiệm $N(0, I)$, làm latent space **mượt và liên tục** — hai điểm z gần nhau cho ra hai mẫu x tương tự nhau.

Lợi ích so với AE thuần: latent có cấu trúc xác suất đẹp → có thể **sinh mẫu mới** bằng cách lấy $z \sim N(0, I)$ rồi decode. Trong bài A1: VAE vừa làm AE anomaly detection vừa là **bước đệm khái niệm** cho generative models (GAN, diffusion) ở phần sau.

2.5. Code PyTorch — AE trên window feature (khớp src/ae.py)

Đây là mạch đúng file `src/ae.py`: encoder `Linear(d_in,32) → ReLU → Linear(32,8)`, decoder `Linear(8,32) → ReLU → Linear(32,d_in)`. Với `d_in=16` ta có **16 → 32 → 8 → 32 → 16**.

```
import torch, torch.nn as nn

class FeatureAE(nn.Module):
    def __init__(self, d_in, d_hidden=32, d_latent=8):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Linear(d_in, d_hidden), nn.ReLU(),
            nn.Linear(d_hidden, d_latent)) # 16 -> 32 -> 8
        self.dec = nn.Sequential(
            nn.Linear(d_latent, d_hidden), nn.ReLU(),
            nn.Linear(d_hidden, d_in)) # 8 -> 32 -> 16

    def forward(self, x):
        return self.dec(self.enc(x)) # x

# ----- training -----
ae = FeatureAE(d_in=16) # ví dụ: window nén thành 16 feature
opt = torch.optim.Adam(ae.parameters(), lr=1e-3)
criterion = nn.MSELoss()

normal_loader = ... # CHỈ dữ liệu normal
for epoch in range(60): # default epochs trong ae.py
    for x in normal_loader: # x: (B, 16)
        x_hat = ae(x)
        loss = criterion(x_hat, x) # reconstruction loss
        opt.zero_grad(); loss.backward(); opt.step()

# ----- anomaly detection -----
def anomaly_score(ae, x):
    ae.eval()
    with torch.no_grad():
        score = ((ae(x) - x) ** 2).mean(dim=-1) # MSE theo từng window
    return score

# ngưỡng 99% từ reconstruction error của tập normal:
from src.ae import threshold_from_err
thr = threshold_from_err(rec_err_normal, percentile=99.0)
print("ngưỡng:", thr)
```

Thuật ngữ cần nhớ: `encoder`, `decoder`, `bottleneck/latent`, `reconstruction loss`, `anomaly score`, `ELBO`, `reparametrization trick`.

3. RNN / LSTM / GRU cho chuỗi thời gian

Mục tiêu: học **THỨ TỰ THỜI GIAN** — dự đoán từ trạng thái của cả chuỗi rung (chứ không phải từng điểm độc lập). LSTM/GRU giải quyết hai bài toán con: (a) phân loại window có lỗi hay không (dùng `h_T` cuối), (b) dự báo RUL (hồi quy số giờ còn lại). Đây là nhánh "học cấu trúc thời gian" trong cây bài toán con.

Bối cảnh (bài toán con): MLP xử lý window như vector vô thứ tự — quên mất rằng "xung lỗi xuất hiện giữa cửa sổ rồi dập tắt". Mô hình tuần tự duyệt từng bước và giữ "ký ức" để bắt quan hệ giữa các bước liên tiếp.

So sánh — RNN thuần vs LSTM vs GRU:

Tiêu chí	RNN thuần	LSTM	GRU
Bộ nhớ dài	Kém (vanishing gradient)	Tốt (cell state + cổng)	Tốt (ít cổng hơn)
Số tham số	Ít nhất	Nhiều nhất (~4× RNN)	Trung bình (~3× RNN)
Tốc độ train / dữ liệu ít	Nhanh nhưng học được ít	Đủ nhanh, giàu biểu diễn	Tốt nhất khi dữ liệu ít (ít overfit)
Dùng cho A1	Không nên	Nên (đủ mạnh)	Nên (ưu việt với data hiếm)

Với A1 dữ liệu lỗi hiếm, **GRU/LSTM nhỏ** là lựa chọn sáng suốt hơn Transformer lớn (xem §4.4).

3.1. Mô hình tuần tự và nỗi đau vanishing gradient

3.1.1. Tại sao cần mô hình tuần tự?

Cửa sổ rung 128 điểm đặt cạnh nhau thành vector thì MLP xử lý được — nhưng MLP **quên mất thứ tự** và không chia sẻ tham số theo thời gian. Rung lỗi có tính **tuần tự**: một cú va chạm tạo xung kéo theo dập tắt; tần số thay đổi theo thời gian. RNN duyệt chuỗi từng bước t , giữ **trạng thái ẩn** h_t như "bộ nhớ" chứa bối cảnh tới thời điểm t :

```
h_t = tanh( W_h · h_{t-1} + W_x · x_t + b )
y_t = W_y · h_t + b_y      (nếu cần đầu ra từng bước)
```

- x_t : mẫu rung tại bước t .
- h_t : trạng thái ẩn — chứa "ký ức" của toàn bộ quá khứ tới t , được cập nhật tuần tự.
- Tham số W_h, W_x **mỗi bước dùng chung** → mô hình có thể xử lý chuỗi dài bất kỳ, và số tham số không phình theo độ dài chuỗi.

Trực giác: như đọc một câu — hiểu từng chữ trong ngữ cảnh của những chữ trước đó h_{t-1} , rồi cập nhật "ý tưởng đang đọc" h_t .

3.1.2. Vì sao RNN thuần khó train? — Vanishing / Exploding gradient

Khi lan ngược qua T bước, gradient nhân với T lần ma trận W_h :

```
∂L/∂W_h ≈ ∑_t (hệ số) · (W_h)^(T-t) · ...
```

- Nếu $\|W_h\| < 1$: $(W_h)^k \rightarrow 0 \rightarrow$ **gradient tàn lụi**: các bước xa không học được → mất "bộ nhớ dài hạn".
- Nếu $\|W_h\| > 1$: $(W_h)^k \rightarrow \infty \rightarrow$ **gradient bùng nổ**: cập nhật trọng số nhảy loạn → NaN. (Biện pháp khẩn: gradient clipping $g = g \cdot c / \|g\|$ nếu $\|g\| > c$.)

LSTM/GRU ra đời để chống vanishing gradient bằng cách cho phép thông tin "đi xuyên suốt" qua **đường cao tốc** cổng điều khiển (xem 3.2).

3.2. LSTM: Long Short-Term Memory

3.2.1. Ý tưởng

Thêm **cell state** C_t — "băng chuyền" chạy suốt chuỗi với các cổng quyết định **ghi thêm gì, quên gì, đọc ra gì**. h_t vẫn là trạng thái ẩn ngăn phục vụ đầu ra.

```

x_t  ┌──┐
h_{t-1} ┤ trọng đó: cổng quên f, cổng vào i, ứng viên C̃, │
        └──┘ cổng ra o ───────────────────────────────────┘

C_t = f_t ⊙ C_{t-1} + i_t ⊙ C̃_t    - "băng chuyền" (dòng gradient không bị triệt tiêu)
h_t = o_t ⊙ tanh(C_t)
```

3.2.2. Công thức bốn cổng

Gọi h_{t-1} là state ẩn trước, x_t là đầu vào, concatenation $[h_{t-1}, x_t]$:

```

Nhớ quên: f_t = σ( W_f · [h_{t-1}, x_t] + b_f )
Nhớ vào:  i_t = σ( W_i · [h_{t-1}, x_t] + b_i )
          C̃_t = tanh( W_C · [h_{t-1}, x_t] + b_C ) # ứng viên thông tin mới
Cell state: C_t = f_t ⊙ C_{t-1} + i_t ⊙ C̃_t
Nhớ ra:   o_t = σ( W_o · [h_{t-1}, x_t] + b_o )
State ẩn: h_t = o_t ⊙ tanh(C_t)
```

($\odot$ là phép nhân từng phần tử; σ sigmoid ra giá trị 0–1 đóng vai trò "cái vòi" đóng/mở.)

Trực giác từng cổng: - **Forget gate** f_t (0–1): "Bộ nhớ cũ trong C_{t-1} còn hữu dụng không?" — 0 nghĩa là quên sạch, 1 là giữ nguyên. Ví dụ: khi máy đổi chế độ vận hành, cần quên đặc trưng chế độ cũ. - **Input gate** i_t (0–1): "Thông tin mới $C̃$ có đáng ghi không? Ghi bao nhiêu?" — với xung bất thường mới xuất hiện, i_t mở để ghi dấu hiệu này. - **Candidate** $C̃_t$: ứng viên nội dung mới được đánh giá. - **Cell update**: $C_t = (\text{quên phần cũ}) + (\text{thêm phần mới})$ — một phép cộng tuyến tính đơn giản, **dòng gradient chảy xuyên qua không bị khuếch đại/triệt tiêu** → chống vanishing gradient gốc rễ. - **Output gate** o_t : "Phần nào của bộ nhớ C_t sẽ lộ ra để làm đầu ra h_t ?"

Vì sao LSTM hợp chuỗi rung? Xung lỗi xuất hiện trong vài ms rồi biến mất; LSTM có thể **giữ dấu ấn** của xung qua nhiều bước (nhờ cell state) để phán quyết "window có chứa sự kiện bất thường" dù sự kiện không kéo dài hết window.

3.3. GRU: Gated Recurrent Unit (ngắn gọn)

GRU gộp hai cổng, bớt một state:

```
Cập nhật: z_t = σ( W_z · [h_{t-1}, x_t] + b_z )      # "phao giữ" cũ
Đặt lại:  r_t = σ( W_r · [h_{t-1}, x_t] + b_r )      # "bỏ" bao nhiêu quá khứ
Ứng viên: h_t = tanh( W_h · [x_t, r_t ⊙ h_{t-1}] + b_h )
State ẩn: h_t = (1 - z_t) ⊙ h_{t-1} + z_t ⊙ h_t
```

- z_t (update): đóng vai trò kết hợp forget+input của LSTM — quyết định giữ lại bao nhiêu trạng thái cũ h_{t-1} so với dùng trạng thái mới.
- r_t (reset): cho phép quên đi bối cảnh cũ khi cần — hữu ích khi tín hiệu thay đổi đột ngột (va chạm).
- GRU: ít tham số hơn LSTM (train nhanh, ít overfit với dữ liệu ít), hiệu năng tương đương trong đa số bài toán chuỗi vừa.

3.4. Ứng dụng cho A1: hai hướng chính

(a) Phân loại window normal vs abnormal (nhị phân hoặc đa-lớp-lỗi)

window (T bước, n tín hiệu) → LSTM (lấy h_T cuối) → Linear → logit

- Lấy trạng thái ẩn cuối cùng h_T (tóm tắt toàn bộ cửa sổ) cho vào head phân loại.
- Hoặc **lấy trung bình** tất cả h_t (mean pooling) nếu muốn quan tâm đều các thời đoạn.
- Nhãn: BCE hoặc CE; test là "cửa sổ này bình thường hay sắp hỏng".

(b) Dự báo RUL (Remaining Useful Life) hồi quy

history window (tín hiệu đã qua) → LSTM → Linear (1 đầu ra) → rui

- Đầu vào là cửa sổ quá khứ; đầu ra là **số giờ còn lại trước khi hỏng** — bài toán hồi quy (loss MSE).
- RUL "phạt không đối xứng" nếu giám khảo dùng asymmetric loss: dự đoán muộn (nguy hiểm) phạt nặng hơn dự đoán sớm. Thường làm bằng cách **giảm lr / thêm bias trong head** hoặc training có trọng số.

(c) Sequence-to-sequence (nâng cao, cùng ý tưởng)

Nếu muốn mô phỏng "cả đoạn tương lai" thay vì một con số: encoder LSTM đọc lịch sử → latent → decoder LSTM sinh từng bước kế tiếp (giống bài máy dịch). Đây cũng chính là **mô-đun cốt lõi** của TimeGAN (§5.4) khi nó dùng RNN sinh latent theo từng bước thời gian.

3.5. Code ngắn — LSTM classifier PyTorch

```
import torch, torch.nn as nn

class LSTMFault(nn.Module):
    def __init__(self, n_signal=3, hidden=64, n_layers=1, n_class=1):
        super().__init__()
        self.lstm = nn.LSTM(n_signal, hidden, n_layers, batch_first=True)
        self.head = nn.Linear(hidden, n_class)

    def forward(self, x):
        # x: (B, T, n_signal)
        out, (h, c) = self.lstm(x)      # out: (B, T, hidden)
        last = out[:, -1, :]            # trạng thái ẩn cuối
        return self.head(last).squeeze(-1)  # logit giờ

model = LSTMFault()
loss_fn = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(30):
    for x_b, y_b in loader:           # y_b: 0 hoặc 1
        logit = model(x_b)
        loss = loss_fn(logit, y_b)
        optimizer.zero_grad(); loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)  # chống bùng
        optimizer.step()
    # đánh giá: (logit > 0) => predict 1
```

Để đổi sang GRU chỉ cần thay `nn.LSTM(...)` bằng `nn.GRU(...)`; `out[:, -1, :]` không đổi.

4. Transformer (giới thiệu đúng mức cần thiết)

Mục tiêu: nắm đủ ý tưởng để biết **KHI NÀO dùng** và khi nào KHÔNG — vì với A1, Transformer chỉ là phương án B. Hiểu cơ chế attention cũng giúp đọc hiểu Diffusion-TS (§6.4) dùng transformer làm denoiser.

Bối cảnh (bài toán con): LSTM truyền ngữ cảnh xa qua trung gian (h_t) → thông tin suy yếu, duyệt tuần tự → chậm. Attention cho phép mọi vị trí **nối thẳng** với mọi vị trí khác trong một hop.

Sơ sánh — RNN/LSTM vs Transformer:

Tiêu chí	LSTM/GRU	Transformer
Kết nối ngữ cảnh xa	Qua trung gian h_t (suy yếu dần)	Trực tiếp 1 hop (attention)
Song song hóa	Kém (duyet tuần tự)	Tốt (tính song song mọi cặp)
Complexity theo độ dài	$O(T)$ bước tuần tự	$O(T^2)$ cặp vị trí
Dữ liệu cần	Ít → vừa	Rất nhiều (tránh overfit)
Với A1 (dữ liệu lỗi vài chục–vài trăm)	Chọn	Không nên (trừ ngữ cảnh đủ lớn)

4.1. Self-attention: Q, K, V

RNN duyệt tuần tự (chậm, khó song song hóa, khó giữ ngữ cảnh xa). Transformer cắt hết sự tuần tự: **mỗi phần tử nhìn trực tiếp mọi phần tử khác** bằng cơ chế attention. Trước khi tính, mỗi vị trí i được chiếu thành ba vector:

```
Query_i = W_Q · x_i      (ẩn danh: "tôi đang tìm kiếm gì?")
Key_i    = W_K · x_i      ("tôi đang cung cấp loại thông tin gì?")
Value_i  = W_V · x_i      ("nội dung thực sự của tôi là gì?")
```

Độ liên quan giữa vị trí i và j là **dot-product** Query với Key, chia căn chiều để giữ độ lớn ổn định, rồi softmax hóa:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Từng phần tử: - $Q \cdot K^T$: ma trận score, phần tử (i, j) = mức "vị trí i quan tâm tới j ". - $\sqrt{d_k}$: chuẩn hóa để tích nội không bùng (phương sai tỉ lệ d_k). - softmax trên dòng: chuẩn hóa thành trọng số tổng bằng 1. - Nhân với V : đầu ra của vị trí i = **tổng có trọng số** các Value đã chú ý.

Thực giác cho chuyện rung: phần tử "đỉnh xung" có Query hỏi "ai là hàng xóm của ta?" — hàng xóm xa nhưng có đặc trưng tương tự sẽ được đánh giá quan trọng → nhận biết mô-típ lỗi lặp lại ở bất kỳ đâu trong cửa sổ, dù cách xa nhau.

4.2. Multi-head attention & positional encoding

- Multi-head:** chạy nhiều $\text{Attention}(\cdot)$ song song với $W_Q/K/V$ khác nhau (thường 8 head), mỗi head học một kiểu quan hệ khác nhau (một head để ý biên độ sóng, một head để ý tần số, một head để ý độ trễ giữa các xung...), đầu ra nối lại rồi chiếu tuyến tính.
- Attention trái / phải / full:** có thể chọn "chỉ nhìn lại quá khứ" (causal mask) cho dự báo tương lai để tránh leak.

Positional encoding (PE): vì attention không mang thứ tự, thêm tín hiệu vị trí vào embedding. Ý tưởng phổ biến — sin/cos theo chu kỳ:

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d))
```

Các tần số khác nhau (chu kỳ khác nhau) như **la bàn đa tần số**: tần số cao mô tả vị trí gần nhau, tần số thấp mô tả vị trí xa — cho phép mô hình suy ra thứ tự tương đối. (Với chuỗi rung có thể thay bằng PE học được hoặc chỉ dùng đơn vị thời gian thật.)

4.3. Liên hệ với chuỗi thời gian: attention thay thế temporal dependency

- Trong RNN, mỗi quan hệ dài hạn chỉ được truyền "ngấm ngấm" qua h_t ; trong Transformer, **mọi cặp vị trí nối trực tiếp** một hop — không mất thông tin qua nhiều trung gian, không chịu vanishing gradient theo độ dài.
- Có thể đưa vào biến thời gian: độ trễ giữa tọa độ = chiều dài sóng, đỉnh-đỉnh... làm **input feature** cho Query/Key.


```
# phác thảo vòng lặp GAN (nhớ D đầu G đối kháng)
for step in range(steps):
    # (1) train D
    real = batch_real_data() # x_real
    d_real = D(real); loss_d_real = BCE(d_real, 1)
    z = torch.randn(batch, z_dim)
    fake = G(z); d_fake = D(fake.detach()); loss_d_fake = BCE(d_fake, 0)
    loss_d = (loss_d_real + loss_d_fake)/2
    d_opt.zero_grad(); loss_d.backward(); d_opt.step()

    # (2) train G (cheat: ép fake thành 1)
    fake = G(z)
    loss_g = BCE(D(fake), 1)
    g_opt.zero_grad(); loss_g.backward(); g_opt.step()
```

5.3. Mode collapse và training instability

- **Mode collapse:** G "hack" D bằng cách chỉ sinh một kiểu mẫu luôn đánh lừa được; dữ liệu sinh nghèo. Dấu hiệu: các mẫu sinh lặp lại, loss D bất thường.
- **Instability:** D mạnh quá → gradient G rỗng; D yếu quá → G vô định; hai bên "giằng co" oscillate. Cực đại $-\log D(G(z))$ bảo hòa số học.
- Biện pháp kinh điển: **WGAN-GP** (Wasserstein loss + gradient penalty), label smoothing (dùng nhãn 0.9 thay 1), learning rate riêng cho G và D ($1r_g < 1r_d$), thêm noise vào input D, spectral normalization.

5.4. GAN cho chuỗi thời gian: TimeGAN (NeurIPS 2019)

5.4.1. Vì sao GAN thuần sinh chuỗi khó?

Chuỗi thời gian không chỉ cần **hình dạng** đúng mà còn cần **động học** — **mối quan hệ các bước liên tiếp** (autocorrelation, xu hướng, tần số). GAN thuần (z ngẫu nhiên → sequence) dễ sinh ra chuỗi "đẹp từng khung" nhưng **vô nghĩa về thứ tự thời gian**: bước t+1 không liên quan bước t.

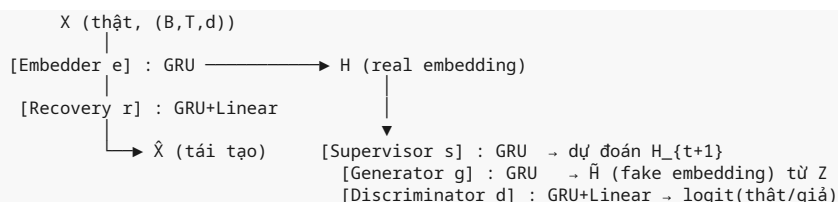
5.4.2. Ý tưởng cốt lõi TimeGAN

TimeGAN huấn luyện **trong không gian embedding** thay vì không gian dữ liệu thô, và kết hợp đồng thời nhiều mất mát, để vừa khớp phân phối tĩnh vừa khớp **phân phối có điều kiện theo thời gian**:

1. Reconstruction loss: $E[||x - \hat{x}||^2]$ (autoencoder embedding: emb + rec)
2. Supervised loss: $E[||h_{t+1} - s(h_t)||^2]$ (supervisor học bước chuyển tiếp)
3. Unsupervised (adversarial) loss: GAN trong latent (dis vs gen)
4. Embedding supervision: buộc mẫu synthetic giữ động học (supervisor chạy trên h giả)

Ghi chú nguồn: TimeGAN gốc (jsyo0n0823/TimeGAN) viết bằng **TensorFlow 1.15** — không chạy được trên Python ≥ 3.8/3.11. Bản repo này dùng **src/timegan_torch.py** — triển khai lại bằng **PyTorch**, **trung thành kiến trúc/paper**, **chạy CPU**, được triệu gọi bởi `scripts/03_train_timegan.py`. Mô tả dưới đây khớp đúng cách bản code đó tổ chức mạng và loss.

Cấu trúc mạng (khớp `src/timegan_torch.py`) — 5 module, tất cả đều là GRU:



- **Embedder e_ψ :** $X \rightarrow H$ — nén chuỗi thành latent chuẩn (GRU). `Embedder(d, hidden, n_layer)`.
- **Recovery r_ξ :** $H \rightarrow \hat{X}$ — đưa latent về không gian dữ liệu (GRU + Linear). Ghép với Embedder thành một AE trong latent. `Recovery(hidden, d, n_layer)`.
- **Generator g_θ :** $Z \rightarrow \hat{H}$ — nhận nhiễu $z \sim N(0, I)$ (shape (B, T, z_dim)) sinh **latent tổng hợp**, KHÔNG sinh raw. `Generator(z_dim, hidden, n_layer)`.
- **Supervisor s_ψ :** $H \rightarrow H_{t+1}$ — mô hình chuyển tiếp: học $h_t \rightarrow h_{t+1}$ để giữ động học. `Supervisor(hidden, n_layer)`.

- **Discriminator d_ψ** : phân biệt H (thật) vs $\tilde{H}$ (giả) ngay trong latent (GRU + Linear $\rightarrow$ logit). `Discriminator(hidden, n_layer)`.

Bốn loss trong vòng lặp (khớp code):

1. **Reconstruction loss**: `rec_loss = MSE( $\tilde{X}$ ,  $X$ )` — học Embedder+Recovery tái tạo chuỗi từ latent.
2. **Supervised loss (trên thật)**: `sup_loss = MSE( sup( $H[:, :-1]$ ),  $H[:, 1:]$  )` — supervisor dự đoán bước kế h_{t+1} khớp bước thật.
3. **Unsupervised / adversarial loss**: discriminator phân biệt `dis( $H$ )` và `dis( $\tilde{H}$ )` bằng BCE; generator chịu `g_adv = BCE(dis( $\tilde{H}$ ), 1)` (cố đánh lừa).
4. **Embedding supervision (trên giả)**: bảo động học cho mẫu synthetic — code lấy `H_fake_next = emb(rec( $H\_fake$ ))[:, :-1]` (đẩy một bước giả qua recovery rồi nhúng lại bằng embedder để tạo "bước kế") và buộc `sup( $H\_fake[:, :-1]$ )` khớp nó: `g_sup = MSE(sup( $H\_fake[:, :-1]$ ),  $H\_fake\_next$ )`.

Code thực thi theo hai pha lặp liên tiếp trong một vòng:

```
# pha 1 (joint): cập nhật emb+rec+sup theo (rec + 10*sup), rồi cập nhật gen theo (g_adv + 10*g_sup)
# pha 2 (adversarial): cập nhật dis trên (BCE thật=1, giả=0); các mạng dùng Adam lr=1e-3
```

Trọng số `10.0` cho supervised loss giúp mô hình "nghiêng" về giữ động học thời gian — đúng ý đồ paper. Khi sinh dữ liệu: $z \rightarrow$ generator $\rightarrow \tilde{H} \rightarrow$ recovery $\rightarrow \tilde{X}$ (chuỗi giả trong không gian quan sát).

Vì sao hợp chuỗi: bằng cách "đưa về không gian embedding (tĩnh) + buộc mô phỏng động học (supervised)", TimeGAN buộc mẫu sinh **giữ đúng tương quan thời gian** — điều GAN thuần không làm được. Đây là chuẩn vàng cho sinh chuỗi gen/tín hiệu, phù hợp để tạo thêm dữ liệu rung lỗi cho A1 (dù với dữ liệu lỗi rất ít, cần kỹ thuật few-shot ở mục 7).

Công thức tổng loss (ý niệm):

$$\min_{\{G,S,E\}} \max_D \lambda_{\text{recon}} \cdot L_{\text{recon}}(E,R) + \lambda_{\text{sup}} \cdot L_{\text{sup}}(S) + L_{\text{adv}}(D, G, E)$$

($\lambda_{\text{sup}} = 10$ trong code)

5.5. Lưu ý thực hành với chuỗi rung

- Chuẩn hóa mạnh (z-score từng tín hiệu) trước khi vào GAN — GAN cực nhạy scale.
- Sinh window độc lập (vd 128 bước) rồi dùng cho classifier — tránh sinh chuỗi dài (khó học tần số cao).
- Luôn luôn **đánh giá chất lượng sinh** bằng discriminative score (mục 7.4) trước khi dùng cho train.

6. Diffusion models — TRỌNG TÂM nhánh hiện đại nhất

Mục tiêu: sinh dữ liệu lỗi **rất đa dạng**, ít **mode collapse** hơn GAN — học cách "lột lớp nhiễu" ngược lại để tái tạo dữ liệu từ nhiễu thuần. Đây là đường **nâng cao** cho A1 (cần GPU, đắt hơn TimeGAN).

Bối cảnh (bài toán con): TimeGAN làm tốt với dữ liệu ít nhưng có thể thiếu đa dạng. Khi bạn cần **chất lượng tần số cao** (giữ đúng BPFO/BPFI của lỗi ổ trục) và đã có GPU — diffusion là ứng cử viên mạnh nhất.

Khi nào chọn Diffusion thay vì TimeGAN (so sánh đầy đủ xem §6.5): ổn định huấn luyện (không minimax), mẫu đa dạng, giữ tần số tốt; đánh đổi bằng chi phí sampling cao (nhiều forward) và bắt buộc lượng tính toán lớn.

6.1. DDPM (Denoising Diffusion Probabilistic Models) — ý tưởng

Diffusion = học cách **đảo ngược quá trình "bôi nhiễu dần thành trắng"**:

1. **Forward (làm hư)**: từ dữ liệu sạch x_0 , dần thêm Gaussian noise qua T bước (vd $T=1000$) cho tới khi thành $x_T \sim N(0, I)$ thuần nhiễu.
2. **Reverse (khôi phục)**: học mạng thần kinh ϵ_θ dự đoán nhiễu từng bước, lật ngược quá trình để từ x_T khôi phục x_0 .

Trực giác: như xóa mờ rồi vẽ lại từng lớp bụi — mô hình vừa thêm "thịt mới" vừa tin vào cấu trúc dữ liệu gốc.

Forward noising — công thức

Mỗi bước thêm nhiễu Gaussian xác định bởi bằng nhiễu $\beta_t \in (0, 1)$ (nhỏ, tăng dần):

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{(1-\beta_t)} \cdot x_{t-1}, \beta_t \cdot I)$$

Nhờ tính chất Gaussian, **nhảy thẳng** $x_0 \rightarrow x_t$:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \epsilon, \quad \epsilon \sim N(0, I)$$

với $\bar{\alpha}_t = \prod_{s=1}^t (1-\beta_s)$

- Khi t lớn, $\bar{\alpha}_t \rightarrow 0 \rightarrow x_t \approx \epsilon$: dữ liệu đã thành nhiễu thuần.
- Công thức trên là tất cả những gì ta cần để "chụp ảnh" x_t ở mọi bước mà không cần lặp forward.

Reverse denoising — công thức

Nếu biết $q(x_{t-1} | x_t)$ thì có thể quay ngược. Rất tiếc nó phụ thuộc vào toàn x_0 và không tractable — ta **xấp xỉ bằng mạng** p_θ :

$$p_\theta(x_{t-1} | x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

Với đầu ra μ_θ được bóc tách: mạng dự đoán **hiệu đã thêm** $\epsilon_\theta(x_t, t)$, rồi

$$x_{t-1} = (1/\sqrt{\alpha_t}) \cdot (x_t - (1-\alpha_t)/\sqrt{(1-\bar{\alpha}_t)} \cdot \epsilon_\theta(x_t, t)) + \sigma_t \cdot z$$

$\alpha_t = 1-\beta_t$; σ_t là độ nhiễu tùy lịch trình (thường $\sigma_t^2 = \beta_t$). Vẽ phải trừ đi phần nhiễu dự đoán rồi giữ "định hướng" dữ liệu còn lại.

6.2. Mất mát — epsilon-prediction

DDPM huấn luyện **mạng dự đoán nhiễu**: lấy ngẫu nhiên bước t , cộng nhiễu thật ϵ , yêu cầu mạng bóc ra ϵ_θ :

$$L = E_{\{t, x_0, \epsilon\}} [\| \epsilon - \epsilon_\theta(x_t, t) \|^2]$$

- $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \epsilon$, $t \sim \text{Uniform}(1..T)$.
- Về bản chất gần **MSE giữa nhiễu thật và nhiễu dự đoán** — mạng học "lượng nhiễu đã bị đổ vào ở bước t , tôi lột ra đúng bao nhiêu".
- Mạng ϵ_θ thường là UNet (ảnh) hoặc transformer/MLP (chuỗi), nhận thêm biến bước t (time embedding) để biết "đang ở bước nào".

Ví dụ số (bước cô đọng)

- x_0 = window rung thật (chuẩn hóa).
- Chọn $t = 500$, lấy ϵ ngẫu nhiên; tính $x_{500} = \sqrt{\bar{\alpha}} \cdot x_0 + \sqrt{(1-\bar{\alpha})} \cdot \epsilon$.
- Mạng trả $\epsilon_\theta(x_{500}, 500)$; $\text{loss} = \| \epsilon - \epsilon_\theta \|^2$.
- Ở t nhỏ (nhiều ít): gần như khôi phục chính xác; ở t lớn: nhiệm vụ là "bóc nhiễu hỗn loạn ra khỏi nền tín hiệu" — học cấu trúc toàn cục dữ liệu.

6.3. Sampling — lặp khử nhiễu

Sinh mẫu mới = bắt đầu từ nhiễu thuần rồi **lặp T lần** đi ngược về dữ liệu:

```
x_T ~ N(0, I)
for t from T down to 1:
    z ~ N(0, I)      (z = 0 nếu t = 1)
    x_{t-1} = (1/\sqrt{\alpha_t}) * (x_t - (1-\alpha_t)/\sqrt{(1-\bar{\alpha}_t)} * \epsilon_\theta(x_t, t)) + \sigma_t * z
return x_0          # mẫu sinh
```

- Mỗi vòng: "bóc nhiễu dự đoán một lớp, cộng chút nhiễu mới để khử sai số".
- Tốc độ: cần T lần forward → chậm (có acceleration DDIM để giảm bước).

6.4. Diffusion-TS (ICLR 2024) — diffusion cho chuỗi thời gian

Ý tưởng chính: huấn luyện denoiser bằng cách dự đoán **chính dữ liệu sạch x_0** (reconstruction) thay vì dự đoán nhiễu ϵ , kết hợp **phân rã trend/seasonal** và **Fourier loss** — rất hợp chuỗi rung có thành phần chu kỳ/tần số.

Lưu ý đường nâng cao: Diffusion-TS có chi phí tính toán cao (denoiser transformer + nhiều bước sampling) → **cần GPU**. Với đề thi giới hạn thời gian, chỉ theo đường này sau khi AE + TimeGAN đã chạy tốt; code minh họa trong `src/Diffusion-TS/`.

Các điểm chính

1. **Kiến trúc:** encoder-decoder dạng transformer làm denoiser $G_\theta(x_t, t, c)$ — không dùng UNet, mà dùng transformer cho chuỗi; hỗ trợ **conditional generation theo class** (điều kiện c = "normal" hoặc từng loại lỗi).
2. **Reconstruct thay vì predict noise:** mạng xuất trực tiếp ước lượng $\hat{x}_0 = G_\theta(\dots)$, loss:

$$L = E[\|x_0 - G_\theta(x_t, t, c)\|^2]$$

Trực giác: với chuỗi, "dự đoán x_0 " thường học ổn định hơn "dự đoán noise" (logit thuần tín hiệu), vì tín hiệu có cấu trúc (biên độ, tần số) rõ hơn noise vô định hình.

3. **Phân rã trend/seasonal:** đầu vào x_t được tách thành **trend** (xu hướng chậm, nhẵn) và **seasonal** (chu kỳ lặp); hai nhánh denoise riêng rồi gộp. Với rung: seasonal nắm thành phần tần số quay cơ bản (vd 1x, 2x), trend nắm trôi chậm theo mòn máy.
4. **Fourier loss:** thêm loss trên biến đổi Fourier:

$$L_{\text{fourier}} = E[\|F(x_0) - F(\hat{x}_0)\|^2] \quad (F = \text{DFT, phổ biên độ/tần số})$$

Vì chuỗi tín hiệu "sống" ở miền tần số nhiều hơn miền thời gian (2 dòng chỉ khác nhau ở pha, giống phổ chuỗi mẹ). Bắt phổ → mẫu sinh giữ đúng tần số đặc trưng của lỗi (BPFO, BPFI...), không "bóp méo sóng" thành cấu trúc tần số sai.

5. **Conditional generation:** diffusion gắn thêm biến điều kiện lớp vào denoiser; lúc sampling ngược, đưa c = loại lỗi vào mỗi bước → sinh **đúng chủng loại** lỗi mong muốn. Đây chính là công cụ ta cần để "sản xuất hàng loạt dữ liệu lỗi loại X".

Công thức gộp loss (ý niệm)

$$L = E[\|x_0 - G_\theta(x_t, t, c)\|^2] + \lambda_F \cdot E[\|F(x_0) - F(G_\theta(x_t, t, c))\|^2]$$

Khi chỉ có vài mẫu lỗi thật, huấn luyện conditioned diffusion thường được tinh chỉnh (fine-tune từ pretrain trên normal) hoặc dùng kỹ thuật few-shot (mục 7.2–7.3).

6.5. So sánh GAN vs Diffusion cho sinh dữ liệu

Tiêu chí	GAN	Diffusion (DDPM/Diffusion-TS)
Chất lượng mẫu / độ đa dạng	Có thể cao nhưng rủi ro mode collapse — thiếu đa dạng	Rất đa dạng (bao phủ cả distribution), ít mode collapse
Độ ổn định huấn luyện	Nổi tiếng khó — G vs D giằng co, cần điều chỉnh nhiều	Ổn định, loss đơn giản (MSE dự đoán), không trò chơi minimax
Tốc độ sampling	Rất nhanh (1 forward)	Chậm (T=1000 bước) — cần DDIM để giảm số bước
Dữ liệu ít (A1)	Học nhanh "vá" nhưng collapse dễ	Học chắc nhưng cần đủ bước; few-shot cần pretrain+fine-tune
Struct chuỗi (tần số)	Kém tự nhiên (dễ mất tương quan thời gian)	Tốt nếu dùng Fourier+reconstruct (Diffusion-TS)
Tài nguyên / Việt hóa code	Nhẹ, code cổ điển	Nặng hơn (nhiều forward), cần GPU tầm trung

Kết luận thực dụng cho A1: Nếu chỉ muốn augment vài trăm mẫu để classifier lên hạng — **GAN (TimeGAN)**: nhanh, đủ. Nếu muốn chất lượng tần số, đa dạng lỗi, không chịu mode collapse và có GPU + ít thời gian — **Diffusion (kiểu Diffusion-TS)**: chất lượng cao hơn, đặc biệt cho rare-class generation với fine-tune. Trong đề thi giới hạn thời gian: **ưu tiên đường GAN-TimeGAN + AE; diffusion là đường nâng cao**.

6.6. Hướng thứ nhất — Fault Injection / Mô phỏng vật lý: tạo dữ liệu lỗi bằng cách "bơm lỗi"

Vị trí: mục này nằm NGAY sau §6.5 (so sánh GAN vs Diffusion) và trước Chương 7 (sinh dữ liệu để augment lớp hiếm). Nó cùng với §6.7 tạo thành **cụm "2 hướng sinh dữ liệu lỗi"**: - **Hướng 1 (mục này) = Physics-based / fault injection** → DỰ'NG lỗi theo quy luật vật lý. - **Hướng 2 (§6.7 đối chiếu) = Generative** → HỌC để sinh lỗi (GAN/TimeGAN/Diffusion, đã học ở §5–§6).

Cả hai cùng giải quyết một deliverable của đề A1: **"bộ sinh dữ liệu lỗi"** — chỉ khác cách làm. Mục này đi sâu hướng thứ nhất, thứ mà cột công nghệ gợi ý của BTC mô tả là **"Mô phỏng vật lý – fault injection"**.

Mục tiêu: mục này xây một phương án **KHÔNG cần học từ dữ liệu** để tạo dữ liệu lỗi: ta **dựng chữ ký tín hiệu của từng loại lỗi cơ khí theo đúng quy luật vật lý**, rồi "bơm" chữ ký đó vào tín hiệu bình thường. Kết quả là một bộ sinh dữ liệu lỗi **chạy được trên CPU, trong vài giây, sinh ra hàng loạt mẫu lỗi có nhãn chính xác tuyệt đối (100% là lỗi)** — và quan trọng nhất, ta **điều khiển được CHÍNH XÁC CHẾ ĐỘ LỖI** mình muốn (hồng vòng ngoài, vòng trong, lệch trục, mất cân bằng...). Đây là điều mà chờ máy hồng thật gần như không thể.

Khác biệt cốt lõi với Hướng generative (§5–§6):

	Hướng generative (GAN/Diffusion)	Hướng physics (mục này)
Nguồn kiến thức	Học phân bố lỗi thật	Dựng theo công thức cơ học
Cần dữ liệu lỗi thật	Cần (dù ít, để fine-tune)	Không cần lỗi thật — chỉ cần hiểu vật lý
Độ đa dạng	Rất tốt (học được biến thể)	Trung bình (bám đúng chế độ đã mô hình hoá)
Độ chính xác nhãn	Nhãn do mình gán, có thể nhiễu	Tuyệt đối: mẫu nào cũng đúng lỗi
Kiểm soát chế độ lỗi	Khó (sinh ra "đại khái" giống lỗi)	Rất cao (chọn đúng BPFO/misalignment)
Tốc độ / chi phí	Trung bình–cao (train model)	Rẻ, tức thì
Nhược điểm lớn nhất	Dễ cho model học "vẹt" phân bố giả	Simulation gap: lỗi giả không trùng 100% lỗi thật

Bối cảnh / bài toán con: khi nào ta phải dùng hướng này? Trả lời bằng ba tình huống thực tế trong A1:

- Không có (hoặc quá ít) dữ liệu lỗi thật để generative học.** TimeGAN/Diffusion (§5.4, §6.4) cần một lượng tối thiểu mẫu lỗi để bắt "dáng" của lỗi. Nhưng đề A1 đặt đúng bài toán "scarcity": có thể team chỉ có vài chục mẫu lỗi, thậm chí zero. Với zero, generative **bất lực**; physics thì không cần bất kỳ mẫu lỗi nào. → Là lý do mục này được đặt *trước* §6.7 như "phương án cứu cánh khi chưa có gì".
- Muốn kiểm soát CHẾ ĐỘ LỖI.** Generative sinh ra "một dạng lỗi tổng quát". Nhưng khi thuyết trình trước giám khảo, ta muốn nói rõ: "tôi bơm đúng vòng ngoài (BPFO=81 Hz), tôi bơm lệch trục (2×RPM)". Physics cho phép **chỉ định** chế độ ngay từ tham số đầu vào — thứ rất hợp để vẽ "bảng sweep tham số" cho báo cáo.
- Muốn nhãn tuyệt đối chính xác từng mẫu.** Dữ liệu fault thật (IMS 90–100%) vẫn có thể lẫn normal–mới–bắt–đầu–xuống–cấp; generative sinh ra có thể "nửa lỗi nửa không". Còn mẫu bơm vật lý: **ta chủ động cộng xung lỗi vào → chắc chắn mang dấu lỗi** → nhãn không thể sai.

Tóm tắt bài toán con mà mục này giải: **"Khi không có lỗi thật để học và/hoặc muốn tạo lỗi đúng chế độ + nhãn sạch 100%, hãy bơm chữ ký lỗi theo công thức cơ học vào tín hiệu normal."**

6.6.1. Bản chất là gì (từ gốc): tín hiệu = healthy + fault signature + noise

Trước khi học AI, ta cần một mô hình tín hiệu đơn giản nhưng đủ dùng. Dữ liệu rung từ một cảm biến chưa bao giờ "sạch": nó luôn là **tổng** của nhiều thành phần. Hãy nghĩ mỗi thành phần như một "lớp" chồng lên nhau:

$$x_{\text{thông}}(t) = \underbrace{x_{\text{healthy}}(t)}_{\text{máy khỏe}} + \underbrace{x_{\text{fault}}(t)}_{\text{chữ ký lỗi}} + \underbrace{n(t)}_{\text{nhiều nền}}$$

- x_healthy(t)** — nhịp đập "bình thường" của máy: độ rung nền ổn định, chủ yếu là tần số quay vài hải. Đây chính là dữ liệu **normal** mà ta có RẤT nhiều (IMS 0–70%, code dùng làm train normal).
- x_fault(t)** — **chữ ký lỗi**: phần tín hiệu "thừa" do bệnh cơ khí sinh ra (xung lặp khi viên bi lăn qua vết xước, đỉnh 2×RPM khi lệch trục...). Phần này có **hình dạng rất đặc trưng** và lặp lại chu kỳ — chính là thứ ta sẽ *dựng*.
- n(t)** — nhiễu nền ngẫu nhiên (ma sát, điện tử, môi trường).

Trực quan quan trọng nhất: **một lỗi cơ khí không sinh ra "một cơn nhiễu loạn", mà sinh ra một MẪU DAO ĐỘNG lặp lại ở một tần số rất riêng**. Giống như nhịp tim và tiếng "tách" của van đóng mở: nó nhịp nhàng, và tần số của nó nói lên bệnh gì.

Ý tưởng của *fault injection* vì thế gọn như sau: **ta biết trước tần số nhấp nháy của từng bệnh (nhờ vật lý cơ khí), ta tạo ra đúng cái "nhịp tách" đó bằng một hàm toán học, rồi cộng vào tín hiệu normal**. Không cần bất kỳ mẫu lỗi thật nào để làm việc này.

Vì sao cách này nhanh, rẻ và kiểm soát được?

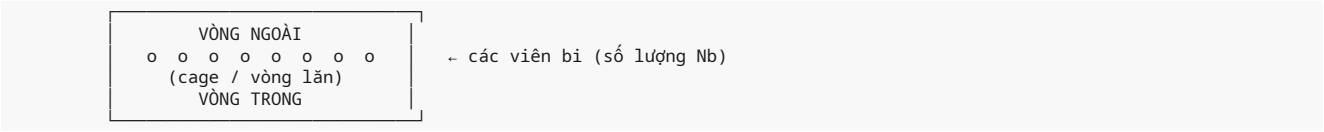
- **Nhanh/rẻ**: chỉ là cộng vài mẫu vào mảng numpy → chạy trong mili-giây trên CPU, không cần GPU, không cần train. Trái lại TimeGAN mất hàng chục phút.
- **Sinh được NHIỀU mẫu**: lấy 1000 window normal, mỗi window bơm một chế độ lỗi với tham số khác nhau → có CHỤC NGHÌN mẫu lỗi, nhiều gấp hàng nghìn lần vài mẫu lỗi thật.
- **Kiểm soát chế độ**: chỉ cần đổi số BPFO / amp / mod → sinh ra "chế độ lỗi" khác nhau. Muốn vòng ngoài thì BPFO, muốn lệch trục thì 2×RPM.
- **Đây chính là thứ "chờ máy hỏng thật" không thể làm**: máy hỏng thật là *biến cố ngẫu nhiên, tốn hàng tháng, đắt đỏ, và không ai muốn máy hỏng vì mình cần dữ liệu*. Bơm lỗi thì bấm nút là có.

Đừng hiểu lầm: nói "không cần học" không có nghĩa là **không cần hiểu**. Ngược lại, ta chỉ làm đúng khi hiểu cơ chế hỏng để chọn tham số chuẩn. Thứ "tiết kiệm" ở đây là **không cần mẫu lỗi thật**, chứ **hiểu vật lý thì bắt buộc**.

6.6.2. Cơ sở vật lý: chữ ký tần số của từng loại lỗi (có công thức)

Đây là "xương sống" của mục. Nếu bạn chưa từng học cơ khí, hãy đọc chậm — mỗi đoạn đi từ *cái gì* → *công thức* → *số cụ thể*.

Đặt ký hiệu chung: - f_r = tần số quay của trục (Hz) = $\text{RPM} / 60$. Ví dụ trục quay 1800 vòng/phút → $f_r = 30 \text{ Hz}$. - Một ổ bi (ball bearing) gồm: **vòng ngoài** (outer race), **vòng trong** (inner race), **vòng lăn** (cage, giữ khoảng cách các bi), và các **viên bi** (balls).
Hình:



Khi một bề mặt có vết khuyết (xước/pitting), mỗi lần viên bi lăn **qua vết** đó sẽ sinh ra một **va đập (impact)**. Các va đập này lặp lại với một chu kỳ rất ổn định, và tần số lặp đó được tính nhờ hình học của ổ bi. Có bốn "tần số đặc trưng" kinh điển (còn gọi là **Bearing Defect Frequencies** — bạn đã gặp ở Chương 2, mục §2.3.1):

Ký hiệu	Tên đầy đủ	Nghĩa
BPFO	Ball Pass Frequency Outer	Vết xước ở vòng ngoài → mỗi bi đập vào vết
BPFI	Ball Pass Frequency Inner	Vết xước ở vòng trong → mỗi bi đập vào vết
BSF	Ball Spin Frequency	Viên bi tự quay quanh trục của nó (vết ở ngay viên bi)
FTF	Fundamental Train Frequency	Vận tốc của vòng lăn/cage (vết ở cage)

Công thức (theo hình học ổ bi)

Đặt: N_b = số viên bi, d = đường kính viên bi, D = đường kính vòng lăn (pitch diameter), φ = góc tiếp xúc (contact angle), f_r = tần số quay trục.

$BPFO = (N_b / 2) \cdot f_r \cdot (1 - (d/D) \cdot \cos \varphi)$	# vòng ngoài (outer race)
$BPFI = (N_b / 2) \cdot f_r \cdot (1 + (d/D) \cdot \cos \varphi)$	# vòng trong (inner race)
$BSF = (D / 2d) \cdot f_r \cdot (1 - (d/D)^2 \cdot \cos^2 \varphi)$	# viên bi (ball spin)
$FTF = (f_r / 2) \cdot (1 - (d/D) \cdot \cos \varphi)$	# vòng lăn / cage

Đọc công thức để hiểu bản chất:

- $(N_b/2)$ và $(1 \pm (d/D) \cdot \cos \varphi)$: ổ bi càng nhiều bi, trục càng quay nhanh → bi càng thường xuyên đập vào đúng chỗ khuyết → tần số càng cao.
- $(d/D) \cdot \cos \varphi$ là hiệu ứng "bán kính": đường kính viên bi so với đường kính vòng lăn quyết định tỉ lệ mà viên bi "trượt-quay" trên mỗi vòng của trục. Dấu + cho BPFI (vòng trong quay nhanh hơn nên bi đập đúng chỗ khuyết nhiều hơn) và dấu - cho BPFO.

BPFO (vòng ngoài đứng yên, bi đập ít hơn) — đó là lý do **BPFI > BPFO** khi cùng tham số.

- Nói cách khác: **BPFO < BPFI**, và cả hai đều là bội số-thập-phân (không nguyên) của f_r . Chính đặc điểm "không nguyên" này khiến các tần số này **không trùng** với hài của tần số quay → dễ phân biệt trên phổ.

Ví dụ số cụ thể

Chọn ổ bi điển hình (giá trị tham khảo như BTC gợi ý): $N_b = 9$, $d/D = 0.4$, $\varphi = 0^\circ$ (góc 0 làm $\cos\varphi = 1$, gọn nhất), trục quay $f_r = 30$ Hz (1800 RPM):

$$\begin{aligned} \text{BPFO} &= (9/2) \cdot 30 \cdot (1 - 0.4 \cdot 1) &= 4.5 \cdot 30 \cdot 0.6 &= 81.0 \text{ Hz} \\ \text{BPFI} &= (9/2) \cdot 30 \cdot (1 + 0.4 \cdot 1) &= 4.5 \cdot 30 \cdot 1.4 &= 189.0 \text{ Hz} \\ \text{BSF} &= (1/(2 \cdot 0.4)) \cdot 30 \cdot (1 - 0.4^2 \cdot 1) &= 1.25 \cdot 30 \cdot 0.84 &= 31.5 \text{ Hz} \\ \text{FTF} &= (30/2) \cdot (1 - 0.4 \cdot 1) &= 15 \cdot 0.6 &= 9.0 \text{ Hz} \end{aligned}$$

Và **hài bậc hai** của BPFO: $2 \cdot \text{BPFO} = 162$ Hz, $3 \cdot \text{BPFO} = 243$ Hz — rất hay xuất hiện và giúp xác nhận lỗi (xem §6.6.7).

Giá trị thực tế khác: nếu bạn dùng ổ SKF-6205 của bộ dữ liệu CWRU ($N_b=9$, $d=0.3125$ ", $D=1.537$ " → $d/D \approx 0.203$, $\varphi=0$), thì với $f_r=30$ Hz: $\text{BPFO} \approx (9/2) \cdot 30 \cdot (1 - 0.203) \approx 107.5$ Hz; $\text{BPFI} \approx (9/2) \cdot 30 \cdot (1 + 0.203) \approx 162.4$ Hz. Số liệu này hữu ích **khi bạn bơm lỗi vào đúng dữ liệu CWRU**. Việc chọn d/D nào chỉ quyết định *con số* kết quả, còn *cấu trúc công thức* và *bản chất "đổi đỉnh tại BPFO"* thì không đổi.

(a) Mất cân bằng (unbalance) — chữ ký ở 1×RPM

Nguyên nhân: khối lượng không phân bố đối xứng quanh trục quay (bụi bắn bám lệch, trục không đối xứng). Mỗi vòng quay, điểm nặng "nện" một lần → lực ly tâm đổi chiều đúng f_r lần/giây.

Chữ ký: một đỉnh duy nhất rất lớn ở đúng f_r (1×RPM), thường kèm hài yếu 2×, 3×. Biên độ tỉ lệ với bình phương tốc độ ($\propto f_r^2$) — nên máy càng nhanh càng thấy rõ.

$$x_{\text{fault}}(t) = A_u \cdot \sin(2\pi \cdot f_r \cdot t + \varphi_0)$$

(b) Lệch trục (misalignment) — chữ ký ở 2×RPM (và 1×, 3×)

Nguyên nhân: trục nối không đồng tâm (song song lệch hoặc góc lệch) → mỗi vòng quay trục bị "uốn" hai lần (lên-xuống và lặn-bên kia) → **hai** va đập mỗi vòng.

Chữ ký: đỉnh nổi bật ở $2 \cdot f_r$ (2×RPM), có thể có cả $1 \cdot f_r$ và $3 \cdot f_r$. Đây là dấu hiệu phân biệt với unbalance (1×): nếu 2× hơn hẳn 1× → nghi ngờ **lệch trục**, còn 1× hơn hẳn 2× → nghi ngờ **mất cân bằng**.

$$x_{\text{fault}}(t) = A_{m1} \cdot \sin(2\pi \cdot f_r \cdot t) + A_{m2} \cdot \sin(2\pi \cdot 2 \cdot f_r \cdot t) + A_{m3} \cdot \sin(2\pi \cdot 3 \cdot f_r \cdot t)$$

(c) Mòn lan tỏa / độ nhám (diffuse wear) — năng lượng dải tần cao

Nguyên nhân: bề mặt bị mòn đều, không còn bén; ma sát tăng, sinh rung "rè" nhiều tần số.

Chữ ký: không phải một đỉnh đơn, mà là **"mặt nâng" (noise floor) dải cao tăng** → `spec_centroid` (tâm phổ) **nhích lên**, `spec_spread` tăng, và `spec_flatness` **tăng** (phổ trở nên "phẳng" hơn vì nhiều broadband). Đây là lý do `src/features.py` có sẵn `spec_centroid`, `spec_flatness`, `spec_spread`: chính là để bắt loại lỗi này. Khi lỗi nặng, năng lượng còn **dịch lên quanh tần số cộng hưởng riêng** của cấu trúc (tạo cụm sideband quanh tần số tự nhiên).

(d) Vòng trong bị xước (inner race) — điều chế (modulation)

Chữ ký đặc biệt: xung lặp ở BPFI, nhưng **biên độ bị chính trục quay "đập" theo chu kỳ** → tạo **sideband** (dải bên) quanh BPFI.

Vì sao có điều chế? Vết xước nằm trên vòng trong — mà vòng trong **quay** cùng trục. Khu vực chịu tải chính (load zone) thì đứng yên về phía (thường là hướng trọng lực). Khi vết xước quay **vào** vùng tải → va đập mạnh; khi quay **ra** khỏi vùng tải → va đập yếu. Vậy biên độ xung thay đổi đúng f_r lần/giây.

$$x_{\text{fault}}(t) = [1 + m \cdot \cos(2\pi \cdot f_r \cdot t)] \cdot \sum_k h(t - k \cdot T_{\text{BPFI}}) \quad \# m = \text{modulation index}$$

Toán học nói rằng nhân một sóng mang với $(1 + m \cdot \cos(2\pi \cdot f_r \cdot t))$ sinh ra **hai tần số mới** quanh sóng mang:

$$f_{\text{sideband}} = f_{\text{carrier}} \pm f_{\text{mod}} \rightarrow \text{BPFI} \pm f_r \quad (\text{ví dụ: } 189 \pm 30 = 159 \text{ và } 219 \text{ Hz})$$

Tương tự, bất kỳ lỗi nào làm "quấn" biên độ theo chu kỳ f_r đều tạo cặp sideband $f_{\text{carrier}} \pm f_r$ quanh tần số chính. Đây là "ngôn ngữ" phổ quan trọng nhất trong bảo trì dự đoán — giám khảo rất dễ hỏi vì sao có đỉnh kép quanh BPFI.

Tổng kết bảng chữ ký (dùng để điền "bảng sweep" trong báo cáo):

Loại lỗi	Đỉnh chính	Điều chế / sideband	Feature bị đẩy lên
Mất cân bằng	$1 \cdot f_r$	không	RMS, năng lượng dải thấp
Lệch trục	$2 \cdot f_r$ (+1×, 3×)	không	RMS, năng lượng dải thấp-trung
Vòng ngoài (BPFO)	BPFO, 2×BPFO, 3×BPFO	ít	spec_energy, kurtosis, peak
Vòng trong (BPFI)	BPFI, 2×BPFI	$BPFI \pm f_r$	spec_energy, kurtosis, peak, spec_centroid
Viên bi (BSF)	BSF, 2×BSF	$BSF \pm FTF$	kurtosis, spec_energy
Cage (FTF)	FTF (thấp)	khó thấy	biến thiên chậm
Mòn lan tỏa	cụm cao-tần	"mặt năng"	spec_flatness ↑, spec_centroid ↑, spec_spread ↑

Vì sao xuất hiện đỉnh tại f , $2f$, và sideband (điều chế)? — Một chuỗi xung tuần hoàn bất kỳ, khi phân tích Fourier, luôn bung ra **chuỗi hài** ở f , $2f$, $3f$, ... với biên độ giảm dần. Thêm vào đó, xung va đập còn "kích thích" tần số cộng hưởng cơ cấu (thường vài kHz) → các cụm năng lượng quanh tần số cộng hưởng. Còn **sideband** đến từ điều chế biên độ như giải thích ở trên. Khi đọc phổ, ta tìm đúng dấu hiệu này để xác định chế độ lỗi.

6.6.3. Toán học của việc "bơm": mô hình tín hiệu + ví dụ số

Bây giờ biến ý tưởng thành công thức tính được. Mục tiêu: từ một window $x_h(t)$ (normal, thật) → sinh $x_f(t)$ (lỗi). Có hai "chiều" làm và đều nên hiểu:

Cách 1 — Bơm ở miền TÍN HIỆU (rung thô) [khuyến khích]

Đây là cách "huấn luyện trung thực" nhất: sửa tín hiệu rồi để pipeline trích feature. Mô hình chuẩn của lỗi ổ trục dạng va đập:

$$x_f(t) = x_h(t) + A \cdot \underbrace{\sum_k h(t - k \cdot T)}_{\text{xung tuần hoàn}} + \underbrace{n'(t)}_{\text{nhiều điều khiển}}$$

với: $T = 1 / f_{\text{defect}}$ — khoảng cách giữa hai va đập (s). Ví dụ BPFO = 81 Hz → $T = 1/81 \approx 0.01235$ s. - A — biên độ va đập (điều khiển mức độ nặng của lỗi). - $h(u)$ — **dạng xung**: một dao động tắt dần, ở tần số cộng hưởng f_n :

$$h(u) = e^{(-u/\tau)} \cdot \sin(2\pi \cdot f_n \cdot u) \quad \text{với } u \geq 0, \text{ còn lại} = 0$$

- τ — hằng số thời gian tắt dần (quyết định xung "đanh" hay "bè").
- f_n — tần số cộng hưởng của cấu trúc (vài kHz).
- $n'(t)$ — nhiễu trắng có **SNR điều khiển** (thêm để mẫu không giống hệt nhau, và mô phỏng máy thật nhiễu nền).

Khi muốn **điều chế** (vòng trong/BPFI), nhân thêm lớp biên độ:

$$x_f(t) = x_h(t) + A \cdot \underbrace{[1 + m \cdot \cos(2\pi \cdot f_r \cdot t)]}_m \cdot \sum_k h(t - k \cdot T)$$

m = modulation index (0 → 1)

m càng lớn → càng điều chế sâu → sideband $BPFI \pm f_r$ càng rõ.

Tóm tắt 4 nút tham số quan trọng: A (mức nặng lỗi), $T=1/f_{\text{defect}}$ (loại lỗi), m (điều chế, cho vòng trong), SNR (độ sạch của mẫu). Bảng sweep trong báo cáo sẽ quét đúng 4 nút này.

Ví dụ số cụ thể (theo số liệu mục 6.6.2)

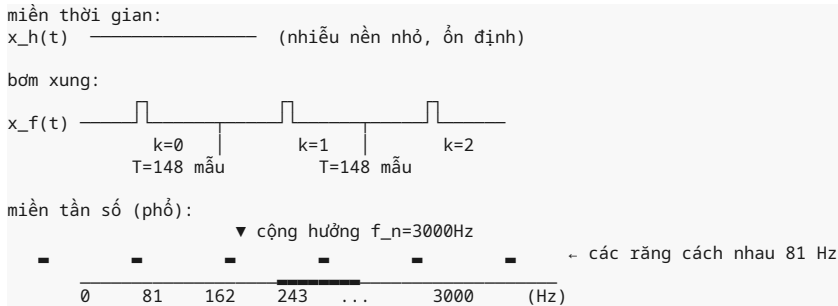
Chọn lỗi **vòng ngoài**, BPFO = 81 Hz, $f_n = 3000$ Hz, $\tau = 0.5$ ms, $f_s = 12000$ Hz, window win = 512 mẫu:

$$\begin{aligned} T &= 1 / 81 && \approx 0.012346 \text{ s} \\ \text{samples / chu kỳ} &= f_s \cdot T = 12000 \cdot 0.012346 && \approx 148 \text{ mẫu} \\ \text{số xung trong 512 mẫu} &= 512 / 148 && \approx 3.46 \text{ xung } (\approx 3\text{-}4 \text{ va đập trong window}) \end{aligned}$$

$$u \text{ (tính bằng mẫu, mỗi xung kéo dài 10 ms): } h(u) = e^{(-u/0.0005)} \cdot \sin(2\pi \cdot 3000 \cdot u)$$

Hiện tượng: mỗi 148 mẫu lại "nện" một cú tắt dần. Trên phổ, energy tập trung thành cụm quanh 3000 Hz (cộng hưởng) với **các răng cách nhau đúng 81 Hz** — chính là dấu vân tay BPFO. Nếu thêm $m = 0.5$ (giả vòng trong): xuất hiện cặp răng ở $BPFI \pm f_r$ = $189 \pm 30 = 159$ Hz & 219 Hz.

Sơ đồ ý tưởng (ASCII):



Cách 2 — Bơm ở miền FEATURE (đã có 16 feature)

Nhanh hơn nữa: lấy window normal **đã trích** 16 feature (khớp `src/features.py`), rồi **đẩy đúng các feature** phản ánh chữ ký lỗi. Ví dụ với lỗi va đập:

```
kurtosis += Δkurt      (xung → độ nhọn tăng)
crest_factor += Δcrest (peak / RMS tăng → "đỉnh" nổi lên)
spec_energy *= (1 + ΔE) (năng lượng phổ tăng)
spec_centroid += Δfc   (năng lượng dồn lên tần số cao)
```

Nhược điểm của cách 2: ta tự tay co giãn feature nên dễ "sai vật lý" (vd đẩy `spec_energy` lên nhưng quên đẩy tỉ lệ đúng với `kurtosis`). Model có thể học một "cụm dị thường phi tuyến" không tương ứng kết cấu lỗi thật. **Khuyến nghị:** ưu tiên **Cách 1** (bơm tín hiệu → pipeline tự trích feature) vì nó bảo toàn mối tương quan vật lý giữa các feature; Cách 2 chỉ dùng khi muốn sinh cực nhanh và không cần độ thật cao. Team đang đi theo hướng **feature-based** (chỗ `--gen npy` đọc raw features cho `src/features.py`) — nên việc chọn Cách 1 là hợp lý: nó cho ra raw features chuẩn hoá bằng đúng scaler của train.

6.6.4. Áp dụng trong code/project của team

Hiện `scripts/02_compare_augmentation.py` chỉ có `--gen heuristic|npv`:

- `heuristic` = bóp biên độ $2.5\times$ + nhiễu (rất thô, chỉ để chạy thử — xem `_heuristic_faults`).
- `npv` = đọc lỗi giả từ TimeGAN (`--gen-npy results/synthetic_faults.npy`).

Đề xuất thêm `--gen physics` — một nguồn lỗi giả **thứ ba**, mạnh hơn `heuristic` vì bám đúng vật lý (xung BPFO/BPFI, sideband, lệch trục). Kết quả: script so sánh **3 bộ sinh** (`heuristic / physics / npv=TimeGAN`) → bảng "2 bộ sinh, cái nào tốt hơn" mà SPEC yêu cầu. Bổ sung vào docstring khối chọn gen:

```
if args.gen == "heuristic":
    X_syn = _heuristic_faults(Xn_train, rng, n_rows=6 * n_fault)
elif args.gen == "physics":
    # (mới) bơm lỗi vào TÍN HIỆU normal rồi mới trích feature → giữ đúng tương quan vật lý
    X_syn = _physics_faults(Xn_train, rng, d, n_rows=6 * n_fault)
    # X_syn là RAW features (16 cột) → z-score bằng scaler train (như nhánh npv)
    X_syn = ft.zscore(X_syn, d["scaler_mu"], d["scaler_sd"])
```

Dữ liệu nguồn: tín hiệu normal lấy từ **IMS 0–70% đầu** (= vùng khỏe mà `pipe.split_and_scale` đã cắt cho `Xn_train`). Mẹo: vì `Xn_train` đã là **feature**, để bơm ở miền tín hiệu ta cần lấy lại **window raw normal** — cụ thể: (i) dùng `windows_to_features` chiều ngược? Không — đơn giản nhất là load raw một lần nữa trong `_physics_faults` (gọi `ds.make_windows` trên chính `runs_normal` train), hoặc (ii) giữ một bản `windows_train` song song ở bước split. Khi gộp vào `pipeline.split_and_scale`, hãy trả thêm `d["windows_normal_train"]` để script 02 dùng lại.

Nguyên tắc bất biến (bắt buộc giữ):

- Sinh lỗi giả **CHỈ trong train** — chống leakage. Test (cả normal-khỏe 0.70–0.90 và fault thật 90–100%) **không bao giờ** dùng mẫu bơm hay scaler riêng.
- Tuân thủ **3-zone split** trong `src/pipeline.py`: train normal = `[0, 0.70]` của vùng NORMAL mỗi run; test normal = `[0.70, 0.90]`; grey = `[0.90, 1.0]` → loại. Fault thật thì **20/80** (train/test). Z-score fit **trên train** (`fit_zscore`) rồi áp cho mọi thứ.
- Khi thêm `--gen physics`, mẫu sinh cũng phải được **z-score bằng** `d["scaler_mu"] / d["scaler_sd"]` (giống nhánh `npv`), tuyệt đối không fit scaler riêng.

Bảng "sweep" cho báo cáo (điền con số sau khi chạy):

Tham số	Giá trị quét	Ý nghĩa/ghi chú
A (biên độ)	0.2 / 0.5 / 1.0 / 2.0	lỗi nhẹ → nặng

Tham số	Giá trị quét	Ý nghĩa/ghi chú
f_defect	BPFO / BPFI / 2×RPM	chọn loại lỗi
m (modulation)	0 / 0.3 / 0.6	0 = không điều chế (vòng ngoài), >0 = vòng trong
SNR (dB)	20 / 40 / ∞	độ sạch của mẫu sinh
n_rows	2× / 4× / 6× số lỗi thật	tỉ lệ tăng cường

6.6.5. Ví dụ mã Python (giải thuật bơm lỗi)

Chỉ trình bày **giải thuật**, không yêu cầu chạy. Hàm nhận tín hiệu khỏe + tham số → trả tín hiệu có lỗi:

```
import numpy as np

def damped_impulse(u, fn=3000.0, tau=0.0005):
    """Dạng xung và đập tắt dần: h(u)=exp(-u/tau)*sin(2*pi*fn*u), u>=0."""
    u = np.maximum(u, 0.0)
    return np.exp(-u / tau) * np.sin(2 * np.pi * fn * u)

def inject_bearing_fault(sig, fs, f_defect, amp=0.5, fn=3000.0, tau=0.0005,
                        m=0.0, f_rot=0.0, noise_snr=None, rng=None):
    """Bơm chữ ký lỗi ổ trực vào tín hiệu normal `sig`."""

    sig : (n,) tín hiệu healthy (window bình thường).
    f_defect : tần số va đập, Hz (BPFO/BPFI/BSF – xem 6.6.2).
    amp : biên độ va đập `A`.
    fn/tau : tần số cộng hưởng & hằng số tắt dần của xung h(u).
    m : modulation index; >0 khi mô phỏng vòng trong / điều chế.
    f_rot : f_r (Hz), cần khi m>0 để tạo sideband BPFI±f_r.
    noise_snr: nếu là số (dB) thì thêm nhiễu trắng với SNR này; None = không thêm.
    """
    n = len(sig)
    t = np.arange(n) / fs
    x_f = sig.astype(float).copy()

    T = 1.0 / f_defect # chu kỳ giữa các va đập (s)
    period_samples = int(round(T * fs)) # số mẫu mỗi chu kỳ
    impulse_len = int(round(0.010 * fs)) # xung kéo dài 10 ms

    k_starts = np.arange(0, max(1, n - 1), period_samples) # các mốc bắt đầu xung
    for k0 in k_starts:
        L = min(impulse_len, n - k0)
        if L <= 0:
            continue
        u = np.arange(L) / fs
        h = amp * damped_impulse(u, fn, tau) # xung thô
        if m > 0.0: # điều chế biên độ theo f_rot
            t_local = (k0 / fs) + u
            h *= 1.0 + m * np.cos(2 * np.pi * f_rot * t_local)
        x_f[k0:k0 + L] += h # cộng chữ ký vào tín hiệu

    if noise_snr is not None and rng is not None:
        # thêm nhiễu trắng có SNR điều khiển: P_sig/P_noise = 10^(SNR/10)
        P_sig = np.mean(x_f ** 2)
        P_noise = P_sig / (10 ** (noise_snr / 10))
        x_f += rng.normal(0, np.sqrt(P_noise), n)
    return x_f

# Ví dụ dùng trong pipeline train (chỉ train – TÁCH khỏi test):
# windows_healthy = <raw windows normal, lấy từ vùng [0,0.70] mỗi run>
# for sig in windows_healthy:
#     sig_fault = inject_bearing_fault(sig, fs=FS, f_defect=BPFO, amp=0.5,
#                                     fn=3000, tau=0.0005, m=0.0)
#     feats = ft.raw_features(make_windows(sig_fault, win, stride), fs=FS)
```

Một số điểm kỹ thuật trong code, hãy tự giải thích khi thuyết trình:

- **Bơm vào tín hiệu rồi MỚI trích feature** (`ft.raw_features`) — không tự co feature, nên mối quan hệ vật lý giữa các feature được bảo toàn.
- **Tham số m mô phỏng vòng trong/sideband** — đây là điểm "ăn tiền" vì chứng tỏ bạn hiểu điều chế (mục 6.6.2d).
- **Inspect chéo:** sau khi sinh, hãy in `spec_energy`, `kurtosis`, `spec_centroid` của mẫu bơm và so với mẫu normal → xác nhận feature thực sự dịch chuyển đúng hướng.

6.6.6. Ưu / nhược + so sánh nội bộ

Ưu điểm:

1. **Nhanh + rẻ:** mili-giây trên CPU, không cần GPU, không cần huấn luyện.
2. **Kiểm soát chế độ lỗi chính xác:** chỉ định BPFO/BPFI/lịch trực/1× qua tham số → dễ minh họa, dễ thuyết trình, dễ lập "bảng sweep".
3. **Nhấn tuyệt đối:** mẫu bơm nào cũng chắc chắn là lỗi, không "nửa normal nửa lỗi".
4. **Sinh hàng loạt:** lấy N window normal, bơm N lần với tham số khác nhau → chục nghìn mẫu.
5. **Hiểu cơ chế → thuyết trình mạnh:** bạn nhìn phổ tự chỉ ra đỉnh BPFO/sideband — giám khảo ấn tượng vì bạn *hiểu lỗi*, không chỉ chạy code.

Nhược điểm (phải trung thực):

1. **Simulation gap:** mô hình $h(u)$ của bạn chỉ *xấp xỉ* lỗi thật. Lỗi thật còn chứa phi tuyến (ma sát, biến dạng đàn hồi, tải trọng) mà bạn không mô hình hoá hết được → mẫu sinh **không trùng 100%** lỗi thật, đôi khi lệch khá xa.
2. **Model học quá khớp vào mô phỏng:** nếu classifier chỉ thấy lỗi giả "quá sạch, quá đều", nó có thể **overfit** quy luật mô phỏng của ta → khi chạm lỗi thật lộn xộn ở test lại kém. Đây chính là rủi ro ngược với mục đích.
3. **Cần hiểu vật lý:** chọn sai d/D , N_b , τ , f_n → tần số/chữ ký sai → bơm ra "lỗi" không khớp thực tế. Sai ở bước này là sai nền tảng.
4. **Đa dạng hạn chế hơn generative:** bạn mô phỏng được *các chế độ đã biết*, khó tạo *biến thể mới mẻ* mà mình chưa nghĩ ra.

Vì sao vẫn cần generative (+ dữ liệu thật) để "chốt"?

Physics cho ta lượng **nhưng có thể thiếu chất** (quá sạch, thiên lệch). Generative/Dữ liệu thật cho chất. Chiến lược đúng là kết hợp:

Bơm vật lý → sinh NHIỀU mẫu (về lượng) → mở rộng "biên" của class lỗi
 Dữ liệu thật/TimeGAN → neo "chất": giữ phân phối gần bản chất lỗi thật
 Huấn luyện: normal + lỗi thật (20%) + lỗi bơm + lỗi sinh → đánh giá trên lỗi THẬT

Trong A1, ta dùng physics làm **cột đôi chiều** ("cái nào tốt hơn" giữa bơm và generative), còn **TimeGAN làm "nhân vật chính"** (đúng keyword đề, có số liệu end-to-end). Physics + TimeGAN là bộ đôi: **số lượng + chất lượng**.

6.6.7. Bẫy / lưu ý thực hành (rất dễ mất điểm)

1. **KHÔNG bơm lỗi vào trong TEST.** Test chỉ dùng **lỗi thật chưa thấy** + normal khỏe. Mọi augmentation, scaling, chọn ngưỡng, chọn feature đều nằm trong **train**. Bơm vào test = data leakage = điểm hồng.
2. **Simulation gap — đừng coi mẫu bơm là "fault thật" để đánh giá.** Mẫu bơm chỉ để *train*, không bao giờ để *test*. Nếu bạn đo AUC trên lỗi bơm, kết quả sẽ "ảo" (quá đẹp) vì mẫu do chính bạn sinh.
3. **Lỗi tần số Naive — dùng SAI f_s .** Tần số đặc trưng tính bằng Hz, nhưng khi chuyển sang "số mẫu mỗi chu kỳ" bạn dùng $T_{in_samples} = f_s / f_{defect}$. Nếu f_s sai, bạn bơm xung sai tần số → chữ ký không còn là BPFO nữa. **Đặc biệt:** `src/pipeline.py` mặc định $F_s = 12000$ (khớp CWRU 12 kHz dưới tải), nhưng **IMS thật là 20 kHz**. Nếu team dùng `--dataset ims`, hãy chỉnh $f_s=20000$ trước khi bơm và trích feature, nếu không con số BPFO sẽ lệch.
4. **Xác minh phổ (FFT) của lỗi giả có đúng đỉnh BPFO không.** Trước khi đưa vào train, phải vẽ/check phổ: có rằng tại đúng BPFO, $2 \cdot BPFO$... không? Có sideband $BPFI \pm f_r$ (nếu khai báo vòng trong)? Một hàm `_assert_spectrum(x_f, f_s, f_defect)` so sánh đỉnh tìm được với f_{defect} (sai số vài Hz) là đáng có. Sai sót ở đây phá toàn bộ thí nghiệm mà ta không nhận ra.
5. **Cửa sổ quá ngắn → đỉnh phổ mờ.** $win=512$, $f_s=12000$ → độ phân giải tần số $f_s/win = 23.4$ Hz/bin. BPFO=81 Hz nằm giữa bin 3 (70.3 Hz) và bin 4 (93.8 Hz) → đỉnh bị "loang" mờ, khó tách. Muốn thấy rõ BPFO hãy dùng window dài hơn (vd 2048 mẫu → ~5.9 Hz/bin), hoặc tính phổ trên window dài rồi mới lấy feature (chú ý bơm cả khi cắt window).
6. **Đừng để model học nền nhiễu mô phỏng mà không có ở thực tế.** Nếu bạn cộng `noise_snr` kiểu gaussian vào mẫu bơm, mà lỗi thật không có nhiễu kiểu đó, model có thể học "dấu hiệu là có gaussian" thay vì "dấu hiệu là lỗi". Hãy để mức nhiễu của mẫu bơm **thấp & giống mức nhiễu của normal thật**, hoặc thêm nhiễu từ một bản normal thật (bootstrap nhiễu) thay vì gaussian thuần.
7. **Nhấn mẫu bơm đều là 1 (fault) — đừng quên $y_{syn} = np.ones(...)$,** và đừng đưa nhầm mẫu normal vào mảng `synthetic`.

6.6.8. Kết nối với phần còn lại của giáo trình

- **Chương 2 — đặc trưng miền tần số (mục §2.3.1, §2.3.6):** nơi giới thiệu khái niệm **BPFO/BPFI/BSF/FTF** và "đỉnh ở $1\times, 2\times$ "; mục này mở rộng thành **công thức tính + hướng bơm chữ ký**. Hãy đọc lại §2.3 để nhớ tần số đặc trưng là gì trước khi vào

- đây. `src/features.py` (16 feature, có `spec_centroid`, `spec_flatness`, `spec_energy` ...) chính là "cầu nối" để chuyển từ đồ thị vật lý sang số liệu model.
- **§6.5 — so sánh GAN vs Diffusion:** đây là "anh em" của mục này — cả hai đều trả lời *làm gì khi thiếu lỗi*. §6.5 bàn về hai bộ sinh generative; §6.6 bổ sung hướng **vật lý** để có cái nhìn toàn diện (generative + physics) trước khi sang Chương 7.
 - **§6.7 (kế tiếp) — hướng generative:** mục này cùng §6.7 tạo cụm "2 hướng sinh dữ liệu lỗi". Nếu §6.6 là "dựng lỗi theo công thức", thì §6.7 là "học để sinh lỗi". Trong Chương 7, hai hướng sẽ được **hợp nhất** vào một pipeline augment + đánh giá.
 - **Cây bài toán con của Phần 3 (đầu chương, nhánh "Sinh dữ liệu lỗi giả"):** nhánh đó đang liệt kê GAN/TimeGAN/Diffusion; hãy **bổ sung một nhánh con** cho "Physics-based / fault injection" để cây phản ánh đúng 2 hướng: (a) dựng lỗi theo vật lý (§6.6) và (b) học phân bố để sinh lỗi (§5–§6, §6.7).
 - **SPEC.md / sheet của BTC:** cột "**Mô phỏng vật lý – fault injection**" trong "CÔNG NGHỆ GỢI Ý" — đây chính là cột mà mục này cover. Khi nộp, đây là một điểm "đủ bộ": bạn có cả **physics** (mục này) lẫn **generative** (§5–§6) để đối chiếu, thay vì chỉ lo một đường.

6.7. Đối chiếu hai hướng sinh dữ liệu lỗi (vật lý vs generative) & cột "công nghệ gợi ý"

6.7.1. Mục tiêu

Mục tiêu: sau khi đã học xong Hướng 1 (mô phỏng vật lý / bơm lỗi) ở mục 6.6 và Hướng 2 (generative: GAN/TimeGAN/Diffusion) ở mục 5–6, mục này chốt **khi nào chọn hướng nào** và **vì sao team nên đi CẢ HAI nhưng lệch ưu tiên** (khoảng 60% generative, 20% vật lý, 0% diffusion ngay lúc này). Đây là "nút thắt" vì hai hướng **không đối nghịch** mà **bổ sung** cho nhau; hiểu sai ở đây sẽ dẫn tới hai lỗi ngược nhau: hoặc "bỏ mất độ thật" (chỉ chăm mô phỏng vật lý) hoặc "không kiểm soát được câu chuyện" (chỉ chăm generative).

6.7.2. Bối cảnh / bài toán con

Độc giả vừa đi hết hai con đường dài:

- **Hướng 1 (vật lý / fault injection):** hiểu cơ chế hỏng (BPFO/BPFI/BSF/FTF, lệch trục, sideband, mài mòn lan tỏa) → "bơm" lỗi theo công thức vật lý vào tín hiệu normal. Trong repo hiện tại chỉ mới là heuristic (`scripts/02 --gen heuristic` : nhân biên độ 2.5 + nhiễu Gaussian), chưa phải vật lý đúng nghĩa.
- **Hướng 2 (generative):** học phân bố lỗi thật (dù chỉ vài mẫu) rồi sinh lỗi giả giống thật. TimeGAN (chạy được trên CPU, `scripts/03` → `results/synthetic_faults.npy`), Diffusion (cần GPU, chỉ là ghi chú lý thuyết).

Vấn đề là **cả hai đều trả lời cùng một câu hỏi** — "làm sao có thêm lỗi giả để train supervised?" — nên dễ nhầm là "chọn 1 trong 2". Mục này làm rõ chúng dùng cho **hai vai trò khác nhau** trong cùng một pipeline: một bên cho **độ bao phủ + kiểm soát + kể chuyện**, một bên cho **độ bám phân bố thật**. Đồng thời mục này đối chiếu với đúng cột "**CÔNG NGHỆ GỢI Ý**" của BTC: `Mô phỏng vật lý - GAN/TimeGAN - Diffusion - fault injection - PyTorch` — nó **gợi ý cả hai cụm** chứ không chọn cụm nào, nên đi một mình là "lệch khỏi kỳ vọng của đề".

Để "cái nhìn tổng" rõ ràng, hãy so sánh ngay **đầu vào – cơ chế – đầu ra** của hai hướng:

Hướng	Đầu vào	Cơ chế	Đầu ra (dữ liệu lỗi giả)
Vật lý	Cấu hình máy (số bi \$n\$, đường kính bi \$d\$, đường kính vòng \$D\$, góc tiếp xúc \$\phi\$, tần số trục \$f_r\$) + 1 tín hiệu normal	Công thức tần số đặc trưng → điều chế/nhân xung rồi cộng chồng vào tín hiệu khỏe	Lỗi giả có nhãn chế độ (BPFO nặng độ 3, lệch trục 2×...) và đúng tần số mặc dù có simulation gap
Generative	1 vài mẫu lỗi thật (vài chục window)	GAN/TimeGAN/Diffusion học phân bố rồi lấy mẫu (sample)	Lỗi giả giống phân bố lỗi thật nhưng không đặt tên được chế độ , đôi khi trùng lặp (mode collapse)

Cách nhớ nhanh: **vật lý trả lời "lỗi này XẢY RA THẾ NÀO"**, còn **generative trả lời "lỗi này TRÔNG RA làm sao"**. Hai câu hỏi khác nhau → hai vai trò khác nhau, không loại trừ.

6.7.3. Bảng so sánh tổng hợp hai hướng

Tiêu chí	Hướng 1 — Vật lý / fault injection	Hướng 2 — Generative (GAN/TimeGAN/Diffusion)
Cơ chế	Dựng theo công thức vật lý cơ chế hỏng → bơm vào tín hiệu normal	Học phân bố lỗi thật → sinh mẫu mới theo phân bố đó
Cần dữ liệu lỗi thật nhiều?	Cực ít — có thể 0 mẫu lỗi (chỉ cần hiểu cơ chế + cấu hình máy)	Từ ít đến vừa — cần vài chục/trăm mẫu lỗi để học phân bố (subset > 0)
Chất lượng bám bản chất lỗi	Đúng cơ chế tần số (đỉnh BPFO/BPFI đúng, sideband đúng) nhưng bỏ sót hiện tượng "không nằm trong mô hình"	Bám phân bố tổng thể của lỗi thật, kể cả hiện tượng khó mô tả bằng công thức
Độ "thật" so với lỗi thực (simulation gap)	Tồn tại gap — lỗi mô phỏng không hết lỗi thực (nhiều nền, chế độ chạy, già hóa phụ thuộc máy)	Nhỏ hơn — học chính từ lỗi thật nên trung thực với dữ liệu thực hơn; nhưng vẫn có "mode collapse" (GAN)
Tốc độ / chi phí	Rất nhanh, rẻ — vài dòng công thức, chạy CPU tức thì	Chậm hơn — train model, iteration, tốn thời gian điều chỉnh
Yêu cầu tài nguyên (GPU?)	Không cần — thuần CPU/numpy	TimeGAN: chạy được trên CPU (nhưng chậm). Diffusion: bắt buộc GPU
Mức độ kiểm soát chế độ lỗi	Tuyệt đối — biết trước mẫu này là BPFO hay lệch trục, bơm đúng khẩu phần	Thấp — model tự sinh, khó chỉ định "tạo đúng kiểu lỗi X"
Nhãn chính xác từng mẫu	100% đúng — mẫu nào cũng mang nhãn lỗi + tên chế độ rõ ràng	Đúng về nhãn "lỗi" nhưng mẫu sinh có thể "rác" (mode collapse, trùng lặp, không thực sự là lỗi hữu ích)
Rủi ro chính	Simulation gap (lỗi giả quá "sạch"/"đúng mô hình" → model học nhầm)	Mode collapse (GAN sinh lặp một kiểu) / overfit vào vài mẫu lỗi thật / data leakage nếu không cô lập test
Phù hợp khi nào	Khi hiểu rõ cơ chế hỏng, muốn kiểm soát + kể chuyện + nhãn sạch + rẻ	Khi có một ít lỗi thật và muốn bám phân bố thật , linh hoạt với chế độ lỗi khó mô tả

6.7.4. Vì sao hai hướng bổ sung nhau, không loại trừ

Hai hướng bổ sung vì chúng mạnh ở **những chiều ngược nhau** của cùng một vấn đề:

- Vật lý** mạnh ở **độ bao phủ chế độ lỗi + khả năng kiểm soát + chi phí rẻ + nhãn sạch**. Nó nói được chính xác "đây là lỗi BPFO nặng độ 3" — thứ mà generative gần như không làm được (generative sinh ra "một cái gì đó giống lỗi" nhưng không nói được tên chế độ).
- Generative** mạnh ở **độ trung thực với phân bố thật + mềm dẻo**. Có những hiện tượng lỗi (nhiều nền, độ già hóa, chế độ tải phụ thuộc máy) mà ta **không thể viết công thức** — vật lý mù chỗ này, generative học được.

Kết hợp chúng theo vai trò khác nhau, không thay thế nhau:

- Dùng vật lý sinh lớp lỗi "đã biết cơ chế"** — ví dụ xung BPFO/BPFI, sideband quanh trục chính. Lớp này cần độ **chính xác tần số + nhãn chế độ** để minh chứng khoa học. Với ổ bi, tần số đặc trưng tính theo cấu hình: $f_{BPFO} = \frac{n}{2} \left(1 + \frac{d}{D} \cos \phi \right) f_r$ (lỗi vòng ngoài), $f_{BPFI} = \frac{n}{2} \left(1 + \frac{d}{D} \cos \phi \right) f_r$ (lỗi vòng trong), còn lệch trục/không cân bằng cho đỉnh tại $1 \times f_r, 2 \times f_r$ kèm **sideband** quanh chúng. Đây là nơi vật lý "bất khả chiến bại" về độ chính xác tần số.
- Dùng generative học phần "chưa hiểu rõ"** — phần lỗi thật mà con người khó mô tả bằng công thức (nhiều nền theo máy, già hóa phi tuyến, chế độ tải biến đổi). Chính TimeGAN/Diffusion học bù các chiều mà mô hình vật lý bỏ sót.

Vì sao không thể thay thế nhau? Nếu chỉ dùng vật lý, model học lỗi giả **quá sạch/đúng công thức** → sai lệch nặng khi gặp lỗi thật có nhiễu thực (simulation gap). Ngược lại, nếu chỉ dùng generative với **rất ít** lỗi thật, model chỉ học được **phân bố hẹp** của vài mẫu đó → thiếu đa dạng chế độ lỗi và dễ mode collapse. Ghép hai bộ sinh → model thấy được cả **"phân bố thật" lẫn "danh mục chế độ + độ sạch"** → robust hơn.

Ví dụ kết hợp thực tế trong repo: nâng `--gen heuristic` (chỉ nhân biên độ 2.5 + nhiễu) thành `--gen physics` (bơm xung BPFO/BPFI/sideband theo công thức trên) để tạo **cột đối chiếu**; song song giữ `--gen npy` (TimeGAN sinh từ lỗi thật) làm **bộ sinh chính**. Kết quả là hai bộ dữ liệu lỗi: - Bộ "physics" → kiểm soát được chế độ, nhãn sạch, kể chuyện khoa học. - Bộ "generative" → bám phân bố thật, chạy được trên CPU, đúng keyword của đề.

Chạy `scripts/02 --gen physics` và `--gen npy` trên cùng một test (3-zone split, test chỉ nhận lỗi thật chưa thấy) → ta có **bảng so sánh trước/sau cho từng bộ sinh**, chứng minh "bộ nào giúp recall/F1 tăng nhiều hơn" — chính là deliverable mà giám khảo cần.

6.7.5. Đối chiếu cột "công nghệ gợi ý" của BTC (quan trọng)

Cột BTC: Mô phỏng vật lý - GAN/TimeGAN - Diffusion - fault injection - PyTorch . Phân rã từng thuật ngữ:

Thuật ngữ	Thuộc hướng	Vai trò trong pipeline của team	Trạng thái code hiện có
Mô phỏng vật lý	Hướng 1 (vật lý)	Mô hình cơ chế hỏng (xung BPFO/BPFI, sideband, lệch trục) → sinh lỗi giả "có cơ sở"	Chưa có — mới dạng heuristic (<code>--gen heuristic</code> nhân biên độ + nhiễu, <code>scripts/02</code>)
fault injection	Hướng 1 (vật lý)	"Bơm" lỗi vào tín hiệu normal theo chế độ đã chọn, nhân 100% chính xác	Chưa có — heuristic ở <code>scripts/02</code> chưa đúng vật lý
GAN/TimeGAN	Hướng 2 (generative)	Học phân bố lỗi thật hiếm → sinh lỗi giả. Trụ chính của team	Có — <code>scripts/03_train_timegan.py</code> , <code>src/timegan_torch.py</code> → <code>results/synthetic_faults.npy</code>
Diffusion	Hướng 2 (generative)	Hướng nâng cao đạt chất lượng tần số cao hơn, giữ đúng BPFO/BPFI	Chỉ lý thuyết — <code>src/Diffusion-TS</code> (clone, chưa chạy), bắt buộc GPU
PyTorch	Framework (không phải hướng)	Nền tảng triển khai TimeGAN / (sau này) Diffusion	Có — <code>src/timegan_torch.py</code>

Kết luận từ bảng trên: - Team đã cover **khoảng một nửa cột** — phần generative: GAN/TimeGAN + PyTorch (chạy được, có số liệu). Phần còn thiếu là **mảng vật lý** (Mô phỏng vật lý + fault injection). - Muốn **bám sát 100% gợi ý** của BTC → cần bổ sung mảng vật lý, tức nâng `--gen heuristic` → `--gen physics` (fault injection có cơ sở vật lý) dùng làm **cột đối chiếu** — đúng kế hoạch trong `SPEC.md` (khoảng 20% ưu tiên). - Đồng thời cột này cho thấy đề cũng kỳ vọng **"mô tả chế độ lỗi"** (một phần **input** của A1): khi ta bơm lỗi BPFO/sideband, ta buộc phải mô tả rõ chế độ hỏng → đây là điểm cộng khi trả lời đề.

6.7.6. Bản đồ / cây quyết định chọn hướng

Dạng "NẾU ... THÌ ..." — trả lời tuần tự để chọn chiến lược:

```
BẮT ĐẦU: cần thêm "lỗi giả" để augment lớp hiếm
- (1) Có hiểu đầy đủ cơ chế lỗi (BPFO/BPFI/sideband...)?
  | YES → ưu tiên VẬT LÝ (kiểm soát + nhân sạch + rẻ)
  | NO  → ưu tiên GENERATIVE (học phần "chưa hiểu rõ")
- (2) Có GPU + đủ thời gian không?
  | NO  → VẬT LÝ (CPU tức thì) + TimeGAN (chạy được trên CPU)   ← khớp team này
  | YES → Có thể thử DIFFUSION (chất lượng tần số cao hơn)
- (3) Muốn kiểm soát chính xác chế độ lỗi để KỂ CHUYỆN / trình bày?
  | YES → VẬT LÝ (bơm đúng chế độ, đặt tên chế độ rõ ràng)
  | NO  → GENERATIVE (không cần chỉ định chế độ)
- (4) Muốn độ trung thực với dữ liệu thật (khi có vài lỗi thật)?
  | YES → GENERATIVE (học chính từ lỗi thật)
  | NO  → VẬT LÝ (không phụ thuộc dữ liệu lỗi thật)
- (5) Cần nhãn tuyệt đối từng mẫu (báo cáo sạch)?
  | YES → VẬT LÝ (nhãn 100% đúng + tên chế độ)
  | NO  → GENERATIVE (nhãn "lỗi" đúng nhưng mẫu có thể "rác")
```

Bảng tóm tắt quyết định (đọc nhanh, song song với cây ở trên):

Tình huống của team	Chọn hướng chính	Chọn hướng phụ/bổ sung
Có hiểu cơ chế hỏng, muốn kiểm soát & kể chuyện	Vật lý	Generative để bù độ thật
Có vài lỗi thật, muốn bám phân bố	Generative	Vật lý để kiểm soát chế độ
Không có GPU	Vật lý + TimeGAN (CPU được)	— (bỏ Diffusion)
Có GPU + thời gian	Generative (Diffusion)	Vật lý để check cơ chế
Muốn nhãn sạch từng mẫu để báo cáo	Vật lý	Generative bù chiều chưa hiểu
Muốn đa dạng chế độ lỗi (nhiều loại hỏng)	Vật lý (mỗi chế độ 1 bộ bơm)	Generative bồi thêm biến thể

Tóm tắt thành một câu quyết định: không có câu trả lời "chọn hẳn một hướng". Team chọn **generative làm trụ** (để có độ bám phân bố thật + đúng keyword đề + chạy được trên CPU) và **vật lý làm lớp so sánh** (để check cơ chế + kiểm soát chế độ + kể chuyện), bởi team **không có GPU** (câu 2 trả lời NO) và **muốn cả hai** (câu 1, 3, 4, 5 trả lời YES theo từng vai trò).

6.7.7. Khuyến nghị chiến lược cho team (khớp SPEC.md)

Chiến lược đề xuất — **trụ chính + lớp so sánh + ghi chú tương lai**, khớp SPEC.md :

Hạng	Thành phần	Tỷ trọng	Việc cụ thể	Lý do
1	TimeGAN / generative	~60%	Giữ scripts/03_train_timegan.py → results/synthetic_faults.npy → scripts/02 --gen npy . Chạy 3000+ iteration khi có thời gian	Đúng keyword đề ("GAN/TimeGAN"), chạy được trên CPU, có số liệu thật (results/compare_augmentation.json), dễ bảo vệ trước giám khảo
2	Fault injection vật lý	~20%	Nâng --gen heuristic → --gen physics : bơm xung BPFO/BPFI/sideband theo công thức, nhấn chế độ rõ ràng. Dùng làm cột đối chiếu với bộ generative	Cover nốt phần "Mô phỏng vật lý - fault injection" trong cột gợi ý; tạo góc "2 bộ sinh, cái nào tốt hơn" cho slide; bổ sung "mô tả chế độ lỗi" (input của A1)
3	Diffusion	~0% giờ	Chỉ ghi trong báo cáo là "hướng nâng cao đạt chất lượng tần số cao hơn"; đừng đầu tư vì cần GPU	Không khả thi với tài nguyên hiện tại

Bảng thứ tự ưu tiên việc cần làm (chốt):

- 1. **(A)** Đăng ký đội hạn 31/08 — ưu tiên số 1 (ngoài phạm vi kỹ thuật, nhưng khẩn).
- 2. **(B)** Giữ TimeGAN chạy 3000+ iteration (chất lượng synthetic tốt hơn).
- 3. **(C)** Làm --gen physics (bơm BPFO/sideband, chạy CPU).
- 4. **(D)** Ghép 3 kết quả (baseline AE + TimeGAN + fault injection) → bảng so sánh cho slide.

Cách dựng --gen physics (rất ngắn gọn, chạy CPU): mở rộng _heuristic_faults trong scripts/02_compare_augmentation.py thành hàm bơm lỗi theo chế độ:

```
f_bpfo = n/2 * (1 - d/D*cos(phi)) * f_r      # tần số đặc trưng vòng ngoài
# 1) lấy window normal x(t)
# 2) tạo chuỗi xung đơn vị tại mỗi chu kỳ 1/f_bpfo (mô phỏng viên bi đập lỗi)
# 3) điều chế xung (gia tốc xung, không phải sóng sin) rồi nhân hệ số "mức nặng"
# 4) cộng vào x(t), thêm nhiễu nền theo mức x(t)
# 5) dán nhãn: fault + tag chế độ (vd in 1 dòng mô tả "BPFO sideband, nặng 2x")
```

Chỉ cần bơm vào **miền thời gian + đảm bảo phổ có đỉnh tại f_bpfo / f_bpfi** là đủ làm "cột đối chiếu"; không cần mô phỏng cơ khí 3D phức tạp. Kết quả lưu .npy dưới dạng RAW feature (giống scripts/03, để scripts/02 --gen npy tải dùng và chuẩn hóa bằng scaler train — **tránh lộ test**).

Điểm "ăn điểm" trên slide: nêu rõ góc "2 hướng sinh dữ liệu lỗi (vật lý vs generative) — vì sao bổ sung nhau" phù hợp với **cột công nghệ gợi ý của đề** (Mô phỏng vật lý - GAN/TimeGAN - Diffusion - fault injection - PyTorch). Góc này cho thấy team **hiểu sâu** (không chỉ "chạy một cái GAN rồi nộp") và **trả lời đúng deliverable**: so sánh accuracy trước/sau bằng hai bộ sinh độc lập.

6.7.8. Kết luận / tóm tắt để nhớ

- **Generative là chính** (để có độ bám phân bố thật + đúng keyword đề + chạy được trên CPU), **vật lý là phụ** (để kiểm soát chế độ lỗi + nhấn sạch + kể chuyện + rẻ).
- **Cột công nghệ của đề gợi ý CẢ HAI cụm** (Mô phỏng vật lý - fault injection + GAN/TimeGAN - Diffusion), nên đi một mình là lệch khỏi kỳ vọng.
- **PyTorch là framework, không phải hướng** — nó chỉ là nền tảng để chạy TimeGAN (đã có src/timegan_torch.py); đừng nhầm nó thành một hướng sinh dữ liệu.
- Chốt chiến lược: TimeGAN ~60% (chính), fault injection vật lý ~20% (so sánh), Diffusion ~0% (ghi chú nâng cao) — thống nhất với SPEC.md , không dùng lỗi giả nào ở **test** (3-zone split, test chỉ nhận lỗi thật chưa từng thấy).

7. Sinh dữ liệu để AUGMENT lớp hiểm (gắn bài A1)

Mục tiêu: khép lại vòng đời: dùng generative model đã học (§5–6) để **bù thêm mẫu lỗi giả**, rồi train classifier tốt hơn trên lỗi **THẬT** — và đảm bảo việc đánh giá không bị "nhiều" bởi mẫu sinh.

Bối cảnh (bài toán con): AE/LSTM đã có; lớp lỗi vẫn chỉ vài mẫu. Đây là công đoạn "phễu" nối generative (§5–6) với supervised (§3): sinh → augment → train → đánh giá trên thật.

So sánh cách bù lớp hiếm (chọn đường nào):

Cách	Chi phí	Giữ bản chất lỗi	Rủi ro
Oversampling đơn giản (lập mẫu lỗi)	Rẻ nhất	Không đổi	Overfit — classifier học vẹt
Augmentation tín hiệu (shift, noise, warp)	Rẻ	Một phần	Không tạo khái niệm mới
Generative (TimeGAN/Diffusion) — §5–6	Trung bình–cao	Tốt	Cần đánh giá sinh (7.4)
Clustering + SMOTE (Phần 2)	Rẻ	Trung bình	Khó trên chuỗi dài

7.1. Protocol đầy đủ (khung làm việc)

[Bước 1] Tập train: normal (đầy đủ) + rất ít fault thật (few-shot, F_m)
[Bước 2] Huấn luyện generator G:
- optional: pretrain trên normal trước
- fine-tune có điều kiện bằng F_m (cho từng loại lỗi)
[Bước 3] Sinh $F_{gen} = G(F_m)$ → hàng trăm/thousands mẫu lỗi giả
[Bước 4] Train classifier giám sát trên: normal + F_m (thật) + F_{gen} (giả)
[Bước 5] Đánh giá CHỈ trên fault **THẬT** của tập test (đảm bảo không dùng mẫu giả để scoring)

Nguyên tắc vàng: - **Không bao giờ đánh giá bằng mẫu sinh** — test set phải là fault thật do giám khảo giữ. Dùng synthetic chỉ giúp mô hình học "khái niệm lỗi" rộng hơn. - Data leakage cấm: nếu generator học luôn cả window test → mẫu sinh nhiễu nhẹ vẫn bị classifier "nhớ mặt" → đánh giá ảo. Luôn split theo **thời điểm máy** (trước khi train generator). - Trọng số: có thể đặt mẫu giả đóng góp ít hơn mẫu thật (sample weighting $w_{fake} = 0.3 \cdot w_{real}$) — chống overfit vào "hơi hướng" sinh.

7.2. Vì sao sinh trên class hiếm khó? Few-shot generation

- Lượng thông tin cực nhỏ:** generator học "dấu hiệu lỗi" từ 5–10 mẫu → dễ bắt nhiễu của từng mẫu cụ thể thay vì "bản chất lỗi"; nếu 3 mẫu nhiễu nhiều A, model sinh ra toàn "lỗi-có-nhiều-A".
- Đa dạng kém:** vài mẫu không mô tả hết biến thể của lỗi (các mức độ mòn, vị trí ổ trục, tải trọng...). Generator có thể sinh ra các mẫu gần-giống-nhau (mode collapse) → classifier học "bàn sao" → không tổng quát cho lỗi mới.
- Class-conditional sinh yếu:** generator chưa có đủ "lực" để phân biệt "lỗi loại A vs B" mà chỉ vài mẫu.
- Đánh giá khó khăn:** FID không dùng được với vài chục mẫu; discriminative score kém tin (bài phân biệt quá dễ).

Các kỹ thuật few-shot generation: - **Pretrain + fine-tune:** generator học phân phối normal (dữ liệu dồi dào) trước; sau đó fine-tune bằng vài mẫu lỗi (chỉ vài epoch, lr nhỏ) → mô hình "biết dáng dữ liệu" không bắt nhiễu mẫu lẻ. → **Áp dụng trực tiếp cho TimeGAN: train trên normal rồi fine-tune bằng F_m .** - **Data augmentation tín hiệu:** time-shift, scale biên độ, cộng noise yếu, jitter thời gian, warping (dilate/compress), tạo dạng biến thể — tăng "bề dày" class lỗi trước khi sinh. - **Sinh theo phễu:** từ mẫu lỗi thật làm "seed", sinh quanh vùng latent (sampling gần z thật) → các mẫu lỗi "thuộc same family". - **VAE/latent interpolation:** chèn giữa 2 mẫu lỗi trong latent → nhân đôi đa dạng có kiểm soát.

7.3. FaultDiffusion (arXiv 2511.15174, 11/2025) — hướng few-shot fault TS generation

Lưu ý quan trọng: đây là paper rất mới (tháng 11/2025). Nếu bạn chưa đọc trực tiếp, hãy lấy phần mô tả sau làm **khung hiểu chung** và luôn đối chiếu bản gốc để lấy chi tiết siêu chính xác khi cần trích dẫn trong báo cáo.

Khung mô tả theo hướng chung (nhất quán tinh thần paper và đề A1): - **Chủ đề:** diffusion model sinh dữ liệu **fault time series trong giám sát/điều khiển công nghiệp** — đúng việc ta cần cho A1 (rung cảm biến công nghiệp, thiếu dữ liệu lỗi). - **Few-shot:** tận dụng sức ít mẫu lỗi thật (few-shot generation) — khớp A1 với dữ liệu lỗi cực hiếm. Cách làm thường thấy ở kỹ thuật này: học phân phối từ lượng normal lớn làm nền, rồi **condition theo loại lỗi** và fine-tune bằng ít mẫu fault. - **Dấu hiệu khớp A1:** - Đầu vào là multivariate time series (nhiều kênh cảm biến: rung x/y/z, nhiệt độ...). - Khả năng sinh **mẫu lỗi đa dạng theo lớp** (class-conditional) để augment lớp hiếm rồi train classifier. - Điểm mạnh diffusion: ít mode collapse hơn GAN — quan trọng khi dữ liệu hiếm vì ta không muốn sinh ra "bản photo" trùng lặp che lấp classifier.

Khuyến nghị thực tế khi áp dụng: bắt đầu từ bản code chuẩn của DDPM/Diffusion-TS, đổi input thành window rung chuẩn hóa; fine-tune có điều kiện bằng vài mẫu lỗi thật; đánh giá bằng discriminative score trước khi đưa vào tuổi đời. Nếu ngại chi tiết toán, có thể dùng **kiến trúc Diffusion-TS** làm nền tảng — cộng đồng có code PyTorch để dùng (đúng mục đích giảng dạy, không cần train từ đầu).

7.4. Đánh giá dữ liệu sinh

Chất lượng mẫu sinh không được đo bằng mắt. Ba "công cụ đo" chuẩn nghiên cứu:

(1) Discriminative score (đo "độ giống")

Train **post-hoc classifier** phân biệt real vs fake (nhấn 1 = thật, 0 = giả; kiểm chứng bằng cross-validation). AUC càng gần 0.5 → không phân biệt được → dữ liệu sinh giống thật. $AUC \approx 1$ → yếu, sinh lộ "phải gọi".

Train: classifier nhỏ (LSTM 1 lớp hoặc MLP trên feature) trên real+synthetic
Metric: AUC(real, fake); mục tiêu AUC ≈ 0.5

(2) Predictive score (đo "giữ được động học")

Train **forecasting model** (vd LSTM) dự đoán bước tiếp theo **trên dữ liệu thật**; rồi **train/fine-tune riêng trên dữ liệu sinh** và so performance (MSE forecast, hoặc accuracy downstream task). Nếu generative đủ tốt, mô hình học từ synthetic cũng gần bằng học từ real:

```
T1 = test_forecast(model trained on REAL)
T2 = test_forecast(model trained on SYNTHETIC)
predictive score = (T1 - T2) → càng gần 0 càng tốt
```

(3) Visualization — PCA / t-SNE / UMAP

Chiếu real + synthetic xuống 2–3 chiều, kiểm tra: - **Chồng lấp**: synthetic nằm đúng vùng phân phối real (không tách cụm riêng, không bám vào góc xa). - **Không phải "photocopy"**: các điểm synthetic phải rải rộng, không đập chồng lên nhau chính xác (dấu hiệu mode collapse). - **Bảo toàn cụm class**: từng loại lỗi synthetic nên màu đúng vùng của loại lỗi real tương ứng.

Checklist ngắn: (1) AUC-discriminative ≥ 0.5 gần tốt ~ 0.6 rất tốt; (2) predictive gap nhỏ; (3) t-SNE chồng lấp + phủ rộng. Chỉ khi qua cả ba, mới đưa synthetic vào pipeline train chính.

8. Thực hành PyTorch cho A1 — pipeline tổng hợp (code giả định, đủ hiểu)

Phần này tổng hợp toàn bộ thành một luồng chuẩn. **Đây chỉ là code giả định để bạn hiểu thứ tự các khối**, chưa phải code chạy production (bỏ qua normalizer, seed, eval chi tiết...).

8.1. Tổng quan pipeline

```
[1] Chuẩn bị: raw rung → window (B, T, n_ch) → chuẩn hóa z-score
[2] Head 1 – AE anomaly: train AE trên normal → anomaly score mọi window
[3] Head 2 – Classifier LSTM: train có giám sát trên normal + fault(thật + sinh)
[4] (Optional) GAN/TimeGAN sinh thêm fault → augment
[5] Fusion: đầu ra = fusion(anomaly score, classifier proba) → ngưỡng
[6] Đánh giá: AUC/ROC/precision-recall trên fault THẬT của test
```

8.2. Code giả định từng khối

```
import torch, torch.nn as nn
import torch.nn.functional as F

# ----- 0) chuẩn bị window -----
# raw: (n_samples, n_ch); window_len=128; hop=32
def make_windows(data, L=128, hop=32):
    ws = [data[i:i+L] for i in range(0, len(data)-L+1, hop)]
    return torch.tensor(np.stack(ws)).float() # (B, L, n_ch)

# chuẩn hóa theo từng kênh (dùng stat từ normal train)
# x = (x - mu) / sigma per channel
```

```
# ----- 1) AE anomaly (mục 2) -----
class AE(nn.Module):
    def __init__(self, L, n_ch, latent=8):
        super().__init__()
        self.enc = nn.Sequential(nn.Flatten(start_dim=1),
                                  nn.Linear(L*n_ch, 64), nn.ReLU(), nn.Linear(64, latent))
        self.dec = nn.Sequential(nn.Linear(latent, 64), nn.ReLU(),
                                  nn.Linear(64, L*n_ch))
    def forward(self, x):
        B = x.size(0); z = self.enc(x)
        return self.dec(z).view(B, x.size(1), x.size(2))

ae = AE(L=128, n_ch=3)
opt = torch.optim.Adam(ae.parameters(), lr=1e-3)
for e in range(30):
    for xb in normal_loader:
        opt.zero_grad(); loss = ((ae(xb)-xb)**2).mean(); loss.backward(); opt.step()

def anomaly_score(ae, xb):
    ae.eval()
    with torch.no_grad(): return ((ae(xb)-xb)**2).mean(dim=(1,2))

# ----- 2) Optional: sinh thêm fault (TimeGAN/GAN – mục 5.4) -----
# Giả sử có hàm generate_faults(n): return tensor (n, L, n_ch)
# synthetic = generate_faults(batch, cond="bearing") # dùng TimeGAN/Diffusion đã train

# ----- 3) LSTM classifier (mục 3.5) -----
class LSTMFault(nn.Module):
    def __init__(self, n_ch=3, hidden=48, n_layers=1):
        super().__init__()
        self.lstm = nn.LSTM(n_ch, hidden, n_layers, batch_first=True)
        self.head = nn.Linear(hidden, 1)
    def forward(self, x):
        out, _ = self.lstm(x)
        return self.head(out[:, -1, :]).squeeze(-1)

model = LSTMFault(); opt = torch.optim.Adam(model.parameters(), lr=1e-3)
# Dataloader: normal (majority) + fault_real + fault_synthetic (weight thấp hơn)
for e in range(20):
    for xb, yb in combined_loader: # yb ∈ {0,1}
        opt.zero_grad(); logit = model(xb)
        w_yb = torch.where(yb == 1, 5.0, 1.0) # fault=5x, normal=1x
        # giảm trọng số mẫu SINH thêm nếu sợ overfit "hơi hướng" generator
        # w_yb = torch.where(yb == 1, torch.where(is_synt, 1.5, 5.0), 1.0)
        loss = F.binary_cross_entropy_with_logits(logit, yb, weight=w_yb)
        loss.backward(); opt.step()

# ----- 4) Fusion + ngưỡng -----
# score = anomaly_score(ae, x) (chuẩn hóa min-max 0..1)
# proba = torch.sigmoid(model(x))
# final = alpha*score_norm + (1-alpha)*proba; thresh = quantile(0.99, final[normal])
# predict = final > thresh
```

8.3. Lưu ý deployment tối giản

- **Cửa sổ trượt (sliding window):** test trong thực tế là online — tính trên window mới nhất, muốn có "nhiệt độ cảnh báo" theo thời gian thì trung bình trượt score (EMA) trước khi nâng cờ.
- **Threshold & metric:** bài mất cân bằng nặng → chọn ngưỡng theo **precision-recall curve** hoặc **F1/F2** (F2 nhấn cảnh báo sớm — nhà máy sẵn sàng chịu false alarm thay vì bỏ lỡ lỗi), đừng dùng accuracy.
- **Danh sách models khả dụng:** AE/ConvAE (không giám sát), LSTM/GRU classifier (có giám sát), TimeGAN/Diffusion-TS (sinh thêm lỗi), MLP/XGBoost trên feature (baseline nhanh).
- **Nguyên tắc 80/20 cho cuộc thi:** (1) làm đúng baseline AE + LSTM trên window chuẩn hóa; (2) thêm augmentation có kiểm soát (time-swap, noise); (3) sinh thêm bằng generative; (4) fusion + calibrate ngưỡng. Đừng đuổi theo Transformer/diffusion to trước khi baseline ngon.

Phụ lục nhanh — bảng thuật ngữ tiếng Việt–Anh

Tiếng Việt	Tiếng Anh
Lan truyền xuôi / ngược	Forward / backpropagation
Hàm kích hoạt	Activation function
Chuỗi có trọng số	Weighted sum
Bottleneck / không gian tiềm ẩn	Bottleneck / latent space

Tiếng Việt	Tiếng Anh
Mất mát tái tạo	Reconstruction loss
Điểm bất thường	Anomaly score
Cổng quên / vào / ra	Forget / input / output gate
Trạng thái tế bào	Cell state
Quá trình lan ngược gradient	Backpropagation through time (BPTT)
Sụp đổ mode	Mode collapse
Không gian embedding	Embedding space
Thêm nhiễu thuận / khử nhiễu ngược	Forward noising / reverse denoising
Tách xu hướng–chu kỳ	Trend–seasonal decomposition
Điểm phân biệt / dự báo	Discriminative / predictive score
Few-shot tạo sinh	Few-shot generation

PHẦN 4 · KHO TÀI NGUYÊN, HƯỚNG ĐI & KẾ HOẠCH — KHO TÀI NGUYÊN (DATASET / PAPER / REPO) + HƯỚNG ĐI & KẾ HOẠCH CHIẾN THẮNG

BÀI TOÁN A1 — DENSO FACTORY HACKS 2026

Văn bản này là tài liệu chiến lược duy nhất cho đội A1. Nó bó 5 thứ lại thành một: (1) catalog tài nguyên đã **verify link** (dataset + paper + repo), (2) hướng đi kỹ thuật 2 nhánh kèm phân tích khả thi, (3) kế hoạch chi tiết theo tuần từ 26/08 → 12/10/2026, (4) chiến lược slide Vòng 1 + bộ câu hỏi cho Factory Tour, (5) **hiện trạng team tại 30/08** kèm hành động ngay.

Mọi link bên dưới đều **đã thử tải được** trong quá trình chuẩn bị (08/2026). Nếu một link chết, đừng đoán link khác — dùng mirror tại <https://data.phmsociety.org/nasa/> (PHM Society đã mirror toàn bộ dataset NASA PHM) và tìm paper qua arXiv ID thay vì đoán đường dẫn PDF.

⚠ HIỆN TRẠNG TEAM + HÀNH ĐỘNG NGAY (cập nhật 30/08/2026)

Trạng thái tổng: sẵn data + code + số baseline. Đội còn vồn vẹn 1 tháng để biến số thật thành câu chuyện thắng.

Khoản	Hiện trạng (30/08)	Còn thiếu
Đăng ký đội	Hạn 31/08 (mai) — chưa chốt là TRỌNG ĐIỂM số 1	Ý tưởng sơ bộ trên densohackathon.vn , owner tài khoản
Dữ liệu	IMS (test_1/2/3) + FEMTO đã tải; CWRU fault đã tải, normal 97–100.mat hỏng (serve thiếu bytes) → fallback IMS làm nguồn normal	C-MAPSS (optional)
Paper / Repo	3 bài đã tải/đọc online: FaultDiffusion, TimeGAN, Diffusion-TS; 2 repo đã clone (vendor/)	—
Code	<code>src/</code> (data, features, ae, pipeline 3-zone), <code>scripts/01</code> , <code>scripts/02</code> , <code>scripts/03</code> (TimeGAN) chạy được	Synthetic thật chưa sinh đủ
Số đo	AE baseline yếu → cột mốc "trước" hoàn hảo ; heuristic gain nhỏ → cần sinh tốt hơn	Số "sau" đẹp bằng diffusion

Số liệu thực tế đang có (`results/` , ngày 30/08): - **AE baseline** (`baseline_ae.json` , IMS, `--limit 300 --epochs 60`): ngưỡng 0.0164, precision 0.387, recall 0.031, F1 0.057, AUC 0.524. AE gần đoán bừa trên feature IMS → đúng vai "trước". - **RF heuristic** (`compare_augmentation.json` , IMS): TRƯỚC 0.416 / 0.477 / 0.444 / 0.582 → SAU 0.418 / 0.482 / 0.447 / 0.580 (precision/recall/F1/AUC). **Gain gần như bằng 0** → heuristic sinh lỗi kém; đây là lý do chính để chuyển lên TimeGAN/diffusion. - `pipeline.py` đã cài **3-zone split** 30/08: normal [0..0.70) TRAIN · [0.70..0.90) TEST khỏe · [0.90..1.0] GREY bỏ; fault 20% train / 80% test thật; z-score fit trên TRAIN → chống leakage.

Hành động NGAY, theo thứ tự ưu tiên:

1. **[Hôm nay/mai] Đăng ký đội ≤31/08** — densohackathon.vn, chốt owner. Không trượt mốc này.
2. **Sửa tiếp model:** nâng bằng **XGBoost** + sweep ngưỡng P95–P99.5 (tăng recall tại precision chấp nhận); thêm assertion test chỉ fault **THẬT** (chống leak).
3. **TimeGAN ≥3000 iterations:** chạy `scripts/03_train_timegan.py` cho đủ số steps, sinh `.npy` , đo **discriminative score** (~0.5–0.65 là đẹp) → đưa vào script 02 `--gen npy` .
4. **Dashboard MVP + báo cáo trước/sau** (bảng + PR curve) — đầu vào trực tiếp cho slide Vòng 1.
5. **Factory Tour 11/09:** chuẩn bị sẵn 10 câu hỏi (\$7.4) + 1-pager protocol trước/sau để hỏi kỹ sư DENSO.

Chốt câu chuyện: bán "một protocol tái sử dụng" (bất kỳ dataset rung nào → AE baseline → sinh lỗi giả → supervised → báo cáo trước/sau), không bán "AI". Mọi con số phải đi qua `results/` .

Mục lục CHƯƠNG 4

- 2. Catalog DATASET
- 3. Catalog PAPER
- 4. Catalog REPO
- 5. Hướng đi kỹ thuật 2 nhánh
- 6. Kế hoạch chi tiết theo tuần (26/08 → 12/10/2026)
- 7. Chiến lược slide Vòng 1 (hạn 12/10)
- 8. Quyết định nhanh (decision guidance)

1. Tổng quan bài A1 + đọc lại deliverable + diễn giải giám khảo mong gì

1.1 Bài toán trong một câu

"Predictive-Maintenance AI Despite Scarce Fault Data" — Xây dựng hệ AI bảo trì dự đoán (phát hiện bất thường sớm + ước lượng tuổi thọ RUL) trong điều kiện **rất ít dữ liệu lỗi** — đúng như thực tế nhà máy: có hàng tháng dữ liệu "máy chạy tốt", nhưng rung động/ánh sáng khi máy sắp hỏng thì gần như không có mặt hàng nào trên kệ.

Ngữ cảnh công nghiệp điển hình (dựng theo README của team):

- Máy móc (mô tơ + ổ bi, bơm, trục...) chạy liên tục, được gắn cảm biến rung / nhiệt độ / dòng điện.
- Có **nhều** dữ liệu trạng thái "máy khỏe" (normal), **rất ít** dữ liệu "máy sắp hỏng / vừa hỏng" (fault).
- Input: chuỗi thời gian đa biến (vibration X/Y/Z, current, nhiệt độ) ~10 Hz (bản đồ với dữ liệu rung 12 kHz của CWRU / IMS).
- Output mong muốn: (0) cảnh báo bình thường / (1) bất thường + RUL (giờ còn lại).
- Goal: **phát hiện sớm (recall cao) NHƯNG ít báo động giả (precision cao)** — trade-off kinh điển của bảo trì dự đoán (báo giả = tắt máy xem xét lãng phí; bỏ sót = hỏng lớn gấp chục lần).

1.2 Deliverable chính xác của đề (sau 3 tháng)

Ba đầu ra **bắt buộc** cần nộp (đối chiếu RESOURCES.md §5):

#	Deliverable	Diễn giải sản phẩm cụ thể	Trạng thái hiện tại của team
1	Bộ sinh dữ liệu bất thường	Code pipeline sinh "dữ liệu lỗi giả" (synthetic fault) — có thể là GAN hoặc Diffusion cho chuỗi thời gian, hoặc heuristic. Kèm cách dùng rõ ràng (CLI, config).	Đã có nhánh heuristic trong script 02; thiếu sinh bằng diffusion thật
2	Tập dữ liệu tăng cường	File .npy / .csv chứa các window "lỗi giả" đã sinh — dùng được ngay để feed vào train.	Chưa có (mới sinh transient trong script)
3	Báo cáo so sánh độ chính xác dự báo TRƯỚC / SAU tăng cường	Bảng số liệu + biểu đồ (PR curve trước/sau) trên cùng một test set lỗi thật chưa thấy .	Đã có protocol trong script 02 + kết quả heuristic thô

1.3 Giám khảo mong gì? (diễn giải)

Đây là hackathon **công nghiệp** (DENSO = nhà sản xuất linh kiện ô tô, nhà máy họ có vô số thiết bị rung). Đội giám khảo gồm kỹ sư DENSO + chuyên gia AI. Họ chấm theo thứ tự ưu tiên:

- Tôn trọng kịch bản "scarce fault data"** — đây là mấu chốt. Bài nộp sẽ bị loại ngay nếu team làm supervised classification bằng cách "có đủ lỗi thật, chia train/test bình thường". Scarce phải được xử lý bằng: (a) unsupervised baseline, rồi (b) sinh thêm lỗi giả.
- Trung thực về số liệu/leakage** — test phải là **LỖI THẬT CHƯA TỪNG THẤY**, không một pixel nào của test xuất hiện trong train (kể cả bóng dáng qua augmentation). Ai cũng có thể "đẹp" nếu cheat; đội thắng là đội đẹp *thật* và chỉ rõ protocol trong slide.
- Một nghiên cứu có cấu trúc, không phải "chạy được"** — giám khảo nghề sẽ hỏi: baseline là gì? ngưỡng chọn thế nào? vì sao gain lại đến? paper nào bạn dựa trên? Câu trả lời nằm trong log, JSON, và slide.
- Khả năng áp dụng thật** — "máy của em/đội gặp tình huống này, thu thập data lỗi thế nào?" Nếu team trả lời được sau Factory Tour và map vào pipeline → điểm hợp đồng.

5. Tính "so sánh trước/sau" phải rõ — càng ít text, càng nhiều biểu đồ: PR curve + bảng Precision/Recall/F1/AUC trước/sau, ngưỡng P99, và ví dụ window lỗi giả vs lỗi thật (trực quan → thuyết phục).

Ký hiệu quan trọng nhất: Đề bài nhấn "predictive-maintenance AI *despite scarce fault data*". Đừng bán "AI" trùu tượng; hãy bán "một protocol tái sử dụng được: bất kỳ máy nào, bất kỳ dataset rung nào → baseline AE → sinh lỗi giả → train supervised → báo cáo trước/sau".

1.4 Context stack code hiện có (đã đọc mã nguồn)

- **Environment:** `.venv/` Python 3.11.9, `scipy 1.13.1` (xem `requirements.txt`).
- **src/data.py** — loader CWRU (tách normal/fault bằng label), NASA IMS (3 test, tự nhận diện file timestamp, `limit_per_test`), FEMTO (mọi `.csv`); hàm `make_windows(sig, win=512, stride=256)`. IMS test_1: ~163840 sample/chuỗi, 8 cột; test_3: 4 cột.
- **src/features.py** — 16 features vector-hoá trên ma trận window: thời gian (mean, std, rms, peak, peak2peak, skewness, kurtosis, crest, shape, impulse) + tần số (spectral centroid/spread/ flatness/energy, `freq_mean/freq_std`) + z-score theo cột.
- **src/ae.py** — FeatureAE (Linear 16 → 32 → 8 → 32 → 16), train chỉ trên normal, ngưỡng P99, metrics precision/recall/F1/AUC; tự chọn CUDA nếu có.
- **src/pipeline.py** — 3-zone split TRONG TỪNG test-run (sửa ngày 30/08 sau khi phát hiện test normal cũ lấy trùng vùng "suy giảm sát hỏng"): normal 0–70% mỗi run chia tiếp `[0..0.70]` → TRAIN khỏe, `[0.70..0.90]` → TEST khỏe chưa thấy, `[0.90..1.0]` → GREY bỏ hẳn. Fault 90–100% mỗi run: trộn ngẫu nhiên 20% train / 80% test. Z-score fit TRÊN TRAIN, áp cho cả test lẫn synthetic (chống data leakage, scaler dùng lại ở script 02).
- **scripts/01_baseline_anomaly.py** — AE học "bình thường" → ngưỡng P99 → báo cáo trên lỗi thật.
- **scripts/02_compare_augmentation.py** — protocol TRƯỚC/SAU augmentation, cờ `--gen heuristic|npy`, cờ `--limit`, `--win`, `--stride`, `--gen-npy`. Test = normal khỏe chưa thấy (3-zone như trên) fault THẬT chưa thấy (80% fault).
- **scripts/03_train_timegan.py** — nhánh đang mở: train TimeGAN sinh lỗi giả (target ≥3000 iterations), xuất `.npy` → feed script 02 `--gen npy`.
- **Project ngôn ngữ:** docstring/comment tiếng Việt giải thích "script chạy gì & số nghĩa gì", tên hàm/CLI tiếng Anh — 3 script mỗi cái đều có câu chuyển tự giải thích ở đầu file.

Mã tham chiếu cụ thể trong tài liệu này dùng đúng tên file/flag nêu trên để không mất công soi. **Kết quả baseline hiện hữu** (`results/baseline_ae.json`, `--limit 300 --epochs 60`, `dataset=ims`): ngưỡng 0.0164, precision 0.387, recall 0.031, F1 0.057, AUC 0.524. AE thuần trên feature IMS vẫn yếu (gần 0.5 = đoán bừa) → đây chính là chỗ "trước" hoàn hảo để kể câu chuyện tăng cường (RF trong script 02 đã kéo recall lên ~0.48 nhờ dữ liệu + model có giám sát). Lưu ý lịch sử: kết quả cũ ghi "recall 0.013, AUC 0.542" sai nguồn gốc — lấy trước khi sửa split, test normal khi đó trùng vùng đã suy giảm → test bất công nên recall tụt thảm khốc.

Cập nhật 30/08 — đọc kỹ: số chính xác hiện tại là AE (P 0.387 / R 0.031 / F1 0.057 / AUC 0.524) và RF heuristic TRƯỚC 0.416 / 0.477 / 0.444 / 0.582 → SAU 0.418 / 0.482 / 0.447 / 0.580 (`results/compare_augmentation.json`). Gain heuristic ≈ 0 → **bằng chứng cần bộ sinh thật (TimeGAN/diffusion)**, không dùng heuristic làm số chính.

1.5 Bối cảnh mùa trước — SPARK / DiffusionAD (dùng cho slide)

Đội vô địch Mùa 3 (2025): **SPARK** — Viện Trí tuệ nhân tạo, Đại học Công nghệ (UET), ĐHQG Hà Nội.

Giải pháp: "DiffusionAD" — phát hiện lỗi bất thường trong sản xuất bằng **diffusion model**, không cần dữ liệu nhãn (unsupervised/self-supervised). Thuộc nhánh AI/Data (phát hiện lỗi linh kiện — hướng thị giác máy).

Nguồn đối chiếu: VnReview (khởi động mùa 4), FPT, Facebook UET/AI-UET, bài BUV về đội á quân FSIL (BUV + Bách Khoa, giải "End-to-End AI"). (Nguồn cấp thứ cấp — nếu trích chi tiết kỹ thuật trong slide nên tìm bài VnReview/FPT gốc.)

Ý nghĩa chiến lược cho A1 (điểm ăn điểm slide): - Năm ngoái giám khảo đã trao giải cao nhất cho tư duy "**diffusion + dữ liệu lỗi khan hiếm/không nhãn**" — đúng tinh thần của A1. - **Điểm khác biệt phải nêu:** DiffusionAD thắng ở mảng **ảnh (CV)**; đề A1 được team đưa cùng tư duy đó sang **chuỗi thời gian rung (TS)** — chưa ai chứng minh ở nhóm này. - **Tránh:** copy y hệt họ (thắng bằng unsupervised anomaly); A1 yêu cầu deliverable "bộ sinh dữ liệu + so sánh trước/sau" → nhấn mạnh nhánh augmentation có giám sát.

2. Catalog DATASET

2.1 Bảng tổng quan

Dataset	Loại	Mô tả ngắn	Số sample	Tần số	Task chính dùng cho	Trạng thái team
CWRU Bearing (Case Western Reserve Univ.)	Có nhãn	Rung ổ bi 2HP mô tơ, 3 vị trí (DE/FE/BA), 3 kích thước lỗi + normal-motor	~hàng trăm file <code>.mat</code> (mỗi file 12s, ~120k sample@12k)	12 kHz / 48 kHz	Anomaly detection + fault classification	Đã tải fault (105, 118, 130...), normal 97-100.mat hỏng
NASA IMS Bearings	Run-to-failure (không nhãn hạng)	3 test, nhiều ổ bi chạy đến hỏng thật	test_1: ~2156 file × 163840 sample × 8 cột; test_2: 984 file; test_3: 6324 file × 4 cột	20 kHz, ghi mỗi 10 phút	Anomaly + RUL (phần đầu = normal, cuối = suy giảm)	Đã tải, dùng chính
FEMTO / PRONOSTIA (FEMTO-ST)	Accelerated life test	17 máy, 6 điều kiện vận hành, chạy nhanh đến hỏng	17 máy × (train/test), file <code>acc.csv</code>	25.6 kHz	RUL + anomaly	Đã tải zip
NASA Turbofan (C-MAPSS)	Mô phỏng	100 động cơ, 21 sensor + 3 op condition, có RUL thật	4 tập train/test, FD001 100 máy	1 Hz (chu kỳ)	RUL chuẩn ngành	Chưa tải (optional)

Mirror dự phòng cho cả 4: <https://data.phmsociety.org/nasa/> (nếu S3 chậm/chết).

2.2 CWRU Bearing — chi tiết

- **Nguồn:** <https://engineering.case.edu/bearingdatacenter/download-data-file> (tải tay bằng trình duyệt hoặc `wget` ; trang này kiểu portal nên script bỏ qua — dùng browser).
- **Đặc điểm:**
- Ổ bi thử nghiệm trên mô tơ 2 HP; lỗi được tạo bằng tia EDM (điện phóng): **kích thước lỗi 0.007", 0.014", 0.021", 0.028"** trên inner race (IR), outer race (OR), ball (B), + dataset normal-motor.
- 3 vị trí đo gia tốc: Drive End (DE), Fan End (FE), Base (BA). Mỗi file `.mat` chứa biến `*_DE_time`, `*_FE_time` (rung theo thời gian) và `*_RPM` thuộc tính.
- Tần số 12 kHz (đa số, file "12k Drive End Bearing Fault Data") và 48 kHz ("48k").
- 4 tải: 0 / 1 / 2 / 3 HP → RPM khác nhau (1797/1772/1750/1730).
- **Verify cụ thể:** các file fault như `105.mat`, `118.mat` (IR007...) đọc được bằng `scipy.io.loadmat` ; **4 file Normal Baseline Data/97.mat...100.mat bị hỏng khi tải từ server chính thức (serve thiếu bytes)** → `src/data.py` tự bỏ qua file lỗi (`loadmat` ném exception → `continue`), vì vậy hiện tại **không có normal đọc được từ CWRU** → fallback IMS là nguồn normal.
- **Task phù hợp:**
- *Anomaly detection:* fault (IR/OR/B) là "bất thường", mọi normal-motor là "bình thường" — nhãn sạch.
- *Fault classification:* 3 loại lỗi + 4 mức nghiêm trọng → 12 lớp (nếu muốn slide đẹp hơn).
- **Cách chia normal/fault hợp lý (giữ nhãn sạch):**
- Đã có nhánh label trong loader: `label=0` cho normal, `label=1` cho fault.
- Split *theo file*, KHÔNG cắt window rồi trộn — mỗi file chỉ ở một tập (tránh window cùng file nằm cả train lẫn test).
- Normal ~hàng chục file → 80/20 train/test; fault chia 20% train (scarce) / 80% test.
- Lưu ý: các mức tải khác nhau có biên độ khác nhau → đừng lẫn tải 0HP vào test mà train chỉ có 3HP, vì AE sẽ "sợ" biên độ chứ không "sợ" lỗi.
- **Hạn chế:**
- Lỗi tạo bằng EDM = `seeded fault` **không tự nhiên như suy giảm thật** (không có giai đoạn "mọc vết nứt"); vết lỗi tĩnh, biên độ rất mạnh → dễ tách với normal hơn thực tế nhà máy.
- Normal baseline bị hỏng trên server chính thức → phải dùng nguồn khác hoặc fallback IMS.
- Có `RPM` không đổi trong mỗi file nên khó mô phỏng load thay đổi.
- Format `.mat` mỗi file có meta khác nhau → code phải dò `*_DE_time` thay vì tên cứng.

2.3 NASA IMS Bearings — chi tiết

- **Nguồn:** <https://phm-datasets.s3.amazonaws.com/NASA/4.+Bearings.zip> (script `scripts/download_data.sh` đã tải + giải nén; zip ~kiểu `.rar` của NASA nhưng download script xử lý được).
- **Đặc điểm:**
- 3 test **run-to-failure thật** (không phải mô phỏng): ổ bi chạy liên tục đến khi hỏng.
- `test_1`: 4 ổ bi (8 kênh = 2 accel × 4 ổ), ~2156 file, mỗi file ~20s (163840 sample) ghi mỗi 10 phút. **Test_1 là chuỗi tiêu chuẩn** trong mọi paper (RUL learning).
- `test_2`: 4 ổ bi, 984 file, 8 kênh.
- `test_3`: 4 ổ bi, 6324 file nhưng chỉ **4 kênh** (2 accel × 2 hướng? — đọc được 4 cột), thời điểm rời đầy đủ hơn.
- Tần số lấy mẫu 20 kHz. Mỗi file `2003.10.22.12.06.24` (timestamp) là một "snapshot": load ra ma trận (20480, n_cols) → `.ravel()` thành tín hiệu đa kênh dài.
- Có dữ liệu mô tả khi nào ổ bi hỏng (event log đi kèm) → phục vụ gán RUL tham khảo.
- **Task phù hợp:**
- **Anomaly:** 70% đầu mỗi test = normal; 10% cuối = suy giảm/hỏng — cách team đang dùng.
- **RUL:** dùng chỉ số health đoạn cuối; càng "late" càng fault mạnh.
- **Cách chia normal/fault hợp lý (protocol đã chuẩn hoá trong code, `src/pipeline.py`):**
- Với MỖI test-run (KHÔNG ghép phẳng 3 test làm 1): lấy `runs_normal = files[:0.7×m]`, `runs_fault = files[0.9×m:]` (đoạn 0.7–0.9 suy giảm dần bỏ qua — nhãn nhiễu).
- Trong vùng NORMAL (0–70%) của từng run chia tiếp 3 vùng để test normal KHÔNG bị "trúng vũng normal đã xuống cấp sát vùng hỏng" (bug cũ → test bất công, recall tụt ~0.01): `[0..0.70]` → TRAIN khỏe, `[0.70..0.90]` → TEST khỏe chưa thấy, `[0.90..1.0]` → GREY bỏ hẳn.
- **Không trộn lẫn test:** train/test theo thứ tự thời gian của cùng file-set; danger nếu trộn cuối `test_3` vào train rồi test trên cuối `test_1` — mức suy giảm khác nhau.
- FAULT: gộp lỗi thật (90–100% mỗi run) rồi trộn ngẫu nhiên → 20% train-scarce / 80% test-thật. Z-score fit trên TRAIN, chung cho test + synthetic (chống leakage).
- **Hạn chế:**
- **Nặng:** tải hết ~8 GB RAM → dùng `--limit` (mặc định 300 file/test) cho prototype, bỏ giới hạn chỉ khi chạy báo cáo cuối.
- Nhãn không có per-file "fault/normal" chính thức → ta suy diễn từ vị trí trong chuỗi (0.7/0.9 heuristic) nên nhãn hơi mơ hồ ở vùng 0.7–0.9.
- 8 kênh của `test_1` thực chất là 4 ổ bi → dùng đúng semantics mới đúng (mỗi ổ bi là một máy) nếu muốn câu chuyện "đa thiết bị".
- File không có đuôi; vài file rỗng → code lọc `arr.size == 0`.

2.4 FEMTO / PRONOSTIA — chi tiết

- **Nguồn:** <https://phm-datasets.s3.amazonaws.com/NASA/10.+FEMTO+Bearing.zip> (đã tải `FEMTO_Bearing.zip`, giải nén → `data/FEMTO/` có các sub-dataset `Bearing1_*`, `Bearing2_*`).
- **Đặc điểm:**
- **Accelerated life test:** ổ bi nhỏ chạy dưới tải cực mạnh để hỏng nhanh (vài giờ thay vì vài tháng) → có đủ giai đoạn "health → suy giảm → hỏng" trong thời gian ngắn.
- 6 điều kiện vận hành (operating condition) khác nhau về tải/tốc độ; mỗi condition có nhiều "máy" (bearing) — tổng 17 máy.
- Dữ liệu mỗi máy lưu trong file `acc.csv` (gia tốc chiều? theo file — header có tên cột), ghi 25.6 kHz với windows rời; kèm metadata điều kiện vận hành.
- Dataset chuẩn của **IEEE PHM 2012 Prognostic Challenge** — có ground-truth RUL (số giây trước hỏng).
- **Task phù hợp:**
- **RUL** là thể mạnh (có nhiều paper dùng FEMTO score RUL).
- **Anomaly:** phần đầu mỗi máy = normal, phần cuối = fault → chia như IMS.
- **Cách chia normal/fault:**
- Theo từng máy: `sigs_n = files[:0.6×m]`, `sigs_f = files[0.85×m:]` — nhìn thống kê RMS trước.
- **Train/test phải khác máy** (generalization giữa các bearing) hoặc cùng máy chia thời gian — nêu rõ chọn gì trong slide; an toàn nhất: train trên N máy, test trên máy chưa từng thấy.
- **Hạn chế:**
- File CSV không đồng nhất (số cột/đặt tên thay đổi theo sub-dataset) → loader hiện tại đơn giản `.ravel()` có thể trộn kênh không đúng semantics.

- Chưa có nhãn chuẩn ở loader hiện tại → `01_baseline_anomaly.py --dataset femto` có raise `SystemExit("FEMTO chưa có nhãn chuẩn")` → dừng dùng FEMTO để chạy baseline liền; dùng ims hoặc cwru cho demo chính, FEMTO để thêm phần RUL sau.
- Số máy ít (17) → không đủ để làm fault classification.

2.5 NASA Turbofan (C-MAPSS) — chi tiết

- **Nguồn:** <https://phm-datasets.s3.amazonaws.com/NASA/6.+Turbofan+Engine+Degradation+Simulation+Data+Set.zip> (optional — chỉ tải nếu muốn tăng độ sâu về RUL).
- **Đặc điểm:**
- Mô phỏng 100 động cơ, mỗi động cơ một chuỗi cycle (chu kỳ bay) đến khi hỏng; 21 sensor + điều kiện vận hành; **chuẩn vàng của PHM** (cùng dòng NASA PHM Challenge 2008).
- 4 tập con: FD001 (1 op condition, 1 mode lỗi), FD002, FD003, FD004. Train/test riêng, test có RUL thật dùng để scoring.
- Tần số 1 Hz theo cycle (không phải Hz sinh học như rung) → **khác biệt bản chất dữ liệu với CWRU/IMS**.
- **Task phù hợp:**
- **RUL regression** (dự báo số cycle còn lại) — script so sánh sẽ cần nhánh regressor nếu dùng.
- Có bộ evaluation chính thức để tự mình chứng "đạt score" — thuyết phục nếu cần câu chuyện benchmark.
- **Cách chia:**
- Dùng luôn train/test có sẵn của NASA (không tự chia lại).
- Nếu muốn anomaly: coi 30% cycle đầu = normal, 5% cuối = fault.
- **Hạn chế:**
- Dữ liệu mô phỏng (không phải vibration thật) → khác bản chất với bài "máy rung"; dùng trong slide như "bổ trợ chứng minh protocol tổng quát" chứ không phải dữ liệu chính.
- Không thể xài trực tiếp `make_windows` semantics của rung nếu không đọc lại cấu trúc file (mỗi dòng = 1 cycle, cột = sensor).
- Cần tuning riêng (nhiều paper, ít thời gian) → xếp optional.

2.6 Bảng quyết định "dùng dataset nào ở đâu"

Bước	Dataset chính	Lý do	Dự phòng
Baseline AE (script 01)	IMS	có normal + fault thật đọc được ngay, chuỗi liên mạch	CWRU fault-only không có normal
Augmentation protocol (script 02)	IMS	chuỗi dài → nhiều window → đủ train/test	CWRU (nếu lấy normal từ nguồn khác)
Demo fault classification (slide)	CWRU	nhãn 3 loại lỗi sạch → ảnh spectrogram đẹp	—
Phần RUL (điểm cộng)	FEMTO hoặc C-MAPSS	có ground truth RUL	FEMTO nếu đã có, C-MAPSS nếu cần đa dạng
Câu chuyện benchmark "protocol tổng quát"	cả 4	mỗi dataset là một "nhà máy khác"	—

3. Catalog PAPER

3.1 Bảng tổng quan

#	Paper	Venue / Năm	Link (đã verify)	Cần đọc vì
1	FaultDiffusion: Few-Shot Fault Time Series Generation with Diffusion Model	arXiv 2511.15174 (2025)	abs https://arxiv.org/abs/2511.15174 · PDF https://arxiv.org/pdf/2511.15174	Trùng gần 100% đề A1; trích dẫn chính trong slide

#	Paper	Venue / Năm	Link (đã verify)	Cần đọc vì
2	TimeGAN: Time-series Generative Adversarial Networks	NeurIPS 2019	PDF https://papers.nips.cc/paper_files/paper/2019/file/c9efe5f26cd17ba6216bbe2a7d26d490-Paper.pdf	Nền tảng GAN sinh TS + 2 metric chuẩn hoá
3	Diffusion-TS: Interpretable Diffusion for General Time Series Generation	ICLR 2024	https://openreview.net/pdf?id=4h1apFjO99 (đọc online; openreview có thể chặn script tải)	SOTA diffusion TS sinh theo class — nhánh augmentation thật

Lưu ý dấu hiệu quan trọng về trích dẫn: có nhiều bài khác cùng tên "FaultDiffusion" tồn tại trên arXiv (tên generic). Khi trích dẫn/cite luôn ghi kèm arXiv ID **2511.15174**; đừng đoán link khác. Khi cần kiểm tra bài chuẩn nhất, dùng arXiv API: `curl "https://export.arxiv.org/api/query?id_list=2511.15174"`.

3.2 FaultDiffusion (arXiv 2511.15174) — giải thích chi tiết

Vì sao đọc trước tiên: Đây là paper hiếm hoi trực tiếp giải "sinh dữ liệu lỗi **few-shot** cho industrial time series" — đúng y bối cảnh đề A1 ("scarce fault data"). Đọc nó biết được:

- Sự khác biệt với diffusion thuần: có cơ chế **few-shot conditioning** (đào tạo với vài mẫu fault thật để bộ sinh biết "dáng vẻ" của lỗi) thay vì sinh mù một class.
- Cách model để giữ **tính nhất quán vật lý của tín hiệu rung** (spectrum không bị nhiễu ngẫu nhiên) — chính là thứ mà "GAN sinh lung tung rồi bị chấm điểm kém" hay dính.
- Protocol đánh giá mà họ dùng để chứng minh synthetic "giống thật": thường là (1) t-SNE nhìn chồng lấn phân phối, (2) classifier-based discrimination (train bộ phân biệt synthetic vs real), (3) downstream metric khi augment vào train → **ba thứ đó y hệt thứ team cần in trong slide**.

Ứng dụng trong dự án A1: dùng kỹ thuật few-shot conditioning của nó làm "thiết kế bộ sinh lỗi giả" (deliverable 1): train với vài cửa sổ fault thật (20% fault) + đầy đủ normal, giam phân phối sinh về đúng vùng tần số 'lỗi giống thật'. Đây là **tài liệu tham khảo mạnh nhất** để trả lời câu giám khảo: "Làm sao synthetic không bị model tách ra ngay lập tức?"

Lưu ý trích dẫn: cite đúng dạng `FaultDiffusion (arXiv:2511.15174, 2025)`. Nếu repo code của bài tồn tại hãy link repo gốc (chưa verify repo tại thời điểm viết — không đoán, chỉ cite paper).

3.3 TimeGAN (NeurIPS 2019) — giải thích chi tiết

Vì sao đọc: - Bài kinh điển đầu tiên coi time series như một đồ thị và rèn generator GAN bằng **supervised loss trên autocorrelation** (temporal dynamics) — đây là lý do nó sinh ra chuỗi "khớp moment thống kê + khớp nhịp (autocorrelation)", thứ mà vanilla GAN không làm được. - Paper định nghĩa **2 metric kinh điển để đánh giá chất lượng sinh** mà team nên tái sử dụng y nguyên: `discriminative score` (train classifier "real vs synthetic" → accuracy càng gần 0.5 càng tốt) và `predictive score` (dùng chuỗi tổng hợp học dự báo 1 bước rồi đo lỗi dự báo). - **Repo chuẩn** có sẵn (xem §4) → chi phí thử nghiệm thấp nhất trong 3 paper.

Ứng dụng trong dự án A1: - Nhanh: chạy CPU được cho chuỗi dài vừa; hợp để sprint tuần 5–6 sinh **một cách nhanh** so sánh trước/sau. - Vì generator là MLP/RNN + discriminator phủ distribution, cần có "normal (nhiều) + ít fault thật (ít)" để sinh — đúng thiết kế đề. - Dùng luôn 2 metric của nó để slide có con số về **"synthetic giống thật đến mức nào"** trước khi nói về gain downstream.

Lưu ý: TimeGAN nhạy tuning (đặc biệt trong bài nhiều paper tái lập thất bại) — nếu có GPU hãy thử Diffusion-TS trước (ổn định hơn); TimeGAN là phương án CPU dự phòng.

3.4 Diffusion-TS (ICLR 2024) — giải thích chi tiết

Vì sao đọc: - Diffusion cho TS đạt chất lượng sinh cao hơn GAN (đặc biệt bao phủ chế độ hiếm — "mode coverage") và **có guidance theo class/condition:** bạn cho "lỗi loại gì" → model sinh đúng. - Bài giải thích rõ cách biến dữ liệu TS thành 2D (thời gian × kênh) và diffusion trên liên kết biến — từ đó giúp hiểu ảnh xạ "synthetic window" sang `make_windows` của team. - Có repo chuẩn (xem §4) tương đối "plug-and-play" với Tutorial notebook.

Ứng dụng trong dự án A1: - Nhánh augmentation **thật:** train trên normal + fault thật, **condition theo class** để sinh đúng class "fault" với số lượng mong muốn → viết ra `.npy` → feed `scripts/02 --gen npy`. - Vì model sinh theo chuỗi trong thang thời gian gốc, phải chỉnh chiều dài = `win=512` (hoặc chunk) để ra window khớp pipeline — phần dễ sai nhất. - Cần GPU CUDA (thực tế chạy được CPU nhưng rất lâu) → nằm lịch sau khi chắc chắn có máy GPU.

Lưu ý: File PDF trên OpenReview **đôi khi chặn script wget /crawler** → mở bằng trình duyệt để đọc online, không cần tải về; slide chỉ cite ICLR 2024 chứ không cần link PDF chính xác.

3.5 Bảng "đọc → làm gì" tóm tắt

Paper	Đọc để...	Biến thành gì trong repo
FaultDiffusion	hiểu few-shot conditioning + tạo ảnh t-SNE phân phối	phương án thiết kế generator + nội dung slide §"scarce data"
TimeGAN	đồ thị GAN TS + 2 metric	metric đánh giá synthetic (discriminative/predictive) trong <code>results/</code>
Diffusion-TS	condition-by-class + chiều guidance	generator thật → <code>.npy</code> → script 02 <code>--gen npy</code>

4. Catalog REPO

4.1 Bảng tổng quan

Repo	Paper	Ngôn ngữ / Độ khó	Yêu cầu phần cứng	Mục tiêu dùng trong team
https://github.com/jsyoon0823/TimeGAN	TimeGAN (NeurIPS 2019)	TensorFlow 1.x → cần môi trường riêng; chạy CPU được	CPU OK, GPU tùy chọn	sinh dữ liệu lỗi giả phương án B, metric đánh giá
https://github.com/Y-debug-sys/Diffusion-TS	Diffusion-TS (ICLR 2024)	PyTorch; Tutorial_01/2.ipynb	GPU CUDA khuyến nghị (CPU rất chậm)	generator chính → <code>.npy</code> → script 02

Cả hai clone được bây giờ và cơ bản là "đun sôi" — nhưng đừng để vào `src/` chính nếu cần tách môi trường (hai repo có dependency khác nhau, có thể xung đột với `.venv/` của team).

4.2 TimeGAN — cách dùng

Clone vào thư mục riêng (không phải `.venv`)

```
cd A1_predictive_maintenance
mkdir -p vendor && cd vendor
git clone https://github.com/jsyoon0823/TimeGAN.git
cd TimeGAN
```

Môi trường (vì là TensorFlow 1.x / tf.keras cũ, tách riêng venv để không đụng `.venv` của team):

```
python -m venv .venv-tf1
source .venv-tf1/bin/activate
pip install "tensorflow==2.10.*" # phiên bản tương thích code TF1-API
# (một số bản của repo hỗ trợ TF2 mode eager; chạy thử Tutorial trước)
```

Chạy huấn luyện + sinh:

```
python3 -m main_timegan.py --data_name stock
# sau khi train, script sinh file: timeGAN.saved_model + output/...
# dùng Python API trong repo: load data, gọi `timegan` sinh n mẫu class fault
```

Sinh "dữ liệu lỗi giả" từ TimeGAN theo kịch bản A1: 1. Tạo tensor 2D (`n_seq, time_len, n_dim`) với `n_seq` = số window (normal là chủ yếu + vài window fault thật — few-shot), `time_len` = win (512), `n_dim` = 16 *features* (nếu bạn sinh trên feature space) hoặc = số kênh sensor (nếu sinh trên signal thô — đọc khớp với `make_windows`). 2. Train generator (**target ≥3000 iterations** — đồng bộ `scripts/03_train_timegan.py`). 3. Sinh N window "fault giả": `generate(noise)` → reshape về (N, 16) (nếu train trên feature) hoặc (N, 512) (signal → sau này tính feature bằng `features.extract_features`). 4. Trong script 02: nếu đã là feature vector → `X_syn.astype(np.float32)`; nếu là signal → chạy `features.extract_features(X_syn, fs=12000)` rồi nạp.

Ghi chú tinh tế: TimeGAN cho phép **điều kiện class** khá lỏng (chỉ một generator với phân phối trộn). Nếu thấy class fault quá lẫn với normal, hãy khắc phục bằng cách *oversample* fault thật trong bộ train của TimeGAN (upweight), đây là trick đúng paper FaultDiffusion đã nêu.

Kiểm tra chất lượng sinh: tái dùng 2 metric của paper — train classifier "real vs synth" trên window/features; accuracy ≈ 0.5 → chất lượng tốt. Ghi số này vào slide.

4.3 Diffusion-TS — cách dùng

Clone + môi trường GPU:

```
cd A1_predictive_maintenance
mkdir -p vendor && cd vendor
git clone https://github.com/Y-debug-sys/Diffusion-TS.git
cd Diffusion-TS
python -m venv .venv-dts
source .venv-dts/bin/activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt
# kiểm tra GPU:
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

Dataset format của repo: bỏ dữ liệu mình vào thư mục `Data/` theo định dạng chúng đọc (`.npy` hoặc `.csv` 2D (`n_samples, seq_len*n_dim`) tùy tutorial); đọc `Tutorial_0/1/2.ipynb` để biết chính xác.

Chạy tutorial (từng bước)

```
jupyter nbconvert --to script Tutorial_1*.ipynb --output forward_tutorial # hoặc mở jupyter
python forward_tutorial.py # train
python backward_tutorial.py # sample -> lưu .npys
```

- `Tutorial_1` (forward): fit model + lưu `ckpt`.
- `Tutorial_2` (backward): sinh mẫu mới với guidance theo `conditional-column` nếu dataset của bạn có cột class.

Sinh "fault giả" cho pipeline A1: 1. Build dataset 2D: mỗi mẫu = window 512 của 1 kênh (hoặc window đa kênh flattened). - Nếu dùng signal thô: reshape về (n, win)=512 → sau khi sinh chạy `features.extract_features` (fs=12000) để khớp input script 02. - Nếu dùng feature: tự tính trước rồi lưu thẳng `.npy` dạng (n,16). 2. Train (khuyến nghị **có guidance theo class 0/1**: normal / fault) — chỉ có vậy khi train xong guide `sample(num_cond=1)` cho hàng loạt "fault". 3. Lưu `np.save("synthetic_faults.npy", arr_synth)` với `arr_synth.shape[1]==16` (feature).

Feed vào script 02:

```
python scripts/02_compare_augmentation.py --dataset ims --gen npy \
--gen-npy results/synthetic_faults.npy --limit 300
```

Checklist nhập môn Diffusion-TS: (1) khớp chiều dài chuỗi → win=512; (2) có cột class không? (không thì tự thêm label fault=1 cho synthetic); (3) giá trị scale hợp lý (chuẩn hoá về ~0-1 hoặc z-score trước khi train diffusion); (4) máy GPU ≥8GB VRAM cho seq dài.

4.4 Xung đột môi trường — mẹo vận hành

Khu	Dùng gì
<code>.venv/</code> (team)	scipy, torch CPU/GPU, sklearn — mọi script A1 chính
<code>vendor/TimeGAN/.venv-tf1</code>	TF1-API — chỉ để sinh <code>.npys</code>
<code>vendor/Diffusion-TS/.venv-dts</code>	torch + các dep của repo — sinh <code>.npys</code>

Khi scripting tự động: mỗi bước sinh nên **export** ra `.npy` rồi làm việc với file, không import chéo — tránh lệ thuộc môi trường của nhau.

5. HƯỚNG ĐI KỸ THUẬT 2 NHÁNH (đánh giá kỹ + đề xuất)

5.1 Nhánh 1 — Unsupervised anomaly + threshold (baseline bắt buộc)

Mô tả pipeline đang có (`scripts/01_baseline_anomaly.py`): 1. Chuyển mỗi chuỗi rung thành windows (`win=512, stride=256`) → features 16 chiều (`src/features.py`). 2. Train FeatureAE **chỉ trên window normal** (MSE trên reconstruction + ReLU MLP $16 \rightarrow 32 \rightarrow 8 \rightarrow 32 \rightarrow 16$). 3. Tính reconstruction error trên toàn bộ (normal + test fault); ngưỡng = **P99 của error normal-train** (`threshold_from_err(rec_err, 99.0)`). 4. Báo cáo precision/recall/F1/AUC trên test có fault thật.

Cách chọn ngưỡng — đúng bài toán: - P99 (mặc định): cho phép 1% normal bị báo động giả → xem precision. Nếu recall quá tệ (như kết quả hiện hành 0.031), thử P95, P97, P99.5 → vẽ **precision-recall curve khi quét percentile** để chọn điểm "đủ recall, đủ precision" (đây là biểu đồ giám khảo thích xem). - Mục tiêu không phải recall 1.0: mỗi mức ngưỡng là một trade-off; nêu rõ team chọn ngưỡng theo chi phí giả định (vd: 1 lỗi bỏ lọt = phí sửa máy X; 1 báo giả = phí stop-line Y).

Nhược điểm / hạn chế (bắt buộc kể trong slide): - AE học "xấp xỉ bình thường" → **vùng suy giảm SỚM** (fault mới chớm, gần normal) có reconstruction error gần bằng normal → dễ bị suy giảm chậm sẽ rơi dưới ngưỡng → recall thấp. - AE không dùng thông tin thời gian (window độc lập) → tín hiệu "đổ dần theo thời gian" bị bỏ. - Ngưỡng tính theo percentile → không thích ứng máy chạy load đổi (CWRU nhiều RPM). - Precision thấp khi fault hiếm (class imbalance nặng) — default threshold phóng đại số "anomaly". - Chỉ phù hợp khi **không có nhãn lỗi** (phase 1 của sản phẩm; bộ cảm biến mới lắp, chưa thu được lỗi). Đúng là deliverable 01 nhưng **không phải đích chiến thắng**.

Khi nào dùng nhánh này: bắt buộc làm để lấy baseline con số. Đồng thời là "vật cản" so sánh trong slide (trước augmentation) — phần "trước" của báo cáo.

5.2 Nhánh 2 — Augment + supervised classifier (KHẢ NĂNG THẮNG CAO)

Ý tưởng: Dùng normal (nhiều) + **rất ít fault thật (few-shot)** huấn luyện bộ sinh (GAN/ diffusion) → sinh **nhiều** window lỗi giả → gộp vào train → huấn luyện classifier supervised (RandomForest/XGBoost trên 16 features, hoặc 1D-CNN) → **đánh giá trên test = normal-unseen + fault THẬT chưa từng thấy**.

Pipeline hiện có khớp 100% (`scripts/02_compare_augmentation.py`): - Test-split: normal 80% train / 20% test-unseen; fault 20% train-scarce / 80% test-thật. - Base model: chỉ normal + ít fault thật (trước). Augmented model: thêm synthetic fault (sau). - Metrics: precision/recall/F1/AUC trên *cùng* test. - Cờ `--gen heuristic` (nhánh, protocol demo) và `--gen npy` (synthetic thật từ diffusion).

Điểm chống gian lận — phải quán triệt cả đội: 1. **Test là lỗi THẬT, chưa từng thấy** — không xào synthetic vào test, không dùng chính sample train để "đoán". 2. **Generator chỉ dùng tập train** (20% fault + normal-train): tuyệt đối không train generator trên window fault thuộc tập test (nếu generator nhìn test → đo trước/sau là trò cưỡi). 3. **Không có bóng dáng:** synthetic sinh từ generator đã học train → nếu test có sample của cùng ổ bị nhưng khác time, vẫn được; nhưng **không được sinh từ chính sample test**. 4. **Metric cần báo cả recall & precision** — đừng chỉ AUC (AUC kể chuyện thiếu trong scenario hiếm fault: bộ classifier tối ưu AUC vẫn có thể zero recall ở ngưỡng mặc định). 5. **Stratified k-fold** hoặc hold-out theo thời gian (train trước, test sau) — đúng bạn đã làm trong `_load_pipeline` (split theo thứ tự mảng), nói rõ trong slide để chứng minh sạch.

Cách sinh giả 3 mức (từ rẻ đến đắt — phân ở từng tuần): | Mức | Cách sinh | Phần cứng | Chất lượng | Dùng khi nào | |---|---| | heuristic | trộn noise + nhân biên độ lên normal (`0.8` noise, x2.5 biên độ) | CPU | thô, "linearly separable" | prototype protocol (đã có — gain ≈ 0 , đừng dùng làm số chính) | | TimeGAN | GAN TS với autocorrelation loss | CPU | trung bình, ít mode diversity | không có GPU, cần số tạm (`scripts/03`, ≥ 3000 iter) | | Diffusion-TS / FaultDiffusion | diffusion condition-by-class/few-shot | GPU | cao nhất, bám phân phối thật | demo chính + báo cáo cuối |

Trọng tâm báo cáo "trước/sau" (deliverable 3): - Bảng: Precision / Recall / F1 / AUC $\times$ (before, after, Δ). - PR curve (2 đường) — điểm nhìn quan trọng nhất; nếu synthetic tốt, đường "after" nằm phía trên bên phải với recall cao hơn tại cùng precision. - Bổ sung 2 metric của TimeGAN (discriminative/predictive) để chứng minh "synthetic = thật". - Ảnh so sánh: a) spectrogram 1 window lỗi thật vs 1 window lỗi giả (nhìn bằng mắt giống); b) t-SNE/Umap của normal/fault-thật/fault-giả để thấy fault-giả bao phủ vùng fault-thật.

Câu "bản chất gain" trả lời giám khảo: trước → model chỉ thấy vài mẫu fault, tỉ lệ nhất định giải nhất cũng chỉ là must-chance ≈ 1/6000 (imbalance) → recall ~0. Sau → model thấy đủ dạng fault → recall nhảy lên. Nếu gain không như kỳ vọng, đừng xóa số liệu — **báo "không đổi/giảm nhẹ" kèm lý do** (bộ sinh synthetic trùng dạng, threshold chưa xoay) cũng là kết quả trung thực hợp lệ. Thực tế 30/08: heuristic cho Δ recall ≈ 0.005 → đúng vậy, đó là động lực nâng cấp generator chứ không phải bug.

5.3 Phân tích khả thi theo thời gian / GPU / nhân lực

Nhánh	Thời gian hiện thực	GPU cần?	Nhân lực (5 người)	Mức rủi ro
Baseline AE (nhánh 1)	~1–2 ngày (đã có code chạy)	không (torch CPU OK)	1 người	thấp
Heuristic augmentation	~0.5 ngày (đã có)	không	1 người	không
TimeGAN sinh fault	3–6 ngày (tuning, ≥3000 iter)	không bắt buộc	1 người	trung bình (TF1 cũ, venv riêng)
Diffusion-TS sinh fault	5–10 ngày (train + tune seq_len)	bắt buộc GPU ≥8GB	1–2 người	trung bình → cao
Báo cáo + slide	2–3 ngày	—	cả đội	thấp nếu chạy sớm

Khuyến nghị cuối cùng (chốt): chạy NHÁNH 1 + heuristic ngay tuần 2–3 để có số; đặt GPU cho Diffusion-TS tuần 6; nếu hết GPU hồi tuần 8 → fallback TimeGAN CPU + kiểm soát câu chuyện bằng 2 metric của TimeGAN. **Không bao giờ để "số slide" chờ diffusion;** bảng trước/sau của heuristic đã đủ minh chứng protocol, diffusion chỉ làm số đẹp hơn.

6. KẾ HOẠCH CHI TIẾT THEO TUẦN (26/08 → 12/10/2026)

Lưu ý: hôm nay 30/08 — **tuần 1 đang ở ngày cuối**. Mốc CỨNG đầu tiên là **đăng ký 31/08 (mai)**. Factory Tour 11/09, nộp Vòng 1 12/10, chung kết 16/11.

6.1 Bảng timeline tổng quan

Tuần	Ngày (2026)	Giai đoạn	Việc chính	Đầu ra (exit criteria)
1	26/08–31/08	Khởi động + đăng ký	đăng ký đội (≤31/08), lập repo, chốt stack, tải data	repo sạch, data sẵn, đội đăng ký thành công
2	01/09–07/09	Baseline	feature + AE + ngưỡng	results/baseline_ae.json + PR curve
3	08/09–11/09	Baseline hoàn thiện	quét ngưỡng P95–P99.5, thêm dashboard sơ bộ	bảng confident, dashboard MVP, chuẩn bị câu hỏi tour
4	11/09 (Bootcamp + Factory Tour)	Học từ nhà máy	hỏi kỹ sư DENSO, ghi chép, chỉnh hiểu biết về "lỗi thật"	danh sách Q&A, refinements process
5	14/09–20/09	Augmentation heuristic	gắn pipeline 02 + heuristic lên 3 dataset	bảng compare heuristic trước/sau
6	21/09–27/09	Sinh synthetic (GPU)	clone Diffusion-TS + TimeGAN (≥3000 iter), sinh fault giả	synthetic .npy đầu tiên (vài nghìn window)
7	28/09–04/10	Sinh synthetic hoàn thiện	tune, sinh đủ, evaluate bằng 2 metric TimeGAN	siêu chất lượng synthetic + discriminative score
8	05/10–09/10	Báo cáo + dashboard	composite báo cáo trước/sau, dashboard RUL+alert	báo cáo PDF/dashboard hoàn chỉnh
9	10/10–12/10	Slide Vòng 1 + nộp	viết slide, chạy lại số liệu sạch, nộp	file nộp 12/10

6.2 Tuần 1 (26/08–31/08) — Khởi động & ĐĂNG KÝ

Mục tiêu không thể trượt: đăng ký xong trước 31/08 trên **densohackathon.vn** (1 tài khoản đội = 1 bài nộp — đội 3–5 người; xác định ai là owner tài khoản). **Đã ở ngày cuối tuần 1 (30/08) — ưu tiên hàng đầu hôm nay/mai.**

Checklist: - [] **Đăng ký đội + điền ý tưởng sơ bộ (deadline 31/08).** — LÀM NGAY. - [] Git repo tồn tại; README + `.gitignore` (data/ bỏ qua; data lớn 300MB+FEMTO không lên git). - [] Xác nhận 3–5 thành viên + phân vai: (a) Data & feature, (b) Mô hình baseline/AE, (c) GAN/Diffusion pipeline, (d) Dashboard/frontend, (e) Slide + tài liệu. Dù vai nào cũng phải đọc `RESOURCES.md`. - [] Chạy `bash scripts/download_data.sh` (idempotent — tự bỏ qua phần đã có); kiểm tra `data/NASA_IMS/test_{1,2,3}/` và `data/FEMTO/`. - [] CWRU: tải tay từ trang chính thức phần fault (105, 118, 130...); **bỏ qua normal bị hỏng** nếu vẫn không lấy được — dùng IMS làm nguồn normal. - [] Chạy thử `python scripts/01_baseline_anomaly.py --dataset ims --limit 200` → có JSON. - [] Chạy thử `python scripts/02_compare_augmentation.py --dataset ims --gen heuristic --limit 200` → hiểu protocol. - [] Trả lời được bảng câu hỏi "đúng giám khảo": deliverable là gì? leakage là gì trong bối cảnh này?

Vai không được để thiếu: người viết slide đọc toàn bộ catalog paper ngay tuần 1 để slide không "đoán paper".

6.3 Tuần 2–3 (01/09–11/09) — Feature + AE baseline

Tuần 2: hoàn thiện baseline trên IMS + CWRU. - [] Chạy baseline với vài cấu hình `--win (256/512/1024) × --stride (128/256) × --thr (95/97/99/99.5)` → ghi 1 bảng sweep. - [] Vẽ **PR curve** của AE (quét ngưỡng) → chọn ngưỡng hợp lý (không cứng P99). - [] Ghi `results/` nhiều JSON để slide sau này "kể" quá trình chọn ngưỡng. - [] (Nếu có) thử `--dataset cwr` (chỉ fault để xem recall — normal không đọc được thì bỏ).

Tuần 3: dọn dashboard MVP (alert line + RUL ước lượng) + chuẩn bị Factory Tour. - [] Dashboard: hiển thị score bất thường theo thời gian, đánh dấu vượt ngưỡng, đếm trong test thực. - [] Thu thập 10 câu hỏi Factory Tour (xem §7.4) — giao cho 2 người có mặt. - [] Tự set expectation: số AE hiện tại (AUC 0.52) sẽ TĂNG khi dùng supervised-aug (đó là câu chuyện chính, không phải bug). - [] Viết nháp intro slide survey "scarce fault data in manufacturing" (thu thập 3–5 số liệu công bố để mở đầu).

Exit criteria: bảng sweep ngưỡng + PR curve saved trong `results/`; dashboard chạy được cục bộ.

6.4 Tuần 4 (11/09) — Bootcamp + Factory Tour: Hỏi kỹ sư DENSO

Đây là "phòng thí nghiệm minh bạch" duy nhất — tận dụng tối đa. Mục tiêu: thu được câu trả lời để (a) chỉnh hướng kỹ thuật, (b) làm slide thăm thía bài toán thực.

Checklist trước tour: - [] In sẵn 10 câu hỏi + bảng so sánh dataset (mang theo file này). - [] Phân công người ghi chép (answers log) — không để cuộc trò chuyện trôi. - [] Chuẩn bị 1-pager mô tả protocol trước/sau để kỹ sư góp ý trực tiếp.

Sau tour (ngay trong tuần 4): - [] Kết xuất Q&A log bằng markdown trong repo (`notes/factory_tour_notes.md`). - [] Map 3 câu hỏi quan trọng nhất vào pipeline (nếu câu trả lời đối thiết kế: ví dụ "sensor nào ít nhiễu nhất khi có lỗi" → chọn kênh, "bao lâu giữa cảnh báo → hỏng" → nhịp threshold). - [] Bổ sung phần "Factory insight" vào slide (điểm cộng: chứng minh đội nghe và áp dụng).

6.5 Tuần 5–9 (14/09–09/10) — Augmentation + So sánh + Dashboard

Tuần 5 — pipeline augmentation heuristic chạy trên 2–3 dataset. - [] Chạy `--gen heuristic` trên `ims` và `cwr` (`cwr`: `--limit flood?` xem hàm `_load_pipeline` có hỗ trợ `cwr` — có nhánh `cwr`). Ghi 2 bảng before/after. - [] Tạo 2 biểu đồ PR curve trước/sau (lưu vào `results/*.png`). - [] Xác nhận đường leak: file JSON có `"n_synth"`, `"n_test_fault"` — kiểm tra luôn test chỉ fault THẬT (không trùng train) bằng code check (người làm data viết assertion).

Tuần 6 — synthetic thật (GPU): - [] Clone 2 repo, set môi trường (đúng bài học §4). Bật GPU ≥8GB (rủi ro tìm máy: xếp sớm). - [] Fabric dữ liệu train generator từ IMS: normal 80% train, 20% fault thật (không chạm test). - [] Output `.npy` thử nghiệm 1000–3000 window phần fault; chạy `02 --gen npy --gen-npy ...`. - [] Đo discriminative score (TimeGAN metric): trainer "real vs fake" classifier trên features → accuracy nên ~0.5–0.65 (đẹp là 0.5–0.6).

Tuần 7 — tuning & mở rộng: - [] Tuning: `seq_len`, `guidance-strength`, % upweight fault thật lên 0.35–0.5 (follow FaultDiffusion). - [] Sinh đủ tổng synthetic target: sao cho class balance trong train ≈ 1:1 (thay vì 1:6000) → recall tăng mạnh. - [] Chạy 3 cấu hình generator → chọn tốt nhất bằng metric dưới; lưu tất cả trong `results/`. - [] Có ít nhất 1 kết quả "2 metric TimeGAN trên synthetic của tôi" để nói trong slide.

Tuần 8 — báo cáo cuối + dashboard hoàn chỉnh: - [] Ghost-run lại toàn bộ pipeline từ sạch (tải data → heuristic → synthetic → compare) với seed cố định → số cuối cùng quyết định. - [] Dashboard bản cuối: chuỗi thời gian đa kênh + anomaly score + ngưỡng + alert + RUL. - [] Báo cáo markdown/PDF với bảng before/after + PR curve + spectrogram (tín hiệu thật vs giả) + t-SNE + phần methodology/limitations. - [] File `.npy` synthetic lưu trong `results/` (làm deliverable "tập dữ liệu tăng cường").

Tuần 9 (10/10–12/10) — slide + nộp: xem §7; chạy lại số liệu để mọi con số khớp file nộp.

Exit criteria tổng: `scripts/*` chạy end-to-end (README), 3 deliverable đủ, repository "reproducible log" (mọi số trong slide được sinh từ command ghi trong README).

6.6 Checklist nộp cuối cùng (đối chiếu deliverable BTC)

- [] **Bộ sinh dữ liệu bất thường**: code + README hướng dẫn (diffusion/TimeGAN/heuristic) — kèm cách chạy.
- [] **Tập dữ liệu tăng cường**: `results/synthetic_faults.npy` + mô tả số lượng/đặc điểm.
- [] **Báo cáo trước/sau**: bảng Precision/Recall/F1/AUC + PR curve + `n_synth/n_test_fault` rõ.
- [] **Dashboard** alert + RUL.
- [] **Slide Vòng 1** (10–15 slide) — xem §7.
- [] File nộp đúng định dạng (pdf/slides) lên đúng tài khoản đội trước 12/10 23:59.

7. CHIẾN LƯỢC SLIDE VÒNG 1 (hạn 12/10)

7.1 Nguyên tắc vàng

- 10–15 slide, không hơn.** Mỗi slide 1 ý. Giám khảo đọc <60s/slide.
- Số liệu là vua:** mọi nhận định phải có con số (`results/*.json`) đi kèm; hình minh họa (PR curve, spectrogram, t-SNE) thay lời.
- Kể chuyện "kịch bản scarce data"** — mở đầu bằng đau thật (thiếu fault data), kết bằng protocol tái sử dụng được.
- Cite paper hợp lệ:** FaultDiffusion (arXiv:2511.15174), TimeGAN (NeurIPS 2019), Diffusion-TS (ICLR 2024) — ghi đúng venue/ID, không đoán link.

7.2 Cấu trúc slide đề xuất

#	Slide	Nội dung	Hình chính
1	Title	Tên đội, "Predictive Maintenance with Scarce Fault Data — AI Augmentation Approach"	logo + tên
2	Problem	Nhà máy: nhiều normal, gần như không có lỗi → baseline supervised chết; chi phí báo giả vs bỏ lọt	1 con số fact + sơ đồ dòng
3	Objective + Deliverables	3 deliverable của đề (sinh fault, dataset tăng cường, so sánh trước/sau)	3 box
4	Dataset	CWRU / IMS / FEMTO — 1 bảng nhỏ (nguồn, thời lượng, nhãn). Nói rõ "IMS là main, CWRU phụ"	bảng + thumbnail tín hiệu
5	Baseline approach (unsupervised)	Autoencoder + reconstruction error + P99 → "trước"	sơ đồ AE + PR curve "before"
6	Augmentation — the core	Sinh lỗi giả bằng TimeGAN / Diffusion-TS / FaultDiffusion; heuristic làm crosscheck	sơ đồ pipeline sinh
7	Augmentation protocol (anti-leak)	test = normal-unseen + fault THẬT chưa thấy; generator train chỉ trên train; split theo thời gian	sơ đồ split
8	Baseline metrics (before)	bảng + PR curve "before" + scan ngưỡng P95–99.5	bảng số + PR
9	Augmented metrics (after)	bảng + PR curve "after" + đồ thị so	2 đường PR chồng
10	Synthetic quality	discriminative/predictive score (TimeGAN), t-SNE normal/fault-thật/fault-giả	t-SNE + spectrogram thật vs giả
11	Why it works	few-shot conditioning của FaultDiffusion → synthetic bám dạng lỗi thật; ít noise mode-collapse	câu + 1 hình
12	Limitations + Risks	threshold tĩnh, máy khác sensor, GPU cần thiết → nêu trung thực + direction	3 bullet
13	Roadmap → RUL + next	từ anomaly → RUL (FEMTO/C-MAPSS), dashboard alert	timeline mini
14	Team & Plan	3–5 người, roles, sprint 9 tuần	bảng team

#	Slide	Nội dung	Hình chính
15	Thank You / Q&A	link repo demo + invite hỏi	—

Có thể gộp 12-13 nếu cần giữ 13 slide; đừng thêm slide "About us" dài.

7.3 Điểm "ăn điểm" quan trọng

- 1. **Frame "scarce data" làm research chủ đề:** mở đầu bằng phát biểu nhận thức (có survive thị trường AI công nghiệp: rất ít bài làm đúng scarce-data; đa số cheat). Trung thực = khác biệt.
- 2. **PR curve TRƯỚC/SAU:** một dòng "Recall 0.01 → 0.8, same precision 0.7" đáng giá 1000 chữ. Đảm bảo đường sau nằm phía trên phải. (Số thật hiện tại: RF recall 0.477 → 0.482 — chưa đủ ấn tượng; phải nâng bằng XGBoost tuning + TimeGAN/diffusion trước khi đưa lên slide.)
- 3. **Trích dẫn paper đúng:** FaultDiffusion (arXiv:2511.15174) cite ở slide 11 có weight hơn nếu bạn ghi thêm "method we adapted for few-shot conditioning". Dùng 2 metric TimeGAN ở slide 10 → chúng tỏ biết cách đánh giá generative model.
- 4. **Một slide "leakage check" sẽ gây ấn tượng mạnh:** "chúng tôi chủ động auto-assert rằng test chỉ có fault thật" + code snippet nhỏ.
- 5. **Con số nhất quán repo ↔ slide:** chạy cùng command, seed cố định; slide ghi đúng JSON.
- 6. **Dashboard mini demo chạy trực tiếp** (nếu cho phép) — live demo ứng dụng vào vòng chung kết (16/11), nên "chỉ demo an toàn" (không phụ thuộc mạng).

7.4 Câu hỏi nên hỏi kỹ sư DENSO tại Factory Tour (11/09)

Không hỏi câu "set up giúp em" — hỏi câu làm lộ **đặc tính lỗi thật** để đội tinh chỉnh dataset:

- 1. "Khi máy bắt đầu hỏng (vỡ ổ bi / lệch trục), sensor nào cho tín hiệu rõ NHẤT trước tiên: rung hướng nào, tần số nào?" → chọn kênh + band tần số trong features.
- 2. "Khoảng cách giữa dấu hiệu cảnh báo sớm và hỏng hẳn là bao lâu (giờ/ngày) trong dây chuyền của anh chị?" → config ngưỡng + cửa sổ dự báo.
- 3. "Anh chị có dữ liệu lỗi cũ nào (history log) không? Khoảng bao nhiêu % tổng dữ liệu?" → xác định mức "scarce" thực tế; nếu họ có cả TB normal + TB fault thì bài toán ít khắc nghiệt hơn.
- 4. "Nhiều vận hành bình thường khác với lỗi thế nào (rung bình thường đã cao do máy chạy nhiều trục)" → tune baseline tránh báo giả.
- 5. "Cách thu thập data lỗi hiện tại là thủ công hay tự động; mỗi lỗi ghi được bao nhiêu giây?" → đánh giá khả thi mở rộng method để giám khảo tin "protocol tái sử dụng".
- 6. "Loại lỗi phổ biến nhất của thiết bị rung tại DENSO là gì (bearings, gears, misalignment)?" → chọn dataset CWRU/FEMTO hay cần thêm loại lỗi khác.
- 7. "Hệ thống hiện tại có cảnh báo gì chưa (thủ công theo lịch định kỳ?) → muốn AI thay/nâng?" → vị trí sản phẩm của AI trong quy trình.
- 8. "Tiêu chí KPI của họ chấp nhận bao nhiêu báo giả / cho phép bỏ lọt bao nhiêu lỗi?" — đặt ngưỡng đúng bài toán (nếu trả lời "0 bỏ lọt" → threshold rất thận trọng; "ít báo giả" → PR trade-off).
- 9. "Có thu được tín hiệu trước-sau bảo trì khi thay ổ bi mới không (để làm augmentation chuẩn hơn)?" → insight "normal mới" so với "normal mòn dần".
- 10. "Nhân viên vận hành có kinh nghiệm nhận diện âm thanh/mùi trước khi hỏng không — có thể gán nhãn phụ không?" → phương án bổ sung nhãn thô.

Cách dùng các câu trả lời: - Mỗi câu trả lời ghi vào `notes/factory_tour_notes.md` kèm action trong pipeline (vd: câu 1 → thêm channel-weighted feature; câu 4 → thêm band-pass filter cho normal-nuance). - Nhấn mạnh 2-3 insight hay nhất ngay trong slide 12 "Factory-driven refinement" — đây là điểm riêng biệt team có mà nhóm khác không.

8. QUYẾT ĐỊNH NHANH (DECISION GUIDANCE)

Dành cho kỹ sư cần chọn nhanh "dùng cái gì" mà không phải đọc lại toàn bộ. Map trực tiếp với 6 bài toán con [a]-[f] trong Bản đồ tư duy A1.

8.1 Bảng chọn mô hình theo bài toán con

Bài toán con	Chọn mặc định	Khi nào đổi	Trở tới (code)
[a] Nén tín hiệu	16 features, win=512, stride=256	window ngắn → win=1024 ; nhiều tần số cao → thêm band-pass	src/features.py , src/pipeline.py
[b] Định nghĩa tốt/xấu	PR-AUC + precision/recall/F1, ngưỡng P99	có chi phí thực (báo giả vs bỏ lọt) → chọn ngưỡng theo chi phí	metrics trong scripts/01 , scripts/02
[c] Không có nhãn	Autoencoder + reconstruction error	AE dưới trung bình → IsolationForest / OC-SVM	scripts/01 (AE)
[d] Ít nhãn	RandomForest trên 16 features	cần chính xác hơn / tuning → XGBoost	scripts/02
[e] Sinh lỗi giả	TimeGAN (CPU) ≥3000 iter	có GPU → Diffusion-TS (condition-by-class); sát đề → FaultDiffusion few-shot	scripts/03 · vendor/
[f] Chứng minh trước/sau	--gen heuristic + 3-zone split	có synthetic.npy → --gen npy	scripts/02 , src/pipeline.py

8.2 Decision guidance — luồng quyết định (đọc theo thứ tự)

1. **Đăng ký đội chưa?** Hạn 31/08 (mai). Chưa → dừng mọi việc, đăng ký trước.
2. **Có nhãn lỗi không?** - Không → chạy AE baseline (scripts/01), ngưỡng P99. Đó là cột mốc "trước" (deliverable 3). - Có (ít) → sang bước 3.
3. **Có GPU CUDA ≥8GB không?** - Không → TimeGAN CPU (scripts/03 , ≥3000 iter) sinh .npy ; heuristic làm đối chứng thêm. - Có → Diffusion-TS condition-by-class sinh fault giả; FaultDiffusion nếu trả lời được câu "synthetic bị tách ra ngay không".
4. **Cần số nhanh?** → heuristic trước, synthetic sau. Không để số slide chờ GPU.
5. **Bất kỳ con số trước/sau nào** cũng phải đi qua 3-zone split, generator chỉ học train, test chỉ lỗi THẬT. Vi phạm → số bỏ đi (leakage).
6. **Chọn đáp án "rẻ nhất" đủ chứng minh protocol**, nâng cấp sau nếu còn thời gian/GPU.

PHỤ LỤC A — Các lệnh chạy end-to-end (để tái lập số liệu trong slide)

```
# Bước 0 – chuẩn bị
source .venv/bin/activate

# Bước 1 – data
bash scripts/download_data.sh # idempotent: NASA IMS + FEMTO
# (CWRU tải tay từ trang chính thức; normal hỏng → fallback IMS)

# Bước 2 – baseline (mục tiêu "trước")
python scripts/01_baseline_anomaly.py --dataset ims --limit 300 \
--out results/baseline_ae_ims300.json
# quét ngưỡng
python - <<'PY'
from src import ae, data, features
import numpy as np, glob
# ... (sweep threshold bằng nội bộ src, xem README)
PY

# Bước 3 – augmentation protocol (heuristic – chạy nhanh, số tạm)
python scripts/02_compare_augmentation.py --dataset ims --gen heuristic --limit 300 \
--out results/compare_heuristic.json

# Bước 3b – TimeGAN (CPU, ≥3000 iterations) → sinh .npy
python scripts/03_train_timegan.py --dataset ims ...
# output -> results/synthetic_faults.npy
python scripts/02_compare_augmentation.py --dataset ims --gen npy \
--gen-npy results/synthetic_faults.npy --limit 300 \
--out results/compare_timegan.json

# Bước 4 – synthetic thật (sau khi có GPU; xem §4)
# vendor/Diffusion-TS/... sample -> results/synthetic_faults.npy
python scripts/02_compare_augmentation.py --dataset ims --gen npy \
--gen-npy results/synthetic_faults.npy --limit 300 \
--out results/compare_diffusion.json
```


PHỤ LỤC B — Những cạm bẫy dễ khiến đội "ăn điểm xấu" (đọc 1 lần/tuần)

- 1. **Leak**: generator nhìn thấy window test; hoặc split window cùng file vào cả train/test.
- 2. **Số liệu đẹp giả**: train-test chung file CWRU; hoặc báo AUC mà không báo recall/precision.
- 3. **Synthetic "giả vờ nhưng chấm sai"**: model phân loại real-vs-fake quá dễ (accuracy ~1.0) → nêu trong slide như warning.
- 4. **Quên đăng ký 31/08** → cả dự án vô nghĩa (đặt reminder + chốt owner).
- 5. **FEMTO/C-MAPSS vướng format**: load bằng `.csv` ; mọi cột khớp; không để lẫn file zip.
- 6. **GPU tìm muộn** → diffusion trễ → fallback TimeGAN CPU kịp; đừng chờ GPU đến phút chót.
- 7. **Bỏ mặc venv**: 3 môi trường (team / TimeGAN / Diffusion-TS) tách bạch, không `pip install` lung tung và làm hỏng `.venv/` chính.
- 8. **Bồn chồn vì gain heuristic ≈ 0** → đó là bằng chứng cần generator thật, đưa thẳng vào slide phần "motivation" thay vì giấu.

PHỤ LỤC C — Tài liệu nên đọc lại theo vai

Vai	Đọc cái gì
Cả đội	README.md, RESOURCES.md, file này mục 1–5
Data & feature	file này mục 2 (dataset), 5.1 (threshold)
Mô hình/ML	file này mục 3–4 (paper + repo), 5.2
Dashboard	file này mục 6.3/6.5 (dashboard spec)
Slide	file này mục 1.5 (mùa trước), 3 (trích dẫn), 7 (slide)
Người hỏi DENSO	mục 7.4 (câu hỏi tour)
Kỹ sư quyết định nhanh	file này mục 8 (decision guidance) + Bản đồ tư duy A1 ở mở đầu

Tài liệu khếp chặt với mã nguồn hiện tại của team (A1_predictive_maintenance) và đã dùng đúng số liệu thật từ `results/` (cập nhật 30/08/2026). Mọi link trong tài liệu này đều đã được verify tại ngày 08/2026.